Doctoral Thesis in Computer Science

Fairness and Diversity-Aware Algorithms: Ranking, Streaming, and Graph Analysis

HONGLIAN WANG

KTH ROYAL INSTITUTE OF TECHNOLOGY

Fairness and Diversity-Aware Algorithms: Ranking, Streaming, and Graph Analysis

HONGLIAN WANG

Academic Dissertation which, with due permission of the KTH Royal Institute of Technology, is submitted for public defence for the Degree of Doctor of Philosophy on Tuesday the 5th of May 2026, at 2:00 p.m. in F3, Lindstedtvägen 26, Stockholm.

Doctoral Thesis in Computer Science
KTH Royal Institute of Technology
Stockholm, Sweden 2026

© Honglian Wang
© Aristides Gionis, Sijing Tu, Lutz Oettershagen, Haoyun Zhou, Florian Adriaens

TRITA-EECS-AVL-2026:34
ISBN 978-91-8106-581-7

Printed by: Universitetsservice US-AB, Sweden 2026

Abstract

As algorithmic systems increasingly shape human experiences, ensuring fairness and diversity has become a central challenge. This thesis studies fairness and diversity through the lens of algorithm design and optimization theory, providing formal frameworks and efficient algorithms across three domains: ranking-based recommendation, streaming recommendation, and graph analysis.

The first part of the thesis investigates diversity maximization in recommender systems with stochastic user engagement. We first study how to rank items in recommendation systems, where users engage with content sequentially and probabilistically. We introduce two novel diversity measures, sequential sum diversity and sequential coverage diversity, which account for uncertainty in user engagement. Our goal is to find a ranking of items that maximizes these sequential diversity measures. We show that sequential coverage diversity is ordered submodular, enabling a greedy $\frac{1}{2}$-approximation. For sequential sum diversity, we provide polynomial-time constant-factor approximation algorithms. Separately, we study a streaming setting where items arrive continuously and users may visit the system multiple times at arbitrary moments. For this setting, we aim to design a streaming algorithm that maximizes a stochastic coverage diversity measure. We show that a classic greedy algorithm achieves a tight $\frac{1}{2}$-competitive ratio but requires memory linear in the stream length. With sublinear memory and an upper bound T' on the number of user visits T, we propose STORM, which achieves a $\frac{1}{4(T'-T+1)}$-competitive ratio. We further propose STORM++ improving the competitive ratio to $\frac{1}{8\delta}$, where the integer parameter δ controls the tradeoff between solution quality and computational cost.

The second part of the thesis studies diversity as a constraint in densest subgraph discovery and addresses the problem of finding dense communities in networks with heterogeneous relationship types. We model relationship types as edge colors and formulate the At Least **h** Colored Edges Densest Subgraph problem (ALHCEDGESDSP), which seeks subgraphs that are both dense and contain at least h_i edges of each color i. We prove that even the simplest variant of this problem is NP-hard and develop constant-factor approximation algorithms. Our key technical contribution links the edge-constrained and node-constrained versions of the densest subgraph problem. We first show that algorithms for the At Least k Nodes Densest Subgraph problem (Dal$k$S) can approximate the At Least h edges Densest Subgraph problem (ATLEASTHEDGESDSP), and then extend the algorithm for Dal$k$S to handle colored edge constraints for solving ALHCEDGESDSP.

The third part of the thesis studies graph interventions for fairness in networks. We examine two fairness measures, PageRank fairness and hitting-time fairness, developing methods to balance influence and improve accessibility across groups. For each demographic group, the sum of PageRank scores within it quantifies the influence of that group. PageRank fairness measures how far the current group-wise influence deviates from a given target. We formulate the PageRank fairness problem as an optimization problem that adjusts edge weights such that the resulting graph achieves a group-wise influence distribution as close to the target as possible. The optimization problem involves a nonconvex objective over a convex feasible set under practical constraints, such as not adding new

edges and limiting the magnitude of weight changes. We solve this PageRank fairness maximization problem using efficient projected gradient descent, proving convergence to a stationary point. For hitting-time fairness in bipartite graphs, we formulate two problems, minimizing the average (BMAH) and the maximum hitting time (BMMH) from one group to another via strategic edge additions. We provide a $(2 + \epsilon)$-approximation for BMAH by combining fast random walk simulation with greedy supermodular minimization. For the more challenging BMMH problem, we develop two approaches, the first leverages its connection to the BMAH problem, and the second employs a method based on the asymmetric k center problem. Both approaches yield provable approximation guarantees for BMMH.

The algorithms and analysis techniques presented in this thesis contribute to the growing body of work on fairness and diversity in algorithmic systems. By formalizing new problem variants that capture realistic constraints in interactive and networked settings, and by providing approximation algorithms with provable guarantees, this work expands the toolkit available for addressing fairness and diversity challenges in computational systems.

Sammanfattning

Eftersom algoritmiska system i allt högre grad formar mänskliga upplevelser har det blivit en central utmaning att säkerställa rättvisa och mångfald. Denna avhandling studerar rättvisa och mångfald genom algoritm design och optimeringsteori, och tillhandahåller formella ramverk och effektiva algoritmer inom tre domäner: rankningsbaserad rekommendation, strömmande rekommendation och grafanalys.

Den första delen av avhandlingen undersöker mångfaldsmaximering i rekommendationssystem med stokastiskt användarengagemang. Vi studerar först hur man rangordnar objekt i rekommendationssystem, där användare engagerar sig med innehåll sekventiellt och probabilistiskt. Vi introducerar två nya mångfaldsmått, sekventiell summa-mångfald och sekventiell täcknings-mångfald, som tar hänsyn till osäkerhet i användarengagemang. Vårt mål är att hitta en rangordning av objekt som maximerar dessa sekventiella mångfaldsmått. Vi visar att sekventiell täcknings-mångfald är ordnad submodulär, vilket möjliggör en girig $\frac{1}{2}$-approximation. För sekventiell summa-mångfald tillhandahåller vi polynomtids konstant-faktor approximationsalgoritmer. Separat studerar vi en strömmande miljö där objekt anländer kontinuerligt och användare kan besöka systemet flera gånger vid godtyckliga tidpunkter. För denna miljö strävar vi efter att designa en strömmande algoritm som maximerar ett stokastiskt täcknings-mångfaldsmått. Vi visar att en klassisk girig algoritm uppnår ett tight $\frac{1}{2}$-konkurrensförhållande men kräver minne linjärt i strömlängden. Med sublinjärt minne och en övre gräns T' på antalet användarbesök T, föreslår vi STORM, som uppnår ett $\frac{1}{4(T'-T+1)}$-konkurrensförhållande. Vi föreslår vidare STORM++ som förbättrar konkurrensförhållandet till $\frac{1}{8\delta}$, där heltalsparametern δ kontrollerar avvägningen mellan lösningskvalitet och beräkningskostnad.

Den andra delen av avhandlingen studerar mångfald som en bivillkor i upptäckt av tätaste delgrafer och behandlar problemet med att hitta täta gemenskaper i nätverk med heterogena relationstyper. Vi modellerar relationstyper som kantfärger och formulerar problemet At Least **h** Colored Edges Densest Subgraph (ALHCEDGESDSP), som söker delgrafer som är både täta och innehåller åtminstone h_i kanter av varje färg i. Vi bevisar att även den enklaste varianten av detta problem är NP-svår och utvecklar konstant-faktor approximationsalgoritmer. Vårt viktigaste tekniska bidrag kopplar samman kant-begränsade och nod-begränsade versionerna av tätaste delgraf-problemet. Vi visar först att algoritmer för problemet At Least k Nodes Densest Subgraph (Dal$k$S) kan approximera problemet At Least h edges Densest Subgraph (ATLEASTHEDGESDSP), och utökar sedan algoritmen för Dal$k$S för att hantera färgade kantbegränsningar för att lösa ALHCEDGESDSP.

Den tredje delen av avhandlingen studerar grafinterventioner för rättvisa i nätverk. Vi undersöker två rättvisemått, PageRank-rättvisa och träfftids-rättvisa, och utvecklar metoder för att balansera inflytande och förbättra tillgänglighet mellan grupper. För varje demografisk grupp kvantifierar summan av PageRank-poäng inom den gruppens inflytande. PageRank-rättvisa mäter hur långt den nuvarande gruppvisa influensfördelningen avviker från ett givet mål. Vi formulerar PageRank-rättvisamproblemet som ett optimeringsproblem som justerar kantvikter så att den resulterande grafen uppnår en gruppvis influensfördelning så

nära målet som möjligt. Optimeringsproblemet involverar ett icke-konvext mål över en konvex genomförbar mängd under praktiska begränsningar, såsom att inte lägga till nya kanter och begränsa kantviktningsskalan. Vi löser detta PageRank rättvisemaximerings-problem med hjälp av effektiv projicerad gradientnedstigning, och bevisar konvergens till en stationär punkt. För träfftids-rättvisa i bipartita grafer formulerar vi två problem, att minimera medelvärdet (BMAH) och det maximala träfftidsvärdet (BMMH) från en grupp till en annan via strategiska kanttillägg. Vi tillhandahåller en $(2 + \epsilon)$-approximation för BMAH genom att kombinera snabb slumpvandrings-simulering med girig supermodulär minimering. För det mer utmanande BMMH-problemet utvecklar vi två tillvägagångssätt, det första utnyttjar dess koppling till BMAH-problemet, och det andra använder en metod baserad på det asymmetriska k center-problemet. Båda tillvägagångssätten ger bevisliga approximationsgarantier för BMMH.

Algoritmerna och analystekniker som presenteras i denna avhandling bidrar till den växande mängden arbete om rättvisa och mångfald i algoritmiska system. Genom att formalisera nya problemvarianter som fångar realistiska begränsningar i interaktiva och nätverksbaserade miljöer, och genom att tillhandahålla approximationsalgoritmer med bevisliga garantier, utökar detta arbete verktygslådan som finns tillgänglig för att hantera utmaningar kring rättvisa och mångfald i beräkningssystem.

List of Papers

Papers Included

A ***Sequential Diversification with Provable Guarantees***
Honglian Wang, Sijing Tu, and Aristides Gionis
Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (2025)

B ***Streaming Stochastic Submodular Maximization with On-Demand User Requests***
Honglian Wang, Sijing Tu, Lutz Oettershagen and Aristides Gionis
Thirty-ninth Annual Conference on Neural Information Processing Systems (2025)

C ***Finding Densest Subgraphs with Edge-Color Constraints***
Lutz Oettershagen, **Honglian Wang** and Aristides Gionis
Proceedings of the ACM Web Conference (2024)

D ***Fairness-aware PageRank via Edge Reweighting***
Honglian Wang, Haoyun Zhou, and Aristides Gionis
Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (2026)

E ***Minimizing Hitting Time Between Disparate Groups with Shortcut Edges***
Florian Adriaens, **Honglian Wang** and Aristides Gionis
Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (2023)

Papers Not Included

i. ***Scalable Graph Classification via Random Walk Fingerprints***
Peiyan Li, **Honglian Wang** and Christian Böhm
IEEE International Conference on Data Mining (2024)

ii. ***Influence Without Authority: Maximizing Information Coverage in Hypergraphs***
Peiyan Li, **Honglian Wang**, Kai Li and Christian Bohm
SIAM International Conference on Data Mining (2023)

Claim of Contribution

In Paper A, the problem was proposed by Aris and formulated by Aris and Sijing. The author and Sijing contributed equally to the theoretical analysis and writing. All authors participated in regular discussions to verify proofs and experimental design. The author conducted the experiments.

In Paper B, all authors contributed to conceptualizing the problem and participated in regular discussions. The author and Sijing contributed to the theoretical analysis. The author conducted the experiments. All authors contributed to the writing.

In Paper C, the problem was proposed and formulated by Lutz and Aris. Lutz is the main contributor to the theoretical analysis, experiments, and writing. The author contributed to the theoretical analysis and writing. All authors participated in regular discussions.

In Paper D, the problem was proposed and formulated by Haoyun and Aris. Haoyun and Aris formed the initial draft of the paper. Haoyun completed the initial version of experiments and wrote the abstract, introduction, methods section, and preliminary experimental results before discontinuing work on the project. The author corrected the initial projected gradient descent algorithm and provided convergence analysis for it. The author verified and revised Haoyun's source code, re-ran all experiments, and added new experiments. Aris and the author participated in regular discussions until the paper was completed. The author rewrote the methods and experimental sections. Aris contributed to writing the abstract and introduction, and polished and revised the author's written drafts.

In Paper E, Florian is the main contributor to the problem formulation, theoretical analysis, and writing. The author conducted most of the experiments. The author and Aris contributed to proofreading and polishing the writing. All authors participated in regular discussions.

Acknowledgements

My doctoral journey has been a profound process of pushing beyond my intellectual comfort zone. I entered this field with an ambitious naivety, which was followed by a humbling phase where I felt like a child playing in the mud outside the grand temple of academia. Eventually, I reached a point where I could contribute something uniquely my own. Through persistence, I dissolved my self-doubt, expanded my horizons, and learned to navigate the pressures of research. Looking back, I am most grateful to the version of myself who came very close to quitting but ultimately chose to keep going.

I am deeply grateful to my supervisor, Aris, who is the best advisor I could have hoped for. He treated me with unwavering respect and granted me great freedom. When progress was slow, he never pressured me, but instead trusted that I would find my way. When I doubted my abilities, he reassured me that my struggles were a common part of the PhD experience rather than a personal failure. I am also sincerely thankful for his generous financial support, which allowed me to participate in various conferences and academic activities.

My growth as a researcher was shaped significantly by my collaborators. Florian, my collaborator on Paper E, worked with me when I felt like a headless fly lost in the research process. Together, we experienced how sustained effort leads to breakthroughs. The shared joy we felt when a long-troubling problem finally moved forward is what first showed me the true beauty of theoretical research. That clarity gave me the lasting confidence that time spent struggling is never wasted.

I also wish to thank Sijing, whose professionalism and attention to detail I deeply admire. Collaborating with her taught me not only how to refine my projects but how to work effectively with different personalities. I am grateful to Lutz for the inspiration of his brilliant writing and presentation skills. I also thank Guangyi, my collaborator from my first year, whose advice on time management and research wisdom I have come to appreciate more with every passing year.

The warmth of the research group made this journey a time of connection as well as study. I am grateful to Tianyi for the fun moments we shared in our office and to Ruochun, whose companionship during our local exercises brought a sense of balance to my daily life. I thank Yifei for his warm approachability and Ilie for the energy he brought to our group activities. Stephan gave presentations so brilliant that he became my academic idol, and I am grateful to Sebastian, Suhas, Ece, and Meher for being such friendly colleagues.

Beyond the group, I am grateful to my closest friends in the department, Yuxin,

Andreas, and Amir. To everyone I shared meals and endless conversations with in the kitchen, you helped me improve my English and gave me a deep affection for this department. Likewise, I thank Shuangjie, Xin, and Qiuya for bringing everyday life back into my PhD years and reminding me that I am a person, not just a research machine.

My deepest gratitude belongs to my family. I thank my parents for their quiet and unconditional support. To my husband, Peiyan, we pursued our PhDs side by side while weathering the pressure and celebrating each other's successes. Through love, setbacks, and perseverance, we have emerged as two new doctors and stronger people than when we began.

Finally, I give all my love to my two cats, Alpha and Eta. Your gentle purrs and quiet companionship have been my comfort in every season. Alpha, my best boy in the world, I hope your soul rests in peace.

Contents

Abstract **iii**

Acknowledgements **vii**

List of Papers **vii**

Contents **xi**

I Thesis **1**

1 Introduction **3**
1.1 Diversity Maximization 5
1.2 Streaming Stochastic Coverage Maximization 6
1.3 Diversified Densest Subgraph Discovery 8
1.4 Fairness-Aware PageRank 9
1.5 Reducing Structural Bias via Hitting-Time Minimization 11
1.6 Thesis Organization 12

2 Preliminaries **13**
2.1 Basics of Approximation Algorithms 13
2.2 Submodular Optimization and Matroid Constraint 15
2.3 Graphs and Graph Measures 17
2.4 Streaming Algorithms 19
2.5 The k-Center Problem 20
2.6 The Asymmetric k-Center Problem 21

3 Diversity Maximization in Interactive Systems **23**
3.1 [Paper A] Sequential Diversity Maximization 23
3.2 [Paper B] Streaming Diversity Maximization 27

4 Diversity-Constrained Densest Subgraph Discovery **33**
4.1 [Paper C] Finding Densest Subgraphs with Edge-Color Constraints . . 33
4.2 Reducing ATLEASTHEDGESDSP to Dal$k$S 35
4.3 Reducing ALHCEDGESDSP to ATLEASTHEDGESDSP 36

4.4 Conclusion 37

5 Graph Interventions for Fairness in Networks 39
5.1 [Paper D] Fairness-Aware PageRank via Edge-Rewiring 39
5.2 [Paper D] Fairness-Aware Structural Bias Reduction 42

6 Conclusion and Future Work 49
6.1 Limitations and Future Work 49

References 53

II Included Papers 63

Paper A 65

Paper B 87

Paper C 113

Paper D 127

Paper E 147

Part I

Thesis

Chapter 1

Introduction

As algorithmic systems increasingly shape how we access information, form opinions, and interact with one another, concerns about fairness and diversity have become central to both public discourse and scientific inquiry. News recommendation algorithms influence political awareness and polarization [1], social media feeds affect whose voices are amplified or marginalized [2], and online marketplaces and hiring platforms shape access to economic opportunities [3]. Well-documented phenomena such as filter bubbles and echo chambers highlight how a lack of diversity in algorithmic outputs can narrow users' exposure to information and reinforce existing beliefs [4]. At the same time, growing evidence shows that algorithmic decision-making can reproduce or even exacerbate societal biases, leading to unfair outcomes for historically disadvantaged groups in areas such as advertising, credit, policing, and employment [5, 6]. These concerns are no longer abstract or purely philosophical. They arise from systems that are already deployed at scale and affect millions of users. Addressing these concerns therefore requires systematic, principled approaches that translate normative notions of fairness and diversity into concrete algorithmic objectives. This thesis contributes to this effort by studying fairness and diversity from an algorithmic and optimization-oriented perspective, with a focus on interactive recommendation system design and graph analysis.

Although fairness and diversity are conceptually distinct, they are deeply intertwined in the design of modern algorithmic systems. Fairness concerns the equitable treatment of individuals and groups, addressing questions of justice, equality, and the distribution of resources or opportunities. In contrast, diversity emphasizes heterogeneity and variety, focusing on the breadth of content, perspectives, or options presented by a system. Understanding how these two objectives relate (where they reinforce one another and where they come into tension) requires both philosophical grounding and technical precision.

In computational settings, fairness is typically formalized in several complementary ways. One common perspective is distributive fairness [7, 8], which asks whether algorithmic outcomes are equitably allocated across different groups. In recommendation systems, this may concern whether users from different demographics receive

equally relevant or valuable recommendations. In social or information networks, it may involve whether visibility, influence, or access is fairly distributed across communities. A second perspective, procedural fairness [9, 10], shifts attention from outcomes to the algorithmic process itself, asking whether individuals or groups are treated symmetrically by the underlying mechanism. For example, this might involve whether a ranking algorithm affords each item a comparable opportunity to be surfaced. A further tension arises between individual fairness and group fairness. Individual fairness [11, 12] requires that similar individuals receive similar treatment, while group fairness [12, 13] enforces statistical parity across predefined groups. These notions are often mathematically incompatible, forcing designers to make explicit normative choices. In this thesis, fairness is primarily studied at the group level within graph-based systems. Our work on PageRank (Paper D) aims to enforce target distributions of importance across groups of nodes, while our study of hitting times (Paper E) seeks to equalize accessibility between disparate groups in a network.

Diversity, by contrast, focuses on promoting heterogeneity and breadth. In algorithmic systems, diversity can improve user satisfaction by offering varied choices, encouraging exploration and serendipity, and mitigating filter bubbles and echo chambers. Like fairness, diversity admits multiple mathematical formulations, each capturing a different aspect of heterogeneity. Distance-based formulations quantify diversity through pairwise dissimilarity [14, 15, 16], often assuming items lie in a metric space and measuring diversity as the sum or minimum of pairwise distances. Coverage-based formulations [17, 18] instead emphasize representational breadth, defining a set as diverse if it spans many topics, attributes, or viewpoints, and naturally connecting to classical problems such as set cover and facility location. In interactive or sequential environments, sequential diversity becomes essential, as diversity must be balanced over time while accounting for uncertainty in user engagement. This thesis explores diversity across several such settings. In interactive recommendation systems (Papers A and B), we study distance-based and coverage-based diversity in sequential and streaming contexts, whereas in our work on densest subgraphs (Paper C), diversity is an explicit constraint requiring dense communities to exhibit heterogeneous edge types.

While fairness and diversity are distinct objectives, they often operate in concert [19, 20]. Increasing diversity can promote fairness by broadening exposure and preventing dominant groups or viewpoints from monopolizing attention. Conversely, fairness constraints can enhance diversity by ensuring that minority groups are adequately represented. At the same time, these objectives can conflict. Maximizing diversity alone may overemphasize a small number of highly dissimilar items while neglecting equitable treatment across the full population, while strict fairness constraints may reduce overall diversity when groups exhibit homogeneous preferences.

In the following sections, we introduce the five papers included in this thesis. For each paper, we provide the relevant technical background and summarize our contributions.

1.1 Diversity Maximization

1.1.1 Background

Diversity maximization is a classical algorithmic problem, and it has been studied since the 1980s. The canonical version of this problem is essentially a subset selection problem, i.e., given a ground set of items, the goal is to select a subset subject to constraints that maximizes a diversity measure. The most basic constraint is the cardinality constraint, and the diversity measure typically refers to a distance function or a coverage function.

We are given a ground set of items U and a ground set of topics $\mathcal{C} = \{c_1, c_2, \ldots, c_d\}$. A non-negative distance function $d : U \times U \to \mathbb{R}_+$ maps any pair of items to a non-negative real value, capturing the dissimilarity between items. A labeling function $c : U \to 2^{\mathcal{C}}$ assigns each item to a set of topics it covers. The diversity maximization problem asks to find a subset $S \subseteq U$ of size $|S| = k$ that maximizes diversity. Diversity can be measured in several ways: the sum of pairwise distances among selected items, the minimum pairwise distance, or the coverage of topics. We formally define these three diversities below.

$$\text{MAX-MIN Diversification:} \quad \max_{S \subseteq U, |S| \leq k} \min_{x,y \in S} \{d(x,y)\} \tag{1.1}$$

$$\text{MAX-SUM Diversification:} \quad \max_{S \subseteq U, |S| \leq k} \sum_{x,y \in S} d(x,y) \tag{1.2}$$

$$\text{MAX-COV Diversification:} \quad \max_{S \subseteq U, |S| \leq k} \bigcup_{x \in S} c(x) \tag{1.3}$$

Hardness and Approximation Algorithms To the best of the author's knowledge, the MAX-MIN diversification problem is first studied by Kuby [21] in 1987, while it is later proved to be NP-hard even if the distance function d satisfies the triangle inequality by Erkut [22] in 1990. Later in 1994, Ravi et al. [15] prove that if the distance function d is not required to satisfy the triangle inequality, then there is no polynomial time relative approximation algorithm for the problem unless P=NP. Given the inapproximability results, research on the MAX-MIN diversification problem focuses on situations where the distance d satisfies the triangle inequality. In their paper, Ravi et al. [15] propose a greedy algorithm that first selects two items furthest apart from each other, then greedily adds the items that maximize the minimum distance from items in the chosen subset. This greedy algorithm provides a 2-approximation to the MAX-MIN diversification problem, and the approximation ratio is tight unless P=NP.

The MAX-SUM diversification problem is proven to be NP-hard even in metric spaces by Hansen and Moon [23]. Regarding approximability, unlike the MAX-MIN diversification problem, polynomial-time approximation algorithms exist for the MAX-SUM diversification problem in metric spaces. To name a few, Hansen and Moon [23] introduce a $1/4$-approximation greedy algorithm; Borodin et al. [14] and Gollapudi and Sharma [16] both propose $1/2$-approximation greedy algorithms under

a cardinality constraint; and Borodin et al. [14] also present a $1/2$-approximation local search algorithm for general matroid constraints. More recently, Cevallos et al. [24] develop an algorithm for the cardinality constraint that provides a PTAS (Polynomial-Time Approximation Scheme, which we introduce in Section 2.1) when the distance function d is extended to a more general type known as negative type.

The MAX-COV diversification problem is also NP-hard since it is a variant of the maximum coverage problem. The objective function of this problem is submodular, and thus, according to Nemhauser et al. [25], a classic greedy algorithm provides a $(1 - 1/e)$ approximation.

1.1.2 Contributions of Paper A

The aforementioned canonical research problems formulate diversity maximization as a subset-selection task and overlook the importance of item ordering within the selected set. However, in real-world user–system interaction settings such as recommender systems, item order plays a critical role due to position-dependent user engagement.

The work most closely related to ours is the foundational study by Carbonell and Goldstein [26], which introduces Maximal Marginal Relevance (MMR). MMR greedily selects items by balancing query relevance with diversity relative to previously selected items. Although MMR has become ubiquitous in practice, it offers no formal approximation guarantees. Moreover, the resulting ranking is merely a by-product of the greedy selection process rather than an explicit optimization objective.

A further limitation shared by most existing diversification approaches is the assumption that users examine a fixed number of items k. In practice, user interaction is sequential, and users may stop at any position depending on satisfaction or fatigue. Coppolillo et al. [27] are the first to model such sequential user behavior in recommender systems. They formalize sequential diversity through the EXPLORE algorithm, modeling user continuation with Weibull distributions. However, their approach is purely empirical and provides no approximation guarantees.

We mitigate these limitations by introducing a theoretical framework for sequential diversification that models position-dependent user engagement. Users examine items sequentially and may stop at any position, with continuation probabilities governing their behavior. We define diversity as the expected diversity experienced by a user, computed over all possible stopping points. This formulation naturally integrates relevance and diversity through continuation probabilities and the underlying diversity measure. We show that the resulting optimization problems are NP-hard and develop approximation algorithms with constant-factor guarantees for both sum-diversity and coverage-diversity objectives.

1.2 Streaming Stochastic Coverage Maximization

Paper B extends the stochastic diversification model introduced in paper A to streaming settings, with a focus on coverage diversity. Since the objective is submodular, we first

review the fundamentals of submodular maximization and then conclude by outlining the main contributions of this paper.

1.2.1 Background

Submodular Optimization In the offline setting, given a ground set of items U, the submodular maximization problem selects a subset $S \subseteq U$ subject to some constraint, such that a given submodular function f is maximized. For this thesis, we focus on the case where f is monotone, and omit the discussion for the non-monotone cases.

The most basic version of this problem is when S is required to satisfy a cardinality constraint, i.e., $|S| \leq k$ for some integer k. For this simple case, Nemhauser et al. [25] propose a standard greedy algorithm, which selects at each step the element yielding the largest incremental in f, and provides a $1 - 1/e$ approximation. Feige [28] prove that improving the ratio $1 - 1/e$ is NP-hard, while Nemhauser and Wolsey [29] show that any improvement over $1 - 1/e$ requires an exponential number of queries in the value oracle setting.

When S is required to satisfy a matroid constraint, Fisher et al. [30] show that the standard greedy algorithm that selects the best elements while maintaining independence is a $1/2$-approximation. This ratio is later improved to the optimal $1 - 1/e$ via the *continuous greedy algorithm* by Calinescu et al. [31] and Vondrák [32]. Filmus and Ward [33] propose a randomized non-oblivious local search algorithm that also achieves the optimal $1 - 1/e$ approximation, and converges in polynomial time.

Streaming Submodular Maximization In the streaming setting, the items of the ground set U arrive sequentially, that is, one item becomes available at each time step. Equivalently, the ground set can be viewed as ordered in an arbitrary manner, and any streaming algorithm must process U in this order. At any point during the stream, the algorithm must be able to output a solution S. In the following, we focus on the case where the submodular function is monotone.

For the streaming submodular maximization problem, the solution set is further required to satisfy given constraints, such as a cardinality constraint $|S| \leq k$ or a matroid constraint $S \in \mathcal{M}$ for some matroid $\mathcal{I} = (U, \mathcal{M})$.

The first streaming algorithm for the monotone submodular maximization problem subject to a cardinality constraint with provable guarantees is SIEVE-STREAMING, proposed by Badanidiyuru et al. [34]. SIEVE-STREAMING achieves a one-pass $1/2 - \epsilon$ approximation with time complexity $\mathcal{O}(n \log(k)/\epsilon)$ and space complexity $\mathcal{O}(k \log(k)/\epsilon)$ for an input stream of length n. Kazemi et al. [35] later improve the space complexity to $\mathcal{O}(k)$ and prove that $1/2$ is tight. Norouzi-Fard et al. [36] further show that $1/2$ is optimal when elements in the stream arrive in an arbitrary order. Subsequent works [36, 37] improve the approximation ratio beyond 0.5 under the assumption that elements arrive in a random order. We do not discuss these works further, as this thesis focuses exclusively on the arbitrary-order setting.

Beyond the cardinality constraint, the streaming submodular maximization problem has also been studied under matroid and matchoid constraints. Chakrabarti and Kale [38] propose a $\frac{1}{4p}$-approximation algorithm when the constraint is the intersection

of p matroids. Later, Chekuri et al. [39] introduce a streaming-greedy algorithm that extends the constraint to the more general p-matchoid setting while maintaining the same approximation ratio. Feldman et al. [40] subsequently propose a randomized version of the streaming-greedy algorithm, which is three times faster while achieving the same $1/4p$ approximation guarantee.

1.2.2 Contributions of Paper B

The aforementioned streaming submodular maximization algorithms are designed to produce solutions only at the end of the stream. In applications that require anytime or on-demand responses, existing methods cannot guarantee approximation bounds at arbitrary query times.

We study a streaming setting in which content arrives online, and users may access the system at any time. Users click on each recommended item with a certain probability, and our goal is to maximize the expected number of topics covered by the items a user clicks on by the end of the interaction.

We introduce algorithms that maintain valid approximation guarantees throughout the stream while accounting for stochastic user behavior. Our algorithms achieve different competitive ratios depending on memory constraints. With memory linear in the stream length, they match offline guarantees. With sublinear memory, we develop two algorithms that make multiple concurrent predictions about the user behavior and output the best prediction greedily, both achieving constant competitive ratios. These results demonstrate that strong theoretical guarantees can be preserved in settings with limited memory and uncertain user behavior.

1.3 Diversified Densest Subgraph Discovery

For paper C, we shift our focus from sets to graphs and move from explicit diversity maximization to diversity imposed as a structural constraint. In many real-world networks, edges carry rich semantic information. Relationships in social networks may represent friendship, family ties, or professional connections, while citations between papers may indicate agreement, disagreement, or extensions of prior work. Ignoring such attributes when searching for dense subgraphs often leads to overly homogeneous communities that fail to capture important forms of heterogeneity. Motivated by these observations, this paper studies the densest subgraph discovery problem under edge color constraints, which can be naturally interpreted as diversity constraints on edge types. We first briefly review the classical densest subgraph problem and then describe our main contributions.

1.3.1 Background

Given an undirected graph $G = (V, E)$ with n vertices and m edges, the unconstrained densest subgraph problem asks for a subset of vertices $S \subseteq V$ that maximizes the ratio $|E[S]|/|S|$, where $E[S]$ denotes the edges induced by S. This problem can be solved in polynomial time using Goldberg's max flow algorithm [41] or by solving

the linear programming formulation introduced by Charikar [42]. Charikar's greedy peeling algorithm achieves a 2 approximation in linear time $\mathcal{O}(n+m)$ by iteratively removing vertices of minimum degree.

Subsequent work has explored several extensions of this problem. Khuller and Saha [43] generalize exact algorithms to directed graphs and showed that size-constrained variants, such as finding a densest subgraph with exactly k vertices, are NP hard. Andersen and Chellapilla [44] study the densest subgraph with at least k vertices and proposed a linear time algorithm with a $\frac{1}{3}$ approximation guarantee.

When diversity or fairness constraints are introduced, the problem becomes significantly more challenging. Anagnostopoulos et al. [45] prove that the fair densest subgraph problem with binary node colors is NP hard. They further show that approximating this problem in polynomial time is at least as hard as approximating the densest subgraph with at most k vertices, for which no constant factor approximation is known. For the special case of fair graphs, where fairness requires an equal number of vertices from each color class, they obtained a 2-approximation. Miyauchi et al. [46] extend this line of work to multiple node colors through the Densest Diverse Subgraph Problem and achieve an $\Omega(1/\sqrt{n})$ approximation that is independent of the number of colors.

1.3.2 Contributions of Paper C

Prior work on fairness and diversity in densest subgraph discovery has focused exclusively on node-level constraints, typically enforcing proportional representation across vertex groups. Paper C is the first to introduce edge-color constraints into the densest subgraph discovery problem. The edge-color constraint setting captures a wide range of real-world scenarios, including social networks where diversity corresponds to multiple types of relationships, multilayer networks with different interaction modalities, and biological networks with distinct types of connections. The paper establishes that the problem is NP-complete even when there are only two edge colors. It also presents linear time $\mathcal{O}(1)$-approximation algorithms for the variant that requires at least h distinct edge colors, under the assumption that the input graph is everywhere sparse.

1.4 Fairness-Aware PageRank

1.4.1 Background

The concept of PageRank fairness was recently introduced by Tsioutsiouliklis et al. [47], who define it as the fair allocation of PageRank weight (formally defined in Section 2.3.2) among nodes. To be specific, let $\mathbf{p}$ be the PageRank vector of a given graph $G = (V, E)$, where the PageRank weight of node $u \in V$ is equal to $\mathbf{p}_u$. According to Tsioutsiouliklis et al. [47], a PageRank algorithm is ϕ-fair if the fraction of the total PageRank weight assigned to the members of a target group is exactly ϕ. To achieve ϕ-fairness, the authors proposed several methods. The FSPR algorithm modifies the restart vector and achieves ϕ-fairness without changing the graph structure, while LFPR modifies both the restart vector and the transition matrix

to ensure fair behavior at each vertex, i.e., fair probabilities to visit all groups from each node. Tsioutsiouliklis et al. [48] later proposed an alternative definition, where PageRank fairness requires the total PageRank weight of a target group to meet a threshold ϕ. To achieve this threshold, they propose a link-recommendation algorithm that adds edges to the graph, increasing the PageRank weight of the target group and thereby maximizing the fairness gain.

Beyond these two works that explicitly optimize PageRank fairness, some studies implicitly promote it. For instance, Fairwalk [49] and Crosswalk [50] focus on fairness in graph embeddings by adjusting random walk probabilities. Their methods revise the transition probabilities of a random walker moving to neighboring nodes, ensuring that each node distributes its influence fairly among neighbors belonging to different groups. Although not explicitly framed in terms of PageRank fairness, such approaches indirectly increase PageRank fairness at the individual-node level and, as a consequence, can also enhance fairness across the entire graph.

1.4.2 Contributions of Paper D

The above-mentioned prior works demonstrate multiple pathways to achieving fair PageRank, which can be broadly categorized as (i) modifying the restart probability, (ii) modifying the restart vector, (iii) adding or recommending edges, and (iv) reweighting existing edges. While each approach has its merit, we focus on edge reweighting for several principled reasons.

First, edge reweighting provides interventions that are intrinsic to the graph structure and directly translatable to actionable operations. In social networks, for instance, edge weights can be adjusted by prioritizing or demoting content from a target source, which are operations that platforms can readily implement. Second, while link recommendation strategies like those in Tsioutsiouliklis et al. [48] are also structure-based, they introduce practical challenges by depending on user acceptance of recommended links, creating uncertainty that complicates reliable deployment. Third, strategies that modify PageRank hyperparameters, such as the restart vector adjustments in FSPR and LFPRs, are extrinsic to the graph structure and often lack clear operational meaning. For example, in personalized PageRank, the restart vector encodes user interests and should not be artificially manipulated solely for fairness objectives.

Our approach, therefore, focuses on edge reweighting as a principled, graph-intrinsic method for fairness intervention. This method preserves the interpretability of PageRank parameters while enabling practical deployment through structural modifications that remain faithful to the original network. Specifically, we modify transition probabilities in the PageRank matrix to bring the resulting group-wise PageRank weight as close as possible to a target. Unlike methods that enforce local fairness at each vertex, we target group-level fairness across the entire graph, enabling smaller modifications to the transition matrix. We formulate the objective as a non-convex function over a convex feasible set, incorporating practical constraints such as preserving the original network topology and bounding the magnitude of weight changes. We solve this optimization problem using efficient projected gradient descent, guaranteeing convergence to a stationary point.

1.5 Reducing Structural Bias via Hitting-Time Minimization

1.5.1 Background

Real-world networks often exhibit structural segregation, a phenomenon where distinct groups form densely connected internal clusters but maintain only sparse connections between each other. This structural bias arises across many domains, including social media and content recommendation platforms, leading to concerning patterns such as polarized political discussions on Twitter, echo chambers in news consumption, and radicalization pathways in video-sharing platforms. In these settings, users predominantly interact with like-minded individuals and infrequently encounter diverse viewpoints. For example, in video-sharing or news recommendation platforms, the content graph may split into clusters of "harmful" and "neutral" content, or into communities reflecting opposing stances on controversial issues.

From a fairness perspective, such segregation raises concerns about equitable access to information across groups. If members of one group can easily navigate to another while the reverse requires significantly more steps, the network exhibits fundamental asymmetry in cross-group accessibility. This asymmetry can restrict user exposure to diverse viewpoints, reinforce polarization, and increase the risk of harmful content consumption. Mitigating structural bias is therefore an important objective for improving the fairness, safety, and informational value of recommendation systems.

Recent research has investigated algorithmic interventions that reduce structural bias by modifying network structure. One line of work considers edge rewiring, where existing recommendations are replaced to reduce exposure to harmful content. Fabbri et al. [51] show that it is NP-hard to find the optimal set of recommendations to rewire to reduce the segregation in a recommendation graph; they further prove that the problem cannot be approximated within any multiplicative factor. Coupette et al. [52] study the related problem of minimizing exposure to harmful content through edge rewiring. They prove the objective is submodular and proposed a greedy algorithm that achieves a $(1 - 1/e)$-approximation. Another line of work focuses on introducing shortcut edges, i.e., edges not present in the original network but added as new recommendations, such as suggesting additional videos, articles, or users to follow. In this direction, Haddadan et al. [53] formulate the problem of maximizing bubble radius reduction as a submodular maximization task and solve it using a greedy approach.

The body of work discussed above relies on random-walk–based measures to quantify structural bias, such as bubble radius, inter-group hitting time, and segregation score. Other studies, however, define structural bias using different frameworks. For instance, Zhu et al. [54] model polarization through a polarization–disagreement index derived from opinion disagreement in the Friedkin–Johnsen model, and reduce this index by strategically adding edges to the network.

1.5.2 Contribution of Paper E

The existing works [51, 52, 53] have several key limitations. First, they aim to reduce structural bias rather than to minimize it directly. Second, they rely on specific assumptions about the graph or the random walk process. For instance, Fabbri et al. [51] assume the graph is d-regular, and Haddadan et al. [53] restrict the random walk to a bounded length.

Our work addresses these limitations and tackles the problem in a more general and direct way. We directly minimize the hitting times between groups without making assumptions about graph structure or the random walk process. We focus on general undirected graphs with uniform random walks and design algorithms that minimize both the average and the maximum hitting time between groups.

The rewiring and shortcut-edge technique can be applied to mitigate structural bias and polarization in real-world networks, particularly in recommender and content-navigation systems such as social media feeds, video platforms, and news aggregators. By strategically adding a small number of shortcut edges between otherwise weakly connected groups, the technique reduces the expected time it takes for users to transition from harmful content to neutral content. Importantly, this approach does not require altering or removing nodes in the graph, making it well-suited for deployment in real-world systems. From a societal perspective, it helps counter echo chambers and polarization by gently promoting exposure to diverse viewpoints, fostering healthier information ecosystems while preserving platform constraints and user experience.

1.6 Thesis Organization

This thesis is organized as follows. Chapter 2 presents the preliminaries that are required to understand the included papers. Chapter 3 presents our work on diversity maximization in interactive systems, covering both the sequential ranking problem (paper A) and the streaming coverage problem (paper B). Chapter 4 presents our study of diversity-constrained densest subgraph discovery (paper C), establishing connections between edge-constrained and node-constrained formulations. Chapter 5 presents our work on graph interventions for fairness, covering both PageRank fairness through edge reweighting (paper D) and hitting-time fairness through strategic edge additions (paper E). Chapter 6 concludes with a synthesis of our contributions, discusses broader implications for the design of fair and diverse algorithmic systems, and outlines directions for future research.

Together, these chapters demonstrate that fairness and diversity, properly formalized, can be pursued through principled algorithmic approaches with provable guarantees, contributing both to the theoretical foundations of algorithmic fairness and to practical system design.

Chapter 2

Preliminaries

This chapter provides the necessary background knowledge for understanding the technical content of this thesis. We will start with the basics of approximation algorithms for papers A, B, C, and E, followed by the definition of submodularity and matroids to understand paper B, and finally graph-theoretic notions needed for the papers.

2.1 Basics of Approximation Algorithms

Approximation algorithms [55, 56, 57] are a class of algorithms designed to tackle hard optimization problems for which no efficient method exists to compute the exact optimal solution, or for which computing the exact solution is prohibitively expensive, even in cases where the problem can be solved in polynomial time but the exact algorithm remains too costly for practical use at scale. Instead of guaranteeing optimality, approximation algorithms aim to produce solutions that are provably close to optimal while running in polynomial time. A *relative* approximation algorithm guarantees a solution within a *multiplicative* factor of the optimal value for every instance of the problem, whereas an *absolute* approximation algorithm guarantees a solution within an *additive* constant of the optimal value for every instance. In this thesis, unless otherwise specified, the term *approximation algorithm* refers to a *relative* approximation algorithm.

Formally, an α-approximation algorithm for an optimization problem is a polynomial-time algorithm that, for any instance of the problem, returns a solution whose value is within a factor α of the optimal value. Let OPT denote the value of an optimal solution and ALG the value returned by the algorithm. Then, for a maximization problem, it holds

$$\mathrm{OPT} \geq \mathrm{ALG} \geq \alpha\,\mathrm{OPT},$$

with $\alpha < 1$, and for a minimization problem, it holds

$$\mathrm{OPT} \leq \mathrm{ALG} \leq \alpha\,\mathrm{OPT},$$

with $\alpha > 1$. We call the quantity α the approximation ratio.

Competitive Ratio Competitive analysis [58, 59] is a framework for evaluating online algorithms, which must make irrevocable decisions as input arrives incrementally, without knowledge of future arrivals. This lack of foresight means that decisions appearing optimal at the moment may prove suboptimal later. In the worst case, an adversary can craft future inputs to exploit every decision the algorithm makes. To assess an online algorithm's quality, we compare it against an offline algorithm with full knowledge of the input sequence in advance. The *competitive ratio* quantifies this worst-case performance gap over all possible input instances.

Formally, for an input instance σ, let $\mathrm{ALG}(\sigma)$ and $\mathrm{OPT}(\sigma)$ denote the objective values achieved by the online and optimal offline algorithms, respectively. For a maximization problem, an algorithm is *α-relatively competitive* if there exists a constant k such that $\mathrm{ALG}(\sigma) \geq \alpha\,\mathrm{OPT}(\sigma) + k$ for every possible input σ, and we call α the *relative competitive ratio*. An algorithm is *k-additively competitive* if $\mathrm{ALG}(\sigma) \geq \mathrm{OPT}(\sigma) - k$ for every possible input σ. For minimization problems, the inequalities are reversed. In this thesis, unless otherwise specified, *competitive ratio* refers to the *relative* competitive ratio.

Optimality of Approximation and Approximability [56] If we call an approximation ratio α *optimal*, it means there is no polynomial-time relative approximation algorithm that can provide a performance guarantee better than α unless P = NP. A problem is called *inapproximable* (or *hard to approximate*) if there exists no polynomial-time relative approximation algorithm for it unless P = NP. More formally, a problem is said to be *inapproximable within factor c* if there is no polynomial-time algorithm that guarantees a solution within a multiplicative factor c of the optimal value, unless P = NP.

Tightness of Approximation An approximation ratio α is said to be *tight* if the worst-case performance guarantee of an algorithm coincides with its performance on a specific instance. Specifically, the algorithm guarantees a solution within a factor α of the optimal for all problem instances (upper bound), and there exists at least one instance for which the algorithm achieves exactly this ratio α (lower bound). When these bounds coincide, the analysis precisely characterizes the algorithm's worst-case behavior, and the approximation ratio cannot be improved without modifying the algorithm.

Polynomial-Time Approximation Scheme (PTAS) [60] A *polynomial-time approximation scheme* is an algorithmic scheme with the following property: for every fixed $\epsilon > 0$ there exists an algorithm A_ϵ that runs in time polynomial in the input size n (the degree of the polynomial may depend arbitrarily on $1/\epsilon$) and returns a $(1+\epsilon)$-approximation for minimization problems, or a $(1-\epsilon)$-approximation for maximization problems.

2.2 Submodular Optimization and Matroid Constraint

Given a set U of items, a set function $f : 2^U \to \mathbb{R}_{\geq 0}$ maps a subset of items to a non-negative value. A typical example is the coverage function which calculates the total number of unique topics spanned by a specific collection of news items.
A set function f is monotonically increasing if for any $S \subseteq T \subseteq U$, we have

$$f(S) \leq f(T).$$

Similarly, f is monotonically decreasing if for any $S \subseteq T \subseteq U$, we have

$$f(S) \geq f(T).$$

2.2.1 Submodular Set Function

A function f is submodular [61, 62, 63] if for all $S \subseteq T \subseteq U$ and for all $\{e\} \in U \setminus T$ we have

$$f(S \cup \{e\}) - f(S) \geq f(T \cup \{e\}) - f(T). \tag{2.1}$$

An equivalent definition of submodularity is that for all $S, T \subseteq U$, we have

$$f(S) + f(T) \geq f(S \cup T) + f(S \cap T). \tag{2.2}$$

Many practical problems naturally exhibit submodularity. For instance, in data summarization, the goal is to choose a small set of items that well represents a large dataset, while in information maximization, the objective is to select a set of seed nodes that maximizes the spread of information across a network. Intuitively, these problems are submodular because they exhibit a diminishing returns property: as the selected set grows, newly added items increasingly overlap with what has already been covered, contributing less marginal benefit. This structure aligns directly with the definition of submodularity, providing a principled foundation for designing efficient algorithms.

2.2.2 Supermodular Set Function

By reversing the inequality in Equation (2.2), we obtain the definition of supermodularity [64]. A set function f is supermodular if for all $S, T \subseteq U$, we have

$$f(S) + f(T) \leq f(S \cup T) + f(S \cap T). \tag{2.3}$$

In contrast to submodularity, supermodularity exhibits increasing marginal gains: adding an element to a larger set yields greater benefit than adding it to a smaller set.

Weakly-α-Supermodular Set Function A non-negative monotonically decreasing set function f is said to be *weakly-α-supermodular* [65] if there exists $\alpha \geq 1$ such that for any two sets $S, T \subseteq U$

$$f(S) - f(S \cup T) \leq \alpha \cdot |T \setminus S| \cdot \max_{\{e\} \in T \setminus S} [f(S) - f(S \cup \{\{e\}\})]. \tag{2.4}$$

The supermodularity states that if adding $T \setminus S$ is beneficial to minimizing f, then there is an element $\{e\} \in T \setminus S$ that contributes at least a fraction of that benefit. Liberty and Sviridenko [65] prove that a non-increasing non-negative supermodular function f is weakly-α-supermodular with parameter $\alpha = 1$.

2.2.3 Oracle Model

In submodular optimization, it is standard to assume access to a value oracle [25, 66], which is a black box that returns $f(S)$ for any $S \subseteq U$. Each oracle query is assumed to require one unit of time, and the time complexity of an algorithm is therefore measured by the number of oracle calls it makes, relative to the ground set size in the offline setting, and to the stream length in the streaming setting. The use of a value oracle is natural because submodular functions arise in diverse domains with vastly different evaluation procedures, and the oracle model captures this generality by enabling a unified analysis that is agnostic to the internal structure of f.

2.2.4 Matroids

In combinatorics, we frequently deal with constrained optimization problems, where the solution is a subset of elements chosen from a ground set that must satisfy some given constraints. A matroid [67, 68] specifies a particular type of constraint by introducing a structure of "independence," meaning that valid solutions are required to satisfy an independence condition. For instance, in the case of a graphic matroid, the selected edges must not form a cycle in the underlying graph.

Formally speaking, a *matroid* is a pair $\mathcal{I} = (U, \mathcal{M})$, where U is called the *ground set*, and $\mathcal{M}$ is called the *independent sets*. $\mathcal{M}$ is a set of the subsets of U, each element in $\mathcal{M}$ is called a independent set. For $\mathcal{I}$ to be a matroid, $\mathcal{M}$ must satisfy three axioms:

(I_1) (hereditary) $\emptyset \in \mathcal{M}$

(I_2) (downward closure) For any $Y \in \mathcal{M}$, if $X \subset Y$, then $X \in \mathcal{M}$

(I_3) (augmentation property) For any $X, Y \in \mathcal{M}$, if $|X| < |Y|$, then there exists $e \in Y$ such that $X \cup \{e\} \in \mathcal{M}$

A matroid generalizes the concept of "independence" from vector spaces to more abstract settings where the ground set is arbitrary rather than limited to vectors. Analogously to the vector space, the notions of rank, basis, and span are also defined. Specifically, the rank of a matroid $\mathcal{I} = (U, \mathcal{M})$ is a function $2^U \to \mathbb{R}_+$ denoted r or r_M such that

$$r(S) = \max\{|X| : X \subseteq S, X \in \mathcal{M}\} \textit{ for any } S \subseteq U$$

Axiom (I_2) implies that all maximal (inclusion-wise) independent sets have the same cardinality. A maximal independent set is called a base (or basis) of the matroid. Given a matroid $\mathcal{I} = (U, \mathcal{M})$ and any set $S \subseteq U$, the span of S, denoted as *span*, is defined as

$$\textit{span}(S) = \{e \in U \ : \ r(S \cup \{e\}) = r(S)\}$$

2.3 Graphs and Graph Measures

In this section, we introduce graphs and some measures and problems that are particularly relevant for analyzing graphs.

2.3.1 Graphs

A graph is defined as $G = (V, E)$, where V is a finite set of *vertices* with $|V| = n$, and $E \subseteq V \times V$ is a set of *edges* connecting pairs of vertices, with $|E| = m$. Graphs provide a natural way to model many real-world systems, such as friendships in social networks or protein interactions in biology.

Depending on the problem being modeled, graphs can be directed or undirected, weighted or unweighted. We use $w(u, v)$ to denote edge weight of edge (u, v), with $w(u, v) \in \{0, 1\}$ for unweighted graphs and $w(u, v) \in \mathbb{R}_{\geq 0}$ for weighted graphs. We use $d(u)$ to denote the degree of a vertex u, which is defined as the number of edges connected to it in unweighted graphs and the sum of edge weights connected to it in weighted graphs.

Graphs can be represented and stored using different data structures:

- The *adjacency matrix* $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$, where $\mathbf{A}_{uv} = 1$ (or $w(u, v)$) if there is an edge from vertex u to vertex v, and 0 otherwise.
- The *adjacency list*, which stores for each node a list of its neighbors.

Given a subset S of the vertices V, we denote by $G(S) = (S, E[S])$ the node-induced subgraph by S, where $E[S] = \{\{u, v\} \in E \mid u, v \in S\}$.

2.3.2 PageRank

PageRank is a well-known algorithm popularized by Larry Page and Sergey Brin in the late 1990s [69]. It was the foundation of Google's search engine. The algorithm takes a graph $G = (V, E)$ as input, and outputs a PageRank vector $\mathbf{p}$, with $\mathbf{p}_u$ denoting the importance of node u in the graph. The intuition of the algorithm is to imagine a random surfer moving from one vertex to another, following the edges, and the rank of a vertex represents how likely the surfer is to land there.

Given a directed graph $G = (V, E)$, let $\mathbf{P}$ denote its transition matrix, where $\mathbf{P}_{uv}$ is the probability that a random surfer moves from node u to node v in a single step. The matrix $\mathbf{P}$ is obtained by normalizing the adjacency matrix, with special handling for sink nodes. Formally,

$$\mathbf{P}_{uv} = \begin{cases} \frac{w(u,v)}{d(u)}, & \text{if } d(u) \neq 0, \\ \frac{1}{|V|}, & \text{otherwise.} \end{cases} \tag{2.5}$$

Thus, for non-sink nodes, the random surfer's transition probability is proportional to the normalized edge weight, while for sink nodes, the surfer jumps uniformly to any node in the graph.

PageRank computes a vector $\mathbf{p}$, which represents the stationary distribution of a random walk on the graph G. The vector $\mathbf{p}$ satisfies the equation

$$\mathbf{p}^\top = (1-\gamma)\mathbf{p}^\top\mathbf{P} + \gamma\mathbf{v}^\top \tag{2.6}$$

where γ denotes the restart probability, which represents the probability that the random surfer restarts at any step. The vector $\mathbf{v}$ is the restart vector, i.e., a probability distribution over V that determines the restart probabilities at each vertex. By convention, γ is often set to 0.15, and $\mathbf{v}$ is often chosen as the uniform distribution.

By solving Equation (2.6) , the PageRank vector $\mathbf{p}$ can be computed as

$$\mathbf{p}^\top = \gamma\mathbf{v}^\top\left[\mathbf{I} - (1-\gamma)\mathbf{P}\right]^{-1}$$

Computationally, the PageRank score vector $\mathbf{p}$ can be computed efficiently using the power iteration method [70], which repeatedly applies Equation (2.6) until convergence.

2.3.3 Hitting Time

The concept of *hitting time* [71] is crucial in the study of random walks on graphs. Given a graph $G = (V, E)$ with transition matrix $\mathbf{P}$, we are interested in random walks on the graph G according to $\mathbf{P}$. Intuitively, the hitting time from a vertex u to v is the expected number of steps a random walker starting at node u takes to reach node v for the first time, and it captures how "accessible" node v is from u.

We denote the hitting time from vertex u to vertex v as $H(u, v)$. Let $\tau_v(u)$ denote the random variable representing the first time step at which a random walk, starting from vertex u, visits vertex v. The hitting time is defined as $H(u, v) = \mathbb{E}\left[\tau_v(u)\right]$, where the expectation is taken over all possible random walk trajectories starting from vertex u.

There are two related measures to hitting time, the commute time $c(u, v)$ and cover time C, defined as follows

- *Commute time* $c(u, v)$: the expected time required for a random walk to start at vertex u, hit v for the first time, then hit u. This can be represented as $c(u, v) = H(u, v) + H(v, u)$.
- *Cover time* C: let $C_u(G)$ denote the expected time required for a random walk to start at vertex u, and visit all nodes in the graph G for at least one time. The cover time $C = \max_{u\in V} C(u, G)$. The cover time of any graph with n nodes is *upper bounded* by $2n(n-1)^2$ [72].

2.3.4 Densest Subgraph Discovery

Given a graph $G = (V, E)$, the densest subgraph discovery problem [41] aims to find a subset $S \subseteq V$ of vertices such that the induced subgraph $G[S]$ maximizes a density measure. Densest subgraphs contain tightly interconnected substructures, making

them valuable for applications including community detection, fraud detection, and web-graph analysis.

One of the most commonly used density measures is *edge density*, defined as $d(S) = \frac{|E[S]|}{|S|}$, where $E[S] = \{(u, v) \in E \mid u, v \in S\}$ represents the edges with both endpoints in S.

Computational Complexity and Algorithms The unconstrained densest subgraph problem (finding $S^* = \operatorname{argmax}_{S \subseteq V} d(S)$) is solvable in polynomial time via max-flow computations [41, 73], even though there are exponentially many subgraphs to consider. For efficient approximate solutions, Asahiro et al. [74], Charikar [42] show that a linear-time greedy *peeling* algorithm that iteratively removes the node of smallest degree and returns the best solution encountered achieves a 2-approximation ratio. More recently, Chekuri et al. [75] provide an almost linear-time flow-based $(1 + \epsilon)$-approximation algorithm and analyze an iterative peeling algorithm by Boob et al. [76], proving its convergence to optimality.

Size-Constrained Variants Research has also investigated densest subgraph discovery under cardinality constraints, seeking subgraphs with at most k nodes (Dam$k$S), at least k nodes (Dal$k$S), or exactly k nodes (DkS). The DkS problem is NP-hard, Khot [77] show that there does not exist any PTAS for the DkS problem under a reasonable complexity assumption. The best-known approximation ratio is $O(n^{1/4})$ [78]. The Dam$k$S problem is also NP-hard, and Andersen and Chellapilla [44] show that it is nearly as hard to approximate as the DkS problem. Khuller and Saha [43] show that an α-approximation for Dam$k$S yields an $\alpha/4$-approximation for DkS.

The Dal$k$S problem seeks the densest subgraph containing at least k vertices and is also NP-hard [43], but easier to approximate than DkS. Andersen and Chellapilla [44] design a linear-time $1/3$-approximation algorithm based on greedy peeling, while Khuller and Saha [43] provide two $1/2$-approximation algorithms using flow computations and linear programming, respectively. However, it was left open as to whether or not this problem is NP-complete.

2.4 Streaming Algorithms

Data is generated at high velocity, and in many cases, accessing the entire dataset is infeasible. This may occur because the ground set is too large to fit into memory or because the data arrives as a continuous stream at a rate that makes complete storage impractical (e.g., sensor measurements, real-time traffic data). Even when the full dataset is available, repeated querying can be computationally expensive. Streaming algorithms are well-suited to such settings, as they require limited memory, typically much smaller than the ground set, and process each incoming item efficiently in real time.

Given memory constraints, a streaming algorithm can store and process only a small fraction of the data at any given time [79]. The available memory must accommodate both the current solution and auxiliary information necessary for producing

a high-quality approximation. For each incoming item, the algorithm must decide whether to retain it, discard it, or use it to replace an existing item in memory.

The performance of a streaming algorithm is typically evaluated according to four criteria: the number of passes over the data stream, memory usage, running time, and approximation ratio. In Paper B, running time is measured by the number of oracle calls to evaluate a submodular function. In other applications, the appropriate measure is context-dependent. For example, finding the maximum integer in a data stream requires a single pass, memory for one element, time proportional to stream length, and achieves an exact solution.

2.5 The k-Center Problem

The *k-center problem* is a classical clustering problem that is well studied in combinatorial optimization. We are given a large dataset containing both similar and dissimilar data points. Our goal is to organize these data into groups by selecting representative data points as cluster centers. Each data point in the dataset is then assigned to its closest cluster center, which naturally defines the clustering structure.

Metric Distance [80] The standard k-center problem is define in a metric space. Let U be a set of n points in a metric space. We are given a distance function $d : U \times U \to \mathbb{R}_{\geq 0}$ that computes the distance between any two points $x, y \in U$. We assume d is a metric, meaning that it satisfy the following properties for any $x, y, z \in U$:

(I_1) (Positive semidefiniteness) $d(x, y) \geq 0$, and $d(x, y) = 0$ if and only if $x = y$.

(I_2) (Symmetric) $d(x, y) = d(y, x)$.

(I_3) (Triangle inequality) $d(x, y) + d(y, z) \geq d(x, z)$.

k-Center Problem [81] Given a set of points U and a distance $d : U \times U \to \mathbb{R}_{\geq 0}$, find a set $S \subseteq U$ of k points in order to minimize the maximum distance of any point of U to its closest center in S. That is,

$$\min_{S \subseteq V, |S|=k} \max_{v \in V} d(x, S),$$

where $d(x, S) = \min_{s \in S} d(x, s)$.

The k-center problem is NP-hard [82], but it admits efficient approximation algorithms. Hsu and Nemhauser [83] prove that in metric space, no polynomial-time algorithm can achieve an approximation factor better than 2 unless P=NP. While several algorithms achieve the optimal approximation ratio of 2, most notably those by Hsu and Nemhauser [83], Plesník [84], Dyer and Frieze [85], and Hochbaum and Shmoys [86], the approach by Gonzalez [81] stands out for its elegant simplicity. Known as the furthest-first traversal, Gonzalez's algorithm iteratively selects the point at the maximum distance from the current set of centers until k points are established.

This greedy strategy is not only computationally efficient with a running time of $O(nk)$, but it also serves as a fundamental benchmark in clustering theory.

2.6 The Asymmetric k-Center Problem

When the distance function in Section 2.5 does not satisfy the symmetry property, but non-negativity and the triangle inequality still hold, we call this space a *quasi-metric* space [87]. The asymmetric k-center problem is the k-center problem defined on a quasi-metric space.

Asymmetric k-Center Problem [88] Given a set of points U in a quasi-metric space and a distance function $d : U \times U \to \mathbb{R}_{\geq 0}$ that satisfies the triangle inequality, find a set $S \subseteq U$ of k points that minimizes the maximum distance from any point in U to its closest center in S. That is,

$$\min_{S \subseteq V, |S|=k} \max_{v \in V} d(v, S),$$

where $d(v, S) = \min_{s \in S} d(v, s)$.

The asymmetric k-center problem is no easier than the standard k-center problem and is thus also NP-hard. Chuzhoy et al. [89] show that there exists a constant $\alpha > 0$ such that the asymmetric k-center problem cannot be approximated within a factor of $\log^* n - \alpha$ unless NP $\subseteq$ DTIME$(n^{\log \log n})$. To approximate this problem, Panigrahy and Vishwanathan [88] proposed an $\mathcal{O}(\log^* n)$-approximation algorithm, which was later improved by Archer [90] to $\mathcal{O}(\log^* k)$.

Chapter 3

Diversity Maximization in Interactive Systems

In this chapter, we examine two distinct interactive systems and explore various notions of diversity maximization for each. We use a news recommendation system as a representative model for both settings. In this common framework, both systems are probabilistic: when a user is presented with a news item, they examine it with a given probability or otherwise ignore it. Our goal is to maximize the expected diversity a user obtains by the time they cease interaction with the system. The two systems differ in their delivery mechanisms: the first is an offline system that, given a ground set of news items, presents the user with a shuffled list to be examined sequentially until quitting; the second is an online system where news items arrive in a streaming fashion, and the system must select a subset of items to present whenever a user submits a request.

This chapter summarizes papers A and B: in Section 3.1, we study the sequential diversity maximization problem for the first system as studied in paper A; in Section 3.2, we study the streaming diversity maximization problem for the second system as investigated in paper B. For each paper, we introduce the problem setting, provide the main theoretical results, and provide some intuition for the proofs.

3.1 [Paper A] Sequential Diversity Maximization

We consider a personalized user–item interaction system as defined below.

> **NEWS ITEMS:** Given a ground set of n news items $U = \{1, \cdots, n\}$ and a ground set of attributes $\mathcal{X}$, each news item i is associated with a probability p_i and a set of attributes $X(i) \subseteq \mathcal{X}$. The distance between two news items i and j is represented as $d(i, j)$, where $d : U \times U \to \mathbb{R}$ is a metric distance.
>
> **RECOMMENDER SYSTEM:** The news recommendation system shuffles the ground set U via a permutation function $\pi : U \to U$, assigning item $\pi(i)$ to position i, and presents the shuffled list $O_{(\pi)} = (\pi(1), \pi(2), \ldots, \pi(n))$ to the

user. We omit the subscript (π) and use O to denote a ranking of U when the context is clear.

USER–ITEM INTERACTION: The user examines the list $O_{(\pi)}$ from top to bottom. Upon examining item $\pi(i)$, the user has two options: they either accept it with probability $p_{\pi(i)}$ and proceed to the next item, or reject it and leave the system permanently. Since the probability p_i determines whether a user continues or quits, we call p_i the continuation probability.

3.1.1 Diversity Measures

Based on this interaction system, we study Sequential Sum Diversity ($\mathcal{S}_+$) and Sequential Coverage Diversity ($\mathcal{S}_c$). These measures are considered "sequential" as they quantify the expected diversity accumulated over the items viewed before a user stops. Our objective is to solve two corresponding problems: maximizing these measures by finding the best ordering of the ground set.

Definition 1 (Sequential sum diversity ($\mathcal{S}_+$))**.** *The sequential sum diversity of a sequence O, denoted by $\mathcal{S}_+(O)$, is defined as the expected sum of pairwise distances that the user accepts before quitting, where the expectation is taken with respect to the probability distribution over prefixes. Formally,*

$$\mathcal{S}_+(O) = \sum_{t=0}^{n} \mathbb{P}\left[\text{accept } O_t \text{ and quit}\right] \sum\nolimits_{i,j \in O_t} d(i,j), \tag{3.1}$$

where $O_t = (\pi(1), \cdots, \pi(t))$ represent the length t prefix of O, $O_0 = \emptyset$, and $\mathbb{P}\left[\text{accept } O_t \text{ and quit}\right] = \prod_{j=1}^{t} p_{\pi(j)}(1 - p_{\pi(t+1)})$ represents the probability that a user examines and accepts O_t, then proceeds to examine and subsequently reject $\pi(t+1)$.

We show that $\mathcal{S}_+(O)$ can be reformulated as

$$\mathcal{S}_+(O) = \sum\nolimits_{i=1}^{n-1} p_{O_{i+1}} d(\pi(i+1), O_i), \tag{3.2}$$

where $p_{O_i} = \prod_{t=1}^{i} p_{\pi(t)}$ and $d(\pi(i+1), O_i) = \sum_{t=1}^{i} d(\pi(i+1), \pi(t))$.

Definition 2 (Sequential coverage diversity ($\mathcal{S}_c$))**.** *The sequential coverage diversity of a sequence O, denoted by $\mathcal{S}_c(O)$, is the expected number of attributes covered by a user before quitting the system, where the expectation is taken over the probability distribution of all possible prefixes of O, i.e.,*

$$\mathcal{S}_c(O) = \sum_{t=0}^{n} \mathbb{P}\left[\text{accept } O_t \text{ and quit}\right] \bigcup\nolimits_{i \in O_t} X(i). \tag{3.3}$$

3.1.2 Problem Definition

Given the definition of $\mathcal{S}_+$ and $\mathcal{S}_c$, we define the Maximizing Sequential Sum Diversity problem (MAXSSD) and Maximizing Sequential Coverage Diversity problem (MAXSCD) as finding the best item ordering that maximizes $\mathcal{S}_+$ and $\mathcal{S}_c$, respectively. Formally, given a finite set $U = \{1, \ldots, n\}$ of n distinct items and associated probabilities $p_1, \ldots, p_n$, associated attributes $X(1), \cdots, X(n)$, find an ordering O^* of the items in U that maximizes $\mathcal{S}_+$ and $\mathcal{S}_c$, i.e.,

$$\text{MAXSSD} : O^* = \arg\max_{O=\pi(U)} \mathcal{S}_+(O), \tag{3.4}$$

$$\text{MAXSCD} : O^* = \arg\max_{O=\pi(U)} \mathcal{S}_c(O). \tag{3.5}$$

The MAXSCD problem is an instance of the ordered submodular maximization problem as introduced by Kleinberg et al. [91], and thus, a simple greedy algorithm proposed by the authors can be applied to obtain a $\frac{1}{2}$-approximation. In the rest of this section, we focus on the main theoretical results for the MAXSSD problem.

3.1.3 Analysis and Intermediate Problem

The sequential sum diversity $\mathcal{S}_+(O)$, defined in Equation (3.2), can be expressed as a sum of $n-1$ marginal contributions. The t-th term is the contribution of placing item $\pi(t+1)$ after the prefix O_t, and is given by the product of (i) the probability p_{O_t} that the user reaches position $t+1$, and (ii) the marginal increase in pairwise distance, $d(\pi(t+1), O_t)$.

Since the reach probability p_{O_t} decreases with t and converges to zero, the value of $\mathcal{S}_+(O)$ is dominated by the early positions in the ordering. This structure motivates a greedy construction: at each step, select an item that simultaneously has a high continuation probability and yields a large marginal distance with respect to the current prefix. The main technical difficulty in analyzing such an approach is that the marginal distance term depends on the entire prefix O_t. Specifically, an item that yields a large marginal distance at the current step may contribute very little to the marginal distances of items added in subsequent steps.

To simplify the analysis, we introduce an intermediate problem in which the marginal distance contribution of an item depends only on the item placed immediately before it, rather than on the entire prefix. We refer to this problem as Maximizing Ordered Hamiltonian Path (MAXOHP). The objective value for a given ordering O is denoted as $\mathcal{H}(O)$, and it is defined as

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d(\pi(i), \pi(i+1)),$$

where $W_{O_i} = \sum_{j=i+1}^{n} \prod_{t=1}^{j} p_{\pi(t)}$.

Formally, given a finite set of n distinct items $U = \{1, \ldots, n\}$ with associated probabilities $p_1, \ldots, p_n$, the goal of MAXOHP is to compute an ordering O^* of the items in U that maximizes the $\mathcal{H}$ objective, that is,

$$O^* = \operatorname*{argmax}_{O=\pi(U)} \mathcal{H}(O). \tag{3.6}$$

Table 3.1: Approximation ratios achieved by different algorithms for the MAXOHP problem under various continuation probability settings

	p **is uniform**			p **is not uniform**
	p is a constant	$p = 1 - \frac{1}{o(n)}$	$p = c - \frac{1}{\Theta(n)}$	$a \leq p_i \leq b$
Approx. ratio	$1 - p^{\tau-1} - p^{n-\tau} + p^n$	$\frac{3(e-1)}{16e^2} - \Theta\left(\frac{1}{t_n}\right)$	$c - \frac{1}{\Theta(n)}$	$\frac{a^2(1-b)(1-b^{k-1})}{a^2+(k-1)b^{k+1}}$
Algorithm	Best-τ items	Greedy matching	Arbitrary ranking	Best τ items

We show that $\mathcal{H}(O)$ can be equivalently formulated as

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} p_{O_{i+1}} d_L(O_{i+1}), \tag{3.7}$$

where $d_L(O_k) := \sum_{t=1}^{k-1} d(\pi(t), \pi(t+1))$.

3.1.4 Main Results

In Theorem 1, we show that any good solution to the MAXOHP problem yields a good solution to the MAXSSD problem. Consequently, it suffices to focus on solving MAXOHP. We begin with the simpler setting of uniform probabilities, where all items have the same continuation probability, and then extend our approach to the more general non-uniform case. In both settings, we design constant-factor approximation algorithms.

Theorem 1. *Let $p_i \in [a, b]$, with $0 < a < b < 1$. An α-approximation solution for* MAXOHP *is a $\frac{a(1-b)}{2b(1-a)}\alpha$-approximation for* MAXSSD.

The proof of Theorem 1 relies on two key bounds: (1) by the triangle inequality, $2\mathcal{S}_+(O) \geq \mathcal{H}(O)$ for any ordering O, and (2) for an optimal sequence $O^* = \operatorname{argmax}_O \mathcal{S}_+(O)$ to the MAXSSD problem, and assuming that $p_i \in [a, b]$, for all $i \in U$, it holds that $\mathcal{S}_+(O^*) \leq \frac{b(1-a)}{a(1-b)}\mathcal{H}(O^*)$. Combining these bounds yields Theorem 1. When all probabilities are equal, $\frac{a(1-b)}{2b(1-a)} = \frac{1}{2}$, so solving MAXOHP incurs only a factor-2 loss for MAXSSD.

Approximation Guarantees As discussed in Section 3.1.3, the early positions in an ordering dominate the value of the sequential diversity objective. A key question, therefore, is how many items constitute this "first few." Let this number be denoted by τ; an appropriate choice of τ depends on the continuation probabilities p_i.

We begin with the uniform case, where $p_i = p$ for all i. In this setting, the coefficient associated with position i is

$$W_{O_i} = p^i + p^{i+1} + \cdots + p^n \approx \frac{p^i}{1-p},$$

which decreases geometrically with i. This behavior gives rise to three qualitatively different regimes:

- **Constant Continuation Probability.** If $p < 1$ is a fixed constant, then it follows that $\lim_{i\to\infty} W_{O_i} = 0$, and only a constant number of early positions contribute significantly to $\mathcal{H}$. In this case, we use a greedy algorithm to compute a high-quality τ-permutation of the n items and order the remaining items arbitrarily. We refer to this approach as the *best-τ items algorithm*.

- **Near-certain Continuation.** If $p = 1 - \frac{1}{o(n)}$, then $W_{O_i} \approx 1/e$ for $i = o(n)$ and decreases to zero only as i approaches n. Thus, the first $o(n)$ terms all contribute substantially to the objective. Since computing the optimal $o(n)$-permutation is computationally intractable, we instead design a greedy matching algorithm that ensures the first $o(n)$ consecutive distances are sufficiently large, yielding a constant-factor approximation.

- **Full-ordering Regime.** If $p = c - \frac{1}{\Theta(n)}$ for a constant $c < 1$, then $W_{O_i} \approx c/e$ even for $i = n$, implying that all positions contribute non-negligibly. In this regime, any ordering achieves a constant-factor approximation. When $p = 1$, all orderings are equivalent.

Finally, we consider the non-uniform case in which the continuation probabilities satisfy $a \leq p_i \leq b$ for constants a and b. The best-τ items algorithm and its analysis extend naturally to this setting, preserving a constant-factor approximation guarantee. We summarize the approximation ratios achieved by our proposed algorithms in Table 3.1.

3.2 [Paper B] Streaming Diversity Maximization

In this section, we investigate another user-item interaction system under a streaming and stochastic setting. We introduce a novel diversity maximization problem called Streaming Stochastic Submodular Maximization (*S3MOR*).

Figure 3.1: Illustration of the streaming stochastic submodular maximization problem.

3.2.1 Problem Setting

Assume items arrive in a news recommendation system in a streaming manner. A user can visit the system at any time and as many times as they want. Whenever the user visits, the system must present a set of at most k items to the user. Each item is associated with a click probability, which represents the likelihood of the user clicking on it when presented. Each item is also associated with a set of topics; if a user clicks an item, we say the user covers all topics of that item. The goal is to maximize the expected number of topics the user covers by the end of the stream.

Figure 3.1 provides a toy example of the system, where each news item is represented as a box with colored dots indicating its topics. The user visits the system three times: after the arrival of items 3, 6, and 9, respectively. At each visit, the system outputs a recommendation set of size 2. After the third visit, the user can cover the dark blue topic with probability $1-(1-0.5)\times(1-0.7)=0.85$, the pink topic with probability $1-(1-0.5)\times(1-0.5)=0.75$, the yellow topic with probability 0.4, and the teal topic with probability $1-(1-0.7)\times(1-0.6)=0.88$. The expected number of topics covered by the user after the third visit is thus $0.85+0.75+0.4+0.88=2.88$.

3.2.2 Problem Formulation

We start by introducing the notation needed to define our problem. We are given an ordered set of N items $\mathcal{V}=\{V_1,V_2,\ldots,V_N\}$, where item V_i arrives at time step i. If the same item appears multiple times, we treat each copy of it as a distinct item. Each news item $V_i\in\mathcal{V}$ covers a subset of topics from a predefined set $\mathcal{C}=\{c_1,c_2,\ldots,c_d\}$, where d denotes the total number of topics. Formally, $V_i\subseteq\mathcal{C}$ for each $i\in\{1,\cdots,N\}$. Additionally, each news item V_i is associated with a probability $p_i\in[0,1]$ that represents the likelihood that the user will click on item V_i when it is presented. We also define a binary indicator variable $\tau_i\in\{0,1\}$ at time step i, with $\tau_i=1$ denoting an access after the arrival of V_i, and $\tau_i=0$ otherwise. Whenever $\tau_i=1$, the system must output a subset of at most k items from the stream $\{V_1,\cdots,V_i\}$.

Assume the user visits the system T times at time steps $\{r_1,\ldots,r_T\}$. At each time step $j\in[j]$, the system outputs a subset $\mathcal{S}^j\subseteq\{V_1,\cdots,V_{t_j}\}$. Let $\mathcal{S}=\{\mathcal{S}^1,\cdots,\mathcal{S}^T\}$, we use $f(\mathcal{S})$ to denote the expected number of covered topics, and $f(\mathcal{S})$ is defined as follows:

$$f(\mathcal{S})=\sum_{j=1}^{d}\left(1-\prod_{t=1}^{T}\prod_{\substack{V_i\in\mathcal{S}^t\\ V_i\ni c_j}}(1-p_i)\right). \tag{3.8}$$

We note that the exact visit times are not required in practice, and they are included in the objective function purely for notational clarity. In Equation (3.8), the term $1-\prod_{t=1}^{T}\prod_{V_i\in\mathcal{S}^t,V_i\ni c_j}(1-p_i)$ represents the probability that topic c_j is covered, i.e., at least one item containing c_j is clicked across all presented sets. Since each topic's coverage is a Bernoulli random variable with this success probability, the expected number of topics covered equals the sum of individual coverage probabilities across all d topics.

We prove that $f(\mathcal{S})$ is a submodular function. Combined with the stochastic nature of the problem setting, this leads us to name our problem Streaming Stochastic Submodular Maximization (*S3MOR*), which we formally define below.

Problem 1 (***S3MOR***). *We define a news item stream* $\mathbf{S}$ *as a sequence of triples:* $\mathbf{S} = ((V_1, p_1, \tau_1), \ldots, (V_N, p_N, \tau_N))$ *where* N *is the (unknown) length of the stream. At each time step* i*, the system receives a triple* (V_i, p_i, τ_i)*, where* $V_i \in \mathcal{V}$*,* $p_i \in [0, 1]$*, and* $\tau_i \in \{0, 1\}$*. Whenever* $\tau_i = 1$*, the system selects up to* k *news items from the set of items* $\{V_1, \ldots, V_i\}$ *received so far to present to the user. Let* $\mathcal{S} = \{\mathcal{S}^1, \ldots, \mathcal{S}^T\}$ *denote the sets of news items presented to the user over* T *accesses. The objective is to maximize the expected number of distinct topics the user is exposed to by the end of the stream, measured by the function* $f(\mathcal{S})$*, that is,*

$$\begin{aligned} \max_{\mathcal{S}=\mathcal{S}^1\cup\ldots\cup\mathcal{S}^T} \quad & f(\mathcal{S}), \\ \textit{subject to} \quad & \mathcal{S}^t \subseteq \{V_1, \ldots, V_{r_t}\}, \quad \forall t \in [T], \\ & |\mathcal{S}^t| \leq k, \quad \forall t \in [T]. \end{aligned} \tag{3.9}$$

3.2.3 Challenges

The main challenge in solving the *S3MOR* problem stems from three key characteristics. First, user access is *on-demand*: we do not know in advance when or how many times users will access the system. The system must therefore always be prepared to provide fast responses for each user access. Second, recommendation results are *irrevocable*: once items are shown to a user, they cannot be modified. Third, operating in a streaming setting, the system has only sublinear memory with respect to the stream length.

3.2.4 Reduction to Partition Matroid Constraints

Returning to the toy example in Figure 3.1, recall that at the first user visit, items $\{V_1, V_2, V_3\}$ have arrived. At the second visit, items $\{V_1, \cdots, V_6\}$ have arrived, and at the third visit, items $\{V_1, \cdots, V_9\}$ have arrived. We must select $\mathcal{S}^1$ from the first set, $\mathcal{S}^2$ from the second, and $\mathcal{S}^3$ from the third. To reformulate this problem, we construct an expanded stream by concatenating the three arrival sets and treating each occurrence of an item as distinct. Specifically, we mark each occurrence with a superscript i to indicate it belongs to the i-th arrival set, yielding:

$$\{V_1^1, V_1^2, V_1^3, V_2^1, V_2^2, V_2^3, V_3^1, V_3^2, V_3^3, V_4^2, V_4^3, V_5^2, V_5^3, V_6^2, V_6^3, V_7^3, V_8^3, V_9^3\}.$$

This expanded stream naturally partitions into three disjoint candidate sets: $\mathcal{C}_1 = \{V_1^1, V_2^1, V_3^1\}$, $\mathcal{C}_2 = \{V_1^2, \cdots, V_6^2\}$, and $\mathcal{C}_3 = \{V_1^3, \cdots, V_9^3\}$. The feasibility constraints of the *S3MOR* problem, namely $\mathcal{S}^t \subseteq \mathcal{C}_t$ and $|\mathcal{S}^t| \leq k$ for $t \in \{1, 2, 3\}$, then define a partition matroid over the expanded stream.

We can generalize the toy example to arbitrary inputs. Assume we know the exact number of visits T and that the user visits the system at (unknown) time steps

$\{r_1, \cdots, r_T\}$. For each item arriving after the $(t-1)$-th visit and before the t-th visit, we create $T-t+1$ copies of it, marking each copy with superscripts $t, t+1, \cdots, T$ respectively. By concatenating all copies of all items, we obtain an expanded stream where all items with superscript t belong to the t-th partition $\mathcal{C}_t$ for each $t \in \{1, \cdots, T\}$. This generalizes the partition matroid structure from the toy example to arbitrary T.

When T is known, our expanded-stream construction and partitioning reduce the *S3MOR* problem to the streaming submodular maximization problem under a partition matroid constraint. Thus, we can use existing algorithms to solve the *S3MOR* problem. However, not all existing algorithms are suitable. Since we present $\mathcal{S}^t = \mathcal{S} \cap \mathcal{C}_t$ upon the t-th visit, we require that $\mathcal{S}^t$ remains fixed once visit t occurs. Algorithms satisfying this property include those by Chakrabarti and Kale [38], Chekuri et al. [39], and Feldman et al. [40], all achieving $\frac{1}{4}$- approximation. In this thesis, we use the Streaming-Greedy algorithm from Chekuri et al. [39].

3.2.5 The STORM Algorithm

In the previous section, we assumed that the exact number of user visits T is known. This assumption enables a reduction from the *S3MOR* problem to a streaming submodular maximization problem subject to a partition matroid constraint. However, assuming T is known a priori is unrealistic in practice.

We therefore consider a more practical setting where T is unknown, but an upper bound $T' \geq T$ is provided as input. In practice, T' can be estimated by eliciting from users the maximum number of visits they anticipate, or by analyzing historical user behavior patterns to determine a reasonable upper bound.

A Naive Approach and Its Failure. Given T', a natural extension of the approach from the previous section would proceed as follows: (1) create $T'-t+1$ copies of each item arriving before the t-th user access, marking each copy with superscript $t, t+1, \ldots, T'$; (2) define a partition matroid where items with superscript j belong to the j-th partition; (3) run the Streaming-Greedy algorithm on the expanded stream and output results from the t-th partition upon the t-th user visit.

Unfortunately, this straightforward adaptation fails to provide meaningful guarantees. We demonstrate this via the following adversarial example. Let $T = 1$, with $V_1 = \{c_1\}$ and $p_1 = 1-\epsilon$, followed by $V_2 = \{c_2\}$ with $p_2 = \epsilon$, where $\epsilon < 1$ satisfies $1-(1-\epsilon)^{T'} < 1-(1-\epsilon)^{T'-1}+\epsilon$. Suppose a user accesses the system after V_2 arrives, and $k = 1$. Initially, the Streaming-Greedy algorithm maintains empty results in each partition. After V_1 arrives, all partition results become $\{V_1\}$. However, after V_2 arrives, the expected coverage of $\{V_2^1, V_1^2, V_1^3, \ldots, V_1^{T'}\}$ exceeds that of $\{V_1^1, V_1^2, \ldots, V_1^{T'}\}$. Consequently, the first partition's result is replaced by $\{V_2\}$. Upon the user's visit, the system outputs the first partition, achieving a competitive ratio of $\frac{\epsilon}{1-\epsilon}$, which can be arbitrarily poor.

The Active Partition Strategy. To address the shortcoming of the previous naive approach, we propose a simple yet effective modification: construct an extended

stream and partition it into T' partitions as described in the naive approach, upon each user visit, greedily select and output results from the best *active* partition. We call this algorithm STORM, and it yields a competitive ratio of $\frac{1}{4(T'-T+1)}$.

The key insight is as follows. Initially, we mark all partitions as active and update each partition with every arriving item. Upon a user visit, the results from all active partitions are valid, as each represents a subset of size k selected from the items that have arrived up to that point. We select the partition with the best objective value, output its solution to the user, mark that partition as inactive, and continue updating all remaining active partitions. When $T' = T$, the competitive ratio is equal to $1/4$, which matches the approximation ratio of the underlying Streaming-Greedy algorithm.

3.2.6 The STORM++ Algorithm

When T' is significantly larger than T, the competitive ratio of $\frac{1}{4(T'-T+1)}$ degrades substantially. To improve the competitive ratio, the key insight is to reduce the gap between T' and T.

Discretization and Parallel Execution. We can reduce $T' - T$ by discretizing the range $[1, T']$ into a geometric sequence of candidate values $\{\delta, 2\delta, \ldots, \lceil \frac{T'}{\delta} \rceil \delta\}$. By construction, at least one of these candidates differs from the true value T by at most δ. If we could execute STORM with this closest guess, we would obtain a competitive ratio of at least $\frac{1}{4\delta}$.

However, since we do not know a priori which guess is closest to T, we run multiple parallel instances of STORM, each configured with a different candidate value from our discretized set. This parallel execution introduces a new challenge: upon each user visit, each STORM instance produces its own output, which one should be presented to the user?

Greedy Selection Across Instances. We answer the above question by applying a greedy selection strategy: upon the t-th user visit, we collect the t-th output from each STORM instance and present to the user the solution with the highest expected coverage. This greedy selection guarantees an expected coverage at least half that of the solution produced by the instance running with the best guess. The best guess produces a solution with a competitive ratio of at least $\frac{1}{4\delta}$, consequently, STORM++ achieves a competitive ratio of $\frac{1}{8\delta}$, where the parameter $\delta \in \mathbb{N}$ controls the trade-off between solution quality and computational efficiency. Smaller values of δ yield better competitive ratios but require running more parallel instances.

3.2.7 Complexity

To evaluate our proposed algorithms, we report three complexity metrics: *space complexity*, *time complexity*, and *response time complexity*. Assuming a common space stores all original items, *space complexity* measures the additional memory each user requires for personalized recommendations, including document identifiers and user-item association probabilities. We assume an *oracle* can evaluate $f(S)$ for any

feasible set S. *Response time complexity* measures the worst-case oracle calls needed to produce a size-k recommendation after a user request, capturing user-experienced latency, while *time complexity* counts oracle calls over the entire stream.

3.2.8 Main Results

Previously in the challenges section, we mentioned the goal of using sublinear memory. However, if we relax this constraint and allow $\mathcal{O}(N)$ memory to store item probabilities for all items in the stream, the problem degrades to an offline setting and reduces to submodular maximization subject to a partition matroid constraint, which is studied by Fisher et al. [30]. In this case, we store all incoming items' probabilities and, upon each user request, greedily select the best k items from all available items. This greedy algorithm achieves a $\frac{1}{2}$-approximation ratio [30]. We name this algorithm LMGREEDY, and report its performance below.

Theorem 2. *The* LMGREEDY *algorithm for Problem 1 has a competitive ratio of* $\frac{1}{2}$*, space complexity* $\Theta(N + kT)$*, time complexity* $\mathcal{O}(NTk)$*, and response time complexity* $\mathcal{O}(Nk)$*. Moreover, the competitive ratio of* $\frac{1}{2}$ *is tight, i.e., for Problem 1, no streaming algorithm can achieve a competitive ratio better than* $\frac{1}{2}$ *without violating the irrevocability constraint.*

We also report the performance of the STORM and STORM++ algorithm below. The proofs of Theorem 3 and Theorem 4 are built on the submodularity of the objective function and the greedy selection criterion.

Theorem 3. *Let* T *be the number of user accesses and* T' *a given upper bound.* STORM *has competitive ratio* $\frac{1}{4(T'-T+1)}$*, space complexity* $\mathcal{O}(T'k)$*, time complexity* $\mathcal{O}(NkT')$*, and response time complexity* $\mathcal{O}$ (T')*.*

Theorem 4. STORM*++ has a competitive ratio of* $\frac{1}{8\delta}$*, space complexity* $\mathcal{O}(T'^2k/\delta)$*, time complexity* $\mathcal{O}(NkT'^2/\delta)$*, and response time complexity* $\mathcal{O}(T')$*.*

Chapter 4

Diversity-Constrained Densest Subgraph Discovery

From this chapter onwards, we shift our attention from interactive recommender systems to problems formulated in graph settings. In this chapter, we study the densest subgraph discovery problem (DSP) with edge-color constraints. In this problem, diversity is not directly maximized but implicitly enforced as an optimization constraint.

4.1 [Paper C] Finding Densest Subgraphs with Edge-Color Constraints

In large social networks, users are connected by many kinds of relationships, such as friendship, family, acquaintance, and work ties, which also vary in strength. Simply finding dense subgraphs ignores this heterogeneity and may miss communities that matter for downstream tasks. For instance, in fraud detection on financial transaction graphs where nodes represent accounts and edges represent transactions with attributes such as amount, merchant category, and geographic location, fraudulent account clusters may be characterized by specific edge-type combinations such as transactions in unusual merchant categories for those accounts, or activity from locations that mismatch the accounts' typical patterns. Such patterns would be missed if all edge types were treated equally. Densest subgraph discovery with edge type constraints allows us to focus on groups that are not only well connected, but also have specific mixes of relationship types and tie strengths. Different applications require different edge-type combinations. For example, contact recommendations seek communities rich in weak professional ties, while identifying functional protein complexes requires groups with strong interaction weights across multiple binding types.

To formalize these diverse relationship types, we adopt the classical graph-theoretic convention of representing edge types as colors. Formally, let $G = (V, E)$ denote an undirected, simple graph where V is a finite vertex set and E is a finite edge set satisfying $E \subseteq \{\{u, v\} \subseteq V \mid u \neq v\}$. The cardinalities of these sets are denoted by $n = |V|$ and $m = |E|$.

An edge-colored graph $G = (V, E, c)$ extends this basic structure by assigning colors to edges through a coloring function $c : E \to 2^{\mathbb{N}}$ that maps each edge to a set of natural numbers representing colors. While this definition permits edges to have multiple colors, we focus on the single-color case throughout this chapter, where each edge has exactly one color. For convenience, if edge $e \in E$ has a single color i, we abbreviate $c(e) = \{i\}$ by writing $c(e) = i$. The extension to multi-colored edges is discussed in our paper C.

For any subset $S \subseteq V$, we use $E[S]$ to denote the edge set $\{\{u, v\} \in E \mid u, v \in S\}$ containing all edges with both endpoints in S. The induced subgraph on S is then written as $G[S] = (S, E[S])$.

We call a graph $G = (V, E)$ sparse if $|E| = O(|V|)$. A graph $G = (V, E)$ is *everywhere sparse* if for any subset $V' \subseteq V$, the induced subgraph $G(V')$ is sparse. Throughout this chapter, we restrict attention to everywhere sparse graphs.

With this notation in place, we can now formalize the edge color constrained densest subgraph problem. Let K denote the maximum number of colors present in the graph. We represent edge-color constraints using a vector $\mathbf{h} \in \mathbb{N}^K$, leading to the following definition:

Definition 3 (ALHCEDGESDSP)**.** *Given an edge-colored graph $G = (V, E, c)$, a number of colors $K \in \mathbb{N}$, and a vector $\mathbf{h} \in \mathbb{N}^K$, find $S \subseteq V$ such that*

- *$E[S]$ contains at least h_i edges e with $i \in c(e)$ for all $i \in [K]$, and*
- *the density $\rho(S) = \frac{|E[S]|}{|S|}$ is maximized.*

*We name the problem the **At Least** $\mathbf{h}$ **Colored Edges Densest Subgraph Problem**, and denote it as* ALHCEDGESDSP.

The simplest case for this problem is when all edges have the same color. Assuming the vector $\mathbf{h}$ sums to an integer h, the ALHCEDGESDSP problem reduces to the **At Least h Edges Problem**, denoted as ATLEASTHEDGESDSP, which we formally define below.

Definition 4 (ATLEASTHEDGESDSP)**.** *Given a graph $G = (V, E)$ and $h \in \mathbb{N}$, find a subset $S \subseteq V$ such that $|E[S]| \geq h$, and the density $\rho(S) = \frac{|E[S]|}{|S|}$ is maximized.*

We prove that the decision version of the ATLEASTHEDGESDSP problem is NP-hard. Thus, the decision version of the ALHCEDGESDSP problem is also NP-hard. To solve these problems, we utilize existing algorithms for the **Densest At Least k Nodes Subgraph Problem**, denoted as Dal$k$S, which we define below.

Definition 5 (Dal$k$S)**.** *Given a graph $G = (V, E)$ and $k \in \mathbb{N}$, find a subset $S \subseteq V$ such that $|S| \geq k$, and the density $\rho(S) = \frac{|E[S]|}{|S|}$ is maximized.*

The main technical approach of our work is as follows: we first prove that we can approximate the ATLEASTHEDGESDSP problem by solving the Dal$k$S problem, then we approximate the ALHCEDGESDSP problem by approximating the ATLEASTHEDGESDSP problem. Both approximation steps achieve constant approximation ratios.

4.2 Reducing ATLEASTHEDGESDSP to Dal$k$S

Below, we state the key lemma for our approach. The aim is to approximate the densest at least h edges DSP via the densest at least k node DSP.

Lemma 1. *Given a graph G and $h \in \mathbb{N}$. Let k be the minimum number of nodes over all graphs that are densest subgraphs of G with at least h edges. Furthermore, let S be an optimal solution for the at least k-nodes DSP problem in G. Then S is also an optimal solution for the densest subgraph with at least h edges.*

Lemma 1 suggests an approach to solve the at least h edge DSP. Indeed, it states that there exists an integer $k < N$ such that the optimal density for the at least h edge DSP is equal to that of the at least k nodes DSP. The proof follows by contradiction and can be found in our paper C.

Consequently, provided an α-approximation solution S of the at least k nodes DSP, $\rho(S)$ is also an α approximation of at least h edge DSP. If S is also feasible, S is the desired solution. Otherwise, if $G[S]$ contains fewer than h edges, we can obtain a feasible solution S' by adding at most h edges to S. It remains to bound the density of S', since adding edges may decrease $\rho(S)$. To facilitate the analysis, we introduce the following definition.

Definition 6 (Constant-Yield Approximation)**.** *Let G be a graph, and let S^* denote an optimal solution to the at least k nodes DSP. Let $\mathcal{A}$ be an α-approximation algorithm for this problem, and let S be the subgraph returned by $\mathcal{A}$ on input G. We say that $\mathcal{A}$ is a **Constant-Yield** algorithm if for any input graph, there exists a constant $c_1 > 0$ such that*

$$c_1 \cdot |E(S)| \geq |E(S^*)|. \tag{4.1}$$

The $\frac{1}{2}$-approximation algorithm of Khuller and Saha [43] and the $\frac{1}{3}$-approximation algorithm of Andersen and Chellapilla [44] are both Constant-Yield approximation algorithms.

Analysis. Let ρ^* denote the optimal density for the at least h edges DSP. Let S be the output of a Constant-Yield approximation algorithm such that $\rho(S) \geq \frac{1}{\alpha}\rho^*$. To lower bound the density $\rho(S') = \frac{|E[S']|}{|S'|}$, we upper bound $|S'|$ and lower bound $|E[S']|$ as follows:

- By the definition of Constant-Yield approximation algorithms, there exists a constant c_1 such that $c_1|E[S]| \geq |E[S^*]| \geq h$.
- Since G is everywhere sparse, we have $|E[S]| \leq c_2|S|$ for some constant c_2. Adding h edges to S introduces at most $2h$ nodes, so

$$|S'| \leq |S| + 2h \leq |S| + 2c_1|E[S]| \leq |S| + 2c_1c_2|S| \leq c|S| \tag{4.2}$$

  holds for some sufficiently large constant c.

Combining Equation (4.2) with $|E[S']| \geq |E[S]|$, we obtain

$$d(S') = \frac{|E[S']|}{|S'|} \geq \frac{|E[S]|}{c|S|} = \frac{1}{c}d(S) \geq \frac{1}{c\alpha}\rho^*. \tag{4.3}$$

To summarize, given an everywhere sparse graph G and $h \in \mathbb{N}$, let k denote the minimum number of nodes over all densest subgraphs of G with at least h edges. Let S be the output of a Constant-Yield approximation algorithm [43, 44]. Then S (or any subgraph S' obtained by adding at most h edges to S) is a constant approximation for the at least h edges DSP. For ease of reference, we refer to this procedure as the EDGEAUG algorithm.

To complete the analysis, one essential question remains: how do we determine k? Determining k exactly is computationally challenging, as it requires either solving the densest subgraph problem with exactly k nodes for all $k \in \{1, \ldots, n\}$, which is NP-hard, or enumerating all densest subgraphs, which can be exponential in number.[1]

Instead, we compute a lower bound $\ell(h)$[2] depending only on h, and run EDGEAUG for all $k \in \{\ell(h), \ldots, n\}$, returning the solution with the highest density. In particular, the peeling algorithm of Andersen and Chellapilla [44] covers all such values of k in a single pass in $O(m + n)$ time, making this approach practical.

The above analysis yields the following theorem.

Theorem 5. *For everywhere sparse graphs, there exists a constant-factor approximation for the at least h edges DSP.*

Proof. To illustrate this point, let $\{G_{\ell(h)}, \ldots, G_n\}$ denote the subgraphs produced by a Constant-Yield approximation algorithm for the at least k nodes DSP for $k \in \{\ell(h), \ldots, n\}$, respectively. Let $\tilde{G}_i$ denote the result of augmenting G_i with additional edges from the input graph G to satisfy the edge constraint h. Then the subgraph

$$G^* = \arg \max_{i \in \{\ell(h), \ldots, n\}} \rho(\tilde{G}_i)$$

provides a constant-factor approximation for the at least h edges DSP. □

4.3 Reducing ALHCEDGESDSP to ATLEASTHEDGESDSP

In this section, we show that given a constant approximation for the at least h edges DSP, we can obtain a constant approximation for the at least $\mathbf{h}$ colored edges DSP.

Let $l = \sum_{i=1}^{K} \mathbf{h}_i$. The previous section provides a solution S' that is a $\frac{1}{\alpha c}$-approximation for the at least l edges DSP. If S' is also valid for the at least $\mathbf{h}$ colored edges DSP, S' is the desired solution. Otherwise, we construct a feasible solution $S'' = (V'', E'')$ by adding edges to S'. Specifically, let f_i be the number of edges

[1] Consider a graph consisting of r disjoint densest subgraphs. The union of any non-empty subset of these components is also a densest subgraph, yielding $2^r - 1$ distinct solutions. In the worst case, $r = \Theta(n)$, resulting in $2^{\Theta(n)}$ densest subgraphs in total.

[2] The details on how to calculate ℓ can be found in our paper C.

of color $i \in [K]$ in S', we need to add $\max\{0, h_i - f_i\}$ edges of color i to make S' feasible.

Let S^* be the optimal solution for the at least l edges DSP, and S_c^* be the optimal solution for the at least $\mathbf{h}$ colored edges DSP. Since the at least l edges DSP relaxes the at least $\mathbf{h}$ colored edges DSP, we have $\rho(S^*) \geq \rho(S_c^*)$. This inequality enables us to lower bound $\rho(S'')$. Since adding one edge introduces at most two nodes, we have $|E''| \leq |E'| + l$ and $|V''| \leq |V'| + 2l \leq |V'| + 2|E'| \leq c|V'|$ for some constant $c \geq 3$. Thus,

$$d(V'') = \frac{|E''|}{|V''|} \geq \frac{|E'|}{c|V'|} = \frac{1}{c} d(V') \geq \frac{1}{c^2 \alpha} d^*.$$

To summarize, we get the following theorem.

Theorem 6. *For everywhere sparse graphs, there exists a constant approximation algorithm for the at least* $\mathbf{h}$ *colored edges DSP. Specifically, let* S' *be a* $\frac{1}{\alpha}$*-approximation for the at least* $\sum_{i=1}^{K} \mathbf{h}_i$ *edges DSP. Then* S' *(or any subgraph* S'' *obtained by adding at most* h_i *edges of color* i *for each* $i \in [K]$ *to* S'*) is a* $\frac{1}{\alpha c}$*-approximation for the at least* $\mathbf{h}$ *colored edges DSP for some sufficiently large constant* c*.*

4.4 Conclusion

In paper C, we introduce new variants of the densest subgraph problem in networks with single or multiple edge attributes. To the best of our knowledge, this is the first work that studies the densest subgraph problem subject to edge color constraints. We define the at least $\mathbf{h}$ colored edges DSP and solve it on everywhere sparse graphs. Our approach first reduces the at least h edges DSP to the at least k nodes DSP, then shows that augmenting the solution with colored edges yields a feasible solution to the at least $\mathbf{h}$ colored edges DSP with only a constant factor loss in approximation. We provide two $\mathcal{O}(1)$-approximation algorithms for the at least h edges DSP and extend them to obtain constant-approximation algorithms for the at least $\mathbf{h}$ colored edges DSP. While our algorithms are designed for graphs where each edge has a single color, we show in the paper C that they can be easily extended to the general case where edges may have multiple colors, maintaining $\mathcal{O}(1)$-approximation guarantees.

Our experimental results provided in the paper C validate the practical efficacy and efficiency of the proposed algorithms on a wide range of real-world graphs. As a case study, we evaluate our methods on the DBLP co-authorship network to identify dense research communities with diverse conference representation. Our approach outperforms existing multilayer-based densest subgraph methods by achieving higher density while ensuring balanced participation across all ten conferences.

Chapter 5

Graph Interventions for Fairness in Networks

In this chapter, we continue our exploration of graphs with a focus on fairness, studying two complementary fairness measures: PageRank fairness (Section 5.1) and hitting-time fairness (Section 5.2). PageRank captures the steady-state importance or centrality of nodes as determined by the graph's link structure, while hitting time measures the expected number of steps a random walk takes to reach a target node, reflecting notions of accessibility and proximity.

For both problem settings, we assume that vertices are partitioned into disjoint groups based on a sensitive attribute. In social or human connection networks, this attribute may represent demographic characteristics such as gender or race, while in video recommendation networks, it may encode content properties such as neutral versus harmful video classifications.

We enhance fairness in both settings through targeted graph interventions. For PageRank fairness, we rebalance influence between vertex groups through edge reweighting, whereas for hitting-time fairness, we introduce a small number of strategic shortcuts to improve accessibility for selected groups. These intervention strategies are motivated by real-world applications where structural modifications can mitigate bias and promote equitable outcomes. In content recommendation systems, adjusting edge weights can rebalance visibility across different types of content, while in polarized social networks, strategic edge additions can reduce segregation and improve connectivity between disparate groups. Together, these two problem settings demonstrate that structural interventions in graphs can serve as powerful tools to address different dimensions of fairness.

5.1 [Paper D] Fairness-Aware PageRank via Edge-Rewiring

The problem of fairness-aware PageRank is to ensure that a graph's influence is distributed equitably among groups. To achieve such fairness, various intervention strategies can be employed, including adjusting algorithmic parameters such as mod-

ifying restart probabilities and restart vectors, and adjusting graph structures such as adding edges and reweighting existing edges. In our work, we focus on edge reweighting.

Consider a video recommendation graph as an example. Here, each node represents a video, and an edge (u, v) indicates that while a user is watching video u, video v is recommended as the next video to watch. Typically, there is a list of recommended next-to-watch videos, and the ranking of each video can be influenced by the edge weights. For instance, a recommendation list $[v_1, v_2, \cdots, v_k]$ corresponds to edge weights $w(u, v_1) \geq w(u, v_2), \cdots, \geq w(u, v_k)$. In this scenario, to enhance or reduce the visibility of videos from a target group, a moderator can adjust the rankings within the recommendation lists, which corresponds to modifying the edge weights in the graph.

5.1.1 The PageRank Fairness Loss

Given a directed graph $G = (V, E)$ with n vertices and m edges, suppose the vertices are partitioned into K disjoint groups $\{V_1, \ldots, V_K\}$. For each group V_k, let $\mathbf{1}_k \in \{0, 1\}^n$ denote its indicator vector, where $\mathbf{1}_{kj} = 1$ if vertex $j \in V_k$ and $\mathbf{1}_{kj} = 0$ otherwise.

Let $\mathbf{P}$ be the row-stochastic transition matrix associated with G, and let $\mathbf{p}$ denote the PageRank vector with restart probability γ and restart distribution $\mathbf{v}$. The PageRank vector satisfies

$$\mathbf{p}^\mathsf{T} = \gamma \mathbf{v}^\mathsf{T} \big(\mathbf{I} - (1-\gamma)\mathbf{P}\big)^{-1}. \tag{5.1}$$

Given a target distribution of group-wise PageRank scores $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_K)$, we define two fairness loss functions.

(1) L_2 PageRank Fairness Loss.

$$L(\mathbf{P}, \gamma, \mathbf{v}, \boldsymbol{\phi}) = \frac{1}{K} \sum_{k=1}^{K} \big(\mathbf{1}_k^\mathsf{T} \mathbf{p} - \phi_k\big)^2, \tag{5.2}$$

where $\mathbf{p}$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v})$ as defined in Equation (5.1).

(2) Group-Adapted L_2 PageRank Fairness Loss.

$$L_g(\mathbf{P}, \gamma, \boldsymbol{\phi}) = \frac{1}{K^2} \sum_{\ell=1}^{K} \sum_{k=1}^{K} \big(\mathbf{1}_k^\mathsf{T} \mathbf{p}_\ell - \phi_k\big)^2, \tag{5.3}$$

where the restart vector $\mathbf{v}_\ell = \frac{1}{|V_\ell|} \mathbf{1}_\ell$ corresponds to a random walk that restarts uniformly within group ℓ, and

$$\mathbf{p}_\ell^\mathsf{T} = \gamma \mathbf{v}_\ell^\mathsf{T} \big(\mathbf{I} - (1-\gamma)\mathbf{P}\big)^{-1}.$$

Both loss functions measure the deviation between the target and the actual group-wise PageRank scores. The first allows an arbitrary restart vector $\mathbf{v}$, whereas the second restricts restarts to occur uniformly within individual groups.

5.1.2 Problem Formulation

Our objective is to find a new transition matrix $\hat{\mathbf{P}}$ that minimizes one of the above loss functions subject to structural constraints. To this end, we define two feasible sets:

$$\mathcal{C}(\mathbf{P}) = \left\{ \hat{\mathbf{P}} \in \mathbb{R}_{\geq 0}^{n \times n} \,\middle|\, \hat{\mathbf{P}}\mathbf{1} = \mathbf{1},\ \hat{\mathbf{P}}_{ij} = 0 \text{ if } \mathbf{P}_{ij} = 0 \right\}, \tag{5.4}$$

$$\mathcal{C}_R(\mathbf{P}) = \left\{ \hat{\mathbf{P}} \in \mathbb{R}_{\geq 0}^{n \times n} \,\middle|\, (1-\delta)\mathbf{P}_{ij} - \epsilon \leq \hat{\mathbf{P}}_{ij} \leq (1+\delta)\mathbf{P}_{ij} + \epsilon \right\}. \tag{5.5}$$

The constraint set $\mathcal{C}(\mathbf{P})$ preserves the original graph structure by forbidding the introduction of new edges, while $\mathcal{C}_R(\mathbf{P})$ further restricts the magnitude of edge reweighting by imposing both relative (δ) and absolute (ϵ) bounds.

Given a transition matrix $\mathbf{P}$, restart probability $0 < \gamma < 1$, groups $V_1, \ldots, V_K$, and a target group-wise PageRank score distribution $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_K)$ satisfying $\|\boldsymbol{\phi}\|_1 = 1$, we consider the following general optimization problem:

$$\mathbf{P}^* = \arg\min_{\hat{\mathbf{P}} \in \mathcal{C}} \mathcal{L}(\hat{\mathbf{P}}, \boldsymbol{\phi}), \tag{5.6}$$

where $\mathcal{L} \in \{L(\cdot, \gamma, \mathbf{v}, \boldsymbol{\phi}), L_g(\cdot, \gamma, , \boldsymbol{\phi})\}$ and $\mathcal{C} \in \{\mathcal{C}(\mathbf{P}), \mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})\}$. Combining the two objectives with the two types of feasibility constraints yields four concrete optimization problems as follows

- **$\boldsymbol{\phi}$-PageRank:** Minimize $L(\hat{\mathbf{P}}, \gamma, \mathbf{v}, \boldsymbol{\phi})$ subject to $\mathcal{C}(\mathbf{P})$.
- **Restricted $\boldsymbol{\phi}$-PageRank:** Minimize $L(\hat{\mathbf{P}}, \gamma, \mathbf{v}, \boldsymbol{\phi})$ subject to $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$.
- **Group-adapted $\boldsymbol{\phi}$-PageRank:** Minimize $L_g(\hat{\mathbf{P}}, \gamma, \boldsymbol{\phi})$ subject to $\mathcal{C}(\mathbf{P})$.
- **Restricted group-adapted $\boldsymbol{\phi}$-PageRank:** Minimize $L_g(\hat{\mathbf{P}}, \gamma, \boldsymbol{\phi})$ subject to $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$.

The motivation for studying $\boldsymbol{\phi}$-PageRank and Restricted $\boldsymbol{\phi}$-PageRank is straightforward: our goal is to redistribute group-wise PageRank scores by reweighting existing edges. In contrast, the motivation for Group-adapted $\boldsymbol{\phi}$-PageRank and Restricted group-adapted $\boldsymbol{\phi}$-PageRank is more subtle. Real-world networks often exhibit strong homophily, whereby vertices within the same group are more densely connected than vertices across groups. As a result, the total PageRank score of a group V_k is largely determined by random walks that restart within that group [92]. The Group-adapted $\boldsymbol{\phi}$-PageRank and Restricted group-adapted $\boldsymbol{\phi}$-PageRank problems consider this homophilic structure in its extreme form, asking whether desired group-wise PageRank distributions can still be achieved solely through edge reweighting when random walks are restricted to restart within individual groups.

5.1.3 Proposed Solution

We prove that the loss functions defined in Equation (5.2) and Equation (5.3) are non-convex, but the feasibility constraints $\mathcal{C}(\mathbf{P})$ and $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$ are convex, so we

propose efficient projected gradient-descent methods for computing locally-optimal edge weights. We prove that the proposed projected gradient update method with constant step size converges to a stationary point, which achieves a local minimum for all four problems we study.

The gradient of the fairness-loss function $L(\mathbf{P}, \gamma, \mathbf{v})$ with respect to the transition matrix $\mathbf{P}$ is given by

$$\frac{\partial}{\partial \mathbf{P}} L(\mathbf{P}, \gamma, \mathbf{v}) = \frac{2(1-\gamma)}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\mathsf{T} \mathbf{p} - \phi_k\right) \mathbf{p} \mathbf{y}_k^\mathsf{T}, \tag{5.7}$$

where $\mathbf{p}$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v})$, and $\mathbf{y}_k = \mathbf{U}^{-T}\mathbf{1}_k = (\mathbf{I} - (1-\gamma)\mathbf{P})^{-1}\mathbf{1}_k$, with $\mathbf{U} = \mathbf{I} - (1-\gamma)\mathbf{P}^\mathsf{T}$. The gradient of the fairness-loss function $L_g(\mathbf{P}, \gamma)$ with respect to the transition matrix $\mathbf{P}$ is given by

$$\frac{\partial}{\partial \mathbf{P}} L_g(\mathbf{P}, \gamma) = \frac{2(1-\gamma)}{K^2} \sum_{k=1}^{K} \sum_{\ell=1}^{K} \left(\mathbf{1}_k^\mathsf{T} \mathbf{p}_\ell - \phi_k\right) \left(\mathbf{p}_\ell \mathbf{1}_k^\mathsf{T} \mathbf{U}^{-1}\right), \tag{5.8}$$

where $\mathbf{p}_\ell$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v}_\ell)$.

After each gradient descent step, $\hat{\mathbf{P}}$ needs to be projected back into the feasible area subject to constraints $\mathcal{C}(\mathbf{P})$ or $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$. Both projections are studied in the literature, with the former constraint type studied by Michelot [93], Pardalos and Kovoor [94], Duchi et al. [95], Condat [96], and the latter by Adam and Mácha [97].

5.2 [Paper D] Fairness-Aware Structural Bias Reduction

As discussed in the previous section, in homophilic networks, a group's PageRank is largely driven by random walks that restart within the group. Beyond this effect on PageRank, homophily also induces structural bias, creating segregation in which disparate groups remain only loosely connected. Such segregation perpetuates inequalities by limiting cross-group information flow and reducing minority group visibility. This phenomenon manifests in polarized social network communities and antagonistic content in video-sharing or news-feed platforms. Increasing connectivity between disparate groups is therefore crucial for minimizing social friction and exposing individuals to diverse viewpoints.

A natural measure for quantifying such structural bias is the hitting time between groups, which captures how quickly a random walk originating in one group is expected to reach another. High inter-group hitting times indicate poor connectivity and limited information flow, both of which perpetuate segregation and deepen inequality. In this paper, we study two optimization problems aimed at reducing structural bias through edge additions, focusing on minimizing the average and the maximum hitting times between disparate groups. The average hitting time captures the overall efficiency of inter-group navigation, while the maximum hitting time addresses fairness by preventing worst-case disparities where certain node pairs remain excessively isolated. Both objectives present significant algorithmic challenges, as average hitting time requires handling supermodular optimization under cardinality constraints

and maximum hitting time lacks supermodularity entirely. Nevertheless, we develop approximation algorithms for both objectives with provable guarantees.

5.2.1 Problem Formulation

Graphs. Let $G = (V, E)$ denote an undirected, connected graph where V is a finite vertex set with $|V| = n$ and E is a finite edge set. The vertex set V is partitioned into two disjoint, non-empty groups R (red nodes) and B (blue nodes). For a subset $X \subseteq V$, let $E[X]$ denote the set of edges with both endpoints in X, and let $G[X] = (X, E[X])$ denote the induced subgraph on X. For a set of non-edges F of G, we write $G + F = (V, E \cup F)$. The degree of a node v in G, denoted $d_G(v)$, is its number of neighbors.
Random walks. We consider uniform simple random walks on G, where at each step a neighbor of the current node is selected uniformly at random. For a subset $A \subseteq V$, the random variable $\tau_u(A)$ denotes the first time a walk starting from node u visits a node in A. If $u \in A$, then $\tau_u(A) = 0$. The expectation $H_G(u, A) = \mathbb{E}_G[\tau_u(A)]$ is the *hitting time* from u to A. We note the following properties: $H_G(u, A) = \min_{a \in A} H_G(u, a)$, $H_G(u, u) = 0$, $H_G(u, A) = 0$ if and only if $u \in A$, and $H_G(u, A) \leq H_G(u, B)$ whenever $B \subseteq A$.

Definition 7 (Maximum and Average Hitting Time)**.** *For inter-group non-edges $F \subseteq (R \times B) \setminus E$, the maximum hitting time from R to B on $G + F$ is*

$$f(F) \triangleq \max_{r \in R} H_{G+F}(r, B),$$

and the average hitting time from R to B on $G + F$ is

$$g(F) \triangleq \frac{1}{|R|} \sum_{r \in R} H_{G+F}(r, B).$$

Problem Definitions. Given an undirected connected graph $G = (V, E)$ with $|V| = n$ nodes, valid bipartition $V = \{R, B\}$ and budget $k \in \mathbb{N}$, we seek to find inter-group non-edges $F \subseteq (R \times B) \setminus E$ with $|F| \leq k$ that minimize hitting time from R to B. We consider two variants:

- **Budgeted Minimum Maximum Hitting Time (BMMH):** Minimize the maximum hitting time $f(F)$ from Definition 7.
- **Budgeted Minimum Average Hitting Time (BMAH):** Minimize the average hitting time $g(F)$ from Definition 7.

5.2.2 Key Properties and Observations

We establish several key properties for solving BMMH and BMAH.

1. **Reduction to Selecting Red Endpoints.** Changing the blue endpoints of edges in a feasible solution F does not affect $f(F)$ or $g(F)$, since random walks

halt upon reaching any blue node. Consequently, both problems reduce to selecting a multiset of red endpoints, where multiple edges may share the same red endpoint.

2. **Monotonicity.** Both f and g are monotonically decreasing: adding inter-group edges can only reduce hitting times from red nodes to blue nodes.

3. **Supermodularity of Individual and Average Hitting Times.** The individual hitting-time function $H_{G+e}(r, B)$ is supermodular for all $r \in R$, which implies that the average hitting-time function g is also supermodular. This supermodularity captures a diminishing returns property for hitting-time reductions when adding shortcut edges. Intuitively, the marginal benefit of adding edge e decomposes into two factors: the expected time saved conditional on e being traversed (which is equal to $H(u, B) - 1$, where u is the red endpoint of edge e), and the probability that the walk traverses e before reaching B. Adding another edge e' before e decreases both factors: e' provides an alternative path that reduces the probability of traversing e, and it also shortens the remaining expected hitting time to B, thereby reducing the potential savings from using edge e. The joint decrease of these two factors ensures diminishing marginal returns from adding shortcut edges.

4. **Non-supermodularity of Maximum Hitting Time.** The maximum hitting-time function f is not supermodular due to the max operator, and it is not weakly-α-supermodular [65] for any $\alpha \geq 1$. A counterexample on a simple path graph establishing this claim is provided in paper E.

5.2.3 Algorithm for BMAH

A Classic Greedy Algorithm

Liberty and Sviridenko [65] show that for a non-increasing weakly-α-supermodular function f, if there exists an η-approximation algorithm for minimizing f, then the greedy algorithm that runs for $\lceil \alpha k \ln(\eta/\epsilon) \rceil$ iterations generates a $(1+\epsilon)$-approximation solution. We can verify that BMAH meets the requirements of this result: (1) As stated in Section 2.2.2, since BMAH is supermodular, it is weakly-α-supermodular with $\alpha = 1$. (2) The empty solution $g(\emptyset)$ provides an n^3-approximation of the BMAH problem, because hitting times in connected graphs are bounded by n^3 [71], and the optimal hitting time is at least one. Thus $\eta = n^3$. Taking $\alpha = 1$ and $\eta = n^3$ yields the following lemma:

Lemma 2. *The Greedy strategy that iteratively selects the edge yielding the largest marginal reduction in g returns a $(1 + \epsilon)$-approximation to an optimal solution of* BMAH *(that adds at most k edges) while adding at most $\lceil k \ln(n^3/\epsilon) \rceil$ edges.*

The proof leverages a classical result on minimizing supermodular functions under cardinality constraints. Exploiting the supermodularity of g, the greedy algorithm reduces the gap to the optimal solution by a multiplicative factor of approximately

$(1 - 1/k)$ at each iteration. After $\tau = \lceil k \ln(n^3/\epsilon) \rceil$ iterations, this yields a $(1+\epsilon)$-approximation, where the n^3 term accounts for the initial approximation ratio between the empty solution and any optimal solution ($g(F^*) \geq 1$).

Accelerating Hitting Time's Computation

The main challenge in applying the greedy algorithm is the computational cost of evaluating g, which requires either solving a linear system with $O(n)$ variables or computing the fundamental matrix $(\mathbf{I} - \mathbf{Q})^{-1}$ of an absorbing Markov chain [98], where $\mathbf{Q}$ is the transition matrix between red nodes [98]. Since evaluating g at any iteration is equally expensive, we focus on $g(\emptyset)$ as a representative case and show below intuitively how to approximate it within a factor of $(1 \pm \epsilon)$ efficiently. The approach decomposes $g(\emptyset)$ as an infinite matrix power series and estimates it through truncation and simulation.

The average hitting time can be expressed as $g(\emptyset) = \frac{1}{r}\mathbf{e}_r^T(\mathbf{I}_r - \mathbf{Q})^{-1}\mathbf{e}_r = \frac{1}{r}\mathbf{e}_r^T \sum_{i=0}^{+\infty} \mathbf{Q}^i \mathbf{e}_r$ [99], where $r = |R|$ and $\mathbf{Q}$ is the transition matrix between red nodes obtained by deleting the rows and columns of corresponding blue nodes in the random walk transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$. This infinite series naturally splits into two parts: $p_1 = \frac{1}{r}\mathbf{e}_r^T \sum_{i=0}^{\ell} \mathbf{Q}^i \mathbf{e}_r$ representing the first ℓ terms [99, 100], and $p_2 = \frac{1}{r}\mathbf{e}_r^T \sum_{i=\ell+1}^{+\infty} \mathbf{Q}^i \mathbf{e}_r$ representing the tail.

The truncated sum p_1 admits a probabilistic interpretation: each entry of $\sum_{i=0}^{\ell} \mathbf{Q}^i \mathbf{e}_r$ represents the expected number of times a bounded random walk of length ℓ visits each red node before absorption by a blue node. We estimate p_1 by simulating multiple bounded walks from each red node and computing empirical averages. This estimation approach is analyzed by Haddadan et al. [53, Lemma 4.3], whose proof uses Hoeffding's bound [101] and the union bound to show that with sufficient trials, the empirical estimate concentrates around the true value with high probability.

The tail p_2 captures contributions from walks exceeding ℓ steps. Since $\mathbf{Q}$ is a substochastic matrix for an absorbing Markov chain, its spectral radius satisfies $\lambda < 1$, implying that $\|\mathbf{Q}^i\|$ decays exponentially as λ^i. By choosing ℓ sufficiently large based on the spectral radius and desired accuracy, we can bound $p_2 \leq \epsilon g(\emptyset)$, rendering the truncation error negligible.

Combining the estimation of p_1 and the bound on p_2, we show that $\hat{g} = \frac{1}{r}\mathbf{e}_r^T \hat{h}$ satisfies $(1-\epsilon)g(\emptyset) \leq \hat{g} \leq (1+\epsilon)g(\emptyset)$ with high probability, where $\hat{h}$ is the empirical average from simulated walks of sufficiently large length ℓ. We omit the technical details and provide the formal statement in Lemma 3.

Lemma 3. *Given an undirected connected graph $G = (V, E)$ with a valid bipartition $V = \{R, B\}$, we can find an estimate $\hat{g}(\emptyset) \in (1 \pm \epsilon)g(\emptyset)$ with probability at least $1 - 1/n^c$ (for some $c \geq 1$) in time*

$$O\left(\frac{cn \ln(n) \log^3(d_R/\epsilon(1-\lambda))}{\epsilon^2 \log^3(1/\lambda)}\right),$$

where d_R is the average degree of red nodes, and λ is the spectral radius of $\mathbf{Q} = \mathbf{D}_R^{-1}\mathbf{A}_R$, with $\mathbf{A}_R$ being the adjacency matrix of the induced subgraph $G[R]$ and $\mathbf{D}_R$ the corresponding degree diagonal matrix.

Accelerating the Greedy Algorithm

Lemma 3 states that we can find good estimates $\hat{g}(\cdot) \in (1 \pm \epsilon)g(\cdot)$ relatively quickly and with high probability. The following theorem shows that using this approximation $\hat{g}$ of g in a greedy algorithm still yields a constant-factor approximation for the BMAH problem.

Theorem 7. *The classic greedy algorithm that iteratively selects at most $\tau = \lceil 2k \ln(\frac{n^3}{\epsilon}) \rceil$ edges maximizing the marginal reduction of the function $\hat{g}$ returns a set of shortcut edges F_τ satisfying*

$$g(F_\tau) \leq (2 + \epsilon)\mathit{OPT}, \tag{3}$$

where $\mathit{OPT} = g(F^*)$ and F^* is an optimal solution to BMAH *with at most k edges.*

The proof follows the classic greedy analysis for minimizing monotone supermodular functions [25]. When using the exact function g in the greedy algorithm, each greedy selection reduces at least $1/k$ of the gap to the optimal value OPT:

$$g(F_{i+1}) \leq \left(1 - \frac{1}{k}\right) g(F_i) + \frac{1}{k}\text{OPT}.$$

When using the estimate $\hat{g} \in (1 \pm \epsilon)g$ for greedy selection, we introduce a constant factor of $\frac{1+\epsilon}{1-\epsilon}$ in reducing this gap:

$$g(F_{i+1}) \leq \frac{1+\epsilon}{1-\epsilon}\left[\left(1 - \frac{1}{k}\right) g(F_i) + \frac{1}{k}\text{OPT}\right].$$

This is a first-order affine recurrence of the form $g(F_{i+1}) \leq ag(F_i) + b$, which has the closed-form bound $g(F_\tau) \leq a^\tau g(\emptyset) + \frac{1-a^\tau}{1-a}b$ [102]. By carefully choosing τ, we ensure the transient term $a^\tau g(\emptyset)$ is bounded by $\epsilon \cdot$ OPT. Regardless of the choice of τ, the steady-state term $\frac{1-a^\tau}{1-a}b$ is bounded by 2OPT. The sum of these two terms yields the desired $(2 + \epsilon)$-approximation guarantee.

5.2.4 Algorithm for BMMH

The BMMH problem is not supermodular, so classic greedy algorithms do not yield constant-factor approximations. Instead, we propose two algorithms that approximate BMMH indirectly by reducing it to related problems.

Relating the Average and the Maximum Hitting Times

We start with the following theorem that bounds the maximum hitting time f via average hitting time g:

Theorem 8. *Given an undirected connected graph G with a valid bipartition $V = \{R, B\}$. Let $F \subseteq (R \times B) \setminus E$ be a set of non-edges. Then,*

$$g(F) \leq f(F) \leq 2|R|^{3/4} g(F), \tag{6}$$

and thus, an α-approximation for BMAH is a $(2|R|^{3/4}\alpha)$-approximation for BMMH.

Intuition Behind the Bound. The key observation is that vertices with very large hitting times must be rare. Let S consist of vertices whose hitting time to the blue set is small (at most $\alpha g(\emptyset)$), and let S^c denote the exceptional vertices with large hitting times. Since the average hitting time from red to blue equals $g(\emptyset)$, the exceptional set satisfies $|S^c| \leq |R|/\alpha$.

To analyze the exceptional set's contribution, we contract all vertices in S into a single supernode. This contraction can only increase the time to reach S from S^c, providing a worst-case estimate. The contracted graph has only $|S^c| + 1$ vertices, allowing us to apply general hitting-time bounds of order $|S^c|^3$ [73, 103].

From any red vertex in S^c, the random walk has two stages. First, it reaches S from within S^c. Second, it proceeds from S to blue. By the strong Markov property, the expected total duration is at most the sum of these two stages. The first stage is bounded by $|S^c|^3$, while the second is at most $\alpha g(\emptyset)$ by definition of S.

This bound, $|S^c|^3 + \alpha g(\emptyset)$, reflects a tradeoff in choosing α. Smaller α reduces the second-stage cost but enlarges S^c, while larger α has the opposite effect. Optimizing this bound yields $\alpha = |R|^{3/4}$, showing that the maximum hitting time exceeds the average by at most a factor of $|R|^{3/4}$.

An Asymmetric k-Center Approach

Hitting times are inherently asymmetric. In general, $H_G(u, v) \neq H_G(v, u)$ [71]. This asymmetry suggests a connection to the asymmetric k-center problem [104], where choosing centers corresponds to selecting which red nodes to connect to blue nodes with shortcuts. We first provide the result, then explain the intuition behind the analysis.

Theorem 9. *Given a β-approximation algorithm for the asymmetric k-center problem, there is an $O(\beta d_m)$-approximation for BMMH, where d_m is the maximum degree of the chosen vertices.*

This approach constructs an asymmetric k-center problem instance with $|R| + 1$ nodes, where all blue nodes are collapsed into a single representative point, and distances between nodes correspond to hitting times in the BMMH instance and form a quasi-metric space. The asymmetric k-center objective of minimizing the maximum distance from any point to its nearest center directly mirrors the BMMH objective of

minimizing the maximum hitting time. The key technical result establishes that the optimal value of k-center on this space lower bounds the optimal value of BMMH. This enables approximation analysis as follows. Let C^* denote the optimal k-center value. A β-approximation algorithm for k-center guarantees that any red node reaches a chosen center within expected time βC^*. From each center, there exists a shortcut edge to the blue nodes. When the random walk reaches a center of degree d, it traverses the shortcut with probability at least $\frac{1}{d+1}$, reaching the blue nodes in one additional step. Analyzing the expected hitting time using this two-phase process yields an $\mathcal{O}(\beta d_m)$-approximation for BMMH, where d_m is the maximum degree among the chosen centers.

Chapter 6

Conclusion and Future Work

This thesis investigates fairness and diversity across two complementary domains: recommender systems and social network analysis. We develop algorithms for improving result diversity through pairwise distance maximization and topic-coverage maximization (papers A and B), for enhancing content exposure fairness via ranking adjustments and graph modifications (papers D and E), and for identifying densely connected communities with diverse relationship types (paper C). All problems are formulated as mathematical optimization problems, and the proposed algorithms come with provable performance guarantees or convergence properties.

From an application perspective, our methods correspond to (1) recommendation algorithms that carefully select, rank, or augment recommendation results on real-world platforms such as news aggregators (e.g., Google News), video platforms (e.g., YouTube and TikTok) and (2) network analysis algorithms capable of detecting fraudulent behaviors or identifying meaningful substructures, such as protein–protein interaction modules, which are valuable for downstream tasks including drug discovery and design. Together, these contributions provide a unified algorithmic framework for improving equity and diversity in large-scale networked systems.

6.1 Limitations and Future Work

We discuss limitations and future directions at both the conceptual level, spanning the thesis as a whole, and the technical level, specific to individual papers.

6.1.1 Broad Limitations and Directions

Definitions of Diversity and Fairness. Our diversity measures assume predefined topic taxonomies (paper B) or edge-color categories (paper C), and our fairness measures are group-based (papers D and E). These are reasonable assumptions, but alternatives exist. Individual fairness, intersectional fairness, and procedural fairness each capture aspects that our definitions do not. Studying alternative definitions of

fairness and diversity within the context of our problems is an interesting direction for future work.

Beyond exploring more definitions of fairness and diversity, there is also concern with whether the definitions we employ fully capture the intended concepts. In our recommender system studies, we quantified diversity through topic coverage and sequential sum diversity. While these metrics are mathematically tractable, they may not fully capture a user's subjective experience of diversity, which can be influenced by cognitive biases or aesthetic variety. Similarly, our fairness measures (PageRank and hitting-time fairness) assume that equity is achieved by rebalancing influence or accessibility across disjoint groups based on sensitive attributes. However, these technical metrics may oversimplify the social reality of fairness by focusing on mathematical parity without necessarily capturing broader qualitative impacts or accommodating the diverse ethical standards that real-world applications require.

Trade-offs with Other Objectives. Promoting diversity or fairness often creates a trade-off where gains in long-term satisfaction or content exposure come at the expense of short-term engagement metrics like click-through rate or subgraph density [19, 105]. While we currently address these trade-offs through constrained optimization, developing explicit multi-objective frameworks such as Pareto-optimal formulations would provide practitioners with more robust tools to navigate this engagement-diversity trade-off.

Data Privacy. Our models often assume access to granular data, such as user-item interaction probabilities (p_i) or sensitive demographic attributes. In real-world applications, this data is subject to strict privacy regulations (e.g., GDPR [106]). A critical future direction is the integration of Differential Privacy into our algorithms. Can we maximize diversity or enforce fairness constraints while ensuring that individual user preferences or sensitive attributes remain protected? Furthermore, the use of automated "graph interventions" to rewire networks raises ethical questions regarding platform transparency and user autonomy that merit deeper exploration.

6.1.2 Technical Open Problems

1. **Advanced Modeling of Continuation Probabilities (Papers A and B).** We assume item click probabilities p_i are given as input. Integrating learned prediction models, or studying the online learning variant where p_i must be estimated concurrently with diversity optimization, would connect our work to the bandit and explore–exploit literature [107, 108, 109, 110].

2. **Diverse Optimization Objectives (Papers A and B)** Topic coverage is the sole optimization objective. Can we integrate additional metrics (e.g., completion rate, click-through rate) within a unified formulation, potentially via constrained submodular optimization?

3. **Explore Different Diversity Measure (Paper A)** Apart from the sequential sum and sequential coverage diversity. Can we design approximation algorithms for other definitions of diversity, such as sequential Max-Min diversity?

4. **Dense Graph Generalization (Paper C)** Our algorithms work only for everywhere sparse graphs. Improving approximation results for dense graphs or establishing stronger lower bounds remains an open challenge.

5. **Dense Graph Generalization (Paper C)** We study the "at least $\mathbf{h}$" colored-edge DSP. Can we develop algorithms for the exact and "at most $\mathbf{h}$" variants? Can we still leverage the reduction to at least k node DSP to solve these variants?

6. **Provable Guarantees for Non-Convex Problems (Paper C)** The non-convex formulation currently requires machine-learning-based solutions. Can we reformulate the problem to enable approximation algorithms with provable guarantees, e.g., via convex relaxation or combinatorial techniques?

7. **Improved Hitting-Time algorithms (Paper E)** For fairness interventions based on hitting time, future research should aim to obtain improved approximation algorithms that avoid bicriteria approaches and tighten the gap between algorithmic guarantees and known lower bounds.

References

[1] Jessica T Feezell, John K Wagner, and Meredith Conroy. Exploring the effects of algorithm-driven news sources on political behavior and polarization. *Computers in human behavior*, 116:106626, 2021.

[2] Sarah J Jackson, Meredith D Clark, Deen Freelon, and Lori Lopez. How black twitter and other social media communities interact with mainstream news. *Knight Foundation*, 2018.

[3] Stefan Kirchner and Elke Schüßler. The organization of digital marketplaces: Unmasking the role of internet platforms in the sharing economy. *Organization outside organization*, pages 131–154, 2019.

[4] Amy Ross Arguedas, Craig Robertson, Richard Fletcher, and Rasmus Nielsen. Echo chambers, filter bubbles, and polarisation: A literature review. Technical report, Reuters Institute for the Study of Journalism, University of Oxford, 2022. URL `https://ora.ox.ac.uk/objects/uuid:6e357e97-7b16-450a-a827-a92c93729a08/files/rzw12z639q`. Accessed: 8 February 2026.

[5] Frederic Gerdon, Ruben L Bach, Christoph Kern, and Frauke Kreuter. Social impacts of algorithmic decision-making: A research agenda for the social sciences. *Big Data & Society*, 9(1):20539517221089305, 2022.

[6] Manish Raghavan. *The societal impacts of algorithmic decision-making*. Cornell University, 2021.

[7] Michal Wiktor Krawczyk. A model of procedural and distributive fairness. *Theory and decision*, 70(1):111–128, 2011.

[8] Arild Underdal and Taoyuan Wei. Distributive fairness: A mutual recognition approach. *Environmental science & policy*, 51:35–44, 2015.

[9] Kees Van den Bos, Henk AM Wilke, and E Allan Lind. When do we need procedural fairness? The role of trust in authority. *Journal of Personality and social Psychology*, 75(6):1449, 1998.

[10] Marie Christin Decker, Laila Wegner, and Carmen Leicht-Scholten. Procedural fairness in algorithmic decision-making: the role of public engagement. *Ethics and Information Technology*, 27(1):1, 2025.

[11] Will Fleisher. What's fair about individual fairness? In *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*, pages 480–490, 2021.

[12] Reuben Binns. On the apparent conflict between individual and group fairness. In *Proceedings of the 2020 conference on fairness, accountability, and transparency*, pages 514–524, 2020.

[13] Tim Räz. Group fairness: Independence revisited. In *Proceedings of the 2021 ACM conference on fairness, accountability, and transparency*, pages 129–137, 2021.

[14] Allan Borodin, Hyun Chul Lee, and Yuli Ye. Max-sum diversification, monotone submodular functions and dynamic updates. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI symposium on Principles of Database Systems*, pages 155–166, 2012.

[15] Sekharipuram S Ravi, Daniel J Rosenkrantz, and Giri Kumar Tayi. Heuristic and special case algorithms for dispersion problems. *Operations research*, 42 (2):299–310, 1994.

[16] Sreenivas Gollapudi and Aneesh Sharma. An axiomatic approach for result diversification. In *Proceedings of the 18th international conference on World wide web*, pages 381–390, 2009.

[17] Shameem A Puthiya Parambath, Nicolas Usunier, and Yves Grandvalet. A coverage-based approach to recommendation diversity on similarity graph. In *Proceedings of the 10th ACM Conference on Recommender Systems*, pages 15–22, 2016.

[18] Azin Ashkan, Branislav Kveton, Shlomo Berkovsky, and Zheng Wen. Optimal greedy diversity for recommendation. In *IJCAI*, volume 15, pages 1742–1748, 2015.

[19] Yuying Zhao, Yu Wang, Yunchao Liu, Xueqi Cheng, Charu C Aggarwal, and Tyler Derr. Fairness and diversity in recommender systems: a survey. *ACM Transactions on Intelligent Systems and Technology*, 16(1):1–28, 2025.

[20] Theodor Cimpeanu, Alessandro Di Stefano, Cedric Perret, and The Anh Han. Social diversity reduces the complexity and cost of fostering fairness. *Chaos, Solitons & Fractals*, 167:113051, 2023.

[21] Michael J Kuby. Programming models for facility dispersion: The p-dispersion and maxisum dispersion problems. *Geographical Analysis*, 19(4):315–329, 1987.

[22] Erhan Erkut. The discrete p-dispersion problem. *European Journal of Operational Research*, 46(1):48–60, 1990.

[23] P. Hansen and I. D. Moon. Dispersing facilities on a network. In *TIMS/ORSA Joint National Meeting*, Washington, DC, 1988. Presentation.

[24] Alfonso Cevallos, Friedrich Eisenbrand, and Rico Zenklusen. Local search for max-sum diversification. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 130–142. SIAM, 2017.

[25] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. An analysis of approximations for maximizing submodular set functions–I. *Mathematical programming*, 14(1):265–294, 1978.

[26] Jaime Carbonell and Jade Goldstein. The use of mmr, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*, pages 335–336, 1998.

[27] Erica Coppolillo, Giuseppe Manco, and Aristides Gionis. Relevance meets diversity: A user-centric framework for knowledge exploration through recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 490–501, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671949.

[28] Uriel Feige. A threshold of ln n for approximating set cover. *Journal of the ACM (JACM)*, 45(4):634–652, 1998.

[29] George L Nemhauser and Laurence A Wolsey. Best algorithms for approximating the maximum of a submodular set function. *Mathematics of operations research*, 3(3):177–188, 1978.

[30] Marshall L Fisher, George L Nemhauser, and Laurence A Wolsey. An analysis of approximations for maximizing submodular set functions–II. In *Polyhedral Combinatorics: Dedicated to the memory of DR Fulkerson*, pages 73–87. Springer, 2009.

[31] Gruia Calinescu, Chandra Chekuri, Martin Pal, and Jan Vondrák. Maximizing a monotone submodular function subject to a matroid constraint. *SIAM Journal on Computing*, 40(6):1740–1766, 2011.

[32] Jan Vondrák. Optimal approximation for the submodular welfare problem in the value oracle model. In *Proceedings of the fortieth annual ACM symposium on Theory of computing*, pages 67–74, 2008.

[33] Yuval Filmus and Justin Ward. A tight combinatorial algorithm for submodular maximization subject to a matroid constraint. In *2012 IEEE 53rd Annual Symposium on Foundations of Computer Science*, pages 659–668. IEEE, 2012.

[34] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, and Andreas Krause. Streaming submodular maximization: Massive data summarization on the fly. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 671–680, 2014.

[35] Ehsan Kazemi, Marko Mitrovic, Morteza Zadimoghaddam, Silvio Lattanzi, and Amin Karbasi. Submodular streaming in all its glory: Tight approximation, minimum memory and low adaptive complexity. In *International Conference on Machine Learning*, pages 3311–3320. PMLR, 2019.

[36] Ashkan Norouzi-Fard, Jakub Tarnawski, Slobodan Mitrovic, Amir Zandieh, Aidasadat Mousavifar, and Ola Svensson. Beyond 1/2-approximation for submodular maximization on massive data streams. In *International Conference on Machine Learning*, pages 3829–3838. PMLR, 2018.

[37] Shipra Agrawal, Mohammad Shadravan, and Cliff Stein. Submodular secretary problem with shortlists. *arXiv preprint arXiv:1809.05082*, 2018.

[38] Amit Chakrabarti and Sagar Kale. Submodular maximization meets streaming: matchings, matroids, and more. *Mathematical Programming*, 154(1):225–247, 2015.

[39] Chandra Chekuri, Shalmoli Gupta, and Kent Quanrud. Streaming algorithms for submodular function maximization. In *International Colloquium on Automata, Languages, and Programming*, pages 318–330. Springer, 2015.

[40] Moran Feldman, Amin Karbasi, and Ehsan Kazemi. Do less, get more: Streaming submodular maximization with subsampling. *Advances in Neural Information Processing Systems*, 31, 2018.

[41] Andrew V Goldberg. *Finding a maximum density subgraph*. University of California Berkeley, 1984.

[42] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In *International workshop on approximation algorithms for combinatorial optimization*, pages 84–95. Springer, 2000.

[43] Samir Khuller and Barna Saha. On finding dense subgraphs. In *International colloquium on automata, languages, and programming*, pages 597–608. Springer, 2009.

[44] Reid Andersen and Kumar Chellapilla. Finding dense subgraphs with size bounds. In *International workshop on algorithms and models for the web-graph*, pages 25–37. Springer, 2009.

[45] Aris Anagnostopoulos, Luca Becchetti, Adriano Fazzone, Cristina Menghini, and Chris Schwiegelshohn. Spectral relaxations and fair densest subgraphs. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, CIKM '20, pages 35–44, New York, NY, USA, 2020. Association for Computing Machinery. doi: 10.1145/3340531.3412036.

[46] Atsushi Miyauchi, Tianyi Chen, Konstantinos Sotiropoulos, and Charalampos E. Tsourakakis. Densest diverse subgraphs: How to plan a successful cocktail party with diversity. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pages 1710–1721, New York, NY, USA, August 2023. Association for Computing Machinery. doi: 10.1145/3580305.3599483.

[47] Sotiris Tsioutsiouliklis, Evaggelia Pitoura, Panayiotis Tsaparas, Ilias Kleftakis, and Nikos Mamoulis. Fairness-aware pagerank. In *Proceedings of the Web Conference 2021*, pages 3815–3826, 2021.

[48] Sotiris Tsioutsiouliklis, Evaggelia Pitoura, Konstantinos Semertzidis, and Panayiotis Tsaparas. Link recommendations for pagerank fairness. In *Proceedings of the ACM Web Conference 2022*, pages 3541–3551, 2022.

[49] Tahleen Rahman, Bartlomiej Surma, Michael Backes, and Yang Zhang. Fairwalk: Towards fair graph embedding. In *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI-19*, pages 3289–3295. International Joint Conferences on Artificial Intelligence Organization, 7 2019. doi: 10.24963/ijcai.2019/456.

[50] Ahmad Khajehnejad, Moein Khajehnejad, Mahmoudreza Babaei, Krishna P Gummadi, Adrian Weller, and Baharan Mirzasoleiman. Crosswalk: Fairness-enhanced node representation learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 11963–11970, 2022.

[51] Francesco Fabbri, Yanhao Wang, Francesco Bonchi, Carlos Castillo, and Michael Mathioudakis. Rewiring what-to-watch-next recommendations to reduce radicalization pathways. In *Proceedings of the ACM Web Conference 2022*, pages 2719–2728, 2022.

[52] Corinna Coupette, Stefan Neumann, and Aristides Gionis. Reducing exposure to harmful content via graph rewiring. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 323–334, 2023.

[53] Shahrzad Haddadan, Cristina Menghini, Matteo Riondato, and Eli Upfal. Repbublik: Reducing polarized bubble radius with link insertions. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*, pages 139–147, 2021.

[54] Liwang Zhu, Qi Bao, and Zhongzhi Zhang. Minimizing polarization and disagreement in social networks via link recommendation. *Advances in Neural Information Processing Systems*, 34:2072–2084, 2021.

[55] Vijay V Vazirani. *Approximation algorithms*, volume 1. Springer, 2001.

[56] David P Williamson and David B Shmoys. *The design of approximation algorithms*. Cambridge university press, 2011.

[57] Dorit S Hochba. Approximation algorithms for NP-hard problems. *ACM Sigact News*, 28(2):40–52, 1997.

[58] Daniel D Sleator and Robert E Tarjan. Amortized efficiency of list update and paging rules. *Communications of the ACM*, 28(2):202–208, 1985.

[59] Allan Borodin and Ran El-Yaniv. *Online computation and competitive analysis*. cambridge university press, 2005.

[60] Giorgio Ausiello, Pierluigi Crescenzi, Giorgio Gambosi, Viggo Kann, Alberto Marchetti-Spaccamela, and Marco Protasi. *Complexity and approximation: Combinatorial optimization problems and their approximability properties*. Springer Science & Business Media, 2012.

[61] Satoru Fujishige. *Submodular functions and optimization*, volume 58. Elsevier, 2005.

[62] László Lovász. Submodular functions and convexity. In *Mathematical Programming The State of the Art: Bonn 1982*, pages 235–257. Springer, 1983.

[63] Alexander Schrijver et al. *Combinatorial optimization: polyhedra and efficiency*, volume 24. Springer, 2003.

[64] Donald M Topkis. *Supermodularity and complementarity*. Princeton university press, 1998.

[65] Edo Liberty and Maxim Sviridenko. Greedy minimization of weakly supermodular set functions. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2017)*, pages 19:1–19:11. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2017.

[66] Krause Andreas and Golovin Daniel. Submodular function maximization. *Tractability*, pages 71–104, 2014. doi: 10.1017/cbo9781139177801.004.

[67] James G Oxley. *Matroid theory*, volume 3. Oxford University Press, USA, 2006.

[68] Dominic JA Welsh. *Matroid theory*. Courier Corporation, 2010.

[69] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems*, 30(1-7):107–117, 1998.

[70] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. The pagerank citation ranking: Bringing order to the web. Technical report, Stanford infolab, 1999.

[71] László Lovász. Random walks on graphs. *Combinatorics, Paul erdos is eighty*, 2(1-46):4, 1993.

[72] Thomas Sauerwald and He Sun. Spectral graph theory and applications, lecture 7: Hitting time and cover time of random walks. Lecture notes, 2011. URL `https://resources.mpi-inf.mpg.de/departments/d1/teaching/ws11/SGT/Lecture7.pdf`. WS 2011/2012, Max Planck Institute for Informatics.

[73] Eugene L Lawler. *Combinatorial optimization: networks and matroids*. Courier Corporation, 2001.

[74] Yuichi Asahiro, Kazuo Iwama, Hisao Tamaki, and Takeshi Tokuyama. Greedily finding a dense subgraph. *Journal of Algorithms*, 34(2):203–221, 2000.

[75] Chandra Chekuri, Kent Quanrud, and Manuel R Torres. Densest subgraph: Supermodularity, iterative peeling, and flow. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1531–1555. SIAM, 2022.

[76] Digvijay Boob, Yu Gao, Richard Peng, Saurabh Sawlani, Charalampos Tsourakakis, Di Wang, and Junxing Wang. Flowless: Extracting densest subgraphs without flow computations. In *Proceedings of The Web Conference 2020*, pages 573–583, 2020.

[77] Subhash Khot. Ruling out PTAS for graph min-bisection, dense k-subgraph, and bipartite clique. *SIAM Journal on Computing*, 36(4):1025–1071, 2006.

[78] Aditya Bhaskara, Moses Charikar, Eden Chlamtac, Uriel Feige, and Aravindan Vijayaraghavan. Detecting high log-densities: an O($n^{1/4}$) approximation for densest k-subgraph. In *Proceedings of the forty-second ACM symposium on Theory of computing*, pages 201–210, 2010.

[79] Mohamed Medhat Gaber, Arkady Zaslavsky, and Shonali Krishnaswamy. Mining data streams: a review. *ACM Sigmod Record*, 34(2):18–26, 2005.

[80] Dmitri Burago, Yuri Burago, Sergei Ivanov, et al. *A course in metric geometry*, volume 33. American Mathematical Society Providence, 2001.

[81] Teofilo F Gonzalez. Clustering to minimize the maximum intercluster distance. *Theoretical computer science*, 38:293–306, 1985.

[82] Michael R Garey, David S Johnson, et al. A guide to the theory of NP-Completeness. *Computers and intractability*, pages 37–79, 1990.

[83] Wen-Lian Hsu and George L Nemhauser. Easy and hard bottleneck location problems. *Discrete Applied Mathematics*, 1(3):209–215, 1979.

[84] Ján Plesník. On the computational complexity of centers locating in a graph. *Aplikace matematiky*, 25(6):445–452, 1980.

[85] Martin E Dyer and Alan M Frieze. A simple heuristic for the p-centre problem. *Operations Research Letters*, 3(6):285–288, 1985.

[86] Dorit S Hochbaum and David B Shmoys. A best possible heuristic for the k-center problem. *Mathematics of operations research*, 10(2):180–184, 1985.

[87] Wallace Alvin Wilson. On quasi-metric spaces. *American Journal of Mathematics*, 53(3):675–684, 1931.

[88] Rina Panigrahy and Sundar Vishwanathan. An O(log*n) approximation algorithm for the asymmetric p-center problem. *Journal of Algorithms*, 27(2): 259–268, 1998.

[89] Julia Chuzhoy, Sudipto Guha, Eran Halperin, Sanjeev Khanna, Guy Kortsarz, Robert Krauthgamer, and Joseph Naor. Asymmetric k-center is log*n-hard to approximate. *Journal of the ACM (JACM)*, 52(4):538–551, 2005.

[90] Aaron Archer. Two O(log*k)-approximation algorithms for the asymmetric k-center problem. In *International Conference on Integer Programming and Combinatorial Optimization*, pages 1–14. Springer, 2001.

[91] Jon Kleinberg, Emily Ryu, and Éva Tardos. Ordered submodularity and its applications to diversifying recommendations. *arXiv preprint arXiv:2203.00233*, 2022.

[92] Sotiris Tsioutsiouliklis, Evaggelia Pitoura, Panayiotis Tsaparas, Ilias Kleftakis, and Nikos Mamoulis. Fairness-Aware PageRank. In *Proceedings of the Web Conference*, pages 3815–3826, 2021.

[93] Christian Michelot. A finite algorithm for finding the projection of a point onto the canonical simplex of $\propto$ n. *Journal of Optimization Theory and Applications*, 50(1):195–200, 1986.

[94] Panos M Pardalos and Naina Kovoor. An algorithm for a singly constrained class of quadratic programs subject to upper and lower bounds. *Mathematical Programming*, 46(1):321–328, 1990.

[95] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. Efficient projections onto the ℓ_1-ball for learning in high dimensions. In *Proceedings of the 25th international conference on Machine learning*, pages 272–279, 2008.

[96] Laurent Condat. Fast projection onto the simplex and the ℓ_1 ball. *Mathematical Programming*, 158(1):575–585, 2016.

[97] Lukás Adam and Václav Mácha. Projections onto the canonical simplex with additional linear inequalities. *Optimization Methods and Software*, 37(2):451–479, 2022.

[98] JG Kemery and JL Snell. Finite markov chains: With a new appendix "generalization of a fundamental matrix", 1960.

[99] Pan Peng, Daniel Lopatta, Yuichi Yoshida, and Gramoz Goranci. Local algorithms for estimating effective resistance. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, pages 1329–1338, 2021.

[100] Alexandr Andoni, Robert Krauthgamer, and Yosef Pogrow. On solving linear systems in sublinear time. *arXiv preprint arXiv:1809.02995*, 2018.

[101] Wassily Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American statistical association*, 58(301):13–30, 1963.

[102] Ronald L Graham. *Concrete mathematics: a foundation for computer science*. Pearson Education India, 1994.

[103] Graham Brightwell and Peter Winkler. Maximum hitting time for random walks on graphs. *Random Structures & Algorithms*, 1(3):263–276, 1990.

[104] Oded Kariv and S Louis Hakimi. An algorithmic approach to network location problems. I: The p-centers*. *SIAM journal on applied mathematics*, 37(3): 513–538, 1979.

[105] Elvin Isufi, Matteo Pocchiari, and Alan Hanjalic. Accuracy-diversity trade-off in recommender systems via graph convolutions. *Information Processing & Management*, 58(2):102459, 2021.

[106] Paul Voigt and Axel Von dem Bussche. The EU general data protection regulation (GDPR). *A practical guide, 1st ed., Cham: Springer International Publishing*, 10(3152676):10–5555, 2017.

[107] Tor Lattimore and Csaba Szepesvári. *Bandit algorithms*. Cambridge University Press, 2020.

[108] Lihong Li, Wei Chu, John Langford, and Robert E Schapire. A contextual-bandit approach to personalized news article recommendation. In *Proceedings of the 19th international conference on World wide web*, pages 661–670, 2010.

[109] James McInerney, Benjamin Lacker, Samantha Hansen, Karl Higley, Hugues Bouchard, Alois Gruson, and Rishabh Mehrotra. Explore, exploit, and explain: personalizing explainable recommendations with bandits. In *Proceedings of the 12th ACM conference on recommender systems*, pages 31–39, 2018.

[110] Marko Balabanović. Exploring versus exploiting when learning user models for text recommendation. *User Modeling and User-Adapted Interaction*, 8(1): 71–102, 1998.

Part II

Included Papers

Paper A

Sequential Diversification with Provable Guarantees

Honglian Wang
KTH Royal Institute of Technology
Stockholm, Sweden
honglian@kth.se

Sijing Tu
KTH Royal Institute of Technology
Stockholm, Sweden
sijing@kth.se

Aristides Gionis
KTH Royal Institute of Technology
Stockholm, Sweden
argioni@kth.se

Abstract

Diversification is a useful tool for exploring large collections of information items. It has been used to reduce redundancy and cover multiple perspectives in information-search settings. Diversification finds applications in many different domains, including presenting search results of information-retrieval systems and selecting suggestions for recommender systems.

Interestingly, existing measures of diversity are defined over *sets* of items, rather than evaluating *sequences* of items. This design choice comes in contrast with commonly-used relevance measures, which are distinctly defined over sequences of items, taking into account the ranking of items. The importance of employing sequential measures is that information items are almost always presented in a sequential manner, and during their information-exploration activity users tend to prioritize items with higher ranking.

In this paper, we study the problem of *maximizing sequential diversity*. This is a new measure of *diversity*, which accounts for the *ranking* of the items, and incorporates *item relevance* and *user behavior*. The overarching framework can be instantiated with different diversity measures, and here we consider the measures of *sum diversity* and *coverage diversity*. The problem was recently proposed by Coppolillo et al. [11], where they introduce empirical methods that work well in practice. Our paper is a theoretical treatment of the problem: we establish the problem hardness and present algorithms with constant approximation guarantees for both diversity measures we consider. Experimentally, we demonstrate that our methods are competitive against strong baselines.

CCS Concepts

• **Information systems** → **Information retrieval**; • **Theory of computation** → **Approximation algorithms analysis**.

Keywords

Diversification, Ranking algorithms, Approximation algorithms

ACM Reference Format:
Honglian Wang, Sijing Tu, and Aristides Gionis. 2025. Sequential Diversification with Provable Guarantees. In *Proceedings of the Eighteenth ACM International Conference on Web Search and Data Mining (WSDM '25), March 10–14, 2025, Hannover, Germany*. ACM, New York, NY, USA, 19 pages. https://doi.org/10.1145/3701551.3703531

This work is licensed under a Creative Commons Attribution International 4.0 License.

WSDM '25, March 10–14, 2025, Hannover, Germany
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1329-3/25/03
https://doi.org/10.1145/3701551.3703531

1 Introduction

Diversification has been widely adopted in information retrieval and recommendation systems to facilitate information exploration, that is, to present users with content that is not only relevant, but also novel and diverse. Diversification benefits both users and service providers [26]: it enhances user experience by mitigating the filter-bubble effect and promoting a fair representation of perspectives, while helping service providers avoid over-exposure of popular items and cater to broader information needs.

Numerous diversification approaches have been proposed [26], including methods that aim to balance the tradeoff between *diversity* and *relevance* [5, 16]. However, most existing approaches have two shortcomings. First, they assume a fixed number of items to be selected, overlooking the fact that users may lose interest and terminate their information-seeking task early. Second, many approaches formulate the diversification task as a *set-selection problem*, rather than a *ranking problem*, thereby ignoring the importance of the ordering of selected items.

These shortcomings arise from failing to incorporate the users' engagement behavior. Users may exit due to tiredness or disinterest, making it challenging for the system to determine beforehand how many content items the users will examine. Additionally, the ranking of selected items is crucial, as users are more likely to examine items that are higher in the ranking. Note that many diversification methods employ greedy strategies [2, 5, 6, 10, 24], which implicitly provide a ranking of the items. However, such a ranking is not an integral part of the problem definition. It is not designed to meet any desirable properties and is merely a by-product of the algorithmic solution.

Our contributions. In this paper, we build on the setting of *sequential diversity*, introduced recently by Coppolillo et al. [11], aiming to address the above-mentioned shortcomings. The sequential-diversity setting defines a new diversity measure, which accounts for the *ranking* of items, and it incorporates *item relevance* together with *user behavior*. While the work of Coppolillo et al. [11] presents practical methods for this problem, our paper offers a rigorous analysis and algorithms with provable guarantees.

The sequential-diversity setting is formalized as follows. We consider that items are presented to users in a specific order and users examine them sequentially. We define the *continuation probability* of an item for a user as the probability that the user will *accept* the item and continue *examining* the next item in the sequence. We assume that continuation probabilities are related to the *item relevance* for a given user, and thus can be learned from observed data.

It follows that the model is *stochastic*, with the number of items examined being a random variable depending on the ranking of items and their continuation probabilities. The *sequential diversity* of an ordered sequence is defined as the *expected diversity* of the items accepted by the user, with the expectation taken over the

continuation probabilities. Our objective is to maximize sequential diversity by computing an optimal ranking of items, which we refer to as *sequential-diversity maximization*. This model effectively captures the interplay between relevance, diversity, and user engagement, as achieving high sequential diversity requires rankings that incorporate *both diverse and relevant items* to maintain user engagement with the platform.

The model needs to be instantiated with a diversity measure for item sets, and any standard measure can be used. In this paper, we consider two commonly-used diversity measures: *pair-wise sum diversity* [5] and *coverage diversity* [4, 24, 28]. For coverage diversity, we prove the problem is *ordered-submodular* [22], and thus, existing techniques can be applied.

For pair-wise sum diversity measure and its corresponding maximization problem (MaxSSD, Problem 1), we need to develop novel methods. We establish the computational hardness of the problem and present algorithms with constant-factor approximation guarantees. Our techniques leverage connections to an *ordered* variant of the *Hamiltonian path* problem (Section 5), which seeks to maximize the sum of edge weights in the Hamiltonian path, where the edge weights depend on their order in the path. Our algorithms are grounded on the observation that the sequential diversity score is largely determined by the top items in the ranking; thus, we prioritize optimizing the selection and positioning of these top items.

Depending on the values of the continuation probabilities, we introduce two distinct algorithms. When continuation probabilities are relatively small, e.g., constant values that are smaller than 1, we design an algorithm that focuses on selecting the top-τ items, where the value of τ controls the trade-off between runtime and approximation quality. Notably, when $\tau = 2$, our approach aligns with a simple greedy approach, and hence, we provide the approximation ratio for this greedy approach. In contrast, when continuation probabilities are arbitrarily close to 1, it's not just the first few items that affect the sequential diversity score; rather, more items ranked toward the front impact the score. To address this case, we propose a second algorithm using a greedy matching approach on the set of all items, and offers a constant approximation guarantee.

In our experiments, we introduce metrics for user satisfaction and engagement. We experimentally compare our methods against baselines and demonstrate their effectiveness in finding high-quality solutions. Due to space limitations, proofs of our theoretical results and a more extensive experimental evaluation are provided in the full version of this paper. [1]

2 Related work

Content diversification is commonly formulated as an optimization problem [2, 4–10, 16, 22, 24]. Two common diversity functions are coverage diversity and pairwise-sum diversity.

Agrawal et al. [2] introduce the concept of user coverage, aiming to recommend a relevant and diverse set while maximizing the number of users who encounter at least one relevant document. Ashkan et al. [4] and Puthiya et al. [24] explore taxonomy coverage, defining diversity as the number of topics or users covered by the selected set of items. These studies formulate their objective as monotone submodular functions and apply greedy strategies with provable guarantees.

For pairwise sum diversity, Carbonell and Goldstein [6] introduce the *marginal maximal relevance* (MMR), which is a function that combines the pairwise distance and the relevance of the selected items to generate a ranking via greedily selection. Borodin et al. [5] and Gollapudi and Sharma [16] investigate the notion of *max-sum diversity* (MSD) by combining sum of pairwise distance with a submodular relevance function. Subsequent works on the MSD problem apply convex-programming [7] and local-search [8, 9] techniques to achieve better approximation ratios. Chen et al. [10] and Gan et al. [13] use a *determinantal point process* (DPP) model by maintaining a kernel matrix whose off-diagonal entries measure the similarity between items. Chen et al. [10] use the maximum a posterior (MAP) inference for DPP and propose a greedy MAP inference algorithm for efficient computation.

The main limitation of the aforementioned studies is the assumption that users consider all results equally, without accounting for the order of items Technically, although a greedy heuristic can produce an ordered sequence [2, 5, 6, 10, 24], the objective value does not depend on item ranking.

Some studies incorporate the ranking of items into their models. Ashkan et al. [3, 4] propose the *diversity-weighted utility maximization* (DUM) approach which ranks items solely based on their utilities (relevance scores, in our setting). Unlike DUM, our method integrates relevance and diversity directly into the ranking. Ieong et al. [20] and Tang et al. [25] address advertisement allocation in a sequential setting similar to ours, where users sequentially view content and exit with fixed probabilities. However, they solve a distinct set of problems.

The most closely related works to our problem are by Coppolillo et al. [11] and Kleinberg et al. [22]. Coppolillo et al. [11] introduce a user-behavior model that simulates user interactions with recommendation systems and propose the EXPLORE algorithm to maximize their proposed diversity measures. Their framework allows users to interact with multiple items at each time step and accept one of them. Moreover, they do not provide any provable guarantees of their approaches. Kleinberg et al. [22] assumes that users' patience decays as they progress through a result list. They propose an *ordered-submodular* coverage function and prove that a greedy algorithm achieves a $\frac{1}{2}$-approximation; this result extends to our problem when the coverage function is selected as the diversity measure. When pairwise distance is used as the diversity measure, we develop novel methods to approach our problem.

Recent works have applied deep-learning techniques to content diversification [1, 19, 27, 29], typically incorporating diversification as part of a broader optimization task. In contrast, our focus is on defining sequential diversity as a standalone optimization problem and developing methods with provable guarantees. Notably, our framework can leverage deep-learning approaches to learn continuation probabilities. Consequently, we view deep-learning approaches as complementary to our framework.

3 Preliminaries

We consider a system where users interact with distinct content items, such as videos or articles, presented in an ordered sequence.

[1] https://arxiv.org/abs/2412.10944

Let $U = \{1, \ldots, n\}$ represent the set of items. For a given user u, each item $i \in U$ has an associated *continuation probability* p_i, representing the probability that, upon examining item i, the user u *accepts* this item and continues examining subsequent items. Conversely, $1 - p_i$ is the probability of terminating the session and quit the system after examining item i.

We assume that p_i is determined by the *relevance* of item i for user u and is provided as input to the problem. While estimating p_i is an interesting task, it lies outside the scope of this work. In our empirical evaluation, we explore simple methods that map relevance scores to continuation probabilities.

The user examines items in U sequentially, following an order determined by the permutation $\pi : U \to U$, where $\pi(i)$ denotes the i-th item in the sequence. Let $O_{(\pi)} = (\pi(i))_{i=1}^{n}$ represent the ordered sequence defined by π; for simplicity, we omit the subscript π and write $O = O_{(\pi)}$. We denote by $O_k = (\pi(i))_{i=1}^{k}$ the k-prefix of O, i.e., the subsequence of O consisting of the first k items. Equivalently, we write $O_k \sqsubseteq O$.

After examining item $\pi(i)$, the user either *accepts* it and proceeds to examine the next item with probability $p_{\pi(i)}$, or rejects it and quits the system immediately.

Given an ordered sequence of items O, let us denote by $A \sqsubseteq O$ the ordered sequence of items the user examines and *accepts* before quitting. Notice that A is a random variable. The probability of the user accepting exactly the items of O_k is $\Pr(A = O_k)$. Let $\mathcal{D}(\cdot)$ denote a *diversity function* measuring the diversity of the accepted items. As we will discuss shortly, our goal is to maximize the expected diversity score $\mathcal{D}(A)$.

In our paper, an algorithm ALG achieves an α-approximation ($\alpha \leq 1$) if, for any instance I, it guarantees $\mathrm{ALG}(I) \geq \alpha\, \mathrm{OPT}(I)$.

3.1 Problem definition

We first introduce the *sequential-diversity* objective and then formalize the *maximization* problem we consider.

Definition 1 (Sequential diversity ($\mathcal{S}$))**.** *Let $U = \{1, \ldots, n\}$ be a finite set of n distinct items, and $p_1, \ldots, p_n$ be continuation probabilities assigned to each item. Let $O = (\pi(i))_{i=1}^{n}$ be an ordered sequence of U according to a permutation π. Let $\mathcal{D}(A)$ be a diversity function of the items in A. The* sequential diversity *of O, denoted by $\mathcal{S}(O)$, is defined as the expectation of the diversity of the items that the user accepts before quitting, i.e., $\mathcal{S}(O) = \mathbb{E}_{A \sqsubseteq O}[\mathcal{D}(A)]$.*

The expectation in Definition 1 is taken over the probability that the user *accepts* exactly $O_k \sqsubseteq O$, where $k = 0, 1, \ldots, n$.

We will work with two commonly-used diversity functions. The first is the *pair-wise sum diversity*: for a distance function $d \colon U \times U \to \mathbb{R}_{\geq 0}$, the diversity function $\mathcal{D}_+(A) = \sum_{i,j \in A} d(i, j)$ sums all pair-wise distances of items in U.

Accordingly, we define the *sequential sum diversity* objective.

Definition 2 (Sequential sum diversity ($\mathcal{S}_+$))**.** *The* sequential sum diversity *of a sequence O, denoted by $\mathcal{S}_+(O)$, is defined as in Definition 1, with diversity function $\mathcal{D} = \mathcal{D}_+$, i.e.,*

$$\mathcal{S}_+(O) = \mathbb{E}_{A \sqsubseteq O}[\mathcal{D}_+(A)] = \mathbb{E}_{A \sqsubseteq O}\left[\sum_{i,j \in A} d(i, j)\right]. \tag{3.1}$$

Next, we define the problem MaxSSD of finding an ordering of items in U that maximizes the sequential sum diversity objective.

Problem 1 (Maximizing sequential sum diversity (MaxSSD))**.** *Given a finite set $U = \{1, \ldots, n\}$ of n distinct items and associated continuation probabilities $p_1, \ldots, p_n$, find an ordering O^* of the items in U that maximizes the sequential sum diversity objective, i.e.,*

$$O^* = \arg\max_{O = \pi(U)} \mathcal{S}_+(O). \tag{3.2}$$

In Definition 2, the items in U can be used to define a *complete weighted graph*, with edge weights being the distances between the items. We make heavy use of this observation as we employ *graph-theoretic ideas*, such as *Hamiltonian paths*, *graph matchings*, etc.

The second (alternative) notion of diversity is based on *coverage* [4, 24]. Here we assume that each item $i \in U$ is associated with a set of attributes $X(i) \subseteq \mathcal{X}$, where $\mathcal{X}$ is the set of all attributes. For example, in a move dataset, $\mathcal{X}$ is the set of all movie genres, and $X(i)$ the genres of movie i. The *coverage diversity* of a sequence A is then defined as the set of attributes of all items in A, i.e., $\mathcal{D}_c(A) = \bigcup_{i \in A} X(i)$.

Analogously to Problem 1, we define the problem of *maximizing the sequential coverage diversity* (MaxSCD, Problem 2).

Definition 3 (Sequential coverage diversity ($\mathcal{S}_c$))**.** *The* sequential coverage diversity *of a sequence O, denoted by $\mathcal{S}_c(O)$, is defined as in Definition 1, with diversity function $\mathcal{D} = \mathcal{D}_c$, i.e.,*

$$\mathcal{S}_c(O) = \mathbb{E}_{A \sqsubseteq O}[\mathcal{D}_c(A)] = \mathbb{E}_{A \sqsubseteq O}\left[\bigcup_{i \in A} X(i)\right]. \tag{3.3}$$

Problem 2 (Maximizing sequential coverage diversity (MaxSCD))**.** *Given a finite set $U = \{1, \ldots, n\}$ of n distinct items and associated probabilities $p_1, \ldots, p_n$, find an ordering O^* of the items in U that maximizes the sequential coverage diversity objective, i.e.,*

$$O^* = \arg\max_{O = \pi(U)} \mathcal{S}_c(O). \tag{3.4}$$

3.2 Reformulation and complexity

We start our analysis with an observation that simplifies our objective. Observation 1 stems from the fact that the probabilities assigned to all items are independent.

To simplify our calculations, we use the notation $p_{O_i} = \prod_{t=1}^{i} p_{\pi(t)}$ and $d(\pi(i+1), O_i) = \sum_{t=1}^{i} d(\pi(i+1), \pi(t))$.

Observation 1. *The probability that the user does not accept any item is $\Pr(A = O_0) = 1 - p_{\pi(1)}$; the probability that the user quits after accepting the first k items is $\Pr(A = O_k) = \prod_{i=1}^{k} p_{\pi(i)}(1 - p_{\pi(k+1)})$, if $k \leq n - 1$; and the probability that the user accepts all items is $\Pr(A = O_n) = \prod_{i=1}^{n} p_{\pi(i)}$.*

Based on Observation 1, we reformulate Equation (3.1).

Lemma 1. *The sequential sum diversity objective in Equation (3.1) can be reformulated as*

$$\mathcal{S}_+(O) = \sum_{i=1}^{n-1} p_{O_{i+1}} d(\pi(i+1), O_i).$$

We give a concrete example of how the ordering of the items influences the sequential sum diversity $\mathcal{S}_+$.

Example 1. *Let $U = \{u_1, u_2, u_3\}$, where $d(u_1, u_2) = 0.3$, $d(u_1, u_3) = 1$ and $d(u_2, u_3) = 1$. Let $p_{u_1} = p_{u_2} = 1$ and $p_{u_3} = 0$. The sequential sum diversity scores differ according to the order of the items, specifically:*

- $\mathcal{S}_+((u_1, u_2, u_3)) = \mathcal{S}_+((u_2, u_1, u_3)) = 0.3$, *as the user accepts both* u_1 *and* u_2.
- $\mathcal{S}_+((u_1, u_3, u_2)) = \mathcal{S}_+((u_2, u_3, u_1)) = 0$, *as the user only accepts* u_1 *or* u_2.
- $\mathcal{S}_+((u_3, u_1, u_2)) = \mathcal{S}_+((u_3, u_2, u_1)) = 0$, *as the user directly quits the system after examining* u_3.

Complexity. We can show that the problem of maximizing sequential sum diversity is **NP**-hard.

Theorem 1. *MaxSSD is* **NP**-*hard, even when all* p_i *are equal.*

This result indicates our problem is non-trivial, and approximation algorithms are required to guarantee high-quality orderings.

4 Sequential coverage diversity

In this section, we show that the problem of maximizing sequential coverage diversity, MaxSCD, is ordered submodular [22].

Definition 4 (Ordered submodularity [22])**.** *A sequence function* f *is ordered-submodular if for all sequence* X *and* Y*, the following property holds for all elements* s *and* $\bar{s}$*:*

$$f(X||s) - f(X) \geq f(X||s||Y) - f(X||\bar{s}||Y) \tag{4.1}$$

where $||$ *denotes the concatenation of sequences and elements.*

Ordered submodularity is introduced by Kleinberg et al. [22], who extend the concepts of monotonicity and submodularity from set functions to sequence functions.

Observation 2. *The sequential coverage diversity function* $\mathcal{S}_c(\cdot)$ *is ordered-submodular.*

As proved by Kleinberg et al. [22], a simple greedy algorithm provides a 1/2 approximation for an ordered submodular function, and thus, for MaxSCD. On the other hand, sequential sum diversity is not ordered submodular.

Observation 3. *The sequential sum diversity function* $\mathcal{S}_+(\cdot)$ *is not ordered-submodular.*

In the remainder of our paper, we focus on the technically more intriguing MaxSSD problem. Nonetheless, it is important to highlight that our definition of the MaxSCD problem, as well, provides a novel perspective to recommender systems and information-retrieval applications.

5 Ordered Hamiltonian path

In the previous section, we establish that MaxSSD is **NP**-hard through a reduction from the well-known *clique problem* [15], highlighting the inherent complexity of directly solving the problem. Furthermore, we observe that MaxSSD is not ordered submodular, precluding the use of standard *greedy* approaches for constant-factor approximations.

To address this, we introduce a new problem, *maximum ordered Hamiltonian path* (MaxOHP), and show in Theorem 2 that MaxSSD can be reduced to MaxOHP with only a constant-factor approximation loss. This reduction allows us to focus on MaxOHP, a more tractable problem, for which we propose approximation algorithms in subsequent sections.

Definition 5 (Ordered Hamiltonian path ($\mathcal{H}$))**.** *We are given a finite set* $U = \{1, \ldots, n\}$ *of* n *distinct items, a distance function* $d(\cdot, \cdot)$*, and probabilities* $\{p_1, \ldots, p_n\}$ *assigned to each item* $i \in U$*. Let* $O = (\pi(i))_{i=1}^n$ *be the ordered sequence of items of* U *according to an order* π*. The ordered Hamiltonian path, denoted by* $\mathcal{H}$*, is defined as*

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d(\pi(i), \pi(i+1)), \tag{5.1}$$

where $W_{O_i} = \sum_{j=i+1}^{n} p_{O_j}$.

Note that the *ordered* Hamiltonian-path differs significantly from the classic Hamiltonian-path [17], where the edge weights do not depend on the *order* in which edges are placed. In contrast, in the ordered Hamiltonian-path, the edge weights $d(\pi(i), \pi(i+1))$ are multiplied by a coefficient W_{O_i}, which depends on the order π as well as on the probabilities of all the items in U.

Next, we define the problem of maximizing the ordered Hamiltonian path (MaxOHP).

Problem 3 (MaxOHP)**.** *We are given a finite set* $U = \{1, \ldots, n\}$ *of* n *distinct items, a distance function* $d(\cdot, \cdot)$*, and probabilities* $p_1, \ldots, p_n$ *assigned to each item* $i \in U$*. The goal is to find an order* O^* *of the items in* U *so as to maximize the* $\mathcal{H}$ *objective, that is,*

$$O^* = \arg\max_{O=\pi(U)} \mathcal{H}(O),$$

where $W_{O_i} = \sum_{j=i+1}^{n} p_{O_j}$.

The main result of this section is the following.

Theorem 2. *Let* $p_i \in [a, b]$*, with* $0 < a < b < 1$*. An* α*-approximation solution for MaxOHP is a* $\frac{a(1-b)}{2b(1-a)}\alpha$*-approximation for MaxSSD.*

Corollary 1. *Consider the special case where* $p_i = p$ *for all* $i \in U$*. An* α*-approximation solution for MaxOHP is a* $\frac{\alpha}{2}$*-approximation for MaxSSD.*

6 Uniform continuation probabilities

In this section, we address the case where all continuation probabilities p_i are equal, and we devise approximation algorithms for MaxSSD under this special case. Specifically, we assume p may vary as a function of n and can asymptotically approach 0 or 1, excluding the trivial cases of $p = 0$ or $p = 1$.

We analyze three regimes for p, and design algorithms that achieve constant-factor approximations for MaxSSD in each regime. Due to space constraints, two regimes are presented in the main content, with the third deferred to the full version of this paper.

As outlined earlier, we reformulate the problem using the intemediate problem MaxOHP, which then leads to a solution for MaxSSD. Recall that for an order O, the ordered Hamiltonian-path objective is defined as $\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d(\pi(i), \pi(i+1))$, where $W_{O_i} = \sum_{j=i+1}^{n} p_{O_j}$. In the uniform probability setting, since $p_i = p$ for all i, the coefficient W_{O_i} is entirely determined by its position in the order, rather than the actual items being ranked. In this case,

$$W_{O_i} = \sum_{j=i+1}^{n} p^j = \frac{p^{i+1} - p^{n+1}}{1 - p}. \tag{6.1}$$

Algorithm 1: Best-τ items (BτI) algorithm

Input: Finite set U, integer τ, probabilities p_i, diversity function d
Output: An ordered sequence of U according to π, $O_n \leftarrow \pi(U)$

if p_i *are uniform* **then**
  $O_\tau \leftarrow$ Solution of Equation (6.2)
if p_i *are non-uniform* **then**
  $O_\tau \leftarrow$ Solution of Equation (7.1)
$O_n \leftarrow$ Extend O_τ to a Hamiltonian path if $\tau < n$
return O_n

6.1 The best-τ items algorithm

We first consider a case where $p < 1$. From Equation (6.1), we observe that the term p^{n+1} in the numerator diminishes as n increases, and the coefficient W_{O_i} is dominated by $p^{i+1}/(1-p)$. Specifically, when $\lim_{n\to+\infty} p^n = 0$, W_{O_i} *decays exponentially*, meaning the value of $\mathcal{H}(O)$ is dominated by the top items in the order O.

To obtain a better intuition for this regime, consider a scenario where p is very small, i.e., close to 0. In this case, a user is likely to quit after examining only a few items, so the ranking of the first few items becomes critical. To maximize diversity, the most diverse items should be placed at the top, as they contribute most significantly to the objective of the ordered Hamiltonian path.

Building on this insight, we propose an algorithm for the general case where p is bounded away from 1. We call this algorithm *best-τ items* (BτI), where τ is a parameter controlling the approximation quality. The algorithm always outputs a ranking of all items, with the pseudocode provided in Algorithm 1.

Concretely, the best-τ items (BτI) algorithm works as follows: for any integer τ, let

$$\hat{\mathcal{H}}(O_\tau) = \sum_{i=1}^{\tau-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right)$$

denote the contribution of O_τ to $\mathcal{H}(O)$. The BτI algorithm seeks the optimal τ-item sequence, denoted as BI_τ, which maximizes $\hat{\mathcal{H}}(O_\tau)$; in other words,

$$\mathrm{BI}_\tau = \underset{O_\tau \sqsubseteq \pi(U)}{\arg\max}\ \hat{\mathcal{H}}(O_\tau). \tag{6.2}$$

The BτI algorithm first construct the ordered sequence of best-τ items, BI_τ, and then extends this sequence to include the remaining items in U. The extension can be done using any arbitrary ordering for the remaining items, as it does not affect the algorithm's approximation ratio. However, a more refined approach, such as a greedy heuristic, could be used to append items that maximize the marginal gain on $\hat{\mathcal{H}}(O_\tau)$ at each step.

The performance of BτI algorithm is as follows.

Theorem 3. *Assume that $p_i = p < 1$, for all $i \in U$. For any integer τ, where $2 \le \tau \le n$, the BτI algorithm obtains a $(1 - p^{\tau-1} - p^{n-\tau} + p^n)$-approximation for MaxOHP.*

Notice that when there is a constant $\epsilon > 0$ such that $p < 1 - \epsilon$, and $\tau \ll n$, both $p^{n-\tau}$ and p^n become negligible. In this case, BτI is a $\left(1 - p^{\tau-1} - \Theta(p^n)\right)$-approximation algorithm. This aligns with our intuition: optimizing the dominant term $\mathcal{H}(O_\tau)$ yields a good approximation for the MaxOHP problem.

Algorithm 2: Greedy matching (GM) algorithm

Input: Finite set U, diversity function d
Output: An ordered sequence of U according to π, $O_n = \pi(U)$

$\tau \leftarrow \lfloor \frac{|U|}{2} \rfloor$, $M \leftarrow$ empty sequence, $S \leftarrow \emptyset$
$E \leftarrow$ all possible pairs (u, v) in decreasing order of $d(u, v)$
for $(u, v) \in E$ **do**
  if $M||(u, v)$ *forms a matching* **then**
    $M \leftarrow M||(u, v)$
    $S \leftarrow S \cup \{u, v\}$
Naming the edges in M:
$M \leftarrow ((u_1, v_1), (u_3, v_3) \cdots, (u_{2\tau-1}, v_{2\tau-1}))$
if $|U| = 2\tau + 1$ **then**
  Let $v_{2\tau+1} \in U \setminus S$, $\pi(2\tau + 1) \leftarrow v_{2\tau+1}$
for $i \leftarrow \tau$ *to* 1 **do**
  if $i = \tau$ *and* $|U| = 2\tau$ **then**
    $\pi(2\tau - 1) \leftarrow v_{2\tau-1}$, $\pi(2\tau) \leftarrow u_{2\tau-1}$
  if $d\left(v_{2i-1}, \pi(2i+1)\right) \ge d\left(u_{2i-1}, \pi(2i+1)\right)$ **then**
    $\pi(2i - 1) \leftarrow u_{2i-1}$, $\pi(2i) \leftarrow v_{2i-1}$
  else
    $\pi(2i - 1) \leftarrow v_{2i-1}$, $\pi(2i) \leftarrow u_{2i-1}$
return $O_n \leftarrow \pi(U)$

Observe that searching the best τ items for BI_τ requires $\mathcal{O}(\binom{n}{\tau}\tau!)$ time. Theorem 3 illustrates the inherent trade-off between approximation-quality and efficiency: increasing τ improves the approximation but also increases the computational cost of the algorithm.

6.2 The greedy-matching algorithm

In this section, we examine the case where p is large, specifically when $\lim_{n\to\infty} p = 1$ and $\lim_{n\to\infty} p^n = 0$. This suggests that as the number of items increases, the user accepts more items, but eventually quits before examining all of them. For instance, setting $p = 1 - \frac{1}{\log\log n}$ implies that, in expectation, the user accepts $\log\log n$ items before quitting. For simplicity, we assume $p = 1 - \frac{1}{t_n}$, where t_n is an increasing function of n.

In this scenario, applying BτI algorithm to find best-τ items becomes computationally infeasible. As Theorem 3 shows, achieving a constant approximation requires setting $\tau = \Omega(t_n)$. To address this, we introduce a polynomial-time *greedy-matching* (GM) algorithm.

The idea behind GM is to construct a matching of size-$(t_n/2)$ from t_n items. We observe that for the first t_n items, the coefficient W_{O_i} does not decrease significantly, since $p^{t_n} = 1/e - \Theta(1/t_n)$. Intuitively, the algorithm should find a path of length t_n such that the sum of the edge weights is large. In addition, the algorithm should assign larger W_{O_i} to the edges with larger weights; in other words, the edge weights should be ordered in decreasing order in the path. The proposed GM algorithm, presented in Algorithm 2, achieves both goals.

The GM algorithm starts by applying a greedy algorithm to obtain a matching M, which is a sequence of edges in decreasing order of edge weights. The size of M is equal to $\lfloor \frac{n}{2} \rfloor$. We denote the matching as $M = ((u_{2j-1}, v_{2j-1}))_{j=1}^{\lfloor \frac{n}{2} \rfloor}$.

The next step is to extend M to a Hamiltonian path. Notice that the (undirected) edge (u_{2j-1}, v_{2j-1}) is always the $(2j-1)$-th edge on the path, and the algorithm needs to decide which item comes first in the path. To make this decision, the algorithm utilizes the triangle inequality.

Let π be order of the items on the Hamiltonian path, as obtained by the GM algorithm. The above-mentioned construction satisfies two properties:

(1) $d((\pi(2i-1), \pi(2i))) \geq d((\pi(2i+1), \pi(2i+2)))$, for $i < \lfloor \frac{n}{2} \rfloor$;
(2) $d((\pi(2i), \pi(2i+1))) \geq \frac{1}{2} d((\pi(2i-1), \pi(2i)))$, for $i \leq \lfloor \frac{n}{2} \rfloor$.

For the approximation quality of GM, the following holds.

Theorem 4. *Assume that $p_i = p$, for any $i \in U$. Let p be a function of n such that $\lim_{n\to\infty} p = 1$ and $\lim_{n\to\infty} p^n = 0$. Moreover, assume that $p = 1 - \frac{1}{t_n}$, where t_n is a increasing function of n and $t_n = o(n)$. The algorithm GM yields a $\left(\frac{3(e-1)}{16e^2} - \Theta\left(\frac{1}{t_n}\right)\right)$-approximation for MaxOHP.*

7 Non-uniform continuation probabilities

In this section, we discuss a more general case when the continuation probabilities p_i are non-uniform. We assume $p_i \in [a, b]$, for all $i \in U$, where $0 < a \leq b < 1$. We start by adapting the BτI algorithm, introduced in Section 6, to this setting. Given the computational complexity of the BτI algorithm, we also propose an efficient greedy algorithm, whose performance guarantee is an implication of the BτI algorithm by setting $\tau = 2$.

7.1 The best-τ items algorithm

Recall that the ordered Hamiltonian-path is defined as $\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d(\pi(i), \pi(i+1))$, where $W_{O_i} = \sum_{j=i+1}^{n} p_{O_j}$. When the continuation probabilities p_i are uniformly equal to p, we observe that the value of W_{O_i} is dominated by $p^{i+1}/(1-p)$, hereby, we can use $p^{i+1}/(1-p)$ to approximate W_{O_i}. However, when p_i are not uniform, it is hard to find a simple and closed-form equation to approximate W_{O_i}, which is convenient for our analysis.

Our solution is to simply truncate W_{O_i}, so that the algorithm can determine the best-τ items by only using the information of the first τ items. In particular, we define

$$\tilde{\mathcal{H}}(O_\tau) = \sum_{i=1}^{\tau-1} \sum_{j=i+1}^{\tau} p_{O_j} d(\pi(i), \pi(i+1)),$$

and we find the best-τ items by maximizing $\tilde{\mathcal{H}}(O_\tau)$. We redefine

$$\mathrm{BI}_\tau = \arg\max_{O_\tau \sqsubseteq \pi(U)} \tilde{\mathcal{H}}(O_\tau). \tag{7.1}$$

The BτI algorithm (Algorithm 1) has the following quality guarantee.

Theorem 5. *Let $p_i \in [a, b]$ for all $i \in U$, where $0 < a \leq b < 1$. Let τ be an integer such that $2 \leq \tau \leq n$. The BτI algorithm provides a $\frac{a^2(1-b)(1-b^{\tau-1})}{a^2+(\tau-1)b^{\tau+1}}$-approximation guarantee for MaxOHP.*

Algorithm 3: Greedy algorithm

Input: Finite set U, probability p_i, diversity function d
Output: An ordered sequence of U according to π, $O_n = \pi(U)$

```
O_2 ← Solution of Equation (7.1) by setting τ = 2
for t = 2; t < n; t = t + 1 do
    u ← arg max_{v∈U\O_t} S_+(O_t||v) − S_+(O_t)
    O_{t+1} ← O_t||u
return O_n.
```

7.2 The greedy algorithm

Since the BτI algorithm is impractical when τ is large, we propose a greedy algorithm, presented in Algorithm 3. Its approximation ratio is obtained by setting $\tau = 2$ in Theorem 5.

Corollary 2. *The greedy algorithm (Algorithm 3) provides a $\frac{a^2(1-b)^2}{a^2+b^2}$-approximation guarantee for MaxOHP.*

8 Experimental evaluation

Our implementation and the datasets are publicly available.[2]

8.1 Datasets

We evaluate our methods on seven public datasets: five commonly used in recommender systems and two, LETOR and LTRC, from the information retrieval field. We refer to these datasets as *recommendation datasets* and *information retrieval datasets*, respectively. Table 1 provides a summary of dataset statistics. Due to space constraints, the dataset URLs are provided in the full paper.

Coat: Coat ratings in the range $[1, 5]$, accompanied by categorical features, such as gender, jacket type, and color.

KuaiRec [14]: Recommendation logs from a video-sharing mobile app, containing each video's duration, categories, and user viewing times. The *watch ratio*, calculated as the user's viewing time divided by the video's duration, and normalized to range [1, 5], serves as personalized rating.

Netflix: Movie ratings in range $[1, 5]$ with genre information. We sample 5 K movies and exclude users with less than 20 interactions.

Movielens: Movie ratings in range $[1, 5]$ with movie genres.

Yahoo: Song ratings in range $[1, 5]$ with song genres. We sample 3 K songs and exclude users with less than 20 interactions.

LETOR: Web-search dataset with relevance scores (in $\{0, 1, 2\}$) on queries-document pairs and document feature vectors.

LTRC: Web-search dataset with relevance scores (in $\{0, 1, 2, 3, 4\}$) on queries-document pairs and document feature vectors.

8.2 Experimental setting

Continuation probability. To better understand how the proposed best-τ algorithm performs across various continuation probabilities, we interpolate these probabilities into four different probability regimes: small ($[0.1, 0.3]$), medium ($[0.4, 0.6]$), large ($[0.7, 0.9]$) and full ($[0.1, 0.9]$). The first three regimes characterize different assumptions about the users' browsing behaviors: users might all be

[2] https://github.com/HongLWang/Sequential-diversification-with-provable-guarantees

Table 1: Dataset statistics of the 5 recommendation datasets and 2 information retrieval datasets. $\mathcal{U}$ and $\mathcal{I}$ represent the set of users (or queries) and items (or documents), respectively. $\#\mathcal{R}$ denotes the number of ratings (or relevance scores), $\text{avg}(\mathcal{R})$ denotes the average number of ratings per user (or documents per query), and $\text{avg}(dist)$ denotes the average item-item (or document-document) distance.

Datasets	$\lvert\mathcal{U}\rvert$	$\lvert\mathcal{I}\rvert$	$\#\mathcal{R}$	$\text{avg}(\mathcal{R})$	$\text{avg}(dist)$
Coat	290	300	6 960	24.0	0.73
KuaiRec	1 411	3 327	4 676 570	3 314.4	0.91
Netflix	4 999	1 112	557 176	1 121.5	0.83
Movielens	6 040	3 706	1 000 208	165.6	0.83
Yahoo	21 181	3 000	963 296	45.8	0.26
LETOR	1 691	65 316	69 614	41.2	0.33
LTRC	5 154	146 995	152 772	29.60	0.26

highly likely to accept the system's recommendations or tend to easily quit the system or in the middle. The last probability regime does not impose any specific assumptions.

We obtain the continuation probabilities in different ways for the recommendation datasets and the information-retrieval datasets. For the former, we first complete the user-item rating matrix using a matrix-factorization approach [23]. The estimated ratings, which range from $[1, 5]$, are then interpolated into the four probability regimes. For the latter datasets, we treat each query and its corresponding documents as a dataset and directly interpolate the relevance scores into probability regimes. This is meaningful as the documents of different queries have almost no intersection.

Distance function. For the recommendation datasets, given the categorical nature of the data, we use the *Jaccard distance* as the item-item distance function. In particular, if C_i denotes the set of categories of item i, the Jaccard distance $d(i, j)$ is defined as $d(i, j) = 1 - |C_i \cap C_j|/|C_i \cup C_j|$. For the information-retrieval datasets, with the provided document feature vectors, we use the $1-$*cosine distance* as the document-document distance function.

Additional metrics. To evaluate the performance of our algorithms in terms of user engagement and satisfaction, we use two metrics: expected discounted cumulative gain (*ExpDCG*) and expected serendipity (*ExpSerendipity*). Discounted cumulative gain [21] measures the accumulated relevance of items in a ranking list, discounted by their position. Serendipity [18] quantifies how relevant and unexpected the recommended items are to the user. We adapt these metrics to account for users' continuation probabilities. For any user, let p_i be the continuation probability of item i.

ExpDCG is defined as $\sum_{j=1}^{|I|} \left(\sum_{t=1}^{j} \frac{p_t}{log_2(t+1)} \right) (1 - p_{j+1}) \prod_{t=1}^{j} p_t$. For serendipity, let H be the set of previously rated items, and $C(H)$ be the set of categories covered by H. *ExpSerendipity* is defined as $\sum_{j=1}^{|I|} \left(\sum_{t=1}^{j} p_t \cdot I(t, u) \right) (1 - p_{j+1}) \prod_{t=1}^{j} p_t$, where $I(t, u) = 1$ if $|C(H \cup \{t\})| > |C(H)|$ and 0 otherwise. Here, p_t assesses the relevance of item t, while $I(t, u)$ captures its unexpectedness.

Note that, by definition, the *ExpSerendipity* score for the LETOR and LTRC datasets is 0 as the sequences for each user comprise only rated items. Similarly, the KuaiRec dataset also has a *ExpSerendipity* score of 0, as each user has already covered all categories in their history of ratings. Therefore, we omit the *ExpSerendipity* scores for these datasets.

8.3 Proposed methods and baselines

First, we present the baseline methods used in our experiments. We reformulate the baselines using our notation to enhance clarity. We consistently denote by R the set of already-selected items. The parameter λ models the trade-off diversity vs. relevance. We conduct a grid search to find $\lambda \in [0, 1]$ that performs the best according to the MaxSSD objective for baselines MSD, MMR, and DPP. We report the results using that optimal value of λ for each method.

Random. Randomly shuffle all items into a sequence.

EXPLORE [11]. Details on the EXPLORE algorithm and its adaptation to our setting are provided in the full paper.

Diversity-weighted utility maximization (DUM) [4]. DUM seeks to find a permutation π that maximizes a diversity-weighted relevance objective: $\sum_{i=1}^{|U|} (|C(O_i)| - |C(O_{i-1})|)\, p_{\pi(i)}$, where $C(O_i)$ denotes the set of categories that O_i covers.

Maximal marginal relevance (MMR) [6]. MMR iteratively selects the item $i \in U$ that maximizes $\lambda\, p_i - (1-\lambda) \max_{j \in R}(1 - d(i, j))$.

Max-sum diversification (MSD) [5]. MSD iteratively selects the item $i \in U$ that maximizes $p_i + \lambda \sum_{j \in R} d(i, j)$.

Determinantal point process (DPP) [10]. DPP iteratively selects the item i that maximizes $\lambda p_i + (1-\lambda)(\log\det(\mathbf{S}_{R \cup \{i\}}) - \log\det(\mathbf{S}_R))$, where $\mathbf{S}$ is a similarity matrix, with $\mathbf{S}_{ij} = 1 - d(i, j)$.

Next, we provide more details of our methods BτI and BτI-H.

BτI. We refer to the BτI algorithm as B2I, B3I, and B4I when τ is set to be 2, 3, and 4. Once the first τ-sequence is chosen, we greedily extend the remaining items that maximize the incremental gain of MaxSSD. Notice that B2I in our implementation is the same as the greedy algorithm we propose in Algorithm 3.

BτI-H. Since BτI can be computationally expensive when τ is large, we adopt a greedy approach by selecting the top 100 items that maximize the incremental gain of MaxSSD, thereby forming a candidate set. The best τ items are exclusively selected within this candidate set. We extend the best τ items in the same way as in BτI.

Throughout the experiments, we evaluate B2I, B3I, and B4I on dataset LETOR and LTRC. For the recommendation datasets, we evaluate B2I and heuristics B3I-H and B4I-H on these datasets.

8.4 Results and discussion

Performance on sequential sum diversity. We conduct experiments on all datasets evaluating our methods and the baselines. For each dataset and method, we obtain a sequence O of all items for each user and calculate the sequential sum diversity $\mathcal{S}_+(O)$. We report the average $\mathcal{S}_+(O)$ values and standard deviations across all users as shown in Table 2. Results for the other regimes are included in full version of this paper.

In Table 2, we observe that B2I achieves excellent results and outperforms all baselines. The only exception is on KuaiRec, where B4I-H marginally outperforms B2I and achieves the best result.

Next, compared with B2I, the performance of B3I (or B3I-H) and B4I (or B4I-H) declines. This decline can be explained by two factors. First, the BτI and BτI-H algorithms select the top τ items by solving the intermediate MaxOHP problem, which leads to some information loss compared to directly solving MaxSSD, as their objectives differ when $\tau > 2$. Second, due to the relatively low continuation

Table 2: Sequential sum diversity $\mathcal{S}_+(O)$ with item continuation probability mapped to $[0.4, 0.6]$. The results marked with * are obtained using the BτI-H heuristic.

Datasets	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
Coat	0.646 ± 0.177	1.276 ± 0.374	1.214 ± 0.348	1.211 ± 0.349	1.282 ± 0.377	1.284 ± 0.377	**1.289 ± 0.376**	1.184 ± 0.368*	1.184 ± 0.367*
KuaiRec	0.383± 0.066	0.681 ± 0.082	0.411 ± 0.086	0.704 ± 0.057	0.725 ± 0.080	0.711 ± 0.073	0.778 ± 0.075	0.776 ± 0.072*	**0.782 ± 0.073***
Netflix	0.921± 0.220	1.861 ± 0.263	1.779 ± 0.241	1.888 ± 0.254	1.937 ± 0.258	1.933 ± 0.258	**1.946 ± 0.264**	1.919 ± 0.270*	1.910 ± 0.272*
Movielens	0.878± 0.246	2.142 ± 0.114	2.080 ± 0.113	2.228 ± 0.046	2.232 ± 0.044	2.230 ± 0.046	**2.235 ± 0.043**	2.234 ± 0.046*	2.233 ± 0.050*
Yahoo	0.231 ± 0.248	1.684 ± 0.465	1.793 ± 0.466	1.793 ± 0.466	1.793 ± 0.466	1.793 ± 0.466	**1.795 ± 0.466**	1.729 ± 0.495*	1.715 ± 0.496*
LETOR	0.208 ± 0.106	0.440 ± 0.190	0.564 ± 0.184	0.577 ± 0.191	0.570 ± 0.191	0.557 ± 0.181	**0.592 ± 0.195**	0.582 ± 0.197	0.588 ± 0.197
LTRC	0.359 ± 0.116	0.597 ± 0.151	0.640 ± 0.134	0.662 ± 0.146	0.644 ± 0.145	0.642 ± 0.138	**0.670 ± 0.144**	0.667 ± 0.145	0.668 ± 0.145

Table 3: *ExpDCG* and *ExpSerendipity* values with item continuation probabilities mapped to $[0.4, 0.6]$. The results marked with * are obtained using the BτI-H heuristic.

Metrics	Datasets	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
ExpDCG	Coat	0.354 ± 0.070	0.496±0.100	**0.513 ± 0.096**	0.489 ± 0.091	0.511 ± 0.095	0.510 ± 0.095	0.494 ± 0.096	0.499 ± 0.096*	0.499 ± 0.096*
	KuaiRec	0.347 ± 0.049	0.483 ± 0.048	**0.590 ± 0.048**	0.459 ± 0.028	0.465 ± 0.030	0.472 ± 0.032	0.409 ± 0.072	0.477 ± 0.031*	0.480 ± 0.032*
	Netflix	0.413 ± 0.065	0.575 ± 0.055	**0.604 ± 0.044**	0.596 ± 0.044	0.600 ± 0.044	0.600 ± 0.044	0.597 ± 0.046	0.600 ± 0.044*	0.600 ± 0.044*
	Movielens	0.394 ± 0.080	0.620 ± 0.026	**0.635 ± 0.005**	**0.635 ± 0.005**	**0.635 ± 0.005**	**0.635 ± 0.005**	**0.635 ± 0.005**	**0.635 ± 0.005***	**0.635 ± 0.005***
	Yahoo	0.360 ± 0.094	0.550 ± 0.100	0.572 ± 0.086	0.572 ± 0.086	0.572 ± 0.086	0.572 ± 0.086	0.565 ± 0.092	**0.577 ± 0.085***	0.577 ± 0.085*
	LETOR	0.274 ± 0.082	0.330±0.104	0.327 ± 0.104	**0.416 ± 0.123**	0.400 ± 0.115	0.406 ± 0.117	0.326 ± 0.104	0.390 ± 0.108	0.393 ± 0.110
	LTRC	0.323 ± 0.068	0.373±0.082	0.378 ± 0.078	0.443 ± 0.084	**0.447 ± 0.088**	0.440 ± 0.081	0.378 ± 0.078	0.402 ± 0.080	0.407 ± 0.080
ExpSerendipity	Coat	0.141 ± 0.138	0.247±0.186	0.204 ± 0.159	0.225 ± 0.147	0.214 ± 0.157	0.218 ± 0.160	0.288 ± 0.202	**0.354 ± 0.238***	0.351 ± 0.240*
	Netflix	0.028 ± 0.068	0.132±0.151	0.133 ± 0.171	**0.152 ± 0.164**	0.148 ± 0.164	0.149 ± 0.163	0.143 ± 0.161	0.139 ± 0.162*	0.139 ± 0.165*
	Movielens	0.043 ± 0.091	**0.141±0.153**	0.136 ± 0.159	0.115 ± 0.144	0.115 ± 0.144	0.115 ± 0.144	0.115 ± 0.144	0.115 ± 0.144*	0.115 ± 0.144*
	Yahoo	0.040 ± 0.079	0.275±0.145	0.309 ± 0.140	0.303 ± 0.136	0.303 ± 0.136	0.303 ± 0.136	**0.325 ± 0.143**	0.264 ± 0.143*	0.259 ± 0.145*

probabilities, the expected number of items a user accepts before quitting is typically not larger than 2. Consequently, B2I, which focuses on optimizing the top 2 items, tends to perform better than B3I or B4I, which may produce suboptimal top-2 sequences.

Next, when comparing the performance of B3I and B4I, we observe that performance increases as τ increases. However, when comparing the performance of B3I-H and B4I-H, no consistent trend in the algorithms' performance is observed. This inconsistency can be attributed to the filtering process applied to these heuristics. It is worth noting that similar observations can be made across all probability regimes, as detailed in the full paper.

When comparing the performances of BτI and BτI-H across different continuation probabilities, we observe both algorithms perform better with smaller continuation probabilities than with larger ones. This observation aligns with our theoretical analysis on approximation guarantees (Theorem 5). With fixed τ, the approximation ratio decreases as the continuation probability increases.

Performance on *ExpDCG* and *ExpSerendipity*. With respect to the *ExpDCG* metric, we observe in Table 3 that DUM has the best performance, achieving the highest or second-highest scores in 5 out of 7 datasets. This result is consistent with the theory, as the algorithm prioritizes selecting items with the highest relevance scores. The BτI and BτI-H algorithms do not achieve the highest scores, which is expected as they do not prioritize optimizing relevance. It is worth noting, however, that the differences across the methods are relatively small. Similar findings hold across all the probability regimes.

Regarding the *ExpSerendipity* metric, we observe no clear winners among all methods (with MMR performing poorly), as each method achieves (at least) second-highest scores in 1 out of 4 datasets (Table 3). However, when considering all the continuation probability regimes, our approaches are better. This suggests that the BτI (-H) algorithm may have a higher potential for generating more exploratory recommendations.

9 Conclusion

In this paper, we study the novel concept of *sequential diversity*, aiming to instill diversity into rankings while considering the relevance of items and modeling user behavior. The framework is designed to find rankings that consist of items that are both diverse and relevant. The formulation gives rise to a novel computational problem, for which we establish a connection with the ordered Hamiltonian-path problem and design approximation algorithms with provable guarantees. Our algorithms offer trade-offs of efficiency vs. approximation quality.

Our paper opens many exciting directions for future work. First, it will be interesting to devise more efficient combinatorial algorithms without losing their approximation guarantees. Additionally, it will be valuable to incorporate user models of higher complexity into the framework, in order to capture more nuanced user behavior. Last, it will be valuable to study the proposed framework with a user study on a real-world system.

Acknowledgments

We sincerely thank Sebastian Dalleiger for his valuable feedback and helpful discussions, which contributed to improving this paper. This research is supported by the ERC Advanced Grant REBOUND (834862), the EC H2020 RIA project SoBigData++ (871042), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

[1] Mustafa Abdool, Malay Haldar, Prashant Ramanathan, Tyler Sax, Lanbo Zhang, Aamir Manaswala, Lynn Yang, Bradley Turnbull, Qing Zhang, and Thomas Legrand. 2020. Managing diversity in airbnb search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2952–2960.

[2] Rakesh Agrawal, Sreenivas Gollapudi, Alan Halverson, and Samuel Ieong. 2009. Diversifying search results. In *Proceedings of the second ACM international conference on web search and data mining*. 5–14.

[3] Azin Ashkan, Branislav Kveton, Shlomo Berkovsky, and Zheng Wen. 2014. Diversified Utility Maximization for Recommendations.. In *RecSys Posters*.

[4] Azin Ashkan, Branislav Kveton, Shlomo Berkovsky, and Zheng Wen. 2015. Optimal Greedy Diversity for Recommendation. In *IJCAI*, Vol. 15. 1742–1748.

[5] Allan Borodin, Hyun Chul Lee, and Yuli Ye. 2012. Max-sum diversification, monotone submodular functions and dynamic updates. In *Proceedings of the 31st ACM SIGMOD-SIGACT-SIGAI symposium on Principles of Database Systems*. 155–166.

[6] Jaime Carbonell and Jade Goldstein. 1998. The use of MMR, diversity-based reranking for reordering documents and producing summaries. In *Proceedings of the 21st annual international ACM SIGIR conference on Research and development in information retrieval*. 335–336.

[7] Alfonso Cevallos, Friedrich Eisenbrand, and Rico Zenklusen. 2015. Max-sum diversity via convex programming. *arXiv preprint arXiv:1511.07077* (2015).

[8] Alfonso Cevallos, Friedrich Eisenbrand, and Rico Zenklusen. 2017. Local search for max-sum diversification. In *Proceedings of the Twenty-Eighth Annual ACM-SIAM Symposium on Discrete Algorithms*. SIAM, 130–142.

[9] Alfonso Cevallos, Friedrich Eisenbrand, and Rico Zenklusen. 2019. An improved analysis of local search for max-sum diversification. *Mathematics of Operations Research* 44, 4 (2019), 1494–1509.

[10] Laming Chen, Guoxin Zhang, and Eric Zhou. 2018. Fast greedy map inference for determinantal point process to improve recommendation diversity. *Advances in Neural Information Processing Systems* 31 (2018).

[11] Erica Coppolillo, Giuseppe Manco, and Aristides Gionis. 2024. Relevance Meets Diversity: A User-Centric Framework for Knowledge Exploration Through Recommendations. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining* (Barcelona, Spain) *(KDD '24)*. Association for Computing Machinery, New York, NY, USA, 490–501. https://doi.org/10.1145/3637528.3671949

[12] Zdravko Cvetkovski. 2012. *Inequalities: theorems, techniques and selected problems*. Springer Science & Business Media.

[13] Lu Gan, Diana Nurbakova, Léa Laporte, and Sylvie Calabretto. 2020. Enhancing recommendation diversity using determinantal point processes on knowledge graphs. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2001–2004.

[14] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 540–550.

[15] Michael R Garey and David S Johnson. 1979. Computers and intractability. *A Guide to the* (1979).

[16] Sreenivas Gollapudi and Aneesh Sharma. 2009. An axiomatic approach for result diversification. In *Proceedings of the 18th international conference on World wide web*. 381–390.

[17] Yuri Gurevich and Saharon Shelah. 1987. Expected computation time for Hamiltonian path problem. *SIAM J. Comput.* 16, 3 (1987), 486–502.

[18] Jonathan L Herlocker, Joseph A Konstan, Loren G Terveen, and John T Riedl. 2004. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems (TOIS)* 22, 1 (2004), 5–53.

[19] Yanhua Huang, Weikun Wang, Lei Zhang, and Ruiwen Xu. 2021. Sliding spectrum decomposition for diversified recommendation. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 3041–3049.

[20] Samuel Ieong, Mohammad Mahdian, and Sergei Vassilvitskii. 2014. Advertising in a stream. In *Proceedings of the 23rd International Conference on World Wide Web* (Seoul, Korea) *(WWW '14)*. Association for Computing Machinery, New York, NY, USA, 29–38. https://doi.org/10.1145/2566486.2568030

[21] Kalervo Järvelin and Jaana Kekäläinen. 2002. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems (TOIS)* 20, 4 (2002), 422–446.

[22] Jon Kleinberg, Emily Ryu, and Éva Tardos. 2023. Calibrated recommendations for users with decaying attention. *arXiv preprint arXiv:2302.03239* (2023).

[23] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[24] Shameem Puthiya, Nicolas Usunier, and Yves Grandvalet. 2016. A coverage-based approach to recommendation diversity on similarity graph. In *Proceedings of the 10th ACM Conference on Recommender Systems*. 15–22.

[25] Shaojie Tang, Jing Yuan, and Vijay Mookerjee. 2020. Optimizing ad allocation in mobile advertising. In *Proceedings of the Twenty-First International Symposium on Theory, Algorithmic Foundations, and Protocol Design for Mobile Networks and Mobile Computing*. 181–190.

[26] Haolun Wu, Yansen Zhang, Chen Ma, Fuyuan Lyu, Bowei He, Bhaskar Mitra, and Xue Liu. 2024. Result Diversification in Search and Recommendation: A Survey. *IEEE Transactions on Knowledge and Data Engineering* (2024).

[27] Yue Xu, Hao Chen, Zefan Wang, Jianwen Yin, Qijie Shen, Dimin Wang, Feiran Huang, Lixiang Lai, Tao Zhuang, Junfeng Ge, et al. 2023. Multi-factor sequential re-ranking with perception-aware diversification. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5327–5337.

[28] ChengXiang Zhai, William W Cohen, and John Lafferty. 2015. Beyond independent relevance: methods and evaluation metrics for subtopic retrieval. In *Acm sigir forum*, Vol. 49. ACM New York, NY, USA, 2–9.

[29] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep interest network for click-through rate prediction. In *Proceedings of the 24th ACM SIGKDD international conference on knowledge discovery & data mining*. 1059–1068.

10 ETHICS STATEMENT

In this work, we introduce BrI, a method designed to generate rankings of items that are both relevant and diverse for users. The proposed BrI and BrI-H algorithms can enhance the user experience by preventing the so-called "rabbit hole" effect, where users may become trapped in a narrow set of recommendations or search results that lack variety. By maximizing the sequential diversity of the top-ranked items in a user's recommendation list or web search results, our approach encourages exploration and a broader discovery of content, thereby reducing the risk of reinforcing cognitive biases or limiting exposure to diverse perspectives.

To ensure the ethical integrity of our work, all datasets utilized in this study are publicly available and have been processed to protect user privacy. Specifically, user IDs have been anonymized and replaced with numeric identifiers, ensuring that no personal or identifying information is disclosed.

11 Omitted proofs from Section 5

Lemma 1. *The sequential sum diversity objective in Equation (3.1) can be reformulated as*

$$\mathcal{S}_+(O) = \sum\nolimits_{i=1}^{n-1} p_{O_{i+1}} d\left(\pi(i+1), O_i\right).$$

Proof. We start with demonstrating that the sequential sum-diversity objective can be precisely reformulated via the expression $2\sum_{i=2}^{n}\sum_{j=1}^{i-1}\prod_{t=1}^{i}\Pr_{\pi(t)} d\left(\pi(i),\pi(j)\right)$. Next, the formulation can be simplified using the definition of p_{O_i} and $d\left(\pi(i), O_{i-1}\right)$.

We write the definition of sequential sum-diversity objective, and proceed by merging the terms according to our observations.

$$\begin{aligned}
&\sum_{k=1}^{n}\Pr(A=O_k)\sum_{\pi(i),\pi(j)\in O_k} d\left(\pi(i),\pi(j)\right)\\
&\overset{(a)}{=}\sum_{k=1}^{n}\Pr(A=O_k)\sum_{i,j\le k} d\left(\pi(i),\pi(j)\right)\\
&\overset{(b)}{=}\sum_{k=1}^{n-1}\Big(\prod_{t=1}^{k}p_{\pi(t)}-\prod_{t=1}^{k+1}p_{\pi(t)}\Big)\sum_{i,j\le k} d\left(\pi(i),\pi(j)\right)\\
&+\prod_{t=1}^{n}p_{\pi(t)}\sum_{i,j\le n} d\left(\pi(i),\pi(j)\right)\\
&=\sum_{k=1}^{n-1}\prod_{t=1}^{k+1}p_{\pi(t)}\Big(-\sum_{i,j\le k} d\left(\pi(i),\pi(j)\right)+\sum_{i,j\le k+1} d\left(\pi(i),\pi(j)\right)\Big)\\
&=\sum_{k=1}^{n-1}\prod_{t=1}^{k+1}p_{\pi(t)}\Big(\sum_{i=k+1,j\le k} d\left(\pi(i),\pi(j)\right)+\sum_{j=k+1,i\le k} d\left(\pi(i),\pi(j)\right)\Big)\\
&=2\sum_{k=1}^{n-1}\prod_{t=1}^{k+1}p_{\pi(t)}\Big(\sum_{i=k+1,j\le k} d\left(\pi(i),\pi(j)\right)\Big)\\
&=2\sum_{k=2}^{n}\prod_{t=1}^{k}p_{\pi(t)}\Big(\sum_{i=k,j\le k-1} d\left(\pi(i),\pi(j)\right)\Big)\\
&=2\sum_{i=2}^{n}\sum_{j=1}^{i-1}\prod_{t=1}^{i}p_{\pi(t)} d\left(\pi(i),\pi(j)\right).
\end{aligned}\tag{11.1}$$

Notice that equality (a) holds by the definition of $\pi(i)$: indeed, $\pi(i)$ indicates the item that is placed at the i-th position. Equality (b) holds by substituting the expression of $\Pr(A=O_k)$ from the observation 1. For the following equations, we re-arrange the simplify the formulas. □

For the proof of Observation 2 and Observation 3, we use two lemma from Kleinberg et al. [22].

Lemma 2 (Kleinberg et al. [22])**.** *If f and g are ordered-submodular, then $\alpha f+\beta g$ is also ordered-submodular for any $\alpha,\beta\ge 0$.*

Lemma 3 (Kleinberg et al. [22])**.** *Suppose h is a monotone submodular set function, Then the function f constructed by evaluating h on the set of the first t elements of S, that is,*

$$f(S)=\begin{cases} h(S) & \text{if } |S|\le t\\ h(S_t) & \text{if } |S|>t\end{cases}$$

is ordered-submodular.

Observation 2. *The sequential coverage diversity function $\mathcal{S}_c(\cdot)$ is ordered-submodular.*

Proof. By the definition of the $\mathcal{S}_c$ objective we have:

$$\mathcal{S}_c(O)=\mathbb{E}_{A\sqsubseteq O}[\mathcal{D}(A)]=\sum_{k=1}^{n}\Pr(A=O_k)\mathcal{D}(O_k),\tag{11.2}$$

where $\mathcal{D}(O_k)$ is a monotone submodular function. Let $f_k(O)=\mathcal{D}(O_k)$ so we can rewrite $\mathcal{S}_c(O)$ as $\mathcal{S}_c(O)=\sum_{k=1}^{n}p_{O_k}f_k(O)$. Since $f_k(O)$ is a monotone submodular function evaluated on the first k items, by Lemma 3, $f_k(O)$ is an ordered submodular function. Since $\Pr(O_k)\ge 0$ for all k, by Lemma 2, $\mathcal{S}_c(\cdot)$ is ordered-submodular. □

Observation 3. *The sequential sum diversity function $\mathcal{S}_+(\cdot)$ is not ordered-submodular.*

Proof. To show $\mathcal{S}_+(\cdot)$ is not ordered submodular, we only need one counter example. Let X and Y be two sequences that satisfy $p_X>0$ and $p_Y>0$. Let s be an item such that $p_s>0$ and $d(s,Y)>0$. Also, let $\bar{s}$ be an item such that $p_{\bar{s}}=0$. It follows that

$$\begin{aligned}\mathcal{S}_+(X||s||Y)-\mathcal{S}_+(X||\bar{s}||Y)&=\mathcal{S}_+(X||s||Y)-\mathcal{S}_+(X)\\&>\mathcal{S}_+(X||s)-\mathcal{S}_+(X).\end{aligned}$$

This example violates the definition of ordered submodularity, which proves that $\mathcal{S}_+(\cdot)$ is not an ordered submodular function. □

Theorem 1. *MaxSSD is* **NP**-*hard, even when all p_i are equal.*

Proof. We provide a reduction from the decision version of the maximum clique problem to the decision version of the MaxSSD problem. Define CLIQUE $=(G,V,k)$, with $|V|=n$ to describe instances of the maximum clique problem, where we want to decide whether there exists a clique of size k in G. Also define an instance of the MaxSSD problem by (U,d,p,θ), where U is the item set, p is a uniform continuation probability, and $d(\cdot,\cdot)$ is a metric distance function, and we want to decide whether there exist an ordering π of U such that $\mathcal{S}_+(\pi(U))>\theta$.

Given an instance of CLIQUE, we construct an instance of MaxSSD in the following way: we set $U=V$, and we assign $d(u,v)=2$ if there is an edge between nodes u and v in G, otherwise we set $d(u,v)=1+\epsilon$, where ϵ is any constant smaller than 1. We set $p=\frac{1-\epsilon}{2n^2}$ and $\theta=2\sum_{i=1}^{k-1}i\times\left(\frac{1-\epsilon}{2n^2}\right)^{i+1}$. One can easily verify that this transformation can be done in polynomial time, and that $d(\cdot,\cdot)$ is a metric.

If there is a clique of size $k\le n$ in G, then for the corresponding item set $U=V$, we can construct an ordered sequence $O=(\pi(i))_{i=1}^{n}$ by assigning O_k to be the k nodes in U that corresponds to the clique in G, and setting in an arbitrary way the rest of the sequence order.

Let $\mathcal{L}(O)$ be a lower bound of $\mathcal{S}_+(A)$. One can verify it is also a lower bound for the optimal objective value of the MaxSSD problem. We can define $\mathcal{L}(O)$ as follows:

$$\begin{aligned}\mathcal{L}(O) &= \sum_{i=1}^{k-1} p_{O_{i+1}} d(\pi(i+1), O_i)\\ &= 2(p^2 + 2p^3 + 3p^4 + \cdots + (k-2)p^{k-1} + (k-1)p^k)\\ &= 2\sum_{i=1}^{k-1} i \times p^{i+1}.\end{aligned}$$

Furthermore, it holds that $\mathcal{L}(O) = \theta$.

On the other hand, if there does not exist a size k clique in G, then the densest possible graph structure of G contains $T = \lfloor \frac{n}{k-1} \rfloor$ size-$(k-1)$ cliques, and one clique of size $L = n-(k-1)T$. A node in a size-$(k-1)$ clique can connect to at most $k-2$ nodes in another size-$(k-1)$ clique, thus the maximum number of edges between any two size-$(k-1)$ cliques is $(k-1)(k-2)$. Similarly, the number of edges between a size-L clique and a size-$(k-1)$ clique is $(k-2)L$. Given this graph structure, we can construct an ordered sequence $\tilde{O} = (\tilde{\pi}(i))_{i=1}^n$, where sequence $[\tilde{\pi}((j-1)(k-1)+1), \cdots, \tilde{\pi}((k-1)\times j)]$ corresponds to the j-th size-$(k-1)$ clique of G, with $j \in [1, T]$, and the remaining sequence corresponds to the size-L clique of G.

Let $\mathcal{U}(\tilde{O}) = \mathcal{S}_+(\tilde{O})$. One can verify that $\mathcal{U}(\tilde{O})$ is an upper bound of the optimal objective value of the corresponding MaxSSD problem. It holds that

$$\begin{aligned}\mathcal{U}(\tilde{O}) &= \sum_{i=1}^{n-1} p_{\tilde{O}_{i+1}} d(\tilde{\pi}(i+1), \tilde{O}_i)\\ &\overset{(a)}{=} 2(p^2 + 2p^3 + \cdots + (k-2)p^{k-1})\\ &+ \sum_{i=1}^{T-1}\sum_{j=0}^{k-2} p^{j+1+i(k-1)}(2[i(k-2)+j] + i(1+\epsilon)) \qquad (11.3)\\ &+ \sum_{j=0}^{L-1} p^{T(k-1)+j+1}(2[T(k-2)+j] + T(1+\epsilon))\\ &\overset{(b)}{<} \mathcal{L}(O) + (\epsilon-1)p^k + 2n(p^{k+1} + p^{k+2} + \cdots + p^n),\end{aligned}$$

where (a) holds because of Lemma 1 and (b) holds because for $j \in [k+1, n]$, the j-th node contributes at most $2np^j$ to $\mathcal{U}(\tilde{O})$.

Furthermore, since

$$\begin{aligned}\mathcal{L}(O) - \mathcal{U}(\tilde{O}) &= \theta - \mathcal{U}(\tilde{O})\\ &> (1-\epsilon)p^k - 2n(p^{k+1} + p^{k+2} + \cdots + p^n)\\ &= (1-\epsilon)p^k - 2np(p^k + p^{k+1} + \cdots + p^{n-1})\\ &> (1-\epsilon)p^k - 2np(n-k)p^k\\ &= p^k(1-\epsilon-2np(n-k)) = \frac{p^k(1-\epsilon)k}{n} > 0\end{aligned}$$

holds for all k, we conclude that a yes instance of the CLIQUE problem indicates a yes instance of the MaxSSD problem, and a no instance indicates a no instance of the MaxSSD problem.

This finishes our reduction from the CLIQUE problem to the MaxSSD problem and we conclude that the MaxSSD problem is **NP**-hard. □

11.1 Proof of Theorem 2

We start by observing that $\mathcal{H}(O)$ can be reformulated according to Lemma 4. The reformulation helps us to relate $\mathcal{H}(O)$ with $\mathcal{S}_+(O)$. Notice that $p_{O_{i+1}}$ is the common coefficient of $d_L(O_{i+1})$ in $\mathcal{H}(O)$ and $d(\pi(i+1), O_i)$ in $\mathcal{S}_+(O)$. Hence, we essentially only compare $d_L(O_{i+1})$ with $d(\pi(i+1), O_i)$, for all $1 \le i \le n-1$.

Lemma 4. *$\mathcal{H}(O)$ can be equivalently formulated as*

$$\mathcal{H}(O) = \sum\nolimits_{i=1}^{n-1} p_{O_{i+1}} d_L(O_{i+1}),$$

where $d_L(O_k) \coloneqq \sum_{t=1}^{k-1} d(\pi(t), \pi(t+1))$.

The following corollary is a direct implication from the triangle inequality, $2d(\pi(i+1), O_i) \ge d_L(O_{i+1})$, for any sequence O.

Corollary 3. *For any ordered sequence O of items in U it holds*

$$2\mathcal{S}_+(O) \ge \mathcal{H}(O).$$

Let O^* be the optimal sequence of $\mathcal{S}_+$: $O^* = \arg\max \mathcal{S}_+(O)$. Let $(\pi(i), \pi(i+1))$ be any pair of items swapped in O^*, and denote $O' = (\ldots, \pi(i+1), \pi(i), \ldots)$. By optimality, it follows that $\mathcal{S}_+(O^*) \ge \mathcal{S}_+(O')$. With some simplification, we get the following lemma.

Lemma 5. *For an optimal solution $O^* = \arg\max \mathcal{S}_+(O)$ to the MaxSSD problem, it holds*

$$d\left(\pi(i+1), O_i^*\right) \le \frac{1-p_{\pi(i+1)}}{p_{\pi(i+1)}} \sum\nolimits_{j=1}^{i} \frac{p_{\pi(j+1)}}{1-p_{\pi(j+1)}} d\left(\pi(j), \pi(j+1)\right),$$

for any $1 \le i \le n-1$.

We observe that $\frac{1-p_i}{p_i} \le \frac{1-a}{a}$, and $\frac{p_i}{1-p_i} \le \frac{b}{1-b}$ if $p_i \in [a, b]$. The following corollary holds directly by the implication of Lemma 5 that $d\left(\pi(i+1), O_i^*\right) \le \frac{b(1-a)}{a(1-b)} d_L(O_{i+1}^*)$.

Corollary 4. *For an optimal sequence $O^* = \arg\max \mathcal{S}_+(O)$ to the MaxSSD problem, and assuming that $p_i \in [a, b]$, for all $i \in U$, it holds that $\mathcal{S}_+(O^*) \le \frac{b(1-a)}{a(1-b)} \mathcal{H}(O^*)$.*

We are ready to prove Theorem 2.

Proof. Let O^* be an optimal sequence for the MaxSSD problem, and O^o be an optimal sequence for the MaxOHP problem. Let O^α be an α-approximation sequence for MaxOHP. It holds

$$\begin{aligned}\mathcal{S}_+(O^*) &\overset{(\dagger)}{\le} \frac{b(1-a)}{a(1-b)} \mathcal{H}(O^*) \le \frac{b(1-a)}{a(1-b)} \mathcal{H}(O^o)\\ &\le \frac{b(1-a)}{a(1-b)} \frac{\mathcal{H}(O^\alpha)}{\alpha} \overset{(\ddagger)}{\le} \frac{2b(1-a)}{\alpha a(1-b)} \mathcal{S}_+(O^\alpha),\end{aligned}$$

where inequality $(\dagger)$ follows from Corollary 4, and $(\ddagger)$ from Corollary 3. Thus we proved $\frac{\mathcal{S}_+(O^\alpha)}{\mathcal{S}_+(O^*)} \ge \frac{a(1-b)}{2b(1-a)}\alpha$. □

Lemma 4. *$\mathcal{H}(O)$ can be equivalently formulated as*

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} p_{O_{i+1}} d_L(O_{i+1}),$$

where $d_L(O_k) := \sum_{t=1}^{k-1} d(\pi(t), \pi(t+1))$.

Proof. We simply re-arrange the formulas.

$$\begin{aligned}
\mathcal{H}(O) &= \sum_{i=1}^{n-1} w_i d(\pi(i), \pi(i+1)) \\
&= \sum_{i=1}^{n-1} \sum_{k=i+1}^{n} p_{O_k} d(\pi(i), \pi(i+1)) \\
&= p_{O_2} d(\pi(1), \pi(2)) + p_{O_3}(d(\pi(1), \pi(2)) + d(\pi(2), \pi(3))) \\
&+ \cdots + p(O) \sum_{t=1}^{n-1} d(\pi(t), \pi(t+1)) \\
&= p_{O_2} d_L(O_2) + p_{O_3} d_L(O_3) + \cdots + p(O) d_L(O) \\
&= \sum_{i=1}^{n-1} p_{O_{i+1}} d_L(O_{i+1}).
\end{aligned} \tag{11.4}$$

□

Corollary 3. *For any ordered sequence O of items in U it holds*

$$2\mathcal{S}_+(O) \geq \mathcal{H}(O).$$

Proof. First, we show that if $2d(\pi(i+1), O_i) \geq d_L(O_{i+1})$ holds for any $i \in [n-1]$, then our lemma is proven:

$$\begin{aligned}
2\mathcal{S}_+(O) &= 2\sum_{i=1}^{n-1} p_{O_{i+1}} d(\pi(i+1), O_i) \\
&= \sum_{i=1}^{n-1} p_{O_{i+1}} 2d(\pi(i+1), O_i) \\
&\geq \sum_{i=1}^{n-1} p_{O_{i+1}} d_L(O_{i+1}) \\
&= \mathcal{H}(O).
\end{aligned}$$

Next, we show that $2d(\pi(i+1), O_i) \geq d_L(O_{i+1})$ holds for any $1 \leq i \leq n-1$:

$$\begin{aligned}
d_L(O_{i+1}) &= \sum_{j=1}^{i} d(\pi(j), \pi(j+1)) \\
&\overset{(a)}{\leq} \sum_{j=1}^{i} [d(\pi(i+1), \pi(j)) + d(\pi(i+1), \pi(j+1))] \\
&= \sum_{j=1}^{i} d(\pi(i+1), \pi(j)) + \sum_{j=1}^{i} d(\pi(i+1), \pi(j+1)) \\
&= d(\pi(i+1), O_i) + d(\pi(i+1), O_{i+1}) - d(\pi(i+1), \pi(1)) \\
&\overset{(b)}{=} d(\pi(i+1), O_i) + d(\pi(i+1), O_i) - d(\pi(i+1), \pi(1)) \\
&< 2d(\pi(i+1), O_i),
\end{aligned}$$

where (a) holds by triangle inequality, and (b) holds since $d(\pi(i+1), \pi(i+1)) = 0$. □

Lemma 5. *For an optimal solution $O^* = \arg\max \mathcal{S}_+(O)$ to the Max-SSD problem, it holds*

$$d\left(\pi(i+1), O_i^*\right) \leq \frac{1 - p_{\pi(i+1)}}{p_{\pi(i+1)}} \sum_{j=1}^{i} \frac{p_{\pi(j+1)}}{1 - p_{\pi(j+1)}} d(\pi(j), \pi(j+1)),$$

for any $1 \leq i \leq n-1$.

Proof. Let $O^* = (\pi(i))_{i=1}^n$ be an optimal sequence, and let us denote $O' = (\ldots, \pi(i+1), \pi(i), \ldots)$ to be the sequence after swapping node pair $\{\pi(i), \pi(i+1)\}$. Then,

$$\begin{aligned}
\mathcal{S}_+(O') = &\sum_{j=1}^{i-2} p_{O^*_{j+1}} d(\pi(j+1), O^*_j) + p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i+1), O^*_{i-1}) \\
&+ p_{\pi(i)} p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i), O^*_{i-1} \cup \pi(i+1)) \\
&+ \sum_{j=i+1}^{n-1} p_{O^*_{j+1}} d(\pi(j+1), O^*_j),
\end{aligned}$$

and

$$\begin{aligned}
\mathcal{S}_+(O^*) = &\sum_{j=1}^{i-2} p_{O^*_{j+1}} d(\pi(j+1), O^*_j) + p_{\pi(i)} p_{O^*_{i-1}} d(\pi(i), O^*_{i-1}) \\
&+ p_{\pi(i)} p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i+1), O^*_{i-1} \cup \pi(i)) \\
&+ \sum_{j=i+1}^{n-1} p_{O^*_{j+1}} d(\pi(j+1), O^*_j).
\end{aligned}$$

Since $O^* = \arg\max \mathcal{S}_+(O)$ is an optimal sequence, it must hold that $\mathcal{S}_+(O^*) - \mathcal{S}_+(O') \geq 0$, that is,

$$\begin{aligned}
\mathcal{S}_+(O^*) - \mathcal{S}_+(O') = &\, p_{\pi(i)} p_{O^*_{i-1}} d(\pi(i), O^*_{i-1}) \\
&- p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i+1), O^*_{i-1}) \\
&+ p_{\pi(i)} p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i+1), O^*_{i-1}) \\
&- p_{\pi(i)} p_{\pi(i+1)} p_{O^*_{i-1}} d(\pi(i), O^*_{i-1}) \geq 0.
\end{aligned}$$

This is equivalent to

$$\begin{aligned}
&p_{\pi(i)} p_{O^*_{i-1}} (1 - p_{\pi(i+1)}) d(\pi(i), O^*_{i-1}) \\
&\geq p_{\pi(i+1)} p_{O^*_{i-1}} (1 - p_{\pi(i)}) d(\pi(i+1), O^*_{i-1}).
\end{aligned} \tag{11.5}$$

Since $p_{O^*_{i-1}} > 0$, we can cancel that term from both sides of Equation (11.5). We then divide both sides by $(1 - p_{\pi(i)}) \times (1 - p_{\pi(i+1)})$ and obtain

$$\frac{p_{\pi(i)}}{1 - p_{\pi(i)}} d(\pi(i), O^*_{i-1}) \geq \frac{p_{\pi(i+1)}}{1 - p_{\pi(i+1)}} d(\pi(i+1), O^*_{i-1}). \tag{11.6}$$

By taking the telescope sum of Equation (11.6) over i from $i = 2$ to $i = j$, we get

$$\sum_{i=2}^{j} \frac{p_{\pi(i)}}{1 - p_{\pi(i)}} d(\pi(i), \pi(i-1)) \geq \frac{p_{\pi(j+1)}}{1 - p_{\pi(j+1)}} d(\pi(j+1), O^*_{j-1}). \tag{11.7}$$

By adding $\frac{p_{\pi(j+1)}}{1 - p_{\pi(j+1)}} d(\pi(j+1), \pi(j))$ on both sides of Equation (11.7), we get

$$\sum_{i=2}^{j+1} \frac{p_{\pi(j)}}{1 - p_{\pi(i)}} d(\pi(i), \pi(i-1)) \geq \frac{p_{\pi(j+1)}}{1 - p_{\pi(j+1)}} d(\pi(j+1), O^*_j). \tag{11.8}$$

After moving $\frac{p_{\pi(j+1)}}{1-p_{\pi(j+1)}}$ to the left hand side of Equation (11.8) and exchanging the indexes of i and j, we get

$$d(\pi(i+1), O_i^*) \leq \frac{1-p_{\pi(i+1)}}{p_{\pi(i+1)}} \sum_{j=1}^{i} \frac{p_{\pi(j+1)}}{1-p_{\pi(j+1)}} d(\pi(j), \pi(j+1)),$$

which completes the proof. □

Corollary 4. *For an optimal sequence $O^* = \arg\max \mathcal{S}_+(O)$ to the MaxSSD problem, and assuming that $p_i \in [a, b]$, for all $i \in U$, it holds that $\mathcal{S}_+(O^*) \leq \frac{b(1-a)}{a(1-b)} \mathcal{H}(O^*)$.*

Proof. Let $O^* = (\pi(i))_{i=1}^n$ denote an optimal sequence. To proof this corollary it suffices to show that $d\left(\pi(i+1), O_i^*\right) \leq \frac{b(1-a)}{a(1-b)} d_L(O_{i+1}^*)$, for all $i \in [n-1]$.

Since we assume that $p_i \in [a, b]$, for all $i \in [n]$, we get

$$\frac{1-p_{\pi(i+1)}}{p_{\pi(i+1)}} \sum_{j=1}^{i} \frac{p_{\pi(j+1)}}{1-p_{\pi(j+1)}} d(\pi(j), \pi(j+1)) \leq \frac{b(1-a)}{a(1-b)} d_L(O_{i+1}^*). \tag{11.9}$$

Substituting Equation (11.9) into Lemma 5 leads to

$$d\left(\pi(i+1), O_i^*\right) \leq \frac{1-p_{\pi(i+1)}}{p_{\pi(i+1)}} \sum_{j=1}^{i} \frac{p_{\pi(j+1)}}{1-p_{\pi(j+1)}} d\left(\pi(j), \pi(j+1)\right) \leq \frac{b(1-a)}{a(1-b)} d_L(O_{i+1}^*),$$

which completes the proof. □

12 Additional preliminaries

In this section, we list a few useful lemmas that we are going to use repeatedly in the following sections. Since they are simple algebraic statements, we state them directly, without proof.

Lemma 6. *Let s_n be the sum of the the first n terms of a geometric series, namely $s_n = \sum_{i=0}^{n-1} ar^i$, where a is a constant, and r is any number such $0 < r < 1$. Then, $s_n = a\frac{1-r^n}{1-r}$.*

Lemma 7. *When r is a constant, $\lim_{n\to\infty} s_n = \frac{a}{1-r}$. In other words, when $n \to \infty$, it is $s_n = \frac{a}{1-r} - \Theta(r^n)$.*

Lemma 8 (Theorem 4.3, Chebyshev's inequality [12])**.** *Let $a_1 \leq \ldots \leq a_n$ and $b_1 \leq \ldots b_n$ be real numbers. Then, it holds*

$$\left(\sum_{i=1}^{n} a_i\right)\left(\sum_{i=1}^{n} b_i\right) \leq n \sum_{i=1}^{n} a_i b_i,$$

and equality holds when $a_1 = \ldots = a_n$ or $b_1 = \ldots = b_n$.

Lemma 9. $\left(1-\frac{1}{n}\right)^n = \frac{1}{e} - \Theta\left(\frac{1}{n}\right)$.

13 Omitted proofs from Section 6

Theorem 3. *Assume that $p_i = p < 1$, for all $i \in U$. For any integer τ, where $2 \le \tau \le n$, the BτI algorithm obtains a $(1 - p^{\tau-1} - p^{n-\tau} + p^n)$-approximation for MaxOHP.*

Proof. We start by introducing some additional notation. Let π be the ordering of U obtained from Algorithm 1, and O be the corresponding sequence. By definition,

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d\left(\pi(i), \pi(i+1)\right).$$

Let π^o be the optimal ordering for MaxOHP, and O^o be the corresponding optimal sequence. Hence,

$$\mathcal{H}(O^o) = \sum_{i=1}^{n-1} W_{O_i^o} d\left(\pi^o(i), \pi^o(i+1)\right).$$

Since $p_i = p$, for all $i \in U$, we notice that $p_{O_j} = p_{O_j^o} = p^j$. Hence, $W_{O_i} = W_{O_i^o} = \sum_{j=i+1}^{n} p^j$, after expansion, we get

$$\begin{aligned}\sum_{j=i+1}^{n} p^j &\overset{(a)}{=} p^{i+1}\frac{1-p^{n-i}}{1-p}\\ &= \frac{p^{i+1}}{1-p} - \frac{p^{n+1}}{1-p},\end{aligned}$$

where equality (a) holds by Lemma 6.

Recall that the BτI algorithm finds the optimal τ-item sequence that maximizes the following equation:

$$O_\tau = \underset{O_\tau \sqsubseteq \pi(U)}{\arg\max} \sum_{i=1}^{\tau-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right). \tag{13.1}$$

For the convenience in our calculations, we let

$$\ell_\pi = \sum_{i=1}^{\tau-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right).$$

Let us derive a lower bound on $\mathcal{H}(O)$ as follows:

$$\begin{aligned}\mathcal{H}(O) &= \sum_{i=1}^{n-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right) - \sum_{i=1}^{n-1} \frac{p^{n+1}}{1-p} d\left(\pi(i), \pi(i+1)\right)\\ &\ge \sum_{i=1}^{\tau-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right) - \sum_{i=1}^{\tau-1} \frac{p^{n+1}}{1-p} d\left(\pi(i), \pi(i+1)\right)\\ &= \ell_\pi - \frac{p^{n+1}}{1-p}\sum_{i=1}^{\tau-1} d(\pi(i), \pi(i+1)).\end{aligned} \tag{13.2}$$

Next, we derive an upper bound on $\mathcal{H}(O^o)$. Let $T = \left\lceil \frac{n}{\tau-1} \right\rceil$,

$$\begin{aligned}\mathcal{H}(O^o) &= \sum_{i=1}^{n-1} \frac{p^{i+1}}{1-p} d\left(\pi^o(i), \pi^o(i+1)\right) - \sum_{i=1}^{n-1} \frac{p^{n+1}}{1-p} d\left(\pi^o(i), \pi^o(i+1)\right)\\ &\le \sum_{i=1}^{n-1} \frac{p^{i+1}}{1-p} d\left(\pi^o(i), \pi^o(i+1)\right)\\ &\le \sum_{t=1}^{T} \sum_{i=(t-1)(\tau-1)+1}^{t\cdot(\tau-1)} \frac{p^{i+1}}{1-p} d(\pi^o(i), \pi^o(i+1))\\ &\overset{(a)}{\le} \ell_\pi(1 + p^{\tau-1} + \cdots + p^{(T-1)(\tau-1)})\\ &\le \frac{\ell_\pi}{1-p^{\tau-1}}.\end{aligned} \tag{13.3}$$

Notice that (a) holds due to the optimality of ℓ_π. Notice that $T\cdot(\tau-1)$ might be greater than n, we can hypothetically set $d(\pi^o(i), \pi^o(i+1)) = 0$ for $i \ge n$, for our analysis.

Combining Equations (13.2) and (13.3), we get

$$\begin{aligned}\frac{\mathcal{H}(O)}{\mathcal{H}(O^o)} &\ge (1-p^{\tau-1})\frac{\ell_\pi - \frac{p^{n+1}}{1-p}\sum_{i=1}^{\tau-1} d(\pi(i), \pi(i+1))}{\ell_\pi}\\ &= (1-p^{\tau-1})\left(1 - \frac{\frac{p^{n+1}}{1-p}\sum_{i=1}^{\tau-1} d(\pi(i), \pi(i+1))}{\sum_{i=1}^{\tau-1} \frac{p^{i+1}}{1-p} d\left(\pi(i), \pi(i+1)\right)}\right)\\ &\ge (1-p^{\tau-1})\left(1 - \frac{\frac{p^{n+1}}{1-p}\sum_{i=1}^{\tau-1} d(\pi(i), \pi(i+1))}{\sum_{i=1}^{\tau-1} \frac{p^{\tau}}{1-p} d\left(\pi(i), \pi(i+1)\right)}\right)\\ &= (1-p^{\tau-1})(1-p^{n-\tau+1})\\ &= 1 - p^{\tau-1} - p^{n-\tau+1} + p^n\\ &\ge 1 - p^{\tau-1} - p^{n-\tau} + p^n.\end{aligned}$$

We have demonstrated that the BτI algorithm provides an asymptotic $(1 - p^{\tau-1} - p^{n-\tau} + p^n)$-approximation. □

13.1 Proof of Theorem 4

To prove Theorem 4, we first analyze the property of the greedy matching M obtained from Algorithm 2.

Lemma 10. *Given a graph $G = (V, E)$ with $|V| = n$, let $w(S)$ denote the sum of edge weights in an edge subset $S \subseteq E$. Let M be the greedy matching of G, M_k denote the first k edges of M, and M_k^* be the maximum weighted size k matching of G. Then it holds that $w(M_k) \ge \frac{1}{2}w(M_k^*)$ for all $k \le \lfloor \frac{n}{2} \rfloor$.*

Proof. Assume there exists an edge $e \in M_k^* \setminus M_k$, then there must exist an edge $e' \in M_k$ such that e' share one end node with e and $w(e) < w(e')$, because otherwise the greedy matching algorithm will choose e instead of e'. Since each such e' can conflict at most two edges from M_k^*, we have that

$$2\sum_{e' \in M_k \setminus M_k^*} w(e') > \sum_{e \in M_k^* \setminus M_k} w(e),$$

and the lemma thus follows. □

Theorem 4. *Assume that $p_i = p$, for any $i \in U$. Let p be a function of n such that $\lim_{n\to\infty} p = 1$ and $\lim_{n\to\infty} p^n = 0$. Moreover, assume that $p = 1 - \frac{1}{t_n}$, where t_n is a increasing function of n and $t_n = o(n)$. The algorithm GM yields a $\left(\frac{3(e-1)}{16e^2} - \Theta\left(\frac{1}{t_n}\right)\right)$-approximation for MaxOHP.*

Proof. We let π be the ordering of U obtained from GM algorithm, and O be the corresponding sequence. Recall that

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d\left(\pi(i), \pi(i+1)\right). \tag{13.4}$$

Since p^i decreases as i increases, we have $p^i \geq p^{t_n}$, for all $i \leq t_n$. Hence, we obtain a simple lower bound on W_{O_i} for any $i \leq t_n$.

$$W_{O_i} = \sum_{j=i+1}^{n} p^j \geq \sum_{j=i+1}^{t_n} p^j \geq (t_n - i)p^{t_n}. \tag{13.5}$$

Recall that, the π we choose satisfies the following two properties, which we are soon going to use to obtain a lower bound of $\mathcal{H}(O)$.

(P1) $d((\pi(2i-1), \pi(2i))) \geq d((\pi(2i+1), \pi(2i+2)))$ for $i < \lfloor \frac{n}{2} \rfloor$, and

(P2) $d((\pi(2i), \pi(2i+1))) \geq \frac{1}{2} d((\pi(2i-1), \pi(2i)))$ for $i \leq \lfloor \frac{n}{2} \rfloor$.

Let us simplify $\mathcal{H}(O)$ by applying the above observation. In the following analysis, we define $\mathcal{T}_n := \lfloor (t_n - 1)/2 \rfloor$.

$$\begin{aligned}
\mathcal{H}(O) &\overset{(a)}{\geq} \sum_{i=1}^{\mathcal{T}_n} \left(W_{O_{2i-1}} d\left(\pi(2i-1), \pi(2i)\right) + W_{O_{2i}} d\left(\pi(2i), \pi(2i+1)\right) \right) \\
&\overset{(b)}{\geq} \sum_{i=1}^{\mathcal{T}_n} \left(W_{O_{2i-1}} + \frac{1}{2} W_{O_{2i}} \right) d\left(\pi(2i-1), \pi(2i)\right) \\
&\overset{(c)}{\geq} \frac{1}{\mathcal{T}_n} \sum_{i=1}^{\mathcal{T}_n} \left(W_{O_{2i-1}} + \frac{1}{2} W_{O_{2i}} \right) \sum_{i=1}^{\mathcal{T}_n} d\left(\pi(2i-1), \pi(2i)\right) \\
&\overset{(d)}{\geq} \frac{p^{t_n}}{\mathcal{T}_n} \sum_{i=1}^{\mathcal{T}_n} \left((t_n - 2i + 1) + \frac{1}{2}(t_n - 2i) \right) \sum_{i=1}^{\mathcal{T}_n} d\left(\pi(2i-1), \pi(2i)\right) \\
&= \frac{p^{t_n}}{\mathcal{T}_n} \sum_{i=1}^{\mathcal{T}_n} \left(-3i + 1 + \frac{3}{2} t_n \right) \sum_{i=1}^{\mathcal{T}_n} d\left(\pi(2i-1), \pi(2i)\right) \\
&= \left(p^{t_n}\right) \left(\frac{-3 - 3\mathcal{T}_n}{2} + 1 + \frac{3}{2} t_n \right) \sum_{i=1}^{\mathcal{T}_n} d\left(\pi(2i-1), \pi(2i)\right).
\end{aligned} \tag{13.6}$$

Inequality (a) holds as we decompose $\mathcal{H}(O)$ into two parts, the series of $W_{O_{2i-1}} d\left(\pi(2i-1), \pi(2i)\right)$ are related with the top-$\mathcal{T}_n$ matching, and the series of $W_{O_{2i}} d\left(\pi(2i), \pi(2i+1)\right)$ are related with the edges that connect this matching.

Inequality (b) holds according to the property **(P2)**.

Inequality (c) holds as we apply Lemma 8 by property **(P1)**.

Inequality (d) holds as we reformulate $W_{O_{2i-1}}$ and $W_{O_{2i}}$ according to Equation (13.5).

Let ℓ_{t_n} denote the largest weights of a path that consists of t_n nodes chosen from U, i.e.,

$$\ell_{t_n} = \max_{\pi} \sum_{i=1}^{t_n - 1} d(\pi(i), \pi(i+1)). \tag{13.7}$$

Notice that as $\{\pi(2i-1), \pi(2i)\}_{i=1}^{\mathcal{T}_n}$ is a greedy matching of the first $\mathcal{T}_n$ edges, by Lemma 10, it contains at least $\frac{1}{2}$ weights of the maximum weighted size $\mathcal{T}_n$ matching. Hence, it consists of at least $\frac{1}{4}$ of the largest weights of the path that consists of $2\mathcal{T}_n$ edges. We can get

$$\sum_{i=1}^{\mathcal{T}_n} d\left(\pi(2i-1), \pi(2i)\right) \geq \frac{1}{4} \frac{t_n - 2}{t_n - 1} \ell_{t_n},$$

where the factor $\frac{t_n - 2}{t_n - 1}$ is added as $2\mathcal{T}_n$ can either be $t_n - 1$ or $t_n - 2$.

We substitute the above result into Equation (13.6), and we get

$$\begin{aligned}
\mathcal{H}(O) &\geq \left(p^{t_n}\right) \left(\frac{-3 - 3\mathcal{T}_n}{2} + 1 + \frac{3}{2} t_n \right) \frac{1}{4} \frac{t_n - 2}{t_n - 1} \ell_{t_n} \\
&\overset{(a)}{>} \left(p^{t_n}\right) \left(\frac{3}{4} t_n - \frac{1}{2} \right) \frac{1}{4} \frac{t_n - 2}{t_n - 1} \ell_{t_n} \\
&= \left(p^{t_n}\right) \left(\frac{3}{16} t_n - \frac{1}{8} \right) \frac{t_n - 2}{t_n - 1} \ell_{t_n},
\end{aligned} \tag{13.8}$$

where inequality (a) holds by substituting an upper bound $\frac{t_n}{2}$ of $\mathcal{T}_n$ into $\mathcal{T}_n$,

We let π^o be the optimal ordering for the MaxOHP, and O^o be the corresponding optimal sequence. Hence,

$$\mathcal{H}(O^o) = \sum_{i=1}^{n-1} W_{O_i^o} d\left(\pi^o(i), \pi^o(i+1)\right).$$

We can obtain a simple upper bound on $W_{O_i^o}$ by

$$\begin{aligned}
W_{O_i^o} &= \sum_{j=i+1}^{n} p^j \\
&= \frac{p^{i+1}}{1-p} - \frac{p^{n+1}}{1-p} \\
&\leq \frac{p^{i+1}}{1-p} \\
&\overset{(a)}{=} p^{i+1} t_n \\
&\leq p^{\lfloor \frac{i+1}{t_n} \rfloor \cdot t_n} t_n,
\end{aligned} \tag{13.9}$$

where equality (a) holds as we can use $p = 1 - \frac{1}{t_n}$.

By letting $T = \left\lceil \frac{n}{t_n - 1} \right\rceil$ we can derive an upper bound of $\mathcal{H}(O^o)$ as follows

$$\begin{aligned}
\mathcal{H}(O^o) &\leq \sum_{t=0}^{T-1} \sum_{i=t\cdot(t_n-1)+1}^{(t+1)\cdot(t_n-1)} W_{A_i^o} d(\pi^o(i), \pi^o(i+1)) \\
&\overset{(a)}{\leq} \sum_{t=0}^{T-1} \sum_{i=t\cdot(t_n-1)+1}^{(t+1)\cdot(t_n-1)} p^{t\cdot t_n} t_n d(\pi^o(i), \pi^o(i+1)) \\
&\overset{(b)}{\leq} t_n \cdot \ell_{t_n} \sum_{t=0}^{T-1} p^{t\cdot t_n} \\
&\leq \frac{t_n \cdot \ell_{t_n}}{1 - p^{t_n}}.
\end{aligned} \tag{13.10}$$

where inequality (a) holds because of Equation (13.9) and inequality (b) holds because of Equation (13.7). We set $d(\pi^o(i), \pi^o(i+1)) = 0$ for $i \geq n$ for the first inequality to hold.

Combining Equations (13.8) and (13.10) leads to

$$\begin{aligned}
\frac{\mathcal{H}(O)}{\mathcal{H}(O^o)} &\geq \frac{(p^{t_n}) \cdot \left(\frac{3}{16}t_n - \frac{1}{8}\right)\frac{t_n-2}{t_n-1}\ell_{t_n}}{\frac{t_n \cdot \ell_{t_n}}{1-p^{t_n}}} \\
&= p^{t_n}(1-p^{t_n}) \cdot \left(\frac{3}{16} - \frac{1}{8t_n}\right)\frac{t_n-2}{t_n-1} \\
&= p^{t_n}(1-p^{t_n}) \cdot \left(\frac{3}{16} - \Theta\left(\frac{1}{t_n}\right)\right)\left(1-\Theta\left(\frac{1}{t_n}\right)\right) \\
&\overset{(a)}{=} \left(\frac{e-1}{e} + \Theta\left(\frac{1}{t_n}\right)\right)\left(\frac{1}{e} - \Theta\left(\frac{1}{t_n}\right)\right)\left(\frac{3}{16} - \Theta\left(\frac{1}{t_n}\right)\right)\left(1-\Theta\left(\frac{1}{t_n}\right)\right) \\
&= \frac{3(e-1)}{16e^2} - \Theta\left(\frac{1}{t_n}\right),
\end{aligned}$$

where (a) holds as we apply Lemma 9, and use

$$p^{t_n} = (1-\frac{1}{t_n})^{t_n} = \frac{1}{e} - \Theta\left(\frac{1}{t_n}\right).$$

Thus we have proven that GM is a $\left(\frac{3(e-1)}{16e^2} - \Theta\left(\frac{1}{t_n}\right)\right)$-approximation algorithm.

□

13.2 The case of an arbitrary ranking

Let us consider a very special case where the user examines all the items. In particular, we assume that $\lim_{n\to\infty} p = 1$ and $\lim_{n\to\infty} p^n = c$, where c is a non-zero constant. For this special case, we notice that the order of items no longer matters — even an arbitrary order will provide a constant-factor approximation to our problem. Intuitively, this is because for such a large value of p the user will examine all the items of U. Thus, we can apply Lemma 1, and obtain a constant-factor approximation algorithm for problem MaxSSD directly, without considering MaxOHP.

Theorem 6. *Assume that $p_i = p$, for all $i \in U$. Moreover, assume that $\lim_{n\to\infty} p = 1$ and $p^n = c - \frac{1}{\Theta(n)}$, where c is a constant. Then any ordering of the items of U yields a $\left(c - \frac{1}{\Theta(n)}\right)$-approximation for the MaxSSD problem.*

Proof. We let π be any ordering of U, and O be the corresponding sequence. We let π^* be the optimal order for the problem MaxSSD, and let O^* be the corresponding sequence. We have

$$\begin{aligned}
\mathcal{S}_+(O) &= \sum_{i=1}^{n-1} p_{O_{i+1}} d\left(\pi(i+1), O_i\right) \\
&\geq \sum_{i=1}^{n-1} \left(c - \Theta\left(\frac{1}{n}\right)\right) d\left(\pi(i+1), O_i\right) \\
&= \left(c - \Theta\left(\frac{1}{n}\right)\right) \sum_{i=1}^{n-1} d\left(\pi(i+1), O_i\right) \\
&\overset{(a)}{\geq} \left(c - \Theta\left(\frac{1}{n}\right)\right) \mathcal{S}_+(O^*),
\end{aligned}$$

where inequality (a) holds because $\mathcal{S}_+(O^*)$ is not larger than the sum of all pairs of diversities.

Hence,

$$\frac{\mathcal{S}_+(O)}{\mathcal{S}_+(O^*)} \geq c - \frac{1}{\Theta(n)}.$$

□

14 Omitted proofs from Section 7

Theorem 5. *Let $p_i \in [a, b]$ for all $i \in U$, where $0 < a \leq b < 1$. Let τ be an integer such that $2 \leq \tau \leq n$. The BrI algorithm provides a $\frac{a^2(1-b)(1-b^{\tau-1})}{a^2+(\tau-1)b^{\tau+1}}$-approximation guarantee for MaxOHP.*

Proof. Let π be the ordering of U obtained from our algorithm. Let $O_\pi = (\pi(i))_{i=1}^n$ denote the ordered sequence according to π and write $O = O_\pi$. Recall that

$$\mathcal{H}(O) = \sum_{i=1}^{n-1} W_{O_i} d\left(\pi(i), \pi(i+1)\right).$$

Let π^o be the optimal ordering for the MaxOHP. Let $O_{\pi^o} = (\pi^o(i))_{i=1}^n$ and write $O^o = O_{\pi^o}$. For the optimal ordering O^o, let $\mathcal{H}(O^o) = \sum_{i=1}^{n-1} W_{O_i^o} d\left(\pi^o(i), \pi^o(i+1)\right)$.

Recall that

$$W_{O_i} := \sum_{j=i+1}^{n} p_{O_j}.$$

Hence,

$$\begin{aligned}
\mathcal{H}(O) &= \sum_{i=1}^{n-1} W_{O_i} d\left(\pi(i), \pi(i+1)\right) \\
&\geq \sum_{i=1}^{\tau-1} W_{O_i} d\left(\pi(i), \pi(i+1)\right) \\
&\geq \sum_{i=1}^{\tau-1} \sum_{j=i+1}^{\tau} p_{O_j} d\left(\pi(i), \pi(i+1)\right).
\end{aligned}$$

We define

$$\ell_\tau = \sum_{i=1}^{\tau-1} \sum_{j=i+1}^{\tau} p_{O_j} d\left(\pi(i), \pi(i+1)\right).$$

It is not hard to see that ℓ_τ is the function value maximized in Equation (7.1). By setting $T = \left\lceil \frac{n-1}{\tau-1} \right\rceil$, we can rewrite $\mathcal{H}(O^o)$ as follows

$$\begin{aligned}
\mathcal{H}(O^o) &= \sum_{i=1}^{n-1} W_{O_i^o} d\left(\pi^o(i), \pi^o(i+1)\right) \\
&= \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} p_{O_j^o} d\left(\pi^o(i), \pi^o(i+1)\right) \\
&= \sum_{t=1}^{T} \sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=i+1}^{n} p_{O_j^o} d\left(\pi^o(i), \pi^o(i+1)\right) \\
&= \sum_{t=1}^{T} \sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \left(\sum_{j=i+1}^{t\cdot\tau} p_{O_j^o} + \sum_{j=t\cdot\tau+1}^{n} p_{O_j^o}\right) d\left(\pi^o(i), \pi^o(i+1)\right)
\end{aligned}$$

Notice that for each fixed t, $\mathcal{H}(O^o)$ is the sum of two parts:

$$\sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=i+1}^{t\cdot\tau} p_{O_j^o} d\left(\pi^o(i), \pi^o(i+1)\right) \tag{14.1}$$

$$\sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=t\cdot\tau+1}^{n} p_{O^o_j} d\left(\pi^o(i), \pi^o(i+1)\right) \tag{14.2}$$

Let us first discuss the part (14.1)

$$\begin{aligned}\sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=i+1}^{t\cdot\tau} & p_{O^o_j} d\left(\pi^o(i), \pi^o(i+1)\right) \\ &\overset{(a)}{\leq} \ell_\tau p_{O^o_{(t-1)(\tau-1)}} \\ &\leq \ell_\tau b^{(t-1)(\tau-1)}.\end{aligned} \tag{14.3}$$

Similarly, for part (14.2)

$$\begin{aligned}\sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=t\cdot\tau+1}^{n} & p_{O^o_j} d\left(\pi^o(i), \pi^o(i+1)\right) \\ &\overset{(b)}{\leq} \sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \sum_{j=t\cdot\tau+1}^{n} b^j d\left(\pi^o(i), \pi^o(i+1)\right) \\ &\leq \sum_{i=(t-1)(\tau-1)+1}^{t(\tau-1)} \frac{b^{t\cdot\tau+1}}{1-b} d\left(\pi^o(i), \pi^o(i+1)\right) \\ &\leq (\tau-1) d_{\max} \frac{b^{t\cdot\tau+1}}{1-b},\end{aligned} \tag{14.4}$$

where $d_{\max} := \max_{u,v\in U} d(u,v)$. Inequality (a) holds by the definition of ℓ_τ and our algorithm, and inequality (b) holds because $p_{O^o_j} \leq b^j$.

We combine Equations (14.3) and (14.4) and we continue to bound $\mathcal{H}(O^o)$:

$$\begin{aligned}\mathcal{H}(O^o) &\leq \sum_{t=1}^{T} \left(\ell_\tau b^{(t-1)(\tau-1)} + (\tau-1) d_{\max} \frac{b^{t\cdot\tau+1}}{1-b}\right) \\ &\leq \frac{\ell_\tau}{1-b^{\tau-1}} + (\tau-1) d_{\max} \frac{b^{\tau+1}}{(1-b)(1-b^\tau)},\end{aligned} \tag{14.5}$$

Next, we derive an upper bound of the second term of Equation (14.5) in terms of ℓ_τ. Let $\hat{\pi}$ be any order, which places the edge with edge weight $d_{\max}$ to the first two positions. Let $\hat{O}$ be the corresponding sequence. We have:

$$\ell_\tau \overset{(a)}{\geq} \ell_2 \overset{(b)}{\geq} p_{\hat{O}_2} d\left(\hat{\pi}(1), \hat{\pi}(2)\right) \geq a^2 d_{\max}. \tag{14.6}$$

Notice that inequality (a) holds as ℓ_τ should consist of at least one edge, and inequality (b) holds because of the optimality of ℓ_τ. Thus,

$$\begin{aligned}\frac{\mathcal{H}(O)}{\mathcal{H}(O^o)} &\geq \frac{1}{\frac{1}{1-b^{\tau-1}} + \frac{\tau-1}{a^2} \cdot \frac{b^{\tau+1}}{(1-b)(1-b^\tau)}} \\ &= \frac{a^2(1-b^{\tau-1})(1-b)(1-b^\tau)}{a^2(1-b)(1-b^\tau) + (\tau-1)b^{\tau+1}(1-b^{\tau-1})} \\ &> \frac{a^2(1-b^{\tau-1})(1-b)(1-b^\tau)}{a^2(1-b)(1-b^\tau) + (\tau-1)b^{\tau+1}(1-b^\tau)} \\ &= \frac{a^2(1-b^{\tau-1})(1-b)}{a^2(1-b) + (\tau-1)b^{\tau+1}} \\ &> \frac{a^2(1-b)(1-b^{\tau-1})}{a^2 + (\tau-1)b^{\tau+1}}\end{aligned}$$

This completes the proof. □

15 More experimental results

In this section, we present more experimental results. We evaluate the performance of the proposed BrI algorithm and the BrI-H heuristic, by running these methods in a continuation probabilities set in small, large and full regime. We present our results in Table 4, Table 5 and Table 6. We also report the *Expnum*, *ExpDCG* and expected *ExpSerendipity* values when continuation probabilities are mapped to small, large and full regimes in Table 7, Table 8 and Table 9 respectively. We present the running time of our methods and the baselines on different datasets in Table 10. All results marked with * are obtained from the BrI-H heuristic

Table 4: Sequential sum diversity $\mathcal{S}_+(O)$ with item continuation probability mapped to $[0.1, 0.3]$.The results marked with * are obtained using the BτI-H heuristic

	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
Coat	0.039 ± 0.020	0.103 ± 0.041	0.102 ± 0.038	0.095 ± 0.039	0.105 ± 0.041	0.105 ± 0.041	**0.109 ± 0.042**	0.106 ± 0.038	0.104* ± 0.037*
KuaiRec	0.022 ± 0.006	0.050 ± 0.000	0.033 ± 0.008	0.042 ± 0.006	0.051 ± 0.010	0.044 ± 0.006	**0.058 ± 0.010**	**0.058 ± 0.009***	**0.058 ± 0.009***
Netflix	0.063 ± 0.026	0.152 ± 0.026	0.145 ± 0.026	0.161 ± 0.024	0.162 ± 0.024	0.162 ± 0.024	**0.163 ± 0.024**	0.160 ± 0.025	0.159* ± 0.025*
Movielens	0.059 ± 0.028	0.173 ± 0.013	0.169 ± 0.018	**0.183 ± 0.003**	**0.183 ± 0.003**	**0.183 ± 0.003**	**0.183 ± 0.003**	**0.183 ± 0.003***	**0.183 ± 0.003***
Yahoo	0.016 ± 0.027	0.133 ± 0.044	**0.148 ± 0.044**	**0.148 ± 0.044**	**0.148 ± 0.044**	**0.148 ± 0.044**	**0.148 ± 0.044**	0.140 ± 0.045*	0.139 ± 0.045*
LETOR	0.008 ± 0.008	0.026 ± 0.021	**0.043 ± 0.028**	0.039 ± 0.026	0.040 ± 0.026	0.040 ± 0.026	**0.043 ± 0.028**	0.042 ± 0.027	0.042 ± 0.027
LTRC	0.019 ± 0.011	0.041 ± 0.018	0.054 ± 0.020	0.051 ± 0.020	0.051 ± 0.019	0.051 ± 0.019	**0.055 ± 0.020**	0.054 ± 0.020	0.054 ± 0.020

Table 5: Sequential sum diversity $\mathcal{S}_+(O)$ with item continuation probability mapped to $[0.7, 0.9]$.The results marked with * are obtained using the BτI-H heuristic

	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
Coat	9.703 ± 3.704	23.044 ± 12.390	22.251 ± 11.328	18.586 ± 8.481	24.151 ± 13.421	**24.213 ± 13.748**	23.030 ± 12.385	22.839 ± 12.428*	22.826 ± 12.415*
KuaiRec	5.737 ± 1.094	10.668 ± 1.874	11.599 ± 3.289	9.119 ± 1.090	**14.654 ± 3.874**	12.565 ± 4.258	10.920 ± 1.728	10.906 ± 1.714*	10.968 ± 1.726*
Netflix	16.901 ± 5.588	47.316 ± 14.231	39.696 ± 9.856	35.729 ± 10.010	48.595 ± 14.424	**50.530 ± 15.407**	47.224 ± 14.648	47.053 ± 14.294*	47.172 ± 14.266*
Movielens	14.677 ± 4.644	65.410 ± 6.715	48.412 ± 4.246	63.242 ± 8.458	69.341 ± 7.604	**71.362 ± 7.006**	71.303 ± 7.748	71.305 ± 7.738*	71.317 ± 7.703*
Yahoo	4.014 ± 3.259	40.505 ± 19.773	38.251 ± 18.866	38.520 ± 19.221	39.134 ± 19.191	38.377 ± 19.021	41.677 ± 21.090	41.924 ± 20.996*	**41.969 ± 21.010***
LETOR	2.784 ± 1.638	5.825 ± 3.450	5.317 ± 2.755	6.140 ± 2.825	**6.523 ± 3.803**	6.025 ± 3.637	6.034 ± 3.006	6.086 ± 3.033	6.165 ± 3.048
LTRC	4.757 ± 1.339	7.026 ± 2.161	6.332 ± 1.691	7.088 ± 1.958	**7.385 ± 2.352**	7.204 ± 2.164	6.972 ± 1.971	7.005 ± 1.990	7.058 ± 2.012

Table 6: Sequential sum diversity $\mathcal{S}_+(O)$ with item continuation probabilities mapped to $[0.1, 0.9]$.The results marked with * are obtained using the BτI-H heuristic

	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
Coat	0.555±0.722	7.665±0.10.628	8.409±10.708	9.660±13.374	**9.661±13.608**	9.559±13.534	9.075±12.593	9.119±12.67*7	9.100±12.686*
KuaiRec	0.255±0.191	1.140±0.554	4.808±3.282	6.388±4.981	**6.768±5.221**	6.739±5.140	3.726±2.575	3.745±2.570*	3.834±2.617*
Netflix	1.704±1.822	27.252±19.761	22.197±12.660	32.270±22.224	32.703±22.895	**33.157±23.552**	30.790±21.842	30.848±21.822*	30.915±21.813*
Movielens	1.157±1.283	61.736±9.103	37.570±5.342	59.667±13.080	68.657±9.789	66.995±13.290	69.476±11.786	69.478±11.780*	**69.484±11.759***
Yahoo	0.394±0.927	26.104±24.181	21.018±20.291	22.055±20.980	26.303±23.451	21.039±20.324	27.567±26.163	27.709±26.100*	**27.746±26.105***
LETOR	0.127 ± 0.137	0.436 ± 0.419	0.583 ± 0.478	0.595 ± 0.491	0.612 ± 0.521	0.592 ± 0.510	0.610 ± 0.515	0.608 ± 0.516	**0.615 ± 0.520**
LTRC	0.189 ± 0.216	0.702 ± 0.640	1.050 ± 0.865	1.096 ± 0.914	**1.136 ± 0.996**	**1.136 ± 1.000**	1.091 ± 0.899	1.099 ± 0.911	1.112 ± 0.928

Table 7: *Expnum*, *ExpDCG* and *ExpSerendipity* values with item continuation probabilities mapped to $[0.1, 0.3]$. The results with * are obtained using the BτI-H heuristic

Metrics	Datasets	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
ExpDCG	Coat	0.040 ± 0.020	0.077±0.027	**0.082 ± 0.026**	0.078 ± 0.025	**0.082 ± 0.026**	**0.082 ± 0.025**	0.075 ± 0.026	0.076 ± 0.026*	0.076 ± 0.026*
	KuaiRec	0.040 ± 0.012	0.080±0.017	**0.101 ± 0.013**	0.085 ± 0.010	0.091 ± 0.014	0.085 ± 0.010	0.070 ± 0.023	0.090 ± 0.011*	0.091 ± 0.011*
	Netflix	0.054 ± 0.020	0.095±0.017	**0.105 ± 0.012**	**0.105 ± 0.012**	**0.105 ± 0.012**	**0.105 ± 0.012**	0.103 ± 0.012	0.104 ± 0.012*	0.104 ± 0.012*
	Movielens	0.054 ± 0.025	0.107±0.010	**0.112 ± 0.001**	**0.112 ± 0.001**	**0.112 ± 0.001**	**0.112 ± 0.001**	**0.112 ± 0.001**	**0.112 ± 0.001***	**0.112 ± 0.001***
	Yahoo	0.044 ± 0.026	0.090±0.027	**0.099 ± 0.022**	**0.099 ± 0.022**	**0.099 ± 0.022**	**0.099 ± 0.022**	0.095 ± 0.025	**0.099 ± 0.022***	**0.099 ± 0.022***
	LETOR	0.022 ± 0.022	0.030±0.028	0.050 ± 0.038	0.065 ± 0.036	0.065 ± 0.036	**0.066 ± 0.036**	0.050 ± 0.038	0.063 ± 0.035	0.063 ± 0.035
	LTRC	0.033 ± 0.019	0.042±0.024	0.056 ± 0.024	**0.070 ± 0.026**	**0.070 ± 0.026**	0.069 ± 0.025	0.056 ± 0.024	0.064 ± 0.025	0.064 ± 0.025
ExpSerendipity	Coat	0.013 ± 0.020	0.029±0.035	0.022 ± 0.029	0.017 ± 0.025	0.019 ± 0.026	0.019 ± 0.026	0.037 ± 0.038	**0.041 ± 0.041***	**0.041 ± 0.041***
	Netflix	0.005 ± 0.015	0.014±0.027	**0.019 ± 0.033**	**0.019 ± 0.032**	**0.019 ± 0.032**	**0.019 ± 0.032**	0.018 ± 0.031	0.017 ± 0.030*	0.017 ± 0.030*
	Movielens	0.006 ± 0.018	0.018±0.029	**0.019 ± 0.032**	0.013 ± 0.026	0.013 ± 0.026	0.013 ± 0.026	0.013 ± 0.026	0.013 ± 0.026*	0.013 ± 0.026*
	Yahoo	0.003 ± 0.012	0.020±0.021	0.021 ± 0.017	0.019 ± 0.015	0.019 ± 0.015	0.019 ± 0.015	**0.027 ± 0.023**	0.017 ± 0.014*	0.017 ± 0.014*

Table 8: *Expnum*, *ExpDCG* and *ExpSerendipity* values with item continuation probabilities mapped to $[0.7, 0.9]$. The results with * are obtained using the BτI-H heuristic

Metrics	Datasets	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
ExpDCG	Coat	1.564 ± 0.275	2.267±0.534	2.324 ± 0.532	2.101 ± 0.437	2.308 ± 0.540	**2.346 ± 0.555**	2.219 ± 0.508	2.233 ± 0.512*	2.232 ± 0.511*
	KuaiRec	1.565 ± 0.162	2.034±0.174	2.789 ± 0.308	1.772 ± 0.104	2.416 ± 0.443	**2.905 ± 0.350**	1.772 ± 0.145	1.887 ± 0.130*	1.891 ± 0.131*
	Netflix	1.889 ± 0.280	2.881±0.365	2.880 ± 0.308	2.681 ± 0.290	**3.003 ± 0.350**	2.993 ± 0.349	2.841 ± 0.338	2.855 ± 0.332*	2.865 ± 0.330*
	Movielens	1.782 ± 0.287	3.280±0.101	3.141 ± 0.069	3.213 ± 0.109	3.282 ± 0.097	**3.327 ± 0.075**	3.286 ± 0.097	3.287 ± 0.096*	3.288 ± 0.093*
	Yahoo	1.633 ± 0.405	**2.704±0.605**	2.636 ± 0.556	2.637 ± 0.556	2.644 ± 0.557	2.637 ± 0.556	2.653 ± 0.578	2.698 ± 0.553*	2.703 ± 0.553*
	LETOR	1.228 ± 0.281	1.578±0.443	1.402 ± 0.355	1.617 ± 0.411	1.748 ± 0.564	**1.834 ± 0.575**	1.400 ± 0.341	1.523 ± 0.365	1.545 ± 0.373
	LTRC	1.438 ± 0.235	1.659±0.278	1.562 ± 0.253	1.744 ± 0.281	**1.833 ± 0.341**	1.814 ± 0.310	1.567 ± 0.253	1.615 ± 0.259	1.637 ± 0.263
ExpSerendipity	Coat	0.924 ± 0.570	1.766±1.091	1.583 ± 0.895	1.862 ± 0.843	1.585 ± 0.895	1.508 ± 0.898	1.879 ± 0.993	**2.054 ± 1.122***	2.040 ± 1.117*
	Netflix	0.280 ± 0.386	0.965±0.892	1.187 ± 0.900	**1.336 ± 0.852**	0.830 ± 0.848	1.042 ± 0.952	1.290 ± 0.936	1.287 ± 0.942*	1.285 ± 0.949*
	Movielens	0.289 ± 0.422	**1.253±1.078**	1.073 ± 0.984	1.152 ± 0.1053	1.201 ± 1.151	1.183 ± 1.146	1.209 ± 1.097	1.209 ± 1.097*	**1.210 ± 1.097***
	Yahoo	0.229 ± 0.300	2.754±1.380	3.858 ± 1.547	3.857 ± 1.546	3.833 ± 1.537	3.852 ± 1.540	**3.859 ± 1.563**	3.675 ± 1.659*	3.643 ± 1.668*

Table 9: *Expnum*, *ExpDCG* and *ExpSerendipity* values with item continuation probabilities mapped to $[0.1, 0.9]$. The results with * are obtained using the BτI-H heuristic

Metrics	Datasets	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
ExpDCG	Coat	0.293 ± 0.255	1.269±0.832	1.370 ± 0.896	**1.393 ± 0.932**	1.381 ± 0.919	1.387 ± 0.924	1.264 ± 0.881	1.269 ± 0.890*	1.317 ± 0.899*
	KuaiRec	0.299 ± 0.181	0.925±0.229	2.100 ± 0.658	**2.259 ± 0.760**	2.201 ± 0.743	2.161 ± 0.717	1.271 ± 0.436	1.399 ± 0.415*	1.453 ± 0.430*
	Netflix	0.581 ± 0.341	2.221±0.772	2.335 ± 0.674	**2.492 ± 0.777**	2.447 ± 0.765	2.481 ± 0.773	2.347 ± 0.749	2.368 ± 0.739*	2.378 ± 0.738*
	Movielens	0.525 ± 0.369	3.251±0.191	3.005 ± 0.129	**3.347 ± 0.083**	3.294 ± 0.119	3.281 ± 0.163	3.274 ± 0.157	3.274 ± 0.156*	3.275 ± 0.152*
	Yahoo	0.407 ± 0.423	2.074±1.064	1.991 ± 0.947	2.020 ± 0.944	**2.278 ± 0.995**	1.991 ± 0.947	2.051 ± 1.031	2.131 ± 0.984*	2.137 ± 0.986*
	LETOR	0.199 ± 0.150	0.304±0.219	0.377 ± 0.258	0.514 ± 0.282	0.513 ± 0.300	**0.523 ± 0.291**	0.376 ± 0.260	0.465 ± 0.253	0.472 ± 0.260
	LTRC	0.233 ± 0.220	0.433±0.359	0.667 ± 0.407	0.835 ± 0.502	0.841 ± 0.518	**0.855 ± 0.525**	0.666 ± 0.408	0.737 ± 0.433	0.751 ± 0.443
ExpSerendipity	Coat	0.124 ± 0.179	0.843±0.912	0.778 ± 0.869	0.815 ± 0.933	0.800 ± 0.890	0.774 ± 0.897	0.863 ± 0.909	0.903 ± 0.990*	**0.931 ± 1.007***
	Netflix	0.068 ± 0.164	0.715±0.790	0.836 ± 0.801	0.680 ± 0.774	0.869 ± 0.893	0.816 ± 0.889	0.871 ± 0.876	0.873 ± 0.881*	**0.876 ± 0.886***
	Movielens	0.055 ± 0.156	**1.265±1.125**	1.030 ± 0.931	0.744 ± 0.998	1.179 ± 1.134	1.164 ± 1.139	1.223 ± 1.099	1.223 ± 1.099*	1.223 ± 1.099*
	Yahoo	0.051 ± 0.143	1.842±1.588	2.461 ± 1.974	2.452 ± 1.975	2.143 ± 1.987	2.448 ± 1.954	**2.505 ± 2.043**	2.439 ± 2.076*	2.426 ± 2.078*

Table 10: Run times of proposed algorithms and baselines (sec). The results marked with * are obtained using the BτI-H heuristic

	Random	EXPLORE	DUM	MSD	MMR	DPP	B2I	B3I	B4I
Coat	0.83	2.81	0.97	1.08	1.24	0.96	1.41	4.70*	53.03*
KuaiRec	21.00	134.82	25.40	43.92	58.37	106.47	198.61	55.27*	283.62*
Netflix	10.76	180.60	14.51	14.71	14.51	50.67	181.57	72.79*	866.86*
Movielens	40.12	787.52	68.91	135.89	87.50	561.19	929.23	154.21*	1121.35*
Yahoo	53.78	1922.62	132.07	252.97	125.09	1316.29	2089.38	388.58*	3807.37*
LETOR	0.17	9.90	2.71	1.94	1.74	2.12	0.93	10.05	873.70
LTRC	0.45	28.30	6.52	4.98	4.97	5.95	2.24	42.11	3584.67

Paper B

Streaming Stochastic Submodular Maximization with On-Demand User Requests

Honglian Wang
KTH Royal Institute of Technology
Stockholm, Sweden

Sijing Tu
KTH Royal Institute of Technology
Stockholm, Sweden

Lutz Oettershagen
University of Liverpool
Liverpool, UK

Aristides Gionis
KTH Royal Institute of Technology
Stockholm, Sweden

Abstract

We explore a novel problem in streaming submodular maximization, inspired by the dynamics of news-recommendation platforms. We consider a setting where users can visit a news website at any time, and upon each visit, the website must display up to k news items. User interactions are inherently stochastic: each news item presented to the user is consumed with a certain acceptance probability by the user, and each news item covers certain topics. Our goal is to design a streaming algorithm that maximizes the expected total topic coverage. To address this problem, we establish a connection to submodular maximization subject to a matroid constraint. We show that we can effectively adapt previous methods to address our problem when the number of user visits is known in advance or linear-size memory in the stream length is available. However, in more realistic scenarios where only an upper bound on the visits and sublinear memory is available, the algorithms fail to guarantee any bounded performance. To overcome these limitations, we introduce a new online streaming algorithm that achieves a competitive ratio of $1/(8\delta)$, where δ controls the approximation quality. Moreover, it requires only a single pass over the stream, and uses memory independent of the stream length. Empirically, our algorithms consistently outperform the baselines.

1 Introduction

Streaming submodular maximization has been extensively studied under various feasibility constraints, including *matchoid* [7, 8, 10], *matroid* [9], *knapsack* [21, 22] and k-*set* [20] constraints. In the classical setting, the goal is to select a single subset of items from a data stream and output it when the stream ends. However, this classical framework falls short in many real-world scenarios, for example, when seeking to maximize *content coverage* in online news recommendation. In such a setting, users visit a news website multiple times and expect to receive a relevant set of articles on each visit. Meanwhile, new articles continuously arrive at the content-provider's server in a stream fashion. The long-term objective is to maximize the total content coverage across user visits. Existing streaming submodular maximization algorithms are ill-suited for this use case, as they are designed to produce a single solution and do not support multiple or on-demand user interactions during the stream.

Motivated by this limitation, we introduce a novel problem called *Streaming Stochastic Submodular Maximization with On-demand Requests* (*S3MOR*). While the setting is broadly applicable to streaming submodular maximization tasks with multiple output requirements, we focus on the news recommendation task for illustration. Therefore, in *S3MOR*, the data stream consists of incoming news items that the system aggregates into a personalized news website. Each item V_i covers a set

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

Table 1: Comparison of our algorithms. $N \in \mathbb{N}$ denotes the stream length, $k \in \mathbb{N}$ number of news items to present per user access, $T \in \mathbb{N}$ and $T' \in \mathbb{N}$ are the exact number and upper bound of user accesses, and $\delta \in \mathbb{N}$ an approximation parameter.

Algorithm	Competitive ratio	Time	Space	Response time
LMGREEDY	$1/2$	$\mathcal{O}(NT'k)$	$\Theta(N + kT)$	$\mathcal{O}(Nk)$
STORM	$1/4(T'-T+1)$	$\mathcal{O}(NT'k)$	$\mathcal{O}(T'k)$	$\mathcal{O}(T')$
STORM++	$1/(8\delta)$	$\mathcal{O}(NT'^2k/\delta)$	$\mathcal{O}(T'^2k/\delta)$	$\mathcal{O}(T')$

of topics and is associated with a probability p_i, representing the probability that a user will click on the item when it is presented. A user may visit the system arbitrarily often and at any time during the stream. At each visit, the system must select and present a subset of at most k items from those that have arrived in the stream up to that point. If a user clicks on item V_i, they are exposed to all topics associated with that item. The objective is to maximize the expected total number of unique topics covered by all user clicks across all visits. This setting is challenging due to its online character:

- **On-demand**: At any point, the system must be ready to output a subset of news items.
- **Irrevocability**: Once a subset is presented to the user, it cannot be modified.

To the best of our knowledge, no existing algorithm directly addresses the *S3MOR* problem.

We evaluate the performance of online streaming algorithms that make irrevocable decisions using the notion of *competitive ratio*. An algorithm is called *c-competitive* if, for every input stream σ, the value $\mathcal{A}(\sigma)$ achieved by algorithm $\mathcal{A}$ satisfies $\frac{\mathcal{A}(\sigma)}{\mathbf{OPT}(\sigma)} \geq c$, where $\mathbf{OPT}(\sigma)$ denotes the value of an optimal offline solution with full access to the input stream.

To evaluate the performance of our algorithms, we clarify the notions of *response time complexity*, *time complexity*, and *space complexity*.

Response-time complexity vs. time complexity. For the *S3MOR* problem, we assume there exists an oracle that can evaluate the objective function $f(S)$ for any feasible candidate set S. We define the *response time complexity* of an algorithm as the worst-case number of oracle calls needed to produce a recommendation result of size k after the user submits a request. This metric captures the latency users experience. In contrast, *time complexity* is defined as the worst-case number of oracle calls required by the algorithm over the entire stream.

Space complexity. In practice, all users share common storage for original news items. We distinguish between two storage types: *shared storage* containing original items accessible to all users (not counted toward space complexity), and *per-user storage* for additional memory each user needs to compute personalized recommendations, including document identifiers and user-item association probabilities. Throughout this paper, "space complexity" refers exclusively to per-user storage.

Our contributions. We reduce *S3MOR* to the problem of *streaming submodular maximization under a partition matroid constraint*, which allows us to leverage existing online streaming algorithms. Based on this reduction, we propose three algorithms and evaluate them across multiple dimensions: competitive ratio, space complexity, time complexity, and response time complexity. We state our main results as follows:

- We prove that provided sufficient memory to store the whole stream, a greedy algorithm, LMGREEDY, can achieve the best possible $1/2$ competitive ratio for the *S3MOR* problem.
- If the exact number T of user visits is known, the reduction preserves the competitive ratio of the underlying online streaming submodular algorithm.
- However, in realistic scenarios, the number of user visits T is unknown and we need to guess the number of visits as $T' > T$. For this case, we introduce an algorithm STORM, which has a competitive ratio of $\frac{1}{4(T'-T+1)}$.
- To achieve a better competitive ratio, we propose a second algorithm STORM++ that makes multiple concurrent guesses of T, maintaining one solution per guess. It then greedily aggregates outcomes across guesses. This algorithm achieves a competitive ratio of $1/(8\delta)$,

where δ is a tunable parameter that balances efficiency and solution quality. The space and time complexity increase by a factor of $\delta T'$ relative to the underlying streaming algorithm.

- STORM and STORM++ achieve worse competitive ratios than LMGREEDY, but they utilize much smaller space and have significantly better response time.

We summarize the results for all algorithms mentioned above in Table 1. We validate our approach through empirical experiments on both large-scale and small-scale real-world datasets. Our results show the effectiveness of our algorithms in terms of coverage quality and memory usage, in comparison to relevant baselines.

Notation and background. We assume familiarity with constrained submodular maximization and competitive analysis. We provide formal definitions in Appendix A.

Related work

Submodular optimization plays a fundamental role in machine learning, combinatorial optimization, and data analysis [13, 26, 29], capturing problems with diminishing returns, and finding applications in data summarization [34, 42], non-parametric learning [17, 45], recommendation systems [1, 33, 38, 44], influence maximization [24], and network monitoring [13]. The classical greedy algorithm achieves a $(1 - 1/e)$-approximation for monotone submodular maximization under cardinality constraints [39], and extends to matroid constraints [6, 12].

We study stochastic submodular coverage functions, where the objective depends on random realizations but selections must be made non-adaptively. This contrasts with adaptive submodularity [16, 18, 37, 41], which models sequential decision-making with feedback and supports strong guarantees for greedy strategies. Related work includes stochastic variants of set and submodular cover, where the objective itself is random; notable examples analyze adaptivity gaps and approximation bounds under oracle access [15, 19].

Classical greedy algorithms for (constrained) submodular optimization [27, 35, 39] assume random access to the full dataset, which is infeasible in large-scale or streaming settings. This challenge has motivated streaming algorithms that make irrevocable decisions under memory constraints [4, 5, 9, 22, 23, 36]. Buchbinder et al. [5] apply the preemption paradigm, i.e., replacing elements with better ones, to achieve a tight $(1/2 - \varepsilon)$-approximation for monotone functions under cardinality constraints. Sieve++ [23] uses thresholding to achieve a $(1/2 - \varepsilon)$-approximation under cardinality constraints using only $\mathcal{O}(k)$ memory. Extensions to matroid and partition constraints include [7, 8, 10]; for instance, Chekuri et al. [8] gives a $1/4p$-competitive online streaming algorithm for monotone p-matchoids. These approaches, however, do not handle stochastic item utilities or repeated, unpredictable user access. We address this gap by studying stochastic submodular coverage under partition constraints in a streaming model with limited memory and unknown user access requests.

To extend submodularity to sequences, Alaei et al. [1] introduce sequence-submodularity and sequence-monotonicity, showing that a greedy algorithm achieves a $(1 - 1/e)$-approximation, with applications in online ad allocation. Similarly, Ohsaka and Yoshida [40] propose k-submodular functions to model optimization over k disjoint subsets, providing constant-factor approximations for monotone cases. In contrast to their settings, our problem involves making irrevocable decisions as an incoming stream of items progresses, requiring each of the k subsets to be constructed incrementally rather than all at once after seeing the entire input.

Finally, diversity-aware recommendation and coverage problems are closely related, as many systems must recommend item sets rather than single items, motivating submodular approaches to maximize coverage and user satisfaction [2, 43]. Ashkan et al. [3] propose a greedy offline method for maximizing utility under a submodular diversity constraint. Yue and Guestrin [44] use submodular bandits for diversified retrieval, learning user interaction probabilities adaptively rather than relying on a known oracle, as in our setting.

2 Streaming submodular maximization with on-demand requests

Problem setting. Let $\mathcal{V} = \{V_1, V_2, \ldots, V_N\}$ be an ordered set of N news items appearing in the stream, where item V_i arrives at time step i. Each news item $V_i \in \mathcal{V}$ covers a subset of topics from a predefined set of topics $C = \{c_1, c_2, \ldots, c_d\}$, where d denotes the total number of topics. Formally,

$V_i \subseteq C$ for each $i \in [N]$. Additionally, each news item V_i is associated with a probability[1] $p_i \in [0, 1]$, modeling the probability that a user will click the item when presented to them. When a user clicks a news item, we say that the user is exposed to all topics in the news item.

Moreover, we assume that we observe a binary variable $\tau_i \in \{0, 1\}$ indicating whether the user accesses the system at time step i. Specifically, $\tau_i = 1$ denotes an access at time step i, and $\tau_i = 0$ otherwise. Whenever $\tau_i = 1$, the system should present at most k news items from the available news items $\{V_1, \ldots, V_i\}$. Now, suppose a user visits the system T times. The system outputs T sets $\mathcal{S}^1, \cdots, \mathcal{S}^T$, where $\mathcal{S}^t$ represents the items presented to the user at their t-th visit, for $t \in [T]$. We allow the same item to be presented at multiple visits; however, we treat each appearance as a distinct copy. We make this choice as the user has multiple chances to click on an item if the item is presented multiple times. Therefore, for any $t \neq s$, we have $\mathcal{S}^t \cap \mathcal{S}^s = \emptyset$. We define the set of all items presented to the user over the entire sequence of visits as $\mathcal{S} = \bigcup_{t=1}^{T} \mathcal{S}^t$.

Objective function. We define $f(\mathcal{S})$ as the expected number of topics the user covers after T visits, where the expectation is taken over click probabilities. Let $\sigma_j \in \{0, 1\}$ indicate whether topic c_j is covered, i.e., the user clicked at least one item that covers c_j from the shown sets $\mathcal{S}^1, \cdots, \mathcal{S}^T$. The probability $\Pr(\sigma_j = 0)$ that topic c_j remains uncovered equals the probability the user clicks no items covering c_j.

Since the user clicks on item V_i with probability p_i, the probability that they do not click on it is $1 - p_i$ and, thus, $\Pr(\sigma_j = 0) = \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)$, leading to the expectation $\mathbb{E}[\sigma_j] = 1 - \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)$. By linearity of expectation, the expected number of topics covered by a user, which we denote by $f(\mathcal{S})$, is given by

$$f(\mathcal{S}) = \mathbb{E}\left[\sum_{j=1}^{d} \sigma_j\right] = \sum_{j=1}^{d} \mathbb{E}[\sigma_j] = \sum_{j=1}^{d} \left(1 - \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i)\right). \tag{1}$$

Lemma 2.1. *$f(\mathcal{S})$ is a non-decreasing submodular set function.*

Problem definition. We formally define our new problem *streaming stochastic submodular maximization with on-demand requests* (*S3MOR*) below.

Problem 2.2 (*S3MOR*). *We define a news item stream $\mathbf{S}$ as a sequence of triples: $\mathbf{S} = ((V_1, p_1, \tau_1), \ldots, (V_N, p_N, \tau_N))$ where N is the (unknown) length of the stream. At each time step i, the system receives a triple (V_i, p_i, τ_i), where $V_i \in \mathcal{V}$, $p_i \in [0, 1]$, and $\tau_i \in \{0, 1\}$. Whenever $\tau_i = 1$, the system selects up to k news items from the set of items $\{V_1, \ldots, V_i\}$ received so far to present to the user. Let $\mathcal{S} = \{\mathcal{S}^1, \ldots, \mathcal{S}^T\}$ denote the sets of news items presented to the user over T accesses. The objective is to maximize the expected number of distinct topics the user is exposed to by the end of the stream, measured by the function $f(\mathcal{S})$.*

There are two key challenges in our streaming setting: (i) the system does not know in advance when or how many times a user will access it; and (ii) the system has to make decisions based on the information available so far, and these decisions are irrevocable. In the following, we show how the problem can be reduced to a submodular maximization problem subject to a partition matroid constraint. We then leverage this connection to develop solutions under various memory constraints.

Reduction to submodular maximization under a partition matroid constraint. We first reduce the *S3MOR* problem (Problem 2.2) to submodular maximization under a partition matroid constraint, which enables us to apply efficient streaming algorithms with provable guarantees. Let $r\colon t \mapsto r_t$ map a user's t-th access to its access time, and $(r_t)_{t=1}^{T}$ be the sequence of times at which the user accesses the system over an unknown time period. Whenever the user accesses the system, it presents up to k news items selected from all the news items available since the beginning of the stream, denoted by $\{V_i\}_{i=1}^{r_t}$.

[1]These probabilities can be interpreted as measures of user-item preference or engagement, and are typically estimated from historical click-through data using machine learning models such as logistic regression, factorization machines, or neural networks trained on user behavior patterns and item features. We assume they are static and remain constant over time.

Let V_i^t be a *copy* of the item V_i associated with the t-th visit. Let $\mathcal{V}^t$ be a collection of these copies until the t-th visit, namely, $\mathcal{V}^t = \{V_i^t\}_{i=1}^{r_t} = \{V_1^t, \ldots, V_{r_t}^t\}$. Without loss of generality, we treat each copy as a distinct item, i.e., $V_i^t \neq V_i^s$ for all $t \neq s$. Clearly, $\mathcal{V}^t \cap \mathcal{V}^s = \emptyset$ for $t \neq s$.

Moreover, let $\mathcal{S}^t \subseteq \mathcal{V}^t$ be the set of news items that the system presents to the user at their t-th visit. Let $\mathcal{S} = \bigcup_{q=1}^{T} \mathcal{S}^q$ be the union of all sets of presented items. Analogously to Equation (1), we adapt the function $f\colon 2^{\cup_{t=1}^{T}\mathcal{V}^t} \to \mathbb{R}$ such that $f(\mathcal{S})$ is the expected number of topics covered by the (copies) of news items that the user clicked on after $\mathcal{S}^1, \ldots, \mathcal{S}^T$ have been presented to the user.

Observation 2.3. *Let $(\cup_{t=1}^{T}\mathcal{V}^t, \mathcal{F})$ be a set system, where $\mathcal{F}$ is a collection of subsets of $\cup_{t=1}^{T}\mathcal{V}^t$, and $\mathcal{F}$ is constructed in the following way: for any set $X \subseteq \cup_{t=1}^{T}\mathcal{V}^t$, if $|X \cap \mathcal{V}^t| \leq k$ for any $1 \leq t \leq T$, then $X \in \mathcal{F}$. Observe that $(\cup_{t=1}^{T}\mathcal{V}^t, \mathcal{F})$ is a partition matroid. Problem 2.2 can be equivalently formulated as $\max_{X \in \mathcal{F}} f(X)$.*

Based on Observation 2.3, we develop a baseline algorithm for the Problem 2.2. Suppose that the system stores all the news items up to when the user accesses the system at the t-th time, and also has stored all the previous news item sets presented to the user, i.e., $\cup_{q=1}^{t-1}\mathcal{S}^q$. In this case, the stored news items up to time r_t constitute one partition of the matroid, i.e., $\mathcal{V}^t = \{V_i^t\}_{i=1}^{r_t}$. We then apply the local greedy algorithm by Fisher et al. [12], i.e., the system greedily selects k news items from $\mathcal{V}^t$ that maximize the marginal gain of the objective function with respect to $f(\cup_{q=1}^{t-1}\mathcal{S}^q)$. This algorithm does not require any knowledge of T, and has a competitive ratio of $1/2$. Since all incoming news items need to be stored, the memory usage is $\Omega(N + kT)$, and the running time is $\sum_{q=1}^{T} \mathcal{O}(|\mathcal{V}^q|k)$. The response time is $\mathcal{O}(Nk)$ since it selects k items iteratively from $\mathcal{O}(N)$ items. We present the algorithm in Algorithm 3 (LMGREEDY) in Appendix B.

Theorem 2.4. *Algorithm 3 for Problem 2.2 has a competitive ratio of $1/2$, space complexity $\Theta(N + kT)$, time complexity $\mathcal{O}(NTk)$, and response time complexity $\mathcal{O}(Nk)$. Moreover, the competitive ratio of $1/2$ is tight, i.e., for Problem 2.2, no streaming algorithm can achieve a competitive ratio better than $1/2$ without violating the irrevocability constraint.*

The drawback of LMGREEDY is that its memory usage depends linearly on the stream length, making it infeasible for large streams. Thus, we consider the setting where the system has limited memory and can store only a user-specific subset of news items.

3 Algorithms with limited memory

We now consider a more realistic setting where the system has limited memory. Our objective is to substantially reduce the memory usage and the response time complexity while achieving a bounded approximation ratio. We first establish that any algorithm achieving reasonably good performance must use at least $\Theta(Tk)$ memory.

To illustrate this lower bound, consider the following example: Suppose that a user makes T requests; the input stream contains more than Tk items, each containing one distinct topic and associated with acceptance probability of 1. At the end of the stream, the user sequentially submits T requests. If an algorithm is allocated with only $\Theta(T'k)$ memory for some $T' \leq T$, it can store at most $\Theta(T'k)$ items, thus, at most $\Theta(T'k)$ distinct topics. However, optimally, the user may access $\Theta(Tk)$ distinct topics. We can conclude that for this example, the competitive ratio is at most $\Theta(T'/T)$, which is arbitrarily poor when $T' \ll T$.

Since the exact number of requests T is typically unknown in advance, we assume the system has access to an upper bound $T' \geq T$ and is allocated $\Omega(kT')$ memory. This allocation is sufficient to store all candidate items throughout the user's interaction period. In practice, T' can be estimated from historical interaction data, allowing the system to set T' close to T within reasonable accuracy, ensuring both efficient memory utilization and meaningful performance guarantees.

3.1 A first memory-efficient streaming algorithm

Our first memory-efficient algorithm processes the incoming stream $\mathbf{S}$ as follows. First, it initializes T' empty candidate sets $\mathcal{A}^i$ for $i \in [T']$, all of which are initially *active*. Upon the arrival of a news item from the stream $\mathbf{S}$, for each active candidate set, the algorithm either adds the item to the set or

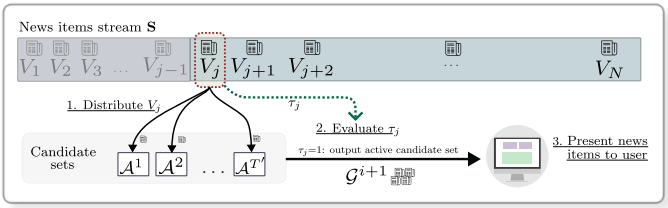


Figure 1: Schematic view of our algorithm STORM. We first initialize T' empty active candidate sets. For each incoming news item in the stream $\mathbf{S}$, we decide whether it can be added to/swapped into each of the active candidate sets. When a user submits a request, i.e., $\tau_j = 1$, we select the best active candidate set, present it to the user, and deactivate it.

Algorithm 1: STORM

Input: Estimated visits $T' \in \mathbb{N}$, budget $k \in \mathbb{N}$.

$\mathcal{A}^1, \cdots, \mathcal{A}^{T'} \leftarrow \emptyset$, $\mathcal{T} = \{1, \cdots, T'\}$, $\mathcal{G}^0 \leftarrow \emptyset$, $i \leftarrow 0$
for (V, τ, p) *in the stream* **do**
  for $j \in \mathcal{T}$ **do**
    if $|\mathcal{A}^j| < k$ **then**
      $\mathcal{A}^j \leftarrow \mathcal{A}^j \cup \{V\}$
    else
      $V' \leftarrow \arg\min_{V \in \mathcal{A}^j} \nu(f, \cup_{q \in [T']} \mathcal{A}^q, V)$
      if $f(V \mid \cup_{q \in [T']} \mathcal{A}^q) \geq 2\nu(f, \cup_{q \in [T']} \mathcal{A}^q, V')$ **then**
        $\mathcal{A}^j \leftarrow \mathcal{A}^j \setminus \{V'\} \cup \{V\}$
  if $\tau = 1$ **then**
    $j^* \leftarrow \arg\max_{j \in \mathcal{T}'} f(\mathcal{A}^j | \bigcup_{t=0}^{i} \mathcal{G}^t)$
    $\mathcal{T} \leftarrow \mathcal{T} \setminus \{j^*\}$ ▷ Deactivate candidate set $\mathcal{A}^{j*}$
    $\mathcal{G}^{i+1} \leftarrow \mathcal{A}^{j^*}$
    output $\mathcal{G}^{i+1}$
    $i \leftarrow i + 1$

replaces an existing item if the set is full. When a user accesses the system, the algorithm selects one of the active candidate sets to present to the user. The chosen candidate set is then marked as *inactivate* and will not be updated further. Figure 1 shows an overview.

Based on our reduction to submodular maximization under a partition matroid constraint, we can adapt any streaming algorithm designed for matroid constraints, provided it produces irrevocable outputs. Notable examples include the algorithms proposed by Chakrabarti and Kale [7], Chekuri et al. [8], and Feldman et al. [10], all of which achieve a $1/4$ approximation ratio. In our work, we specifically adapt the algorithm introduced by Chekuri et al. [8].

Our algorithm, named STORM, is shown in Algorithm 1. It keeps track of the number of visits using a counter i. Assume the user has visited i times when news item V_j arrives. This implies that, prior to the arrival of V_j, i active sets from i partitions have already been presented to the user, and thus, neither V_j nor any subsequent item will be added to these partitions. The algorithm iterates over the remaining $T' - i$ active candidate sets, and determines whether V_j should be added to any of them, with replacement if necessary.

The decision to replace an existing item V with the current item V_j is based on the comparison of their incremental value with respect to set $\cup_{t \in [T']} \mathcal{A}^t$. Specifically, we adopt the notation $\nu(f, \cup_{t \in [T']} \mathcal{A}^t, V)$ from Chekuri et al. [8] and define

$$\nu(f, \cup_{t \in [T']} \mathcal{A}^t, V) = f_{\widehat{\cup_{t \in [T']} \mathcal{A}^t}}(V), \text{ with } \widehat{\cup_{t \in [T']} \mathcal{A}^t} = \{V' \in \cup_{t \in [T']} \mathcal{A}^t : V' < V\}.$$

Algorithm 2: STORM++

Input: Estimated visits $T' \in \mathbb{N}$, budget $k \in \mathbb{N}$, parameter $\delta \in \mathbb{N}$.

1 $\mathcal{G}^0 \leftarrow \emptyset, \mathcal{B}^0 \leftarrow \emptyset, i \leftarrow 0, \mathcal{P} \leftarrow \left\{\delta, 2\delta, \ldots, \lceil \frac{T'}{\delta} \rceil \delta\right\}$

2 **for** $\tilde{T} \in \mathcal{P}$ **do** Initialize STORM $(\tilde{T}, k)$ ▷ Maintain STORM for different $\tilde{T}$

3 **for** (V, τ, p) *in the stream* **do**

4   **for** $\tilde{T} \in \mathcal{P}$ **do**

5         $\mathcal{G}^{i+1,(\tilde{T})} \leftarrow$ STORM $(\tilde{T}, k)$.`step`(V, τ, p) ▷ Process (V, τ, p) by executing lines 3-15 of Alg 1

6   **if** $\tau = 1$ **then**

7         $\tilde{T}^* \leftarrow \arg\max_{\tilde{T} \in \mathcal{P}} f(\mathcal{G}^{i+1,(\tilde{T})} \mid \bigcup_{j=0}^{i} \mathcal{B}^j)$

8         $\mathcal{B}^{i+1} \leftarrow \mathcal{G}^{i+1,(\tilde{T}^*)}$

9   **output** $\mathcal{B}^{i+1}$

10         $i \leftarrow i + 1$

Here, $V' < V$ indicates item V' is added to the set $\cup_{t \in [T']} \mathcal{A}^t$ before item V. For a specific set $\mathcal{A}^{t'}$, V_j replaces an item in $\mathcal{A}^{t'}$ if $f_{\cup_{t \in [T']} \mathcal{A}^t}(V_j) \geq 2 \cdot \min_{V \in \mathcal{A}^{t'}} \nu(f, \cup_{t \in [T']} \mathcal{A}^t, V)$.

Upon the t-th user visit, the system selects which active candidate set to output in a greedy manner. To be specific, let $\mathcal{G}^1, \cdots, \mathcal{G}^{t-1}$ denote the candidate sets that have already been presented to the user, and $\mathcal{T}$ represent the indices of the remaining active sets. The system selects the active set that offers the highest incremental gain with respect to the union of previously presented sets. Formally, let $j^* = \arg\max_{j \in \mathcal{T}} f(\mathcal{A}^j | \cup_{j=1}^{t-1} \mathcal{G}^j)$, and the system outputs $\mathcal{G}^t = \mathcal{A}^{j^*}$.

Theorem 3.1. *Let T be the number of user accesses and T' a given upper bound. Algorithm 1 has competitive ratio $\frac{1}{4(T'-T+1)}$, space complexity $\mathcal{O}(T'k)$, time complexity $\mathcal{O}(NkT')$, and response time complexity $\mathcal{O}$ (T').*

If the exact number of visits T is known, we can set the upper bound $T' = T$ and obtain a $1/4$-competitive ratio using Algorithm 1. However, the competitive ratio of Algorithm 1 can be arbitrarily bad if T' is significantly larger than T. Note that our analysis is tight. We can demonstrate that for any fixed T, there exists an example such that the competitive ratio of Algorithm 1 is at most $\frac{1}{T'-T+1}$, which is only a constant factor away from our analysis. We provide the proof in Appendix C.

3.2 A memory-efficient streaming algorithm with bounded competitive ratio

As shown in the previous section, to achieve a large enough competitive ratio with STORM, the system must have a good enough estimate of the number of user visits T. However, in practice, this information is typically unavailable in advance. To address this issue, we discretize the range $[T']$ to generate multiple guesses for the value of T, and run a separate instance of Algorithm 1 for each guess. When a user visits the system, each copy of Algorithm 1 outputs a solution, and we select the best solution over all the outputs.

Specifically, Algorithm 2 (named STORM++) first initializes the set of guesses of T as $\mathcal{P} = \{\delta i \mid i \in [\lceil T'/\delta \rceil]\}$, where $\delta \in \mathbb{N}$ is a parameter controlling the trade-off between competitive ratio as well as the space and time complexity. In this construction, there always exists a guess $\tilde{T} \in \mathcal{P}$ that is between T and $T + \delta$. Therefore, if we would run Algorithm 1 with $\tilde{T}$ as the input parameter, and present the results to the user, we are guaranteed to have a competitive ratio of at least $1/4\delta$ as shown in Theorem 3.1; however, we do not know the value of $\tilde{T}$ in advance. Thus, we adopt a greedy strategy to aggregate the results from all the copies of Algorithm 1 to determine an output for each user visit. At the i-th user visit, we collect the i-th output from each running copy of STORM with a different guess $\tilde{T} \in \mathcal{P}$ and select the one that maximizes the incremental gain. This selection rule leads to the following result.

Theorem 3.2. *Algorithm 2 has a competitive ratio of $1/(8\delta)$, space complexity $\mathcal{O}(T'^2 k/\delta)$, time complexity $\mathcal{O}(NkT'^2/\delta)$, and response time complexity $\mathcal{O}(T')$.*

4 Empirical evaluation

We empirically evaluate the performance of our proposed algorithms STORM and STORM++. We denote STORM(T) and STORM(T') as the STORM algorithm with the input parameter set to T (i.e., exact number of user accesses) and T' (i.e., upper bound of user accesses), resp.

We design our experiments to answer the following research questions: **(Q1)** How effective are the solutions provided by STORM++ and STORM(T') compared to other baselines and to each other? **(Q2)** What is the impact of parameters δ and k on the performance of STORM++ and STORM(T')? **(Q3)** How sensitive are STORM++ and STORM(T') to the upper bound T' and do our algorithms require highly-accurate estimation of T' to perform well?

We use the following **baselines**:

- LMGREEDY: Our upper-bound baseline, which assumes linear memory size in the stream length, i.e., potentially unbounded memory (Algorithm 3).
- SIEVE++: Based on Kazemi et al. [23], we implement a heuristic version of the SIEVE++ algorithm to solve the *S3MOR* problem. The adaptation proceeds as follows: (1) Before the first user visit, we run the exact SIEVE++ algorithm and obtain output set $\mathcal{S}^1$. (2) For each subsequent i-th output, we run SIEVE++ on the data stream between the $(i-1)$-th and i-th user visits, using the marginal gain function $f(\cdot|\bigcup_{j=1}^{i-1}\mathcal{S}^i)$ as the objective.
- PREEMPTION: We adapt the algorithm proposed by Buchbinder et al. [5] for solving the *S3MOR* problem in the same way as described above for SIEVE++.

Datasets: We use six real-world datasets, including four rating datasets KuaiRec, Anime, Beer and Yahoo, and two multi-label dataset RCV1 and Amazon. The number of items these dataset contains vary from thousands to one million, and the topic sets size vary from around 40 to around 4000. We provide the details in Appendix D.1.

Experimental setting: To simulate the news item stream, we randomly shuffle the items to form a stream of length N. To simulate the user visit sequence, we start with an all-zeros sequence and then randomly select T indices, and set those positions to one. We establish an upper bound on the number of visits by setting $T' = T + \Delta T$ for some positive ΔT. We generate synthetic click probabilities, and provide the details in Appendix D.1.

In all experiments, we randomly selected 50 users and report the average expected coverage for the Yahoo, RCV1, and Amazon datasets. Standard deviations and expected coverage results for the other three datasets are provided in Appendix D. We also report runtime and memory usage on the largest dataset, Amazon.

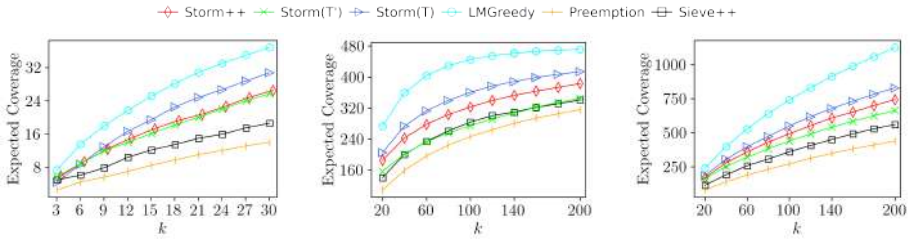
All experiments ran on a MacBook Air (Model Mac15,12) equipped with an Apple M3 chip (8-core: 4 performance and 4 efficiency cores), 16 GB of memory, and a solid-state drive (SSD). The source code and datasets are available.[2]

Results and discussion

Impact of k and T on the expected coverage. As shown in Figure 2, LMGREEDY achieves the best performance across all datasets and values of k. Among our proposed methods, STORM(T) consistently performs the best, followed by STORM++, then STORM(T'). Compared to baselines SIEVE++ and PREEMPTION, our algorithms generally perform better across datasets and values of k, with exceptions on RCV1, where PREEMPTION performs comparably to STORM(T'). In Figure 3, we observe a trend shift: STORM++ slightly outperforms STORM(T') and STORM(T) across datasets and T values. STORM(T') and STORM(T) perform comparably. This performance shift is due to smaller δ and ΔT in this experiment, which lead to improved competitive ratios for STORM(T') and STORM++. Although STORM(T) offers the strongest theoretical guarantee of the three algorithms, STORM(T') and STORM++ can empirically outperform it (we give an example of this effect in Appendix D.2).

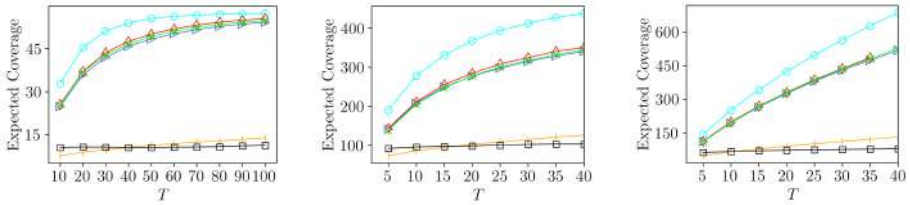
Performance regarding parameters δ and ΔT. Figure 4a shows that STORM++'s expected coverage decreases as δ increases, consistent with theory: as δ grows from 3 to 30, the performance guarantee

[2] https://github.com/HongLWang/Streaming-Submodular-maximization-with-on-demand-user-request



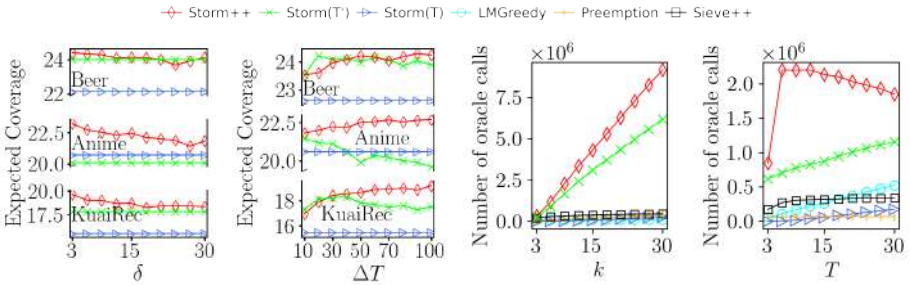
(a) Yahoo (b) RCV1 (c) Amazon

Figure 2: Empirical variation in expected coverage as a function of budget k on all datasets. Parameter $T = 5$, $\delta = 25$ and $\Delta T = 45$ are fixed



(a) Yahoo (b) RCV1 (c) Amazon

Figure 3: Empirical variation in expected coverage as a function of number of visits T on all datasets. Parameter $k = 10$, $\delta = 10$ and $\Delta T = 10$ are fixed.



(a) (b) (c) (d)

Figure 4: Left two: Impact of ΔT and δ on expected coverage. In (a) and (b), we fix $k = 5$ and $T = 10$. When varying δ, we fix $\Delta T = 60$; when varying ΔT, we fix $\delta = 10$. Right two: Oracle call counts on the Beer dataset. We fix $\delta = 25$ and $\Delta T = 45$. When varying k, we fix $T = 5$; when varying T, we fix $k = 5$.

Table 2: Runtime and mem. usage for the Amazon dataset. We set k, T, and δ to 10, $T' = 20$.

	STORM++	STORM(T')	STORM(T)	LMGREEDY	SIEVE++	PREEMPTION
Runtime (s)	35.13	7.94	4.38	88.13	307.43	136.04
Memory (MB)	2.00	1.28	0.72	28.99	0.91	0.03

drops by a factor of 10. This is reflected empirically with coverage declines across datasets, for example, from 130 to 120 on Amazon.

Figure 4b shows that STORM(T')'s performance declines with increasing ΔT, matching the theoretical drop in competitive ratio (e.g., 10-fold decrease from $\Delta T = 10$ to 100). In contrast, STORM++ is theoretically unaffected.

Number of oracle calls, runtime and memory. Figure 4c shows that the number of oracle calls increases rapidly and linearly with budget k for both STORM(T') and STORM++, while remaining

relatively stable for other algorithms. Similarly, Figure 4d indicates a linear increase in oracle calls with user visits T for STORM(T'), STORM(T), and LMGREEDY, but a much slower growth for SIEVE++ and PREEMPTION. Notably, Figure 4d also reveals a non-monotonic trend: oracle calls first rise, then decline with increasing T. This increase is due to the guessing strategy for T under $\delta = 25$ and $\Delta T' = 45$, where the number of guesses $\mathcal{T}$ increases ($\mathcal{T} = \{25, 50\}$ for $T = 3$ and $\mathcal{T} = \{25, 50, 75\}$ for $T \in [6, 30]$). The decrease is due to STORM++'s irrevocability—as more subsets are presented to the user, fewer items remain in $\mathcal{S}$ for potential swap, thereby reducing the number of subsequent oracle calls.

Table 2 shows that our proposed algorithms require much less memory than LMGREEDY, and use comparable memory to the other two baselines. STORM(T') and STORM(T) are also significantly faster than the remaining algorithms.

5 Conclusion

We investigate a novel online setting for streaming submodular maximization where users submit on-demand requests throughout the stream. In this context, we introduce the *Streaming Stochastic Submodular Maximization with On-demand Requests* (*S3MOR*) problem with the goal of maximizing the expected coverage of the selected items. We first propose a memory-efficient approximation algorithm that highlights the trade-off between memory usage and solution quality, but its competitive ratio can be poor. To solve this, we propose a parameter-dependent memory-efficient approximation algorithm to trade off between the competitive ratio and the memory and time usage. Our empirical evaluations show that our proposed algorithms consistently outperform the baselines. We believe our approach will have a positive social impact. For instance, it can be applied to enhance the diversity of news content recommended to the users, thereby helping to expose them to multiple viewpoints and broaden their knowledge. While our approach optimizes topic coverage and can be used to promote content diversity, its misuse, such as deliberately optimizing a submodular utility function that favors polarized or biased content, could risk amplifying existing biases. However, such outcomes would require intentional manipulation of the objective function. When applied responsibly, our approach does not pose such risks.

Acknowledgments and Disclosure of Funding

This research is supported by the ERC Advanced Grant REBOUND (834862), the Swedish Research Council project ExCLUS (2024-05603), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

[1] Saeed Alaei, Ali Makhdoumi, and Azarakhsh Malekian. 2021. Maximizing sequence-submodular functions and its application to online advertising. *Management Science* 67, 10 (2021), 6030–6054.

[2] Bushra Alhijawi, Arafat Awajan, and Salam Fraihat. 2022. Survey on the objectives of recommender systems: Measures, solutions, evaluation methodology, and new perspectives. *Comput. Surveys* 55, 5 (2022), 1–38.

[3] Azin Ashkan, Branislav Kveton, Shlomo Berkovsky, and Zheng Wen. 2015. Optimal Greedy Diversity for Recommendation.. In *IJCAI*, Vol. 15. 1742–1748.

[4] Ashwinkumar Badanidiyuru, Baharan Mirzasoleiman, Amin Karbasi, and Andreas Krause. 2014. Streaming submodular maximization: Massive data summarization on the fly. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 671–680.

[5] Niv Buchbinder, Moran Feldman, and Roy Schwartz. 2019. Online submodular maximization with preemption. *ACM Transactions on Algorithms (TALG)* 15, 3 (2019), 1–31.

[6] Gruia Calinescu, Chandra Chekuri, Martin Pal, and Jan Vondrák. 2011. Maximizing a monotone submodular function subject to a matroid constraint. *SIAM J. Comput.* 40, 6 (2011), 1740–1766.

[7] Amit Chakrabarti and Sagar Kale. 2015. Submodular maximization meets streaming: matchings, matroids, and more. *Mathematical Programming* 154 (2015), 225–247.

[8] Chandra Chekuri, Shalmoli Gupta, and Kent Quanrud. 2015. Streaming algorithms for submodular function maximization. In *Automata, Languages, and Programming: 42nd International Colloquium, ICALP 2015, Kyoto, Japan, July 6-10, 2015, Proceedings, Part I 42*. Springer, 318–330.

[9] Marwa El Halabi, Federico Fusco, Ashkan Norouzi-Fard, Jakab Tardos, and Jakub Tarnawski. 2023. Fairness in streaming submodular maximization over a matroid constraint. In *International Conference on Machine Learning*. PMLR, 9150–9171.

[10] Moran Feldman, Amin Karbasi, and Ehsan Kazemi. 2018. Do less, get more: Streaming submodular maximization with subsampling. *Advances in Neural Information Processing Systems* 31 (2018).

[11] Moran Feldman, Paul Liu, Ashkan Norouzi-Fard, Ola Svensson, and Rico Zenklusen. 2021. Streaming submodular maximization under matroid constraints. *arXiv preprint arXiv:2107.07183* (2021).

[12] Marshall L Fisher, George L Nemhauser, and Laurence A Wolsey. 1978. *An analysis of approximations for maximizing submodular set functions—II*. Springer.

[13] Satoru Fujishige. 2005. *Submodular functions and optimization*. Vol. 58. Elsevier.

[14] Chongming Gao, Shijun Li, Wenqiang Lei, Jiawei Chen, Biao Li, Peng Jiang, Xiangnan He, Jiaxin Mao, and Tat-Seng Chua. 2022. KuaiRec: A fully-observed dataset and insights for evaluating recommender systems. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 540–550.

[15] Michel Goemans and Jan Vondrák. 2006. Stochastic covering and adaptivity. In *Latin American symposium on theoretical informatics*. Springer, 532–543.

[16] Daniel Golovin and Andreas Krause. 2011. Adaptive submodularity: Theory and applications in active learning and stochastic optimization. *Journal of Artificial Intelligence Research* 42 (2011), 427–486.

[17] Ryan Gomes and Andreas Krause. 2010. Budgeted Nonparametric Learning from Data Streams.. In *ICML*, Vol. 1. Citeseer, 3.

[18] Alkis Gotovos, Amin Karbasi, and Andreas Krause. 2015. Non-Monotone Adaptive Submodular Maximization. In *IJCAI*. 1996–2003.

[19] Fabrizio Grandoni, Anupam Gupta, Stefano Leonardi, Pauli Miettinen, Piotr Sankowski, and Mohit Singh. 2008. Set covering with our eyes closed. In *2008 49th Annual IEEE Symposium on Foundations of Computer Science*. IEEE, 347–356.

[20] Ran Haba, Ehsan Kazemi, Moran Feldman, and Amin Karbasi. 2020. Streaming submodular maximization under a k-set system constraint. In *International Conference on Machine Learning*. PMLR, 3939–3949.

[21] Chien-Chung Huang, Naonori Kakimura, and Yuichi Yoshida. 2020. Streaming algorithms for maximizing monotone submodular functions under a knapsack constraint. *Algorithmica* 82, 4 (2020), 1006–1032.

[22] Chien-Chung Huang, Theophile Thiery, and Justin Ward. 2021. Improved multi-pass streaming algorithms for submodular maximization with matroid constraints. *arXiv preprint arXiv:2102.09679* (2021).

[23] Ehsan Kazemi, Marko Mitrovic, Morteza Zadimoghaddam, Silvio Lattanzi, and Amin Karbasi. 2019. Submodular streaming in all its glory: Tight approximation, minimum memory and low adaptive complexity. In *International Conference on Machine Learning*. PMLR, 3311–3320.

[24] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[25] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix factorization techniques for recommender systems. *Computer* 42, 8 (2009), 30–37.

[26] Andreas Krause and Daniel Golovin. 2014. Submodular function maximization. *Tractability* 3, 71-104 (2014), 3.

[27] Jure Leskovec, Andreas Krause, Carlos Guestrin, Christos Faloutsos, Jeanne VanBriesen, and Natalie Glance. 2007. Cost-effective outbreak detection in networks. In *Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining*. 420–429.

[28] David D Lewis, Yiming Yang, Tony G Rose, and Fan Li. 2004. Rcv1: A new benchmark collection for text categorization research. *Journal of machine learning research* 5, Apr (2004), 361–397.

[29] Yajing Liu, Edwin KP Chong, Ali Pezeshki, and Zhenliang Zhang. 2020. Submodular optimization problems and greedy strategies: A survey. *Discrete Event Dynamic Systems* 30, 3 (2020), 381–412.

[30] Julian McAuley and Jure Leskovec. 2013. Hidden factors and hidden topics: understanding rating dimensions with review text. In *Proceedings of the 7th ACM conference on Recommender systems*. 165–172.

[31] Julian McAuley, Jure Leskovec, and Dan Jurafsky. 2012. Learning attitudes and attributes from multi-aspect reviews. In *2012 IEEE 12th International Conference on Data Mining*. IEEE, 1020–1025.

[32] Julian John McAuley and Jure Leskovec. 2013. From amateurs to connoisseurs: modeling the evolution of user expertise through online reviews. In *Proceedings of the 22nd international conference on World Wide Web*. 897–908.

[33] Anay Mehrotra and Nisheeth K Vishnoi. 2023. Maximizing submodular functions for recommendation in the presence of biases. In *Proceedings of the ACM Web Conference 2023*. 3625–3636.

[34] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, and Amin Karbasi. 2016. Fast constrained submodular maximization: Personalized data summarization. In *International Conference on Machine Learning*. PMLR, 1358–1367.

[35] Baharan Mirzasoleiman, Ashwinkumar Badanidiyuru, Amin Karbasi, Jan Vondrák, and Andreas Krause. 2015. Lazier than lazy greedy. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29.

[36] Baharan Mirzasoleiman, Stefanie Jegelka, and Andreas Krause. 2018. Streaming non-monotone submodular maximization: Personalized video summarization on the fly. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32.

[37] Marko Mitrovic, Ehsan Kazemi, Moran Feldman, Andreas Krause, and Amin Karbasi. 2019. Adaptive sequence submodularity. *Advances in Neural Information Processing Systems* 32 (2019).

[38] Houssam Nassif, Kemal Oral Cansizlar, Mitchell Goodman, and SVN Vishwanathan. 2018. Diversifying music recommendations. *arXiv preprint arXiv:1810.01482* (2018).

[39] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functions—I. *Mathematical programming* 14 (1978), 265–294.

[40] Naoto Ohsaka and Yuichi Yoshida. 2015. Monotone k-submodular function maximization with size constraints. *Advances in Neural Information Processing Systems* 28 (2015).

[41] Shaojie Tang and Jing Yuan. 2020. Adaptive cascade submodular maximization. *arXiv preprint arXiv:2007.03592* (2020).

[42] Kai Wei, Rishabh Iyer, and Jeff Bilmes. 2015. Submodularity in data subset selection and active learning. In *International conference on machine learning*. PMLR, 1954–1963.

[43] Qiong Wu, Yong Liu, Chunyan Miao, Yin Zhao, Lu Guan, and Haihong Tang. 2019. Recent advances in diversified recommendation. *arXiv preprint arXiv:1905.06589* (2019).

[44] Yisong Yue and Carlos Guestrin. 2011. Linear submodular bandits and their application to diversified retrieval. *Advances in Neural Information Processing Systems* 24 (2011).

[45] Ghahramani Zoubin. 2013. Scaling the indian buffet process via submodular maximization. In *International Conference on Machine Learning*. PMLR, 1013–1021.

Appendix

A Preliminaries

We use $[j]$, with $j \in \mathbb{N}$, to denote the set $\{1, 2, \ldots, j\}$. A *set system* is a pair $(U, \mathcal{I})$, where U is a ground set and $\mathcal{I}$ is a family of subsets of U.

Definition A.1 (Matroid)**.** *A set system $(U, \mathcal{I})$ is a* matroid *if*

(i) $\emptyset \in \mathcal{I}$,

(ii) *for $B \in \mathcal{I}$ and any $A \subseteq B$ we have $A \in \mathcal{I}$, and*

(iii) *if $A \in \mathcal{I}$, $B \in \mathcal{I}$ and $|A| < |B|$, then there exists $e \in B \setminus A$ such that $A \cup \{e\} \in \mathcal{I}$.*

Definition A.2 (Partition matroid)**.** *A matroid $\mathcal{M} = (U, \mathcal{I})$ is a* partition matroid *if there exists a partition $\{E_i\}_{i=1}^r$ of U, i.e., $U = \cup_{i=1}^r E_i$ and $E_i \cap E_j = \emptyset$ for all $i \neq j$, and non-negative integers $k_1, \ldots, k_r$, such that $\mathcal{I} = \{I \subseteq U \colon |I \cap E_i| \leq k_i$ for all $i \in [r]\}$.*

A non-negative set function $f \colon 2^U \to \mathbb{R}_{\geq 0}$ is called *submodular* if for all sets $A \subseteq B \subseteq U$ and all elements $e \in U \setminus B$, it is $f(A \cup \{e\}) - f(A) \geq f(B \cup \{e\}) - f(B)$. The function f is *monotone* if for all $A \subseteq B$ it is $f(A) \leq f(B)$. Let U be a finite ground set, and let $f \colon 2^U \to \mathbb{R}_{\geq 0}$ be a *monotone submodular* set function. The *constrained submodular maximization problem* is to find $\max_{S \in \mathcal{I}} f(S)$, where $\mathcal{I} \subseteq 2^U$ represents the feasibility constraints.

For ease of notation, for any two sets $A \subseteq U$ and $B \subseteq U$, and any item $e \in U$, we write $A + B$ for $A \cup B$, $A + e$ for $A \cup \{e\}$, $A - B$ for $A \setminus B$, and $A - e$ for $A \setminus \{e\}$. We write $f(A \mid B)$ and $f(e \mid B)$ for the incremental gain respectively defined as $f(A \cup B) - f(B)$ and $f(A \cup \{e\}) - f(A)$.

B Omitted proofs of Section 2

Lemma 2.1. *$f(\mathcal{S})$ is a non-decreasing submodular set function.*

Proof. For any set $\mathcal{S}$, we can rewrite Equation (1). Denote $\mathcal{S} = \cup_{t=1}^T \mathcal{S}^t$ and we allow multiple copies of V_i to appear in $\mathcal{S}$ (the copies come from each $\mathcal{S}^t$). Note that $\mathcal{S}$ is a multi-set, and

$$f(\mathcal{S}) = \sum_{j=1}^{d} \left(1 - \prod_{t \in [T]} \prod_{V_i \in \mathcal{S}^t, V_i \ni c_j} (1 - p_i) \right) = \sum_{j=1}^{d} \left(1 - \prod_{V_i \in \mathcal{S}, V_i \ni c_j} (1 - p_i) \right).$$

Let $g_j(\mathcal{S}) = 1 - \prod_{V_i \in \mathcal{S}, V_i \ni c_j} (1 - p_i)$. Then $f(\mathcal{S}) = \sum_{j=1}^d g_j(\mathcal{S})$. To prove the lemma, it suffices to show that g_j is non-decreasing and submodular for all $j \in [d]$.

Non-decreasing property: For any two sets $\mathcal{X} \subseteq \mathcal{Y}$ and for all $j \in [d]$, it is

$$g_j(\mathcal{Y}) - g_j(\mathcal{X}) = \left(1 - \prod_{V_i \in \mathcal{Y} \setminus \mathcal{X}, V_i \ni c_j} (1 - p_i) \right) \prod_{V_i \in \mathcal{X}, V_i \ni c_j} (1 - p_i) \geq 0.$$

Submodularity: Let $\mathcal{X} \subseteq \mathcal{Y}$, and let $Z \in \overline{\mathcal{Y}}$ be an element not in $\mathcal{Y}$. Then, for all $j \in [d]$, the following holds:

$$\begin{aligned}
&g_j(\mathcal{Y} \cup \{Z\}) - g_j(\mathcal{Y}) - [g_j(\mathcal{X} \cup \{Z\}) - g_j(\mathcal{X})] \\
&= \left(1 - \prod_{V_i \in \{Z\}, V_i \ni c_j} (1 - p_i) \right) \prod_{V_i \in \mathcal{Y}, V_i \ni c_j} (1 - p_i) - \left(1 - \prod_{V_i \in \{Z\}, V_i \ni c_j} (1 - p_i) \right) \prod_{V_i \in \mathcal{X}, V_i \ni c_j} (1 - p_i) \\
&= \left(1 - \prod_{V_i \in \{Z\}, V_i \ni c_j} (1 - p_i) \right) \left(\prod_{V_i \in \mathcal{Y} \setminus \mathcal{X}, V_i \ni c_j} (1 - p_i) - 1 \right) \prod_{V_i \in \mathcal{X}, V_i \ni c_j} (1 - p_i) \leq 0.
\end{aligned}$$

□

Algorithm 3: LMGREEDY

Input: Budget $k \in \mathbb{N}$.

1 $\mathcal{G}^0, \cdots, \mathcal{G}^T \leftarrow \emptyset, \mathcal{G} \leftarrow \emptyset, \mathcal{V}^1 \leftarrow \emptyset, t = 1$
2 **for** (V, τ, p) *in the stream* **do**
3 $\quad \mathcal{V}^t \leftarrow \mathcal{V}^t \cup \{V\}$
4 $\quad$ **if** $\tau = 1$ **then**
5 $\quad\quad \mathcal{N}^t \leftarrow \mathcal{V}^t$
6 $\quad\quad$ **for** $j = 1$ *to* k **do**
7 $\quad\quad\quad V^* \leftarrow \arg\max_{V \in \mathcal{N}^t} f(\mathcal{G} \cup \mathcal{G}^t \cup \{V\})$
8 $\quad\quad\quad \mathcal{G}^t \leftarrow \mathcal{G}^t \cup \{V^*\}$
9 $\quad\quad\quad \mathcal{N}^t \leftarrow \mathcal{N}^t \setminus V^*$
10 $\quad\quad$ **output** $\mathcal{G}^t$
11 $\quad\quad \mathcal{G} = \mathcal{G} \cup \mathcal{G}^t$
12 $\quad\quad \mathcal{V}^{t+1} \leftarrow \mathcal{V}^t$
13 $\quad\quad t \leftarrow t + 1$

Theorem 2.4. *Algorithm 3 for Problem 2.2 has a competitive ratio of $1/2$, space complexity $\Theta(N + kT)$, time complexity $\mathcal{O}(NTk)$, and response time complexity $\mathcal{O}(Nk)$. Moreover, the competitive ratio of $1/2$ is tight, i.e., for Problem 2.2, no streaming algorithm can achieve a competitive ratio better than $1/2$ without violating the irrevocability constraint.*

Proof. We first prove that Algorithm 3 is $1/2$ competitive for Problem 2.2.

Let $\mathcal{O}^1, \cdots, \mathcal{O}^T$ be the optimal output sets, and let $\mathcal{G}^1, \cdots, \mathcal{G}^T$ be the output sets obtained by Algorithm 3. Since we treat each copy of the same item as a distinct item, it follows that for any $i \neq j$, $\mathcal{O}^i \cap \mathcal{O}^j = \emptyset$ and $\mathcal{G}^i \cap \mathcal{G}^j = \emptyset$. We let $\mathcal{G} = \bigcup_{t=1}^T \mathcal{G}^t$ and $\mathcal{O} = \bigcup_{t=1}^T \mathcal{O}^t$.

We prove that the following holds:

$$\begin{aligned} f(\mathcal{O}) &\overset{(a)}{\leq} f(\mathcal{G}) + \sum_{V \in \mathcal{O} \setminus \mathcal{G}} f(V \mid \mathcal{G}) \overset{(b)}{=} f(\mathcal{G}) + \sum_{i=1}^{T} \sum_{V \in \mathcal{O}^i \setminus \mathcal{G}} f(V \mid \mathcal{G}) \\ &\overset{(c)}{\leq} f(\mathcal{G}) + \sum_{t=1}^{T} \sum_{V \in \mathcal{O}^i} f(V \mid \mathcal{G}) \overset{(d)}{\leq} f(\mathcal{G}) + \sum_{t=1}^{T} \sum_{V \in \mathcal{O}^i} f(V \mid \bigcup_{j=1}^{i} \mathcal{G}^j). \end{aligned} \tag{2}$$

Inequalities (a) and (d) holds by submodularity, (c) holds by monotonicity, and equality (b) holds because $\{\mathcal{O}^1, \cdots, \mathcal{O}^T\}$ form a partition of $\mathcal{O}$.

Next, we bound the final term of Equation (2). We denote $\mathcal{O}^i = \{O_1^i, \cdots, O_k^i\}$, and $\mathcal{G}^i = \{G_1^i, \cdots, G_k^i\}$, with G_j^i representing the j-th item that is added to $\mathcal{G}^i$ during the greedy selection steps. Note that, we can assume $|\mathcal{G}| = |\mathcal{O}| = k$, because f is monotone. The following holds

$$\begin{aligned} \sum_{V \in \mathcal{O}^i} f(V \mid \bigcup_{j=1}^{i} \mathcal{G}^j) &= \sum_{l=1}^{k} f(O_l^i \mid \bigcup_{j=1}^{i} \mathcal{G}^j) \overset{(a)}{\leq} \sum_{l=1}^{k} f(O_l^i \mid \bigcup_{j=1}^{i-1} \mathcal{G}^j + \bigcup_{s=1}^{l-1} G_s^i) \\ &\overset{(b)}{\leq} \sum_{l=1}^{k} f(G_l^i \mid \bigcup_{j=1}^{i-1} \mathcal{G}^j + \bigcup_{s=1}^{l-1} G_s^i) = f(\bigcup_{j=1}^{i} \mathcal{G}^j - \bigcup_{j=1}^{i-1} \mathcal{G}^j), \end{aligned} \tag{3}$$

where inequality (a) holds by submodularity, and inequality (b) holds by design of the greedy algorithm (Line 7 of Algorithm 3).

Combining Equation (2) and Equation (3) finishes the proof:

$$f(\mathcal{O}) \leq f(\mathcal{G}) + \sum_{t=1}^{T} f(\bigcup_{j=1}^{i} \mathcal{G}^j - \bigcup_{j=1}^{i-1} \mathcal{G}^j) = 2f(\mathcal{G}) - f(\mathcal{G}^0) \leq 2f(\mathcal{G}). \tag{4}$$

Furthermore, we prove that the competitive ratio of $1/2$ is tight for Problem 2.2.

Consider a simple adversarial example with budget $k = 1$ and a stream given by

$$\mathbf{S} = ((V_1, 0, 1),\ (V_2, 1, 1),\ (V_3, 1, 1)),$$

where $V_1 = \{c_1, c_2\}$ and $V_2 = \{c_3, c_4\}$. For any algorithm, if it selects $\mathcal{S}^1 = \{V_1\}$ as the first set to present, the adversary sets $V_3 = \{c_1, c_2\}$. Conversely, if the algorithm selects $\mathcal{S}^1 = \{V_2\}$, then the adversary sets $V_3 = \{c_3, c_4\}$.

In either case, the algorithm can only cover two topics, while the optimal solution covers all four. Thus, the competitive ratio is at most $\frac{2}{4} = \frac{1}{2}$, which completes the proof.

□

C Omitted proofs of Section 3

Before presenting the omitted proofs in Section 3, we provide a detailed description of the reductions used in our analysis.

We begin by formally defining the streaming submodular maximization problem subject to a partition matroid constraint (*S2MM*). Our definition adapts the streaming submodular maximization framework under a p-matchoid constraint, as introduced by Chekuri et al. [8], and is stated as follows:

Problem C.1 (***S2MM***). *Let $\mathcal{M} = (\bar{\mathbf{S}}, \mathcal{I})$ be a partition matroid and $\hat{f}$ a submodular function. The elements of $\bar{\mathbf{S}}$ are presented in a stream, and we order $\bar{\mathbf{S}}$ by order of appearance. The goal of the* S2MM *problem is to select a subset of items $\mathcal{A} \in \mathcal{I}$ that maximizes $\hat{f}(\mathcal{A})$.*

We demonstrate two types of reductions from an instance of *S3MOR* to an instance of *S2MM*, depending on whether we know the exact value of T. Algorithm 1, as presented in the main content, is based on the second reduction, where we only know an upper bound T' of T.

C.1 The case when the exact number of visits T is known

First, we show that if the exact number of user visits T is known, the *S3MOR* problem reduces to the *S2MM* problem. Specifically, any streaming algorithm for *S2MM* that satisfies the *irrevocable output requirement* can solve *S3MOR* while maintaining the same competitive ratio. The *irrevocable output requirement* requires that the output subset for any partition becomes fixed irrevocably once all its items have appeared in the stream.

The reduction. Given an instance of the *S3MOR* problem, where the stream is given by

$$\mathbf{S} = ((V_1, p_1, \tau_1), \ldots, (V_N, p_N, \tau_N)),$$

with a budget k, and the function f as defined in Equation (1), we can construct a corresponding instance of the *S2MM* problem.

Recall that r_t denotes the time step at which the user submits the t-th visit (with $r_0 = 0$), and V_i^t denotes a copy of item V_i with copy identifier t. We use $\bigsqcup$ to denote the concatenation of streams. An instance of the *S2MM* problem with input stream $\bar{\mathbf{S}}$ can be constructed as follows:

1. Let $\bar{\mathbf{S}}_t = \bigsqcup_{i=r_{t-1}+1}^{r_t} (V_i^t, \cdots, V_i^T)$ for all $t \in [T]$, to be specific,

$$\bar{\mathbf{S}}_t = \left(\underbrace{V_{r_{t-1}+1}^t, \cdots, V_{r_{t-1}+1}^T}_{i=r_{t-1}+1}, \underbrace{V_{r_{t-1}+2}^t, \cdots, V_{r_{t-1}+2}^T}_{i=r_{t-1}+2}, \cdots, \underbrace{V_{r_t}^t, \cdots, V_{r_t}^T}_{i=r_t} \right).$$

2. Let $\bar{\mathbf{S}} = \bigsqcup_{t=1}^{T} \bar{\mathbf{S}}_t$ be the concatenation of sub-streams $\bar{\mathbf{S}}_1, \cdots, \bar{\mathbf{S}}_T$, i.e., $\bar{\mathbf{S}} = (\bar{\mathbf{S}}_1, \cdots, \bar{\mathbf{S}}_T)$.
3. Let $\mathcal{V}^t = \{V_i^t\}_{i=1}^{r_t} = \{V_1^t, \ldots, V_{r_t}^t\}$.
4. Construct $\mathcal{I}$ in the following way: for any set $X \subseteq \bigcup_{t=1}^{T} \mathcal{V}^t$, if $|X \cap \mathcal{V}^t| \leq k$ for all $1 \leq t \leq T$, then $X \in \mathcal{I}$.

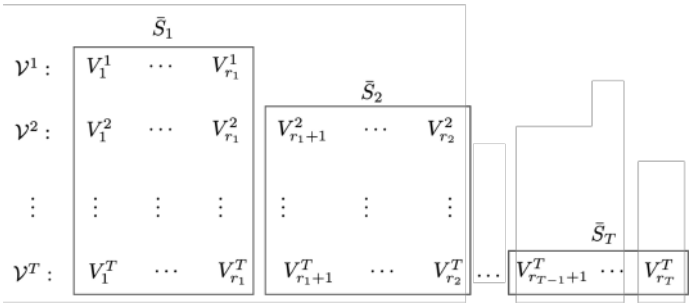


Figure 5: Illustration of the reduction when the exact number of visits T is known. The original item stream for the *S3MOR* problem is $(V_1, \ldots, V_{r_T})$, with user accesses occurring at time steps $r_1, \ldots, r_T$. The constructed stream for the *S2MM* problem is $\bar{\mathbf{S}} = \bigsqcup_{t=1}^{T} \bar{\mathbf{S}}_t$.

5. Let $p_i^t = p_i$ denote a copy of p_i with copy identifier $t \in [T]$. Define $\hat{f} : 2^{\bigcup_{t=1}^{T} \mathcal{V}^t} \to \mathbb{R}_{\geq 0}$. Specifically, for any $\mathcal{A} \subseteq \bigcup_{t=1}^{T} \mathcal{V}^t$, let $\mathcal{A}^t = \mathcal{A} \cap \mathcal{V}^t$. Then, $\hat{f}$ is defined as follows:

$$\hat{f}(\mathcal{A}) = \sum_{j=1}^{d} \left(1 - \prod_{t \in [T]} \prod_{V_i^t \in \mathcal{A}^t, V_i^t \ni c_j} (1 - p_i^t) \right), \tag{5}$$

and the goal of *S2MM* is to find $\mathcal{A} \in \mathcal{I}$ that maximizes $\hat{f}(\mathcal{A})$.

We observe that $\mathcal{M} = (\bar{\mathbf{S}}, \mathcal{I})$, as defined above, forms a partition matroid on the constructed stream $\bar{\mathbf{S}}$, with $\bar{\mathbf{S}} = \bigcup_{t=1}^{T} \mathcal{V}^t$ and $\mathcal{V}^i \cap_{i \neq j} \mathcal{V}^j = \emptyset$. By construction, for each visit $t \in [T]$, the partition $\mathcal{V}^t$ contains exactly the set of items that arrived in the system before the t-th user visit in the *S3MOR* problem. Since the *S2MM* problem and the *S3MOR* problem share equivalent objective functions and equivalent constraints, we conclude the following: for any optimal solution $\bar{\mathcal{O}} \in \mathcal{I}$ of the *S2MM* problem, the sequence of subsets

$$\bar{\mathcal{O}} \cap \mathcal{V}^1, \ \ldots, \ \bar{\mathcal{O}} \cap \mathcal{V}^T$$

yields optimal output for the *S3MOR* problem at each corresponding t-th user visit.

We can further conclude that any streaming algorithm achieving an α-competitive solution for the *S2MM* problem also yields an α-competitive solution for the *S3MOR* problem. For this equivalence to hold, the streaming algorithm for *S2MM* must satisfy the *irrevocable output requirement*: for each partition $\mathcal{V}^t$ of the stream, once the final item from this partition has arrived, the solution that belongs to this partition, $\mathcal{A} \cap \mathcal{V}^t$, can no longer be changed. This requirement excludes algorithms like that of Feldman et al. [11], whose solution set only becomes available after the stream terminates. In contrast, the algorithms proposed by Chakrabarti and Kale [7], Chekuri et al. [8], and Feldman et al. [10] are suitable. These three algorithms all meet the requirement while providing $1/4$-competitive solutions for the *S2MM* problem.

C.2 The case when only an upper bound T' of T is available

Consider the case where only an upper bound T' of the true number of user visits T is known. We prove the following: (1) An instance of the *S2MM* problem can be constructed such that any feasible solution to the *S2MM* problem contains a valid solution for the *S3MOR* problem. (2) Any streaming algorithm that solves the *S2MM* problem and satisfies the *irrevocable output requirement* can be adapted to solve the *S3MOR* problem, albeit with a reduced competitive ratio. Specifically, the competitive ratio decays by a factor of $1/(T' - T + 1)$.

In contrast to the reduction we present in Appendix C.1, the construction of the stream for the *S2MM* problem depends on both the original stream $\mathbf{S}$, and the algorithmic procedure described in Algorithm 1. We will prove that:

1. Algorithm 1 generates a corresponding stream $\hat{\mathbf{S}}$ for the *S2MM* problem

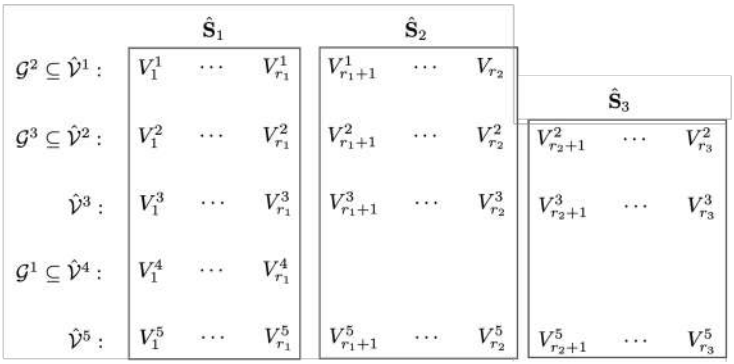


Figure 6: An example illustrating the reduction when only an upper bound T' of the true number of visits T is available. In this example, $T = 3$ and $T' = 5$. The original item stream for the *S3MOR* problem is given by $(V_1, V_2, \ldots, V_{r_3})$, with user visits occurring at time steps r_1, r_2, and r_3. The constructed stream for the *S2MM* problem is $\hat{\mathbf{S}} = \bigsqcup_{t=1}^{3} \hat{\mathbf{S}}_t$, where $\hat{\mathbf{S}}_1 = \bigsqcup_{i=1}^{r_1} \bigsqcup_{a\in[5]} (V_i^a)$, $\hat{\mathbf{S}}_2 = \bigsqcup_{i=r_1+1}^{r_2} \bigsqcup_{a\in\{1,2,3,5\}} (V_i^a)$, and $\hat{\mathbf{S}}_3 = \bigsqcup_{i=r_2+1}^{r_3} \bigsqcup_{a\in\{2,3,5\}} (V_i^a)$. Algorithm 1 maintains and updates $\mathcal{A}^t \in \hat{\mathcal{V}}^t$ for all $t \in [5]$ upon the arrival of each item in $\hat{\mathbf{S}}$. At the visit time steps r_1, r_2, and r_3, it outputs $\mathcal{G}^1 = \mathcal{A}^4$, $\mathcal{G}^2 = \mathcal{A}^1$, and $\mathcal{G}^3 = \mathcal{A}^2$, respectively. By the end of the stream, the union $\bigcup_{t=1}^{5} \mathcal{A}^t$ constitutes a $1/4$-competitive solution for the *S2MM* problem on input $\hat{\mathbf{S}}$. The combined solution $\bigcup_{t=1}^{3} \mathcal{G}^t$ is a $\frac{1}{4\times(5-3+1)}$-competitive solution for the original *S3MOR* problem.

2. The candidate sets $\{\mathcal{A}^1, \cdots, \mathcal{A}^{T'}\}$ maintained by Algorithm 1 forms a solution to the *S2MM* problem; the output $\{\mathcal{G}^1, \cdots, \mathcal{G}^T\}$ of Algorithm 1 forms a solution to the *S3MOR* problem.

By Algorithm 1 line 13, we can construct a bijective mapping π between the index sets of $\{\mathcal{G}^1, \ldots, \mathcal{G}^T\}$ and $\{\mathcal{A}^1, \ldots, \mathcal{A}^{T'}\}$. This mapping satisfies the equality:

$$\mathcal{G}^t = \mathcal{A}^{\pi(t)}, \quad \text{for all } t \in [T].$$

Let $\mathcal{T}^0 = \{1, 2, \ldots, T'\}$ denote the initial index set of all active candidate sets. For each $t \geq 1$, let $\mathcal{T}^t$ represent the index set of active candidate sets before the t-th visit and after the $(t-1)$-th visit. The active index sets evolve as follows:

(1) For all $t \in [T]$: $\mathcal{T}^t = \mathcal{T}^{t-1} \setminus \{\pi(t)\}$, and

(2) for all $t \in \{T+1, \ldots, T'\}$: $\mathcal{T}^t = \mathcal{T}^T$.

The reduction.

1. We construct the stream $\hat{\mathbf{S}}$ for *S2MM* as follows

$$\hat{\mathbf{S}}_t = \begin{cases} \bigsqcup_{i=r_{t-1}+1}^{r_t} \bigsqcup_{a\in\mathcal{T}^t} (V_i^a), & \text{for } t \in [T], \\ \bigsqcup_{i=r_{T-1}+1}^{r_T} \bigsqcup_{a\in\mathcal{T}^T} (V_i^a), & \text{for } t \in \{T+1, \cdots, T'\}. \end{cases}$$

2. Let $\hat{\mathbf{S}} = \bigsqcup_{t=1}^{T} \hat{\mathbf{S}}_t$ be the concatnation of $\hat{\mathbf{S}}_1, \cdots, \hat{\mathbf{S}}_T$.
3. Let $\rho(t) = \pi^{-1}(t)$, and

$$\hat{\mathcal{V}}^t = \begin{cases} \{V_i^t\}_{i=1}^{r_{\rho(t)}} = \{V_1^t, \ldots, V_{r_{\rho(t)}}^t\}, & \text{for } t \in \{\pi(1), \cdots, \pi(T)\}, \\ \{V_1^t, \cdots, V_{r_T}^t\}, & \text{for } t \in \mathcal{T}^0 \setminus \{\pi(1), \cdots, \pi(T)\}. \end{cases}$$

4. Construct $\hat{\mathcal{I}}$ in the following way: for any set $X \subseteq \bigcup_{t=1}^{T'} \hat{\mathcal{V}}^t$, if $|X \cap \hat{\mathcal{V}}^t| \leq k$ for all $1 \leq t \leq T'$, then $X \in \hat{\mathcal{I}}$.

5. Let $p_i^t = p_i$ denote a copy of p_i with copy identifier $t \in [T']$. The goal is to select $\mathcal{A} = \bigcup_{t=1}^{T'} \mathcal{A}^t \in \hat{\mathcal{I}}$ that maximizes $f(\mathcal{A})$. The submodular function $\hat{f}$ is given as follows:

$$\hat{f}(\mathcal{A}) = \sum_{j=1}^{d} \left(1 - \prod_{t \in [T']} \prod_{V_i^t \in \mathcal{A}^t, V_i^t \ni c_j} (1 - p_i^t) \right). \tag{6}$$

Given the above construction, we can verify that $\mathcal{M} = (\hat{\mathbf{S}}, \hat{\mathcal{I}})$ is an instance of a partition matroid defined on the constructed stream $\hat{\mathbf{S}}$, with $\hat{\mathbf{S}} = \bigcup_{t=1}^{T'} \hat{\mathcal{V}}^t$ and $\hat{\mathcal{V}}^i \cap_{i \neq j} \hat{\mathcal{V}}^j = \emptyset$. By construction, the partitions $\hat{\mathcal{V}}^{\pi(t)}$ of $\hat{\mathbf{S}}$ satisfy: (1) for each $t \in [T]$, $\hat{\mathcal{V}}^{\pi(t)}$ contains exactly the set of items that arrived before the t-th user visit in the *S3MOR* problem (2) The remaining $T' - T$ partitions each contain the same set of items that arrived before the final user visit. The design ensures that each output $\mathcal{G}^t$ satisfies $\mathcal{G}^t \subseteq \hat{\mathcal{V}}^{\pi(t)}$ and $|\mathcal{G}^t| \leq k$, for all $t \in [T]$. Consequently, the sequence $\mathcal{G}^1, \ldots, \mathcal{G}^T$ constitutes feasible output for the *S3MOR* problem.

Following Algorithm 1, we conclude that for all $t \in [T']$, we have $\mathcal{A}^t \subseteq \hat{\mathcal{V}}^t$ with $|\mathcal{A}^t| \leq k$. Consequently, the union $\bigcup_{t=1}^{T'} \mathcal{A}^t$ forms a feasible solution to the *S2MM* problem.

Since Algorithm 1 essentially applies the streaming algorithm of Chekuri et al. [8] to the constructed stream $\hat{\mathbf{S}}$, the solution $\bigcup_{t=1}^{T'} \mathcal{A}^t$ achieves a $1/4$-approximation guarantee for the *S2MM* problem.

C.3 Omitted proofs

Theorem 3.1. *Let T be the number of user accesses and T' a given upper bound. Algorithm 1 has competitive ratio $\frac{1}{4(T'-T+1)}$, space complexity $\mathcal{O}(T'k)$, time complexity $\mathcal{O}(NkT')$, and response time complexity $\mathcal{O}$ (T').*

Proof. Following the notations and conclusions we have in Appendix C.2, we are ready to prove Theorem 3.1. Let $\mathcal{O}^1, \cdots, \mathcal{O}^T$ be the optimal solution for the *S3MOR* problem, and let $\hat{\mathcal{O}} = \bigcup_{t=1}^{T'} \hat{\mathcal{O}}^t$, where $\hat{\mathcal{O}}^t = \hat{\mathcal{O}} \cap \hat{\mathcal{V}}^t$, denotes the optimal solution for the *S2MM*. We have

$$\begin{aligned}
f(\bigcup_{t=1}^{T} \mathcal{G}^t) &= f(\bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)} + \mathcal{A}^{\pi(T)}) = f(\mathcal{A}^{\pi(T)} \mid \bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) + f(\bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) \\
&\overset{(a)}{\geq} \frac{1}{T'-T+1} \sum_{s \in \mathcal{T} \setminus \cup_{t=1}^{T-1}\{\pi(t)\}} f(\mathcal{A}^s \mid \bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) + f(\bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) \\
&\geq \frac{1}{T'-T+1} \sum_{s \in \mathcal{T} \setminus \cup_{t=1}^{T-1}\{\pi(t)\}} f(\mathcal{A}^s \mid \bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) + \frac{1}{T'-T+1} f(\bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)}) \\
&\overset{(b)}{} \frac{1}{T'-T+1} f(\bigcup_{t=1}^{T'} \mathcal{A}^t) \overset{(c)}{\geq} \frac{1}{4(T'-T+1)} f(\bigcup_{t=1}^{T'} \hat{\mathcal{O}}^t) \overset{(d)}{\geq} \frac{1}{4(T'-T+1)} f(\bigcup_{t=1}^{T} \mathcal{O}^t),
\end{aligned} \tag{7}$$

where inequality (a) holds because by line 11 of Algorithm 1, that $\mathcal{A}^{\pi(T)}$ is selected with the largest marginal gain in terms of $f(\bigcup_{t=1}^{T-1} \mathcal{A}^{\pi(t)})$. Inequality (b) holds by submodularity. Inequality (c) holds because, as concluded in Appendix C.2, $\bigcup_{t=1}^{T'} \mathcal{A}^t$ achieves a $1/4$-approximation guarantee for the *S2MM* problem. Finally, inequality (d) holds by contradiction: if $f(\bigcup_{t=1}^{T} \mathcal{O}^t) > f(\bigcup_{t=1}^{T'} \hat{\mathcal{O}}^t)$, then there exists a feasible solution for the *S2MM* problem where for all $t \in [T]$, $\mathcal{A}^t = \mathcal{O}^t$, and regardless of the choice of $\mathcal{A}^{T+1}, \cdots, \mathcal{A}^T$, it holds that $f(\bigcup_{t=1}^{T'} \mathcal{A}^t) \geq f(\bigcup_{t=1}^{T} \mathcal{O}^t) > f(\bigcup_{t=1}^{T'} \hat{\mathcal{O}}^t)$, which contradicts that $\hat{\mathcal{O}}$ is an optimal solution.

Complexity analysis. The space complexity is in $\mathcal{O}(T'k)$ because STORM only needs to maintain T' candidate sets of size k. Upon the arrival of each item, STORM makes $T'k$ oracle calls, thus the

time complexity is in $\mathcal{O}(NT'k)$. When a user submit a visit request, STORM selects one partition out of at most T' partitions and output it, thus the response time complexity is $\mathcal{O}(T')$.

□

Tightness of our results. Note that the analysis in Equation (7) is tight. In the following, we show that for any value of T, we can construct a stream such that STORM cannot do better than $\frac{1}{T'-T+1}$, which is only a constant factor away from our analysis in Theorem 3.1.

To see this, consider an instance of the *S3MOR* problem with budget $k = 1$, the user visits one time at the end of the stream, and we have an estimated number of user visits $T' > 1$, the stream is constructed as below:

$$\mathbf{S}_1 = ((\{1\}, 1, 0), (\{2\}, 1, 0), \cdots, (\{T'\}, 1, 0), (\{1, \cdots, T'\}, 1, 1))$$

The STORM(T') algorithm outputs one of the first T' items and achieves an expected topic coverage of value 1, while the optimal solution is to select the last item and obtain an expected coverage of value T'. The competitive ratio in this instance is $\frac{1}{T'}$, while Alg. 1 has competitive ratio $\frac{1}{4(T'-T+1)} = \frac{1}{4T'}$.

Note that for all possible numbers of user visits T and $T' > T$, we can construct a stream such that STORM cannot do better than $\frac{1}{T'-T+1}$. For example, when $T = 2$, we can construct a sub-stream $\mathbf{S}_2$ as follows

$$\mathbf{S}_2 = ((\{T'+1, T'+2\}, 1, 0), \cdots, (\{2T'-1, 2T'\}, 1, 0), (\{T'+1, \cdots, 3T'-3\}, 1, 1))$$

By concatenating $\mathbf{S}_1$ and $\mathbf{S}_2$, we obtain a new stream where the user visits for 2 times. With budget $k = 1$, STORM(T') achieves an expected coverage 3, while the optimal expected coverage is $3(T'-1)$, leading to a competitive ratio of $\frac{1}{T'-1}$, with is again a $1/4$ factor away from $\frac{1}{4(T'-T+1)} = \frac{1}{T'-1}$. We can repeat this process such that for any T, we can construct a stream such that the competitive ratio is equal to $\frac{1}{T'-T+1}$.

Lemma C.2. *Let T be the number of user accesses and T' be a given upper bound. Algorithm 2 generates a set of guesses of T as $\mathcal{P} = \{\delta i \mid i \in [\lceil T'/\delta \rceil]\}$. Let T^* be the smallest integer in $\mathcal{P}$ that is larger than or equal to T. Let $\mathcal{G}^1, \cdots, \mathcal{G}^T$ be the output of* STORM(T^*)*, and let $\mathcal{B}^1, \cdots, \mathcal{B}^T$ be the output of Algorithm 2, then it holds that*

$$f(\bigcup_{t=1}^{T} \mathcal{B}^t) \geq f(\bigcup_{t=1}^{T} \mathcal{G}^t) + \sum_{t=1}^{T-1} \Big[f(\bigcup_{s=1}^{t} \mathcal{B}^s \mid \bigcup_{s=1}^{t+1} \mathcal{G}^s) - f(\bigcup_{s=1}^{t} \mathcal{B}^s \mid \bigcup_{s=1}^{t} \mathcal{G}^s) \Big]. \tag{8}$$

Proof. We prove Equation (8) via induction.

When $T = 1$, according to line 7 of Algorithm 2 that $\mathcal{B}^1$ is chosen with the largest marginal gain, it holds that $f(\mathcal{B}^1) \geq f(\mathcal{G}^1)$.

The base case for the induction starts from $T = 2$. It is

$$\begin{aligned}
f(\mathcal{B}^1, \mathcal{B}^2) &= f(\mathcal{B}^1) + f(\mathcal{B}^2 \mid \mathcal{B}^1) \overset{(a)}{\geq} f(\mathcal{G}^1) + f(\mathcal{G}^2 \mid \mathcal{B}^1) \\
&= f(\mathcal{G}^1 \cup \mathcal{B}^1) - f(\mathcal{B}^1 \mid \mathcal{G}^1) + f(\mathcal{G}^2 \mid \mathcal{B}^1) \\
&= f(\mathcal{B}^1) + f(\mathcal{G}^1 \mid \mathcal{B}^1) - f(\mathcal{B}^1 \mid \mathcal{G}^1) + f(\mathcal{G}^2 \mid \mathcal{B}^1) \\
&\overset{(b)}{\geq} f(\mathcal{G}^1 \cup \mathcal{G}^2 \mid \mathcal{B}^1) + f(\mathcal{B}^1) - f(\mathcal{B}^1 \mid \mathcal{G}^1) = f(\mathcal{B}^1, \mathcal{G}^1, \mathcal{G}^2) - f(\mathcal{B}^1 \mid \mathcal{G}^1) \\
&= f(\mathcal{G}^1, \mathcal{G}^2) + f(\mathcal{B}^1 \mid \mathcal{G}^1 \cup \mathcal{G}^2) - f(\mathcal{B}^1 \mid \mathcal{G}^1),
\end{aligned}$$

where inequality (a) holds because $f(\mathcal{B}^1) \geq f(\mathcal{G}^1)$ and $f(\mathcal{B}^2 \mid \mathcal{B}^1) > f(\mathcal{G}^2 \mid \mathcal{B}^1)$. Inequality (b) holds by submodularity.

We then prove the induction step. Assume for $T = j$, it holds

$$f(\bigcup_{t=1}^{j} \mathcal{B}^t) \geq f(\bigcup_{t=1}^{j} \mathcal{G}^t) + \sum_{t=1}^{j-1} \Big[f(\bigcup_{s=1}^{t} \mathcal{B}^s \mid \bigcup_{s=1}^{t+1} \mathcal{G}^s) - f(\bigcup_{s=1}^{t} \mathcal{B}^s \mid \bigcup_{s=1}^{t} \mathcal{G}^s) \Big]. \tag{9}$$

We can use ϕ_j to represent the second term in Equation (9), i.e., $f(\bigcup_{t=1}^{j}\mathcal{B}^t) \geq f(\bigcup_{t=1}^{j}\mathcal{G}^t) + \phi_j$, we can then show that for $T = j+1$ it holds

$$
\begin{aligned}
f(\bigcup_{t=1}^{j+1}\mathcal{B}^t) &= f(\mathcal{B}^{j+1} \mid \bigcup_{t=1}^{j}\mathcal{B}^t) + f(\bigcup_{t=1}^{j}\mathcal{B}^t) \overset{(a)}{\geq} f(\mathcal{G}^{j+1} \mid \bigcup_{t=1}^{j}\mathcal{B}^t) + f(\bigcup_{t=1}^{j}\mathcal{G}^t) + \phi_j \\
&= f(\bigcup_{t=1}^{j}\mathcal{G}^j \mid \bigcup_{t=1}^{j}\mathcal{B}^t) + f(\bigcup_{t=1}^{j}\mathcal{B}^t) - f(\bigcup_{t=1}^{j}\mathcal{B}^t \mid \bigcup_{t=1}^{j}\mathcal{G}^t) + f(\mathcal{G}^{j+1} \mid \bigcup_{t=1}^{j}\mathcal{B}^t) + \phi_j \\
&\overset{(b)}{\geq} f(\bigcup_{t=1}^{j+1}\mathcal{G}^t \mid \bigcup_{t=1}^{j}\mathcal{B}^t) + f(\bigcup_{t=1}^{j}\mathcal{B}^t) - f(\bigcup_{t=1}^{j}\mathcal{B}^t \mid \bigcup_{t=1}^{j}\mathcal{G}^t) + \phi_j \\
&= f(\bigcup_{t=1}^{j+1}\mathcal{G}^t) + f(\bigcup_{t=1}^{j}\mathcal{B}^t \mid \bigcup_{t=1}^{j+1}\mathcal{G}^t) - f(\bigcup_{t=1}^{j}\mathcal{B}^t \mid \bigcup_{t=1}^{j}\mathcal{G}^t) + \phi_j \\
&= f(\bigcup_{t=1}^{j+1}\mathcal{G}^j) + \sum_{t=1}^{j}\Big[f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s) - f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t}\mathcal{G}^s)\Big].
\end{aligned}
$$

Here, inequality (a) holds by the induction assumption, and (b) holds by submodularity. This completes the induction. □

Lemma C.3. *Let T be the number of user accesses and T' be a given upper bound. Algorithm 2 generates a set of guesses of T as $\mathcal{P} = \{\delta i \mid i \in [\lceil T'/\delta \rceil]\}$. Let T^* be the smallest integer in $\mathcal{P}$ that is larger than or equal to T. Let $\mathcal{G}^1, \cdots, \mathcal{G}^T$ be the output of* STORM(T^*)*, and let $\mathcal{B}^1, \cdots, \mathcal{B}^T$ be the output of Algorithm 2. It holds that*

$$
f(\bigcup_{t=1}^{T}\mathcal{B}^t) \geq \frac{1}{2} f(\bigcup_{t=1}^{T}\mathcal{G}^t). \tag{10}
$$

Proof. Lemma C.3 is a direct result of Lemma C.2, as we can show that

$$
\sum_{t=1}^{T-1}\Big[f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s) - f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t}\mathcal{G}^s)\Big] \geq -f(\bigcup_{t=1}^{T}\mathcal{B}^t).
$$

To prove that the above inequality holds, we proceed to show that

$$
\sum_{t=1}^{T-1}\Big[-f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s) + f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t}\mathcal{G}^s)\Big] \leq f(\bigcup_{t=1}^{T}\mathcal{B}^t).
$$

Indeed, it is

$$\sum_{t=1}^{T-1}\Big[-f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)+f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t}\mathcal{G}^s)\Big] \tag{11}$$

$$\begin{aligned}
&= \sum_{t=1}^{T-1}\Big[f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t}\mathcal{G}^s)-f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)\Big]\\
&= f(\mathcal{B}^1 \mid \mathcal{G}^1)+\sum_{t=1}^{T-2}\left[f(\bigcup_{s=1}^{t+1}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)-f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)\right]-f(\bigcup_{s=1}^{T-1}\mathcal{B}^s \mid \bigcup_{s=1}^{T}\mathcal{G}^s)\\
&\le f(\mathcal{B}^1)+\sum_{t=1}^{T-2}\left[f(\bigcup_{s=1}^{t+1}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)-f(\bigcup_{s=1}^{t}\mathcal{B}^s \mid \bigcup_{s=1}^{t+1}\mathcal{G}^s)\right]\\
&= f(\mathcal{B}^1)+\sum_{t=1}^{T-2}\left[f(\bigcup_{s=1}^{t+1}\mathcal{B}^s+\bigcup_{s=1}^{t+1}\mathcal{G}^s)-f(\bigcup_{s=1}^{t+1}\mathcal{G}^s)-f(\bigcup_{s=1}^{t}\mathcal{B}^s+\bigcup_{s=1}^{t+1}\mathcal{G}^s)+f(\bigcup_{s=1}^{t+1}\mathcal{G}^s)\right]\\
&= f(\mathcal{B}^1)+\sum_{t=1}^{T-2}f(\mathcal{B}^{t+1} \mid \bigcup_{s=1}^{t}\mathcal{B}^s+\bigcup_{s=1}^{t+1}\mathcal{G}^s)\le f(\mathcal{B}^1)+\sum_{t=1}^{T-2}f(\mathcal{B}^{t+1} \mid \bigcup_{s=1}^{t}\mathcal{B}^s)\\
&= f(\bigcup_{t=1}^{T-1}\mathcal{B}^t)\le f(\bigcup_{t=1}^{T}\mathcal{B}^t)
\end{aligned} \tag{12}$$

Combining Equation (12) with Equation (8), we can obtain Equation (10).

□

Theorem 3.2. *Algorithm 2 has a competitive ratio of* $1/(8\delta)$*, space complexity* $\mathcal{O}(T'^2k/\delta)$*, time complexity* $\mathcal{O}(NkT'^2/\delta)$*, and response time complexity* $\mathcal{O}(T')$*.*

Proof. Let T^* be the smallest integer in $\mathcal{P}=\{\delta i \mid i\in[\lceil T'/\delta\rceil]\}$ that is larger than or equal to T. Let $\mathcal{G}^1,\cdots,\mathcal{G}^T$ be the output of STORM(T^*)

Observe that $T^*\le T+\delta-1$, otherwise we can set T^* as $T^*-\delta$ and it is still an upper bound of T. By Theorem 3.1, it holds that

$$f(\bigcup_{t=1}^{T}\mathcal{G}^t)\ge\frac{1}{4(T^*-T+1)}f(\bigcup_{t=1}^{T}\mathcal{O}^t)\ge\frac{1}{4\delta}f(\bigcup_{t=1}^{T}\mathcal{O}^t). \tag{13}$$

Combining Equation (13) with Lemma C.3, we conclude that

$$f(\bigcup_{t=1}^{T}\mathcal{B}^t)\ge\frac{1}{2}f(\bigcup_{t=1}^{T}\mathcal{G}^t)\ge\frac{1}{8\delta}f(\bigcup_{t=1}^{T}\mathcal{O}^t). \tag{14}$$

Since STORM++ maintains $\lceil T'/\delta\rceil$ copies of STORM, each copy of STORM has time complexity in $\mathcal{O}(NkT')$ and space complexity in $\mathcal{O}(T'k)$, the space complexity for the STORM++ is $\mathcal{O}(T'^2k/\delta)$, and the time complexity is $\mathcal{O}(NkT'^2/\delta)$. When a user submits a visit request, each copy of STORM selects one partition out of at most T' partitions as a candidate result set, then STORM++ selects the best set among T'/δ candidates as output. Thus, the response time of STORM++ is $\mathcal{O}(T')$. □

D Additional content for Section 4

D.1 Dataset and experimental setting

We use the following six datasets:

- **KuaiRec** [14]: Recommendation logs from a video-sharing mobile app, containing metadata such as each video's categories and watch ratio. We use the *watch ratio*, computed as the user's viewing time divided by the video's duration, as the user rating score. The dataset includes 7 043 videos across 100 categories.
- **Anime**:[3] A dataset of animation ratings in the range $[1, 10]$, consisting of 7745 animation items across 43 genres.
- **Beer**:[4] This dataset contains BeerAdvocate reviews [32, 31], along with categorical attributes for each beer. After filtering out beers and reviewers with fewer than 10 reviews, the final dataset includes 9 000 beers across 70 categories.
- **Yahoo**:[5] A music rating dataset with ratings in the range $[1, 5]$. After filtering out users with fewer than 20 ratings, the dataset includes 136 736 songs across 58 genres.
- **RCV1** [28]: A multi-label dataset, each item is a news story from Reuters, Ltd. for research use. We sample 462 225 stories across 476 unique labels.
- **Amazon** [30]: A multi-label dataset consisting of 1 117 006 Amazon products over 3 750 unique labels. This dataset is a subset of the publicly available dataset.

Experimental setting To obtain the click probabilities for each item, we generate them uniformly at random in the range $[0, 0.2]$ for the RCV1 and Amazon datasets. For the four item-rating datasets, we estimate click probabilities by computing a low-rank completion of the user-item rating matrix using matrix factorization [25]. This yields latent feature vectors w_u for each user u and v_m for each item m, where the inner product $w_u^\top v_m$ approximates user u's rating for item m. We then apply standard min-max normalization to the predicted ratings and linearly scale the results to the range $[0, 0.5]$ to obtain click probabilities. We set the latent feature dimension to 15 for the KuaiRec dataset, and to 20 for the remaining three rating datasets. While click probability ranges can vary in real-world scenarios, where larger ranges can lead to faster convergence toward the maximum expected coverage value, we intentionally set the probabilities to be small in our experiments. This allows us to observe performance changes over a wider range of parameter settings.

Implementation To enhance the computational efficiency of the LMGREEDY algorithm (described in Algorithm 3), we implement lines 6–9 using the STOCHASTIC-GREEDY approach introduced by Mirzasoleiman et al. [35]. For the implementations of STORM and STORM++, we incorporate the sampling technique from Feldman et al. [10]. Specifically, when adding each item (with potential replacement) to each candidate set, we ignore it with probability 2/3.

D.2 Omitted results

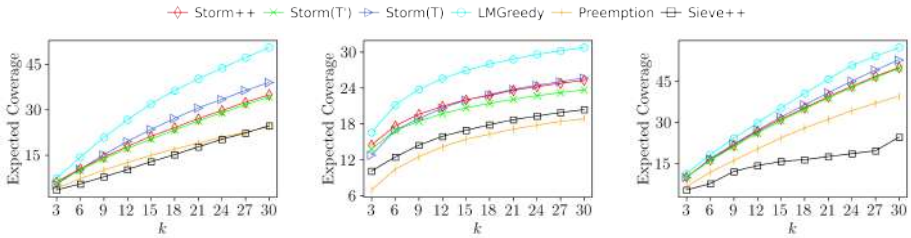
In this section, we present additional experimental results and analyses. Specifically, we show how the expected coverage changes as the budget k varies in Figure 7, and as the number of user visits T varies in Figure 8. The standard deviation of the expected coverage, as k and T are varied, is reported in Figure 10 and Figure 11, respectively. Finally, we present the effect of varying δ and ΔT on the expected coverage in Figure 9.

Why STORM++ and STORM(T') outperform STORM(T)? We present a simple example where STORM(T') outperforms STORM(T). Consider the input stream $\mathbf{S} = (\{\{1\}, 1, 0), (\{2\}, 1, 0), (\{3, 4\}, 0.9, 1), (\{5, 6\}, 1, 1))$ with budget $k = 1$ and actual number of visits $T = 2$. The algorithm STORM(2) selects either $\{1\}$ or $\{2\}$ for the first visit, and $\{5, 6\}$ for the second, achieving expected coverage of 3.

If we overestimate the visits and set $T' = 3$, STORM(3) instead selects $\{3, 4\}$ first and $\{5, 6\}$ second, yielding an expected coverage of 3.8, which is higher than when the number of visits is known exactly. Likewise, STORM++ with $T' = 3$ and $\delta = 3$ matches STORM(3), and achieves the same coverage of 3.8.

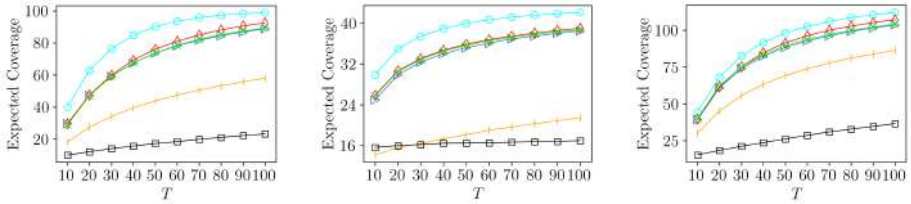
This example shows that while STORM(T) has the strongest theoretical guarantee, STORM(T') and STORM++ can outperform it empirically, which explains the trends observed in our experiments.

[3] `https://www.kaggle.com/datasets/CooperUnion/anime-recommendations-database`
[4] `https://cseweb.ucsd.edu/~jmcauley/datasets.html`
[5] `https://webscope.sandbox.yahoo.com/catalog.php?datatype=i&did=67`



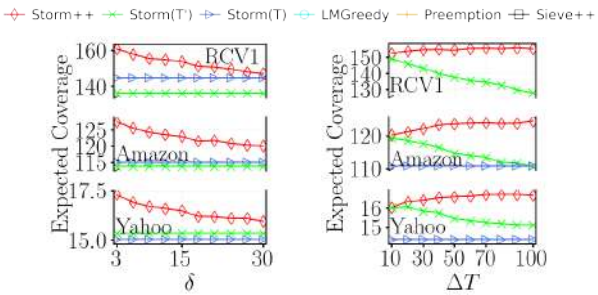
(a) KuaiRec (b) Anime (c) Beer

Figure 7: Empirical variation in expected coverage as a function of budget k on three smaller datasets. Parameter $T = 5$, $\delta = 25$ and $\Delta T = 45$ are fixed



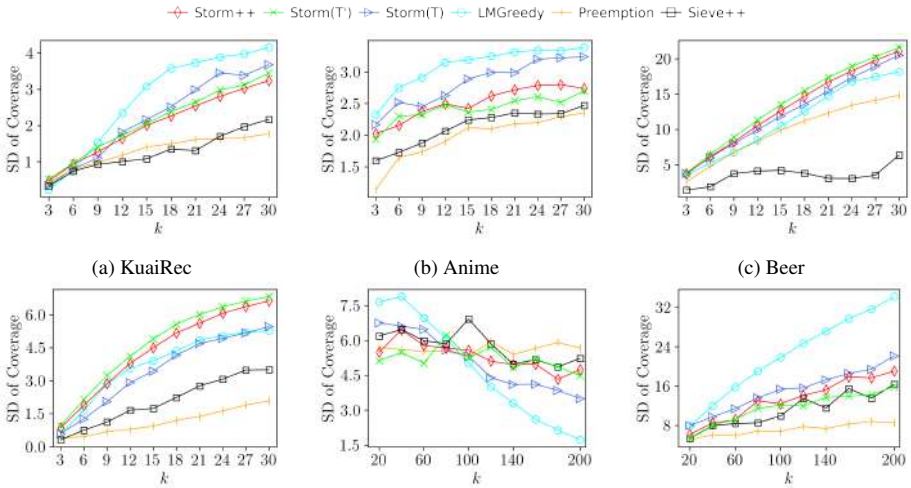
(a) KuaiRec (b) Anime (c) Beer

Figure 8: Empirical variation in expected coverage as a function of number of visits T on three smaller datasets. Parameter $k = 10$, $\delta = 10$ and $\Delta T = 10$ are fixed.



(a) (b)

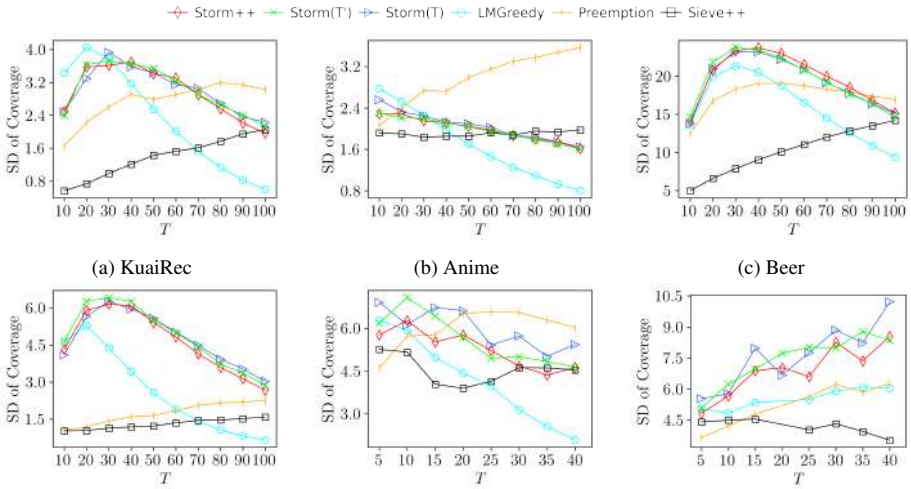
Figure 9: Impact of ΔT and δ on expected coverage on three larger datasets. In (a) and (b), we fix $k = 5$ and $T = 10$. When varying δ, we fix $\Delta T = 60$; when varying ΔT, we fix $\delta = 10$.



(a) KuaiRec (b) Anime (c) Beer

(d) Yahoo (e) RCV1 (f) Amazon

Figure 10: Empirical variation in standard deviation for the expected coverage as a function of budget k on all datasets. Parameter $T = 5$, $\delta = 25$ and $\Delta T = 45$ are fixed.



(a) KuaiRec (b) Anime (c) Beer

(d) Yahoo (e) RCV1 (f) Amazon

Figure 11: Empirical variation in standard deviation for the expected coverage as a function of the number of user visits T on all datasets. Parameter $k = 10$, $\delta = 10$ and $\Delta T = 10$ are fixed.

Paper C

Finding Densest Subgraphs with Edge-Color Constraints

Lutz Oettershagen
lutzo@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Honglian Wang
honglian@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Aristides Gionis
argioni@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

ABSTRACT

We consider a variant of the densest subgraph problem in networks with single or multiple edge attributes. For example, in a social network, the edge attributes may describe the type of relationship between users, such as friends, family, or acquaintances, or different types of communication. For conceptual simplicity, we view the attributes as *edge colors*. The new problem we address is to find a *diverse densest subgraph* that fulfills given requirements on the numbers of edges of specific colors. When searching for a dense social network community, our problem will enforce the requirement that the community is diverse according to criteria specified by the edge attributes. We show that the decision versions for finding *exactly*, *at most*, and *at least* **h** colored edges densest subgraph, where **h** is a vector of color requirements, are **NP**-complete, for already two colors. For the problem of finding a densest subgraph with *at least* **h** colored edges, we provide a linear-time constant-factor approximation algorithm when the input graph is sparse. On the way, we introduce the related at least h (non-colored) edges densest subgraph problem, show its hardness, and also provide a linear-time constant-factor approximation. In our experiments, we demonstrate the efficacy and efficiency of our new algorithms.

CCS CONCEPTS

• **Information systems** → **Social networks**; • **Theory of computation** → *Graph algorithms analysis.*

KEYWORDS

Density, Densest subgraph, Diversity, Social networks

ACM Reference Format:
Lutz Oettershagen, Honglian Wang, and Aristides Gionis. 2024. Finding Densest Subgraphs with Edge-Color Constraints. In *Proceedings of the ACM Web Conference 2024 (WWW '24), May 13–17, 2024, Singapore, Singapore.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3589334.3645647

1 INTRODUCTION

Graph analysis plays a pivotal role in understanding the intricate structure of the World Wide Web, offering insights into the relationships and connections that underpin its vast digital landscape. Finding densest subgraphs is a classical graph-theoretic problem and one of the most fundamental issues in graph data mining and social network analyses [20, 32]. In one of its most basic versions of the *densest-subgraph problem* (DSP), we are given an undirected finite graph $G = (V, E)$, and the goal is to find a subset of nodes $S \subseteq V$ such that the induced subgraph maximizes the ratio between edges and nodes $|E(S)|/|S|$. Examples of the many Web-related applications are, e.g., community detection in social networks [11, 15, 43], real-time story identification [3], identifying malicious behavior in financial transaction networks [31] or link-spam manipulating search engines [19], and team formation in social networks [17, 38]. The problem also has applications in other domains such as, e.g., analyzing biological networks [39, 45], or general applications in data structures like indexing of reachability and distance queries [12]. Recently, the increasing interest in algorithms that ensure fairness or diversity [30, 36] has been extended to finding *diverse dense subgraphs*. Anagnostopoulos et al. [1] and Miyauchi et al. [37] discuss variants of the DSP that include fairness and diversity properties in graphs with respect to the node attributes.

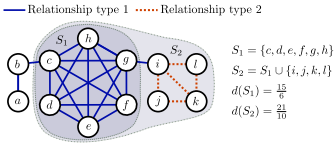


Figure 1: Example for the at least h colored edges densest subgraph problem in a toy social network with two relationship types. The subgraph induced by S_1 is the densest unconstrained subgraph. If we require the densest subgraph to contain at least four edges of type two (red dashed), the graph induced by S_2 is optimal.

Our work: We introduce new problem definitions for finding *edge-diverse* dense subgraphs in graphs with *categorical edge attributes*, which we, for conceptual simplicity, denote as edge colors. Specifically, we introduce the problems of finding a densest subgraph with *at least* **h** *colored* edges, where the vector **h** contains for each attribute, i.e., color, the minimum number of edges that are required to be in the solution. Similarly, we define two variants for *exactly* and *at most* **h** colored edges. Figure 1 shows a small toy example of a network containing two different relationship types. Computing the standard densest subgraph leads to the monochrome subgraph induced by S_1. To obtain a diverse subgraph that also contains edges of relationship type two, we apply the at least **h** colored edges densest subgraph variant and can identify the densest subgraph induced by S_2 that contains edges of relationship type two. We can apply our new problem in the following scenarios in Web-related networks.

Web-graph analysis: Consider a large web graph in which nodes represent online articles published on websites or blogs and edges represent relations between articles. The edges can represent, e.g.,

This work is licensed under a Creative Commons Attribution International 4.0 License.

WWW '24, May 13–17, 2024, Singapore, Singapore
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0171-9/24/05.
https://doi.org/10.1145/3589334.3645647

citations, extensions, or hyperlinks between the websites. Additionally, the edges are annotated with meta-information further describing the relationship, e.g., shared topic, type of relationship between the articles, agreement, refusal, or sentiment between articles. Now, a typical task is to obtain a summary highlighting the most interconnected parts of the network [31, 32]. However, without further restrictions on the edge attributes, the resulting subgraph may completely ignore or exclude specific attributes that are not part of the unconstrained densest subgraph. Using the at least **h** colored densest subgraph, a user has the possibility to include specific attributes into the network summary by setting the corresponding entries in the requirement vector **h** to the minimum number of included relations between the articles.

Online social-network analysis: The participants of large-scale social networks are commonly connected to hundreds or even thousands of other users. Typically, user relationships are heterogenic and can be distinguished in, e.g., friendship, family membership, acquaintance, or work colleague [24]. Additionally, the strength of the relationship is often classified into weak and strong ties, where weak ties often have the capacity to bridge diverse social groups and facilitate the flow of information [22, 41]. By requiring specific numbers of edge attributes, we identify densest subgraphs related across multiple attribute dimensions. In addition to mere interconnections, the resulting dense subgraphs embody communities with diverse relationships. Moreover, identifying dense subgraphs with minimal specific relationships can be advantageous for subsequent tasks, such as content recommendation. For instance, a dense subgraph including many weak professional connections can form the foundation for recommending new professional contacts bridging into new social groups.

Fraud detection in financial networks: Here, nodes represent users and edges represent financial transactions between users. The goal is to identify a dense subgraph of transactions that indicates potentially fraudulent activity by including a minimum number of high-value transactions, transactions in specific locations, and transactions during non-business hours. A dense subgraph, in this case, can represent a group of users with transactions that might be indicative of coordinated fraudulent activity.

Contributions: Our contributions are the following.

1. We introduce new variants of diverse densest subgraph problems in edge-colored graphs. We are interested in finding densest subgraphs that contain exactly h_i, at most h_i, or at least h_i edges of color $i \in [\pi]$ where π is the number of colors in the graph. We discuss variants of the problems in which each edge either has a single or multiple colors. We show that the corresponding decision problems are **NP**-complete.
2. For the problem of at least **h** colored edges in sparse graphs, we introduce a linear-time $O(1)$-approximation algorithm.
3. As an additional result, we introduce the densest subgraph problem with at least h (non-colored) edges. We show that the problem is **NP**-hard as well, and also provide a linear-time $O(1)$-approximation algorithm.
4. We evaluate our algorithms on real-world networks and demonstrate that (i) our approximation algorithms have very low relative approximation errors, in most cases under one percent, and (ii) are highly efficient.

Please refer to Appendix A for the omitted proofs.

2 RELATED WORK

Finding densest subgraphs. Finding densest subgraphs is a fundamental problem in network analysis and has a variety of applications. The problem has gained increasing interest in recent years, both in theoretical computer science and data-mining communities. We discuss only the most relevant work. For a recent survey on the topic, we refer the reader to Lanciano et al. [31].

The unconstrained version of the problem, when the density of a subgraph induced by a subset of vertices $S \subseteq V$ of a graph $G = (V, E)$ is defined as $d(S) = |E(S)|/|S|$, is solvable in polynomial time via max-flow computations [21]. For a more efficient but approximate solution, a linear-time greedy algorithm, which removes iteratively the node of the smallest degree and returns the best solution encountered, provides an approximation ratio equal to two [4, 9]. That type of greedy algorithm is often referred to as *peeling*. Recently, Chekuri et al. [10] provided an almost linear-time flow-based algorithm, approximating the densest subgraph problem within $(1 + \epsilon)$. Chekuri et al. [10] also analyzed an iterative peeling algorithm proposed by Boob et al. [7] and showed that it converges to optimality. Research has also focused on the problems of finding densest subgraphs with at most k nodes (Dam$k$S), at least k nodes (Dal$k$S), and exactly k nodes (DkS). The DkS problem is **NP**-hard, even when restricted to graphs of maximum degree equal to 3 [16], and the best-known approximation ratio is $O(n^{1/4})$ [6]. With respect to the upper-bound variant, Khuller and Saha [28] showed that an α-approximation for the Dam$k$S problem leads to an $\alpha/4$-approximation for DkS.

More related to our work is the Dal$k$S problem, which is also **NP**-hard [28]. Andersen and Chellapilla [2] designed a linear-time 1/3-approximation algorithm based on greedy peeling, while Khuller and Saha [28] provided two algorithms, both yielding a 1/2-approximation, using flow computations and solving an LP, respectively.

Our work is also related to finding densest subgraphs in multilayer networks, see, e.g., [23, 25, 29]. Galimberti et al. [18] discussed the k-core decomposition and densest subgraph problems for multilayer networks and provided an approximation algorithm for a different formulation than the one we study in this paper. We experimentally compare our algorithm with the method of Galimberti et al. [18] and show that our approach finds denser subgraphs.

Diverse densest subgraphs. Two recent works consider diversity in finding densest subgraphs. Anagnostopoulos et al. [1] introduce the *fair densest subgraph problem*. The authors consider graphs with nodes labeled by two colors, and the goal is to find a subset of nodes that contains an equal number of colors. They show that their problem is at least as hard as the Dam$k$S problem. Moreover, they propose a spectral algorithm based on ideas by Kannan and Vinay [27]. In the second work, Miyauchi et al. [37] discuss generalizations of the problem introduced by Anagnostopoulos et al. [1]. They introduce two problem variants. The first problem guarantees that no color represents more than some fraction of the nodes in the output subgraph. The second problem is the "node version" of the problem we discuss, i.e., they study the densest subgraph problem, in which, given a vector of cardinality demands for each color class, the task is to find a densest subgraph fulfilling the demands.

Both of these works focus on node-attributed graphs, while we study the densest subgraph problem for edge-attributed graphs.

3 PROBLEM DEFINITIONS

We use $\mathbb{N}$ to denote the natural numbers (without zero). Furthermore, for $x \in \mathbb{N}$, we use $[x]$ to denote the set $\{1, \ldots, x\}$. Vectors are denoted in boldface, e.g., $\mathbf{h} \in \mathbb{N}^x$, and h_i represents the i-th entry of $\mathbf{h}$. An undirected, simple graph $G = (V, E)$ consists of a finite set of vertices V and a finite set $E \subseteq \{\{u, v\} \subseteq V \mid u \neq v\}$ of undirected edges. We define $n = |V|$ and $m = |E|$. An edge-colored graph $G = (V, E, c)$ is a graph with an additional function $c : E \to 2^{\mathbb{N}}$ assigning sets of colors to the edges. For notational convenience, we write $c(e) = i$ instead of $c(e) = \{i\}$ if edge $e \in E$ is assigned a single color. For $S \subseteq V$, we define $E(S) = \{\{u, v\} \in E \mid u, v \in S\}$ and $G(S) = (S, E(S))$ the subgraph induced by S .

Definition 1. *Given an edge-colored graph $G = (V, E, c)$, a number of colors $\pi \in \mathbb{N}$, and a vector $\mathbf{h} \in \mathbb{N}^{\pi}$, find $S \subseteq V$ such that*

- *$E(S)$ contains at least h_i edges e with $i \in c(e)$ $\forall$ $i \in [\pi]$, and*
- *the density $d(S) = \frac{|E(S)|}{|S|}$ is maximized.*

We denote the problem as ALHCEDGESDSP.

Similarly, we define the *exactly* $\mathbf{h}$ and the *at most* $\mathbf{h}$ colored edges densest subgraph problem variants.

We can check the feasibility for the at least $\mathbf{h}$ and the exactly $\mathbf{h}$ colored edges variants in linear time by counting the occurrences of colors $i \in [\pi]$ at all edges in G. In the case of the at most $\mathbf{h}$ colored edges variant, the empty subgraph is a feasible solution. Hence, in the following, we do not make the feasibility check explicit and consider instances to be feasible. We assume, without loss of generality, that for each $i \in [\pi]$ the requirement is non-trivial, i.e., $h_i > 0$.

Complexity. In contrast to the standard variant of the densest subgraph problem, which can be solved optimally in polynomial time, adding constraints on the numbers of colored edges makes the problems hard. Indeed, we show hardness already for the case that each edge is colored by one of only two colors.

Theorem 1. *The decision versions of the exactly, at most, and at least* $\mathbf{h}$ *colored edges version are* **NP**-*complete.*

4 APPROXIMATION IN SPARSE GRAPHS

In this section, we present a $O(1)$-approximation for at least $\mathbf{h}$ colored edges densest subgraph problem (ALHCEDGESDSP) for *everywhere sparse graphs.*

We call a graph $G = (V, E)$ sparse if $|E| = O(|V|)$. A graph $G = (V, E)$ is *everywhere sparse* if for any subset $V' \subseteq V$ the by V' induced subgraph $G(V')$ is sparse. We first focus on the case that each edge in the graph is assigned a single color and discuss the general case in Section 4.3.

Our approximation for ALHCEDGESDSP is based on finding densest subgraphs with at least h edges (ignoring colors of the edges). We define the problem as follows.

Definition 2. *Given a graph $G = (V, E)$ and $h \in \mathbb{N}$, find a subset $S \subseteq V$ such that $|E(S)| \geq h$, and the density $d(S) = \frac{|E(S)|}{|S|}$ is maximized.*

Algorithm 1: Algorithm for ATLEASTHEDGESDSP

Input: Graph $G = (V, E)$ and $h \in \mathbb{N}$
Output: Densest subgraph with at least h edges
for $i \in \{\ell(h), \ldots, n\}$ **do**
  Compute the at least i nodes DSP S_i
return S_i with maximal density and at least h edges

Algorithm 2: Algorithm for ATLEASTHEDGESDSP

Input: Everywhere sparse graph $G = (V, E)$ and $h \in \mathbb{N}$
Output: Approx. of densest subgraph with at least h edges
for $i \in \{\ell(h), \ldots, n\}$ **do**
  α-approximate the at least i-nodes DSP and obtain solution S_i
  if *$G(S_i)$ does not have at least h edges* **then**
    add edges to $G(S_i)$ to obtain G'_i with at least h edges
return G'_i with maximum density

We denote the problem with ATLEASTHEDGESDSP and show that this problem without colors is already hard.

Theorem 2. *The decision problem of the at least h edges DSP is* **NP**-*complete.*

4.1 Solving the at least h edges densest-subgraph problem

First, assume we have an algorithm for the at least k-nodes DSP problem. We can use it to solve the at least h-edges DSP problem (ATLEASTHEDGESDSP). To this end, let $\ell(h)$ a lower bound on the number of nodes of a graph with at least h edges. For generality, define the lower bound $\ell(h, p)$ for graphs with up to p parallel edges between two nodes and define $\ell(h) = \ell(h, 1)$ (we use the general version for the case of multigraphs as discussed in Section 4.3).

Lemma 1. *Let $G = (V, E)$ be a graph with $|E| \geq h$ and at most p parallel edges between each pair of nodes. Then $\ell(h, p) = \frac{1}{2} + \frac{\sqrt{p^2+8hp}}{2p}$.*

The following lemma establishes a connection between the at least k nodes and at least h edges DSP.

Lemma 2. *Given a graph G and $h \in \mathbb{N}$. Let k be the minimum number of nodes over all graphs that are densest subgraphs of G with at least h edges. Furthermore, let S be an optimal solution for the at least k-nodes DSP problem in G. Then S is also an optimal solution for the densest subgraph with at least h edges.*

Based on Lemma 2 Algorithm 1 computes the solution of the at least h edges DSP.

Theorem 3. *Algorithm 1 is optimal for* ATLEASTHEDGESDSP.

Now, let G be an everywhere sparse graph, and assume we have an α-approximation algorithm for the at least k-nodes DSP problem. We can obtain an $O(1)$-approximation for the least h-edges DSP problem (ATLEASTHEDGESDSP).

Theorem 4. *Algorithm 2 gives an $O(1)$-approximation for* ATLEASTHEDGESDSP.

Algorithm 3: Approximation for atLeastHEdgesDSP

Input: Everywhere sparse graph $G = (V, E)$ and $h \in \mathbb{N}$
Output: Approx. of densest subgraph with at least h edges

$G_0 \leftarrow G$, $i \leftarrow 0$, and $i_{\max} \leftarrow 0$
while $|E(G_i)| \geq h$ **do**
  $i_{\max} \leftarrow i$
  Increment i
  Let v_i be a node with minimum degree
  $G_i \leftarrow G_{i-1} \setminus \{v_i\}$ *// remove v_i and all incident edges*
return G_i for $i \in \{0, \ldots, i_{\max}\}$ with maximum density

Proof. Let k be the minimum number of nodes over all graphs that are densest subgraphs with at least h edges. And, let $d(S)$ be an α-approximation of the at least k-nodes DSP. Then, from Lemma 2, it follows that $d(S)$ is also an α-approximation of the at least h-edges DSP. However, S may not be a feasible solution because it may have less than h edges, i.e., $|E(S)| < h$. Assume that for a constant $c_1 \in \mathbb{N}$ the result of the α-approximation, it holds that $c_1|E(S)| \geq |E(S^*)| \geq h$, where S^* is the optimal solution for the at least i-nodes DSP. Algorithm 2 adds the possibly missing edges to obtain the subgraphs $G'_i = (S'_i, E'_i)$ (line 4). At most h edges and $2h$ nodes are added, and $|S'_i| \leq |S| + 2h \leq |S| + 2c_1|E(S)| \leq c|S|$ where the last inequality holds due to the everywhere sparseness property of G for a large enough constant $c \in \mathbb{N}$. Consequently,

$$d(S') = \frac{|E(S')|}{|S'|} \geq \frac{|E(S)|}{c|S|} = \frac{1}{c}d(S) \geq \frac{1}{c\alpha}d^*,$$

where d^* is the optimal density. □

The assumption that $c_1|E(S)| \geq |E(S^*)| \geq h$ with $c_1 \in \mathbb{N}$ holds for example for the 2-approximation and 3-approximation algorithms provided by Khuller and Saha [28] and Andersen and Chellapilla [2], respectively.

The running time complexity of Algorithm 2 is in $O(n(T_{\text{appr}} + h))$ with T_{appr} being the running time of the at least i nodes DSP approximation as we have n rounds and in each round we call the approximation and have to add at most h edges to obtain G'_i. Using the 3-approximation algorithm by Andersen and Chellapilla [2] based on the k-core computation, we can obtain an approximation with total running time in $O(n + m)$.

Theorem 5. *Algorithm 3 is a $O(1)$-approximation for atLeastHEdgesDSP with running time in $O(n + m)$.*

Proof. Algorithm 3 peels away low degree nodes and thus obtains $G_0, \ldots, G_{i_{\max}}$. Assume we similarly computed the remaining graphs $G_{i_{\max}+1}, \ldots, G_n$ (as in the standard k-core decomposition). Andersen and Chellapilla [2] showed that for each possible $j \in [n]$ one of the $G_0, \ldots, G_{n-j}$ is a 3-approximation for the at least j-nodes DSP, and the number of edges is at least 1/3 of the optimal solution.

Now, let k be the minimum number of nodes over all graphs that are densest subgraphs with at least h edges. And, let $d(S)$ be the 3-approximation of the at least k-nodes DSP. Then, from Lemma 2, it follows that $d(S)$ is also an 3-approximation of the at least h-edges DSP. There are two cases:

Algorithm 4: Approximation for alhcEdgesDSP

Input: Everywhere sparse graph $G = (V, E)$ and $\mathbf{h} \in \mathbb{N}^\pi$
Output: Approx. of densest subgr. with at least h_i edges of color $i \in [\pi]$

Approximate the densest subgraph $G' = (V', E')$ with at least $\sum_{i=1}^{\pi} h_i$ edges
Let f_i be the number of edges of color i in G' for $i \in [\pi]$
$G'' \leftarrow G'$
for $1 \leq i \leq \pi$ **do**
  Add $\max\{0, h_i - f_i\}$ edges of color i to G''
return G''

1. $G(S)$ contains at least h edges, i.e., corresponds to one of the graphs in $\{G_0, \ldots, G_{i_{\max}}\}$. In this case, we are done, and $G(S)$ is a 3-approximation for atLeastHEdgesDSP.
2. $G(S)$ contains less than h edges (but at least 1/3 of the optimal solution), i.e., corresponds to a graph H in $G_{i_{\max}+1}, \ldots, G_{n-k}$. In this case, we need to add edges such that $G(S)$ is feasible. Let S' be the resulting vertex set. With similar arguments as in the proof of Theorem 4, it follows that $d(S') \geq \frac{1}{c}d^*$ for a large enough constant $c \in \mathbb{N}$ and with d^* being the optimal density. Now, as we can choose the edges that we add to achieve feasibility, we choose exactly the edges in $E(G_{i_{\max}}) \setminus E(H)$ such that $G(S') = G_{i_{\max}}$. Note that we add in the worst case at most $2h$ nodes. Hence, $G_{i_{\max}}$ is a c-approximation.

As Algorithm 3 returns the $G_i \in \{G_0, \ldots, G_{i_{\max}}\}$ with maximum density, in either case, we obtain a $O(1)$-approximation for atLeastHEdgesDSP. The running time complexity of Algorithm 3 is equal to the peeling-based k-core decomposition, which is $O(|V| + |E|)$ as shown by Batagelj and Zaversnik [5]. □

4.2 Approximation of the at least h colored edges densest-subgraph problem

We now can use Algorithm 2 or Algorithm 3 to obtain a $O(1)$-approximation for the alhcEdgesDSP problem in everywhere sparse graphs as shown in Algorithm 4.

Theorem 6. *Algorithm 4 gives an $O(1)$-approximation for alhcEdgesDSP.*

Proof. Let $G'' = (V'', E'')$ be the final resulting subgraph. Because atLeastHEdgesDSP relaxes alhcEdgesDSP, we have $d(V') \geq \frac{d^*}{\alpha}$ using an α-approximation for the at least h edges subgraph, and where d^* is the optimal density of atLeastHEdgesDSP. We add in total

$$\ell = \sum_{i=1}^{\pi} \max\{0, h_i - f_i\} \leq \sum_{i=1}^{\pi} h_i \leq |E'|$$

edges to $G'' = (V'', E'')$ to ensure feasibility for alhcEdgesDSP. Because each edge adds at most two vertices and $|V''| \leq |V'| + 2\ell \leq |V'| + 2|E'| \leq c|V'|$ with $3 \leq c \in \mathbb{N}$ and it follows $d(V'') = \frac{|E''|}{|V''|} \geq \frac{|E'|}{c|V'|} = \frac{1}{c}d(V') \geq \frac{1}{c\alpha}d^*$. □

In Algorithm 4, after obtaining G' from Algorithm 3 we might need to add missing edges. Note that G' is node-induced, and we

have to insert at least one new node. Of course, adding all edges of the new node to $G'' = (V'', E'')$ only improves the density. To add the missing nodes, we store the nodes that are removed in the call of Algorithm 3 that lead to missing edges. Therefore, we use a slightly modified version Algorithm 3 as a subroutine to approximate the at least h edges DSP. During the iterations of the while-loop in Algorithm 3, vertices and their incident edges are removed from the graph. Here, if in iteration i we have to remove an edge $e = \{u, v\}$ with $c(e) = c$ and the remaining edges of color c is smaller than h_c, i.e., removing e leads to a deficit of color c edges, then we add both endpoints of e to a set B_i, where $B_i = B_{i-1} \cup \{u, v\}$ (and $B_0 = \emptyset$). At the end of the subroutine, we return $G_i = (V_i, E_i)$ for $i \in \{0, \ldots, i_{\max}\}$ with maximum density and the corresponding set B_i. Note that $|B_i| \leq 2\ell$, i.e., for each missing colored edge, at most two nodes are in B_i. Therefore, we can just add the nodes in B_i and return as a final result of Algorithm 4 the graph $G'' = (V' \cup B_i, E')$, which leads to a running time of $O(n + m)$ of Algorithm 4.

4.3 Graphs with multiple edge colors

Up to this point, our focus was on graphs with single-colored edges. However, in practical contexts, edges often bear multiple distinct colors, each representing diverse aspects of node interactions. Discovering the densest subgraph that illuminates these varying types of node interactions holds intrinsic value. To accommodate this case, we transform a simple graph with multiple edge colors into a colored multigraph, where each edge is assigned a single color.

An undirected edge-colored multigraph M is defined as a tuple $M = (V, E_M, f, c_M)$ where V is a finite set of vertices, E_M is a finite set of edges, $f : E_M \to \{\{u, v\} : u, v \in V\}$ is a function mapping edges to pairs of vertices, and $c_M : E_M \to \mathbb{N}$ is a function assigning to each edge a single color. If $e_1, e_2 \in E_M$ and $f(e_1) = f(e_2)$, then we call e_1 and e_2 multiple or parallel edges. For a subset S of vertices $S \subseteq V$, we define $E_M(S) = \{e \in E_M \mid f(e) \subseteq S\}$ and $M(S) = (S, E_M(S), f, c_M)$ the multigraph induced by S. The density of $M(S)$ is defined as $d(S) = \frac{|E_M(S)|}{|S|}$.

Given an edge-colored simple graph $G = (V, E, c)$ with multiple edge colors, we construct an associated multigraph $M = (V, E_M, f, c_M)$ sharing the same set of nodes, V. For each edge $e \in E$ and for each color $i \in c(e)$, we introduce an edge e' into E_M and assign $c_M(e')$ to be equal to i. If G is everywhere sparse, then it follows that M is also everywhere sparse if we consider the number of distinct colors π to be a small constant, which is commonly the case for real-world networks (see, e.g., Table 1). Furthermore, note that the density of the obtained multigraph can be larger than that of the original graph. By counting the parallel edges separately, we account for the fact that edges with many colors have higher importance as they cover more of the color requirements compared to edges with single or few colors.

The insights and results derived in previous sections extend seamlessly to this multigraph framework, resulting in Algorithm 5 and the following theorem.

Theorem 7. *Algorithm 5 gives an $O(1)$-approximation for* ALHCEDGESDSP *on graphs with multiple edge colors.*

Algorithm 5: Approximation for ALHCEDGESDSP on graphs with multiple edge colors

Input: Everywhere sparse graph $G = (V, E, c)$ and $\mathbf{h} \in \mathbb{N}^{\pi}$
Output: Approximation of densest subgraph with at least h_i edges of color i for $i \in [\pi]$

Transform $G = (V, E, c)$ into multigraph $M = (V, E_M, f, c_M)$
Use Line 1-3 from Algorithm 4 applied to M
while *there exists* $i \in [\pi]$ *such that* $f_i < h_i$ **do**
  choose two nodes $u, v \in V$ that are connected by an edge of color i and add all parallel edges between u, v to G''
  Update f_i for $i \in [\pi]$
return G''

5 EXPERIMENTS

In this section, we evaluate our algorithms in terms of their efficacy and efficiency by discussing the following **research questions:**

- **Q1:** How does our approximation for at least h edges problem perform in terms of approximation quality?
- **Q2:** How does increasing h affect the density and running time?
- **Q3:** How are the colors in the data sets and the unconstrained densest subgraphs distributed?
- **Q4:** How does our approximation for the at least $\mathbf{h}$ color edges problem perform in terms of approximation quality?
- **Q5:** How do increasing color requirements affect the densities?
- **Q6:** How is the efficiency of our approximation for the at least $\mathbf{h}$ color edges problem in terms of running time?

Additionally, we discuss in Section 5.2, as a use case, the identification of densest subgraphs that contain publications at popular data mining conferences in a coauthor graph.

Algorithms: We use the following algorithms for our evaluation.

- `AtLeastHApprox` is the implementation of Algorithm 4 for the at least h edges DSP.
- `ColApprox` is the implementation of Algorithm 4 for the at least h colored edges DSP.
- `AtLeastHILP` and `ColILP` are the exact integer linear programs for finding the densest subgraph with at least h edges and the densest subgraph with at least $\mathbf{h}$ colored edges, respectively. The ILPs are provided in Appendix B.
- `Heuristic` is a new baseline algorithm. As there is no baseline available for the at least $\mathbf{h}$ colored edges problem (Definition 1), we introduce a heuristic, serving for comparison and benchmarking. Given an edge colored graph $G = (V, E, c)$, the heuristic peels away nodes with the lowest degree as long as all color requirements are fulfilled.
- `Mlds` is a state-of-the-art approximation for the multilayer densest subgraph problem defined by Galimberti et al. [18]. The authors provided the source code.

We implemented our algorithms in C++ using GNU g++ compiler 11.4.0. with the flag `-O3`. The ILPs were implemented in Python 3.9 and ran with Gurobi 9.1.2. The experiments ran on a single machine with an Intel i5-1345U CPU and 32GB of main memory. The source code is available at https://gitlab.com/densest/diverse.

Table 1: Statistics of the data sets. $d(S^*)$ denotes the density of the unconstrained optimal densest subgraph and $\psi(G)$ denotes the maximal number of colors per edge.

Data set	$\|V(G)\|$	$\|E(G)\|$	$d(S^*)$	#Colors	$\psi(G)$	Category	Ref.
AUCS	61	353	6.2	5	5	Multilayer social	[35]
Hospital	75	1 139	16.3	5	5	Temporal face-to-face	[44]
HtmlConf	113	2 196	20.5	3	3	Temporal face-to-face	[26]
Airports	417	2953	16.5	37	5	Multilayer transportation	[8]
Rattus	2 634	3 677	3.7	6	4	Multilayer biological	[13]
FfTwYt	6 401	60 583	39.5	3	3	Multilayer social	[14]
Knowledge	14 505	210 946	34.8	30	4	Knowledge graph	[42]
HomoSap	18 190	137 659	38.5	7	5	Multilayer biological	[18]
Epinions	131 580	592 013	85.6	2	2	Signed (trust/no trust)	[33]
DBLP	344 814	1 528 399	57.0	168	21	Multilayer collaboration	[34]
Twitter	346 573	1 088 260	45.1	2	2	Signed (fact/non-fact)	[40]
FriendFeed	505 104	18 319 862	500.1	3	3	Multilayer social	[14]

Table 2: Results and relative approximation errors in percent (%) for the at least h-edges densest-subgraph problem.

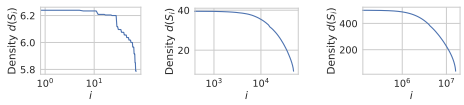
			Relative approximation errors (%)			
Data set	Opt. solved (%)	Within 1% err. (%)	Mean	Std. dev.	Median	Max.
AUCS	38.9	93.1	0.59	0.31	0.42	1.24
Hospital	91.7	100	0.44	0.23	0.47	0.81
HtmlConf	83.0	100	0.14	0.13	0.10	0.83

Data Sets: We use twelve real-world data sets from different domains and a wide range of different numbers of attributes, i.e., colors. Table 1 gives an overview of the statistics. We provide detailed descriptions in Appendix C.

5.1 Results and discussion

Q1 — approximation quality of `AtLeastHApprox`. To evaluate the approximation error of `AtLeastHApprox`, we computed the at least h edges DSP for the *AUCS*, *Hospital*, and *HtmlConf* data sets using `AtLeastHApprox` and the exact ILP approach (`AtLeastHILP`) for all values of $w < h \leq |E(G)|$ with w is the number of edges in the optimal unconstrained DSP. Table 2 shows the percentage of runs that were optimal, i.e., relative approximation error of zero, the percentage of the runs with a relative approximation error of at most one, and the statistics of the relative approximation errors in percent (%) for the runs that were not optimal. For *Hospital* and *HtmlConf* over 90% and 80%, respectively, of the instances are solved perfectly by `AtLeastHApprox`, and all instances are solved with a relative error of less than one. In the case of the *AUCS* data set, this value is lower with 38.9%. However, here, the mean and median relative approximation errors are also less than one percent, and more than 93% of the instances are solved with an error of at most one. The maximum relative error is at 1.24%.

Q2 — density and running times for increasing h. We computed the at least h-edges DSP where we choose $h = w + i$ with w being the number of edges in the unconstrained DSP and $1 \leq i \leq |E| - w$. Figure 2 shows the results for *AUCS*, *FfTwYt*, and *FriendFeed*, and Figure 7 in the appendix shows results for the remaining data sets. In all data sets, the densities of the subgraphs S_i strongly decrease for larger i. As the size of h increases, the peeling process can stop earlier, which leads to shorter running times: Table 3 shows the running times for the four largest data sets and increasing ratios r such that $i = r\,(|E| - w)$. The running times are in seconds, and

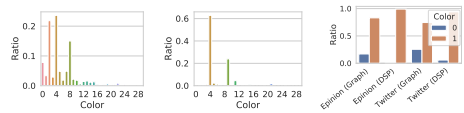


(a) *AUCS* (b) *FfTwYt* (c) *FriendFeed*

Figure 2: The density computed with `AtLeastHApprox` for increasing numbers of required edges.

Table 3: Mean running times and standard deviations of `AtLeastHApprox` in seconds (s).

	$i = r \cdot (\|E\| - w)$								
Data set	$r = 0.1$	$r = 0.2$	$r = 0.3$	$r = 0.4$	$r = 0.5$	$r = 0.6$	$r = 0.7$	$r = 0.8$	$r = 0.9$
Epinions	$0.31_{\pm 0.0}$	$0.28_{\pm 0.0}$	$0.25_{\pm 0.0}$	$0.22_{\pm 0.0}$	$0.19_{\pm 0.0}$	$0.16_{\pm 0.0}$	$0.14_{\pm 0.0}$	$0.10_{\pm 0.0}$	$0.07_{\pm 0.0}$
DBLP	$1.28_{\pm 0.0}$	$1.17_{\pm 0.0}$	$1.04_{\pm 0.0}$	$0.92_{\pm 0.0}$	$0.81_{\pm 0.0}$	$0.69_{\pm 0.0}$	$0.56_{\pm 0.0}$	$0.43_{\pm 0.0}$	$0.29_{\pm 0.0}$
Twitter	$0.84_{\pm 0.0}$	$0.76_{\pm 0.0}$	$0.69_{\pm 0.0}$	$0.63_{\pm 0.0}$	$0.56_{\pm 0.0}$	$0.49_{\pm 0.0}$	$0.42_{\pm 0.0}$	$0.32_{\pm 0.0}$	$0.19_{\pm 0.0}$
FriendFeed	$18.99_{\pm 0.5}$	$16.73_{\pm 0.4}$	$14.45_{\pm 0.4}$	$12.31_{\pm 0.5}$	$9.85_{\pm 0.3}$	$7.74_{\pm 0.3}$	$5.66_{\pm 0.2}$	$3.90_{\pm 0.4}$	$2.02_{\pm 0.1}$



(a) *Knowledge* (Graph) (b) *Knowledge* (DSP) (c) Binary data sets

Figure 3: The distributions of colors in various data sets.

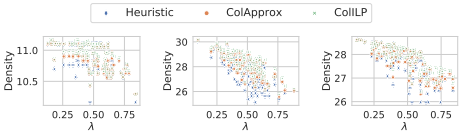
the mean values and standard deviations are over ten repetitions. As expected, the running time decreases with increasing value of i.

Q3 — distribution of colors in unconstrained densest-subgraph problem. We empirically verify the necessity of diversity in edge-colored graphs and the densest subgraphs by assessing the distribution of colors in the graphs and densest subgraphs. The findings consistently show that the fractions of the different colors are not equal in the data sets. Furthermore, often, the distribution of the colors in the unconstrained DSP differs significantly from the distribution in the complete graph. Figure 3 shows the distributions of the *Knowledge* and all data sets with two colors. Figure 3a and Figure 3b show the color distributions in the *Knowledge* data set and the (unconstrained) densest subgraph. The distribution between the graph and the DSP differs for most colors. Similarly, we see in the bichromatic data sets significant differences in the fractions of colors between the complete graph and the densest subgraph. Hence, even if there are many edges of a specific color in a graph, this does not generally mean that the densest subgraph contains a particular number of edges of this color. This validates the motivation for our new problems and algorithms. Moreover, even if the distributions of the colors in the graph and the DSP are similar, we still might want to find a subgraph with specific numbers of edges of given colors, which further justifies our approaches.

Q4 — approximation quality of `ColApprox`. To evaluate the approximation quality of `ColApprox`, we first computed the at least h colored edges DSP for the *AUCS*, *Hospital*, and *HtmlConf* data sets

Table 4: Results and relative approximation errors in percent (%) for the at least h colored edges densest-subgraph problem.

				Relative approximation errors (%)			
Algorithm	Data set	Opt. solved (%)	Within 1% err. (%)	Mean	Std. dev.	Median	Max.
Heuristic	*AUCS*	21	76	0.65	0.82	0.27	3.58
	Hospital	1	19	2.64	1.94	2.14	9.10
	HtmlConf	6	62	0.99	0.87	0.79	3.69
ColApprox	*AUCS*	22	95	0.30	0.34	0.15	1.30
	Hospital	7	74	0.76	0.41	0.72	1.74
	HtmlConf	9	86	0.46	0.40	0.36	1.89



(a) *AUCS* (b) *Hospital* (c) *HtmlConf*

Figure 4: Comparison of the heuristic, approximation algorithm, and exact ILP.

using Heuristic, ColApprox, and the exact ILP approach (ColILP). We chose 100 random problem instances. To this end, let f_i and g_i be the number of edges of color i in the unconstrained DSP and graph G, respectively. For each color $c \in [\pi]$, we chose h_c uniformly at random from the interval $[f_c, g_c]$. Furthermore, let $\lambda = \frac{\sum_{c\in[\pi]} h_c}{\sum_{c\in[\pi]}(g_c - f_c)}$ denote the fraction of edges that are required from all possible additional edges. Figure 4 shows the densities of the solved problem instances with respect to λ. Moreover, Table 4 shows the statistics of the relative approximation errors in percent as well as the percentage of instances solved optimally or within a relative error of at most one. We see that ColApprox solves both more instances optimally and in the range of an error of at most one than Heuristic. The relative approximation errors are generally lower for ColApprox with mean values (much) smaller than one and maximum values of at most 1.89%.

Q5 — density for increasing color requirements. We computed the densities of the densest subgraph for increasing color requirements using ColApprox and Heuristic. To this end, we first computed the unconstrained DSP $H = (S, F)$. Let f_c be the number edges of colors $c \in [\pi]$ in F, and t_c be the total number of color $c \in [\pi]$ edges in G. For each color $c \in [\pi]$, we split define $r_c = (t_c - f_c)/10$. We then defined $h^i_c = i \cdot r_c$ for $i \in [10]$, leading to ten color requirement vectors h^i with $i \in [10]$. Figure 5 shows the densities computed with ColApprox and Heuristic. For increasing numbers of required edges, the densities decrease. Compared to the Heuristic, our approximation algorithm ColApprox leads to higher or at least as high densities for all data sets. For some data sets, e.g., *AUCS*, *HtmlConf*, *Epinions*, and *FriendFeed*, the Heuristic performs similarly good as our approximation. The reason is that for these data sets, all required colored edges are in the densest subgraphs. As the heuristic peels away nodes while the color requirements are not violated, the densest, or close to the densest, subgraph can be obtained in many cases. Also see **Q4** for a comparison between

Table 5: Running times in seconds (s) for computing the at least h colored edges densest-subgraph problem.

	$i = 2$		$i = 4$		$i = 6$		$i = 8$	
Data set	Heuristic	ColApprox	Heuristic	ColApprox	Heuristic	ColApprox	Heuristic	ColApprox
Epinions	0.45±0.0	0.45±0.0	0.37±0.0	0.39±0.0	0.33±0.0	0.33±0.0	0.28±0.0	0.26±0.0
DBLP	1.25±0.0	1.74±0.0	1.01±0.0	1.48±0.0	0.94±0.0	1.21±0.1	0.84±0.1	0.88±0.0
Twitter	1.00±0.0	1.21±0.0	0.86±0.0	1.07±0.0	0.74±0.0	0.91±0.0	0.61±0.0	0.69±0.0
FriendFeed	18.85±0.2	21.06±0.3	16.69±0.2	16.50±0.2	13.97±0.2	12.11±0.2	10.25±0.1	8.25±0.1

Heuristic and ColApprox with the optimal solutions, showing that for random instances ColApprox consistently outperforms Heuristic. Furthermore, Heuristic is bound to fail if the color requirements are violated early in the peeling process. In the following, we additionally show how the baseline fails in this case. To this end, we modified each data set by adding two nodes connected by a single edge of a new additional color. The results for Heuristic are shown in Figure 5 labeled Heuristic*. Because the nodes of the additional edge have a degree of one, the heuristic will try to remove them early on. But because the color of the new edge is required in the solution, the nodes cannot be removed, and Heuristic stops processing the graph, leading to much lower densities compared to ColApprox. In contrast, for ColApprox, the density only changes minimally by the one additional edge and two additional nodes that need to be considered by the algorithm.

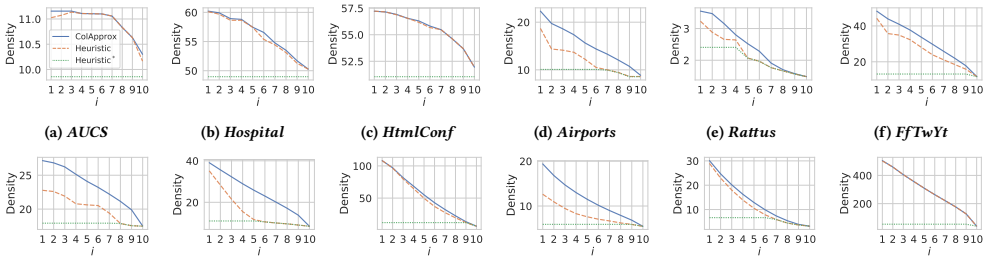
Q6 — running times. Table 5 shows the mean running times and standard deviations for ten repetitions of computations of the densities for $i \in \{2, 4, 6, 8\}$ where i and the color requirements are chosen as in **Q5**. We show the results in seconds for the four largest data sets. The running times are only a fraction of a second for all other data sets. For both Heuristic and ColApprox the running time decreases for increasing i and larger requirements of the colors. In the case of Heuristic, the reason is that the higher the requirements, the earlier the algorithm encounters a vertex whose removal would lead to a color deficit, and it stops. For ColApprox, the reason is that the higher the total color requirements, the earlier the subroutine that finds the at least h edges densest subgraph can stop the peeling process.

5.2 Use case: diverse co-authorship

In the following, we use a subgraph of the *DBLP* data set in which edges describe the coauthorship of publications published at one of ten data mining conferences. The network contains in total 44 823 nodes and 170 659 edges, each colored with one of ten colors representing the conference, and we are interested in finding densest subgraphs that contain publications of all conferences. Another view is that edges of color c belong to layer c of a multilayer graph with ten layers. Hence, we want to obtain densest subgraphs of the multilayer coauthor graph that contain at least h_i edges in layer i. We compare our approximation algorithm ColApprox to the algorithm for the multilayer densest subgraph problem Mlds. The multilayer densest subgraph problem is defined as finding a subset $S \subseteq V$ such that

$$\max_{\hat{L}\subseteq L} \min_{\ell\in\hat{L}} \frac{|E_\ell(S)|}{|S|} \cdot |\hat{L}|^\beta$$

is maximized, where $\beta \in \mathbb{R}$ is a parameter controlling the importance of adding few or many layers and E_ℓ are the edges in layer



(a) *AUCS* (b) *Hospital* (c) *HtmlConf* (d) *Airports* (e) *Rattus* (f) *EfTwYt*

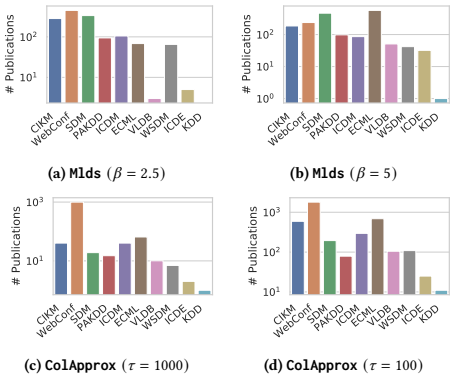
(g) *Knowledge* (h) *HomoSap* (i) *Epinions* (j) *DBLP* (k) *Twitter* (l) *FriendFeed*

Figure 5: Densities for increasing color requirements (the common legend is shown in (a)).

Table 6: Computing densest coauthor subgraphs.

Algorithm		Density	Nodes	Edges	Layers
Unconstrained DSP		20.0	41	820	2
`Mlds`	$\beta = 1$	20.0	41	820	2
	$\beta = 2.2$	6.1	232	1407	9
	$\beta = 5$	4.5	396	1766	10
`ColApprox`	$\tau = 1000$	8.7	137	1194	10
	$\tau = 100$	9.5	409	3890	10
	$\tau = 10$	9.4	2692	25324	10

ℓ [18]. We computed the unconstrained densest subgraph, the multilayer densest subgraph for $\beta \in \{1, 2.2, 5\}$, where we chose the values of β by increasing in 0.1 steps starting from one until all layers are in the densest subgraph. Only for the values of 2.2 and five do the results change. Furthermore, we use the at least **h** colored edges densest subgraph with the following color requirements. Let t_c be the total number of color $c \in [\pi]$ edges in G. For each color $c \in [10]$, we define $h_c = t_c/\tau$ with $\tau \in \{10, 100, 1000\}$. Table 6 shows the results of the by S induced subgraphs, including the density, where we use the standard definition of density, i.e., $d(S) = |E|/|S|$. For $\beta = 1$, the result of `Mlds` equals the unconstrained DSP. For $\beta = 2.2$, nine of the ten layers are included. Figure 6a shows the distribution of the publications. There are no publications from the *KDD* conference in the densest subgraph, and the density dropped significantly to 6.1 from the initial 20. Further increasing β to five leads finally leads to a densest subgraph containing publications from all conferences; however, the density further dropped to 4.5. Moreover, the *KDD* conference is still underrepresented with only one publication (see Figure 6b). For our `ColApprox`, we obtain the densest subgraphs with 1/1000th, 1/100th, and 1/10th of the edges of each color while obtaining higher density values. Figures 6c and 6d show the color distributions for $\tau = 1000$ and $\tau = 100$, respectively.



(a) `Mlds` ($\beta = 2.5$) (b) `Mlds` ($\beta = 5$)

(c) `ColApprox` ($\tau = 1000$) (d) `ColApprox` ($\tau = 100$)

Figure 6: Distributions of conferences in the densest coauthor subgraphs computed with `Mlds` and `ColApprox`.

6 CONCLUSION AND FUTURE WORK

We introduced new variants of diverse densest subgraph problems in networks with single or multiple edge attributes. We established the **NP**-completeness of decision versions for finding *exactly, at most*, and *at least* **h** colored edges densest subgraphs. Furthermore, we presented a linear-time constant-factor approximation algorithm for the problem of finding a densest subgraph with at least **h** colored edges in sparse graphs. As an additional result, we introduced the related at least h non-colored edges densest subgraph problem and provided a linear-time constant-factor approximation for it. Our experimental results validated the practical efficacy and efficiency of the proposed algorithms on a wide range of real-world graphs. Future work directions include improving the approximation results for non-sparse graphs and introducing algorithms for the exact and at most **h** variants.

ACKNOWLEDGEMENTS

This research is supported by the ERC Advanced Grant REBOUND (834862), the EC H2020 RIA project SoBigData++ (871042), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

REFERENCES

[1] Aris Anagnostopoulos, Luca Becchetti, Adriano Fazzone, Cristina Menghini, and Chris Schwiegelshohn. 2020. Spectral relaxations and fair densest subgraphs. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*. 35–44.

[2] Reid Andersen and Kumar Chellapilla. 2009. Finding dense subgraphs with size bounds. In *International workshop on algorithms and models for the web-graph*. Springer, 25–37.

[3] Albert Angel, Nick Koudas, Nikos Sarkas, Divesh Srivastava, Michael Svendsen, and Srikanta Tirthapura. 2014. Dense subgraph maintenance under streaming edge weight updates for real-time story identification. *The VLDB journal* 23 (2014), 175–199.

[4] Yuichi Asahiro, Kazuo Iwama, Hisao Tamaki, and Takeshi Tokuyama. 2000. Greedily finding a dense subgraph. *Journal of algorithms* 34, 2 (2000), 203–221.

[5] Vladimir Batagelj and Matjaz Zaversnik. 2003. An O(m) algorithm for cores decomposition of networks. *arXiv preprint cs/0310049* (2003).

[6] Aditya Bhaskara, Moses Charikar, Eden Chlamtac, Uriel Feige, and Aravindan Vijayaraghavan. 2010. Detecting high log-densities: an O (n 1/4) approximation for densest k-subgraph. In *Proceedings of the forty-second ACM symposium on Theory of computing*. 201–210.

[7] Digvijay Boob, Yu Gao, Richard Peng, Saurabh Sawlani, Charalampos Tsourakakis, Di Wang, and Junxing Wang. 2020. Flowless: Extracting densest subgraphs without flow computations. In *Proceedings of The Web Conference 2020*. 573–583.

[8] Alessio Cardillo, Jesús Gómez-Gardenes, Massimiliano Zanin, Miguel Romance, David Papo, Francisco del Pozo, and Stefano Boccaletti. 2013. Emergence of network features from multiplexity. *Scientific reports* 3, 1 (2013), 1344.

[9] Moses Charikar. 2000. Greedy approximation algorithms for finding dense components in a graph. In *International workshop on approximation algorithms for combinatorial optimization*. Springer, 84–95.

[10] Chandra Chekuri, Kent Quanrud, and Manuel R Torres. 2022. Densest subgraph: Supermodularity, iterative peeling, and flow. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. SIAM, 1531–1555.

[11] Jie Chen and Yousef Saad. 2010. Dense subgraph extraction with application to community detection. *IEEE Transactions on knowledge and data engineering* 24, 7 (2010), 1216–1230.

[12] Edith Cohen, Eran Halperin, Haim Kaplan, and Uri Zwick. 2003. Reachability and distance queries via 2-hop labels. *SIAM J. Comput.* 32, 5 (2003), 1338–1355.

[13] Manlio De Domenico, Vincenzo Nicosia, Alexandre Arenas, and Vito Latora. 2015. Structural reducibility of multilayer networks. *Nature communications* 6, 1 (2015), 6864.

[14] Mark E. Dickison, Matteo Magnani, and Luca Rossi. 2016. *Multilayer Social Networks*. Cambridge University Press.

[15] Yon Dourisboure, Filippo Geraci, and Marco Pellegrini. 2007. Extraction and classification of dense communities in the web. In *Proceedings of the 16th international conference on World Wide Web*. 461–470.

[16] Uriel Feige and Michael Seltser. 1997. *On the densest k-subgraph problem.*

[17] Amita Gajewar and Atish Das Sarma. 2012. Multi-skill collaborative teams based on densest subgraphs. In *Proceedings of the 2012 SIAM international conference on data mining*. SIAM, 165–176.

[18] Edoardo Galimberti, Francesco Bonchi, and Francesco Gullo. 2017. Core decomposition and densest subgraph in multilayer networks. In *Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*. 1807–1816.

[19] David Gibson, Ravi Kumar, and Andrew Tomkins. 2005. Discovering large dense subgraphs in massive graphs. In *Proceedings of the 31st international conference on Very large data bases*. 721–732.

[20] Aristides Gionis and Charalampos E Tsourakakis. 2015. Dense subgraph discovery: Kdd 2015 tutorial. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2313–2314.

[21] Andrew V Goldberg. 1984. Finding a maximum density subgraph. (1984).

[22] Mark S Granovetter. 1973. The strength of weak ties. *American journal of sociology* 78, 6 (1973), 1360–1380.

[23] Farnoosh Hashemi, Ali Behrouz, and Laks VS Lakshmanan. 2022. FirmCore decomposition of multilayer networks. In *Proceedings of the ACM Web Conference 2022*. 1589–1600.

[24] Julia Heidemann, Mathias Klier, and Florian Probst. 2012. Online social networks: A survey of a global phenomenon. *Computer networks* 56, 18 (2012), 3866–3878.

[25] Xinyu Huang, Dongming Chen, Tao Ren, and Dongqi Wang. 2021. A survey of community detection methods in multilayer networks. *Data Mining and Knowledge Discovery* 35 (2021), 1–45.

[26] Lorenzo Isella, Juliette Stehlé, Alain Barrat, Ciro Cattuto, Jean-François Pinton, and Wouter Van den Broeck. 2011. What's in a crowd? Analysis of face-to-face behavioral networks. *Journal of theoretical biology* 271, 1 (2011), 166–180.

[27] Ravindran Kannan and V Vinay. 1999. *Analyzing the structure of large graphs*. Universität Bonn. Institut für Ökonometrie und Operations Research.

[28] Samir Khuller and Barna Saha. 2009. On finding dense subgraphs. In *International colloquium on automata, languages, and programming (ICALP)*. 597–608.

[29] Mikko Kivelä, Alex Arenas, Marc Barthelemy, James P Gleeson, Yamir Moreno, and Mason A Porter. 2014. Multilayer networks. *Journal of complex networks* 2, 3 (2014), 203–271.

[30] Jon Kleinberg, Jens Ludwig, Sendhil Mullainathan, and Ashesh Rambachan. 2018. Algorithmic fairness. In *Aea papers and proceedings*, Vol. 108. American Economic Association 2014 Broadway, Suite 305, Nashville, TN 37203, 22–27.

[31] Tommaso Lanciano, Atsushi Miyauchi, Adriano Fazzone, and Francesco Bonchi. 2023. A survey on the densest subgraph problem and its variants. *arXiv preprint arXiv:2303.14467* (2023).

[32] Victor E Lee, Ning Ruan, Ruoming Jin, and Charu Aggarwal. 2010. A survey of algorithms for dense subgraph discovery. *Managing and mining graph data* (2010), 303–336.

[33] Jure Leskovec, Daniel Huttenlocher, and Jon Kleinberg. 2010. Signed networks in social media. In *Proceedings of the SIGCHI conference on human factors in computing systems*. 1361–1370.

[34] Michael Ley. 2002. The DBLP computer science bibliography: Evolution, research issues, perspectives. In *International symposium on string processing and information retrieval*. Springer, 1–10.

[35] Matteo Magnani, Barbora Micenkova, and Luca Rossi. 2013. Combinatorial analysis of multiple networks. *arXiv preprint arXiv:1303.4986* (2013).

[36] Shira Mitchell, Eric Potash, Solon Barocas, Alexander D'Amour, and Kristian Lum. 2021. Algorithmic fairness: Choices, assumptions, and definitions. *Annual Review of Statistics and Its Application* 8 (2021), 141–163.

[37] Atsushi Miyauchi, Tianyi Chen, Konstantinos Sotiropoulos, and Charalampos E. Tsourakakis. 2023. Densest Diverse Subgraphs: How to Plan a Successful Cocktail Party with Diversity. 1710–1721.

[38] Syama Sundar Rangapuram, Thomas Bühler, and Matthias Hein. 2013. Towards realistic team formation in social networks based on densest subgraphs. In *Proceedings of the 22nd international conference on World Wide Web*. 1077–1088.

[39] Barna Saha, Allison Hoch, Samir Khuller, Louiqa Raschid, and Xiao-Ning Zhang. 2010. Dense subgraphs with restrictions and applications to gene annotation graphs. In *Research in Computational Molecular Biology (RECOMB)*. 456–472.

[40] Chengcheng Shao, Pik-Mai Hui, Lei Wang, Xinwen Jiang, Alessandro Flammini, Filippo Menczer, and Giovanni Luca Ciampaglia. 2018. Anatomy of an online misinformation network. *Plos one* 13, 4 (2018).

[41] Stavros Sintos and Panayiotis Tsaparas. 2014. Using strong triadic closure to characterize ties in social networks. In *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*. 1466–1475.

[42] Kristina Toutanova, Danqi Chen, Patrick Pantel, Hoifung Poon, Pallavi Choudhury, and Michael Gamon. 2015. Representing text for joint embedding of text and knowledge bases. In *Proceedings of the 2015 conference on empirical methods in natural language processing*. 1499–1509.

[43] Charalampos Tsourakakis, Francesco Bonchi, Aristides Gionis, Francesco Gullo, and Maria Tsiarli. 2013. Denser than the densest subgraph: extracting optimal quasi-cliques with quality guarantees. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 104–112.

[44] Philippe Vanhems, Alain Barrat, Ciro Cattuto, Jean-François Pinton, Nagham Khanafer, Corinne Régis, Byeul-a Kim, Brigitte Comte, and Nicolas Voirin. 2013. Estimating potential infection transmission routes in hospital wards using wearable proximity sensors. *PloS one* 8, 9 (2013), e73970.

[45] Serene WH Wong, Chiara Pastrello, Max Kotlyar, Christos Faloutsos, and Igor Jurisica. 2018. Sdregion: Fast spotting of changing communities in biological networks. In *Proceedings of the 24th ACM SIGKDD international conference on Knowledge discovery & data mining*. 867–875.

A OMITTED PROOFS

In this section, we provide the omitted proofs.

A.1 Proofs of Section 3

First, we provide the **NP**-completeness results for the decision versions of our edge-diverse densest subgraph problems.

Theorem 8. *The decision version of the exactly* **h** *colored edges version (*ehcEdgesDSPdec*) is* **NP***-complete.*

Proof. We show the result for the special case of graphs with two colors (i.e., 1 and 2), and we set $\pi = 1$. In this case, the decision version ehcEdgesDSP asks to decide if there is a subset S such that the induced subgraph has exactly h color 1 edges and a density of at least α. It is clearly in **NP**. We use a reduction from k-clique. Given an instance (G, k) of k-clique, we construct the following instance of ehcEdgesDSPdec:

- Let $G' = (V', E')$ with $V' = V \cup \{t\}$ and $E' = E \cup \{\{t, v\} \mid v \in V\}$, and $c(e) = 2$ if $e \in E' \cap E$, and $c(e) = 1$ otherwise,
- furthermore, $h = k$, and $\alpha = \frac{\binom{k}{2}+k}{k+1} = \frac{k}{2}$.

Now, if G contains a k-clique, then ehcEdgesDSPdec has a *yes* answer. Because t is connected to all $v \in V$ in G', i.e., also to a set C containing k nodes of a k-clique, we find a subgraph $S = C \cup \{t\}$ containing a $(k+1)$-clique. Because $G(S)$ is complete, $d(S) = \frac{\binom{k}{2}+k}{k+1}$.

For the other direction, assume ehcEdgesDSPdec has a *yes* answer. This means that $|S| = k + 1$ because otherwise, the induced subgraph would contain more than h color 1 edges as t is connected to all other vertices. Furthermore, in order to achieve the density of $\alpha \geq \frac{\binom{k}{2}+k}{k+1}$, there have to be $\binom{k}{2}$ edges between the nodes $S \setminus \{t\}$ and k edges from t to the nodes $S \setminus \{t\}$. Because the graph G' is simple, the by S induced subgraph is a $(k + 1)$-clique. And due to the construction, $S \setminus \{t\}$ is a k-clique in G. □

Similarly, the decision version of the at-most h colored edges DSP (mhcEdgesDSPdec) is **NP**-complete. We now show the hardness for the at-least h colored edges variant.

Theorem 9. *The decision version of the at least* **h** *colored edges version (alhcEdgesDSPdec) is* **NP***-complete.*

Proof. We show the result for the special case of graphs with two colors (i.e., 1 and 2), and we set $\pi = 1$. The decision version of alhcEdgesDSPdec asks to decide if there is a subset S such that the induced subgraph has at least h color 1 edges and a density of at least α. It is clearly in **NP**. We use again a reduction from k-clique.

Given an instance (G, k) of k-clique, we construct the following instance of alhcEdgesDSPdec:

- Let $n = |V(G)|$ and let K' be the complete graph K_{n^4} in which all edges are colored 1. Furthermore, we construct G' as in the proof of Theorem 8. The graph H is the union of K' and G'.
- Moreover, we set $h = \binom{n^4}{2} + k$, and $\alpha = \frac{\binom{n^4}{2}+\binom{k}{2}+k}{n^4+k+1}$.

Let $S \subseteq V(H)$ be a witness of alhcEdgesDSPdec. (i) First note that K' is completely in $G(S)$. Assume that K' is not completely in $G(S)$ and let T be the nodes from K' that are in S, then T provides $\binom{|T|}{2}$ color 1 edges. Thus we need to choose $h - \binom{|T|}{2}$ color 1 edges from G', which is impossible because $h - \binom{T}{2} > n$ when $|T| < n^4$.

(ii) Next, we show that $|S \cap V(G')| = k+1$. Because $|S| \geq n^4+k+1$ and n^4 nodes belong to K', the number of nodes in $S' = S \cap V(G')$ is at least $k+1$. Now, we show that $|S'| \leq k+1$, by showing that adding additional edges beside the necessary k edges would decrease the density $d(S)$. Let $\beta = d(S')$, $s = |S'|$, $e_k = |E(K')|$, and $v_k = |V(K')|$. We can express the density $d(S)$ as follows

$$d(S) = \frac{\beta \cdot s + e_k}{s + v_k}.$$

If we add another node $u \in V(G)$ to S', we also add new edges into the induced subgraph $G(S')$. Let p be the number of additional edges. Then the density changes to

$$d(S \cup \{u\}) = \frac{\beta \cdot s + e_k + p}{s + v_k + 1}.$$

Note that by adding vertex u, we also add a new edge of color 1 between u and t. We show that $d(S) - d(S \cup \{u\}) > 0$, hence adding additional nodes and, therefore, additional edges of color 1 leads to decreased density.

$$d(S) - d(S \cup \{u\}) = \frac{\beta \cdot s + e_k}{s + v_k} - \frac{\beta \cdot s + e_k + p}{s + v_k + 1} > 0 \quad (1)$$

$$\Leftrightarrow \beta s^2 + \beta s v_k + \beta s + e_k s + e_k v_k + e_k \quad (2)$$

$$-(\beta s^2 + \beta s v_k + e_k s + e_k v_k + ps + pv_k) > 0 \quad (3)$$

$$\Leftrightarrow \beta s + e_k - p(s + v_k) > e_k - p(s + v_k) > 0 \quad (4)$$

To see why the last inequality holds we upper bound p with n^2 and s with n such that we have

$$\binom{n^4}{2} - n^2(n + n^4) > 0$$

which holds for $n \geq 2$.

Now let G contain a k-clique C. Hence, by setting $S = C \cup \{s\}$, we have at least h edges with color 1, and we obtain a total density of α.

If alhcEdgesDSPdec has a *yes* answer, then it holds that $\alpha = \frac{\binom{n^4}{2}+\binom{k}{2}+k}{n^4+k+1}$ due to the previous observations (i) and (ii). Specifically, $|S| = n^4 + k + 1$ because otherwise, the induced subgraph would contain more than h color 1 edges, which would reduce the density. In order to achieve the density of α, there need to be $\binom{k}{2} + k$ edges in G', which means that the nodes in G' are completely connected. Hence, there is a k-clique in G. □

Proof of Theorem 2. Given a graph $G = (V, E)$, $h \in \mathbb{N}$, and $\alpha \in \mathbb{R}$, the decision version asks if there there exists a densest subgraph with at least h edges and density at least α. It is clearly in **NP**. We use a reduction from the decision version of the at least k nodes DSP, which asks for a densest subgraph with at least $k \in \mathbb{N}$ nodes and density of at least $\beta \in \mathbb{R}$. For our reduction, we construct a family of at most $|V|^2$ instances of the at least h_i edges densest subgraph problem with $h_i = i$ for $i \in [|V|^2]$ and $\alpha = \beta$. We show that the at least k nodes instance has a *yes* answer iff. one of the at least h_i edges instances has at least k nodes and a density of at least β.

First we show that if the at least k nodes instance has a *yes* answer, one of the at least h_i edges instances has at least k nodes and a density of at least β. Let ϕ be the minimum number of edges over all graphs that are densest subgraphs of G with at least k nodes. Because $1 \leq \phi \leq |V|^2$ there is a at least ϕ edges densest subgraph instance. Let S be an optimal solution for the at least ϕ edges DSP and assume it would not be optimal for the at least k nodes DSP. Let S' be optimal for the latter with the density of $d^* \geq \beta$. Note that $|E(S')| \geq \phi$ which leads to a contradiction to the optimality of S. Hence, $d(S) \geq d^* \geq \beta$.

For the other direction, it is clear that if there is a solution S to one of the at least h_i edges instances that has at least k nodes and a density of at least β then S is also a solution to the at least k nodes instance. □

A.2 Proofs of Section 4

Proof of Lemma 1. The number of nodes of a graph with $|E| = h$ is minimized if G is complete and there are p parallel edges

between each pair of nodes, i.e., $h = p\binom{|V|}{2}$. Solving for $|V|$ leads to $\ell(h, p)$. □

Proof of Lemma 2. Assume S is optimal for the at least k-nodes DSP but not optimal for the at least h-edges DSP. Let S' be optimal for the latter. Now because $|S'| \geq k$ and $d(S') > d(S)$, we have a contradiction to the optimality of S. □

Proof of Theorem 3. Algorithm 1 computes an optimal solution of the at least i-nodes DSP for each $i \in \{\ell(h), \ldots, n\}$. We know that the optimal solution of atLeastHEdgesDSP has at most n nodes. Due to Lemma 2, Algorithm 1 discovers at least one optimal solution S of the densest subgraph with at least h edges. Finally, an optimal solution will be returned as Algorithm 1 returns the S_i with maximal density and at least h edges. □

Before we prove Theorem 7, we introduce two lemmas.

Lemma 3. *Algorithm 1 and Algorithm 2 can be adapted to everywhere sparse multigraph input M.*

Proof. To adapt Algorithm 1 and Algorithm 2 for use with the multigraph M, we need to make the following adjustments: In Algorithm 1 and Algorithm 2, we use the general lower bound of nodes $\ell(h, \pi)$ instead of $\ell(h)$. Furthermore, in Algorithm 2, when adding an edge to $M(S_i)$, ensure that all parallel edges between a pair of nodes are added. It is noteworthy that Lemma 2 remains applicable to the multigraph M. This follows directly from the subgraph definition and density. Consequently, we can conclude that the adapted Algorithm 1 is optimal for the multigraph M.

To demonstrate that Algorithm 2 provides a $O(1)$-approximation for the multigraph M we need to establish the following:

(1) If $M(S_i)$ does not contain at least h edges, the addition of parallel edges introduces fewer new nodes compared to the addition of simple edges. Therefore, it can only enhance the density of the resultant subgraph.

(2) Regardless of edge colors, a multigraph can be treated as a weighted graph where edge weights correspond to the number of parallel edges between a pair of nodes (u, v). As a result, the 3-approximation algorithm introduced by [2] is applicable to multigraphs. This implies that the assumption $c_1|E(S)| \geq |E(S^*)| \geq h$ used in the proof of Theorem 4 is valid.

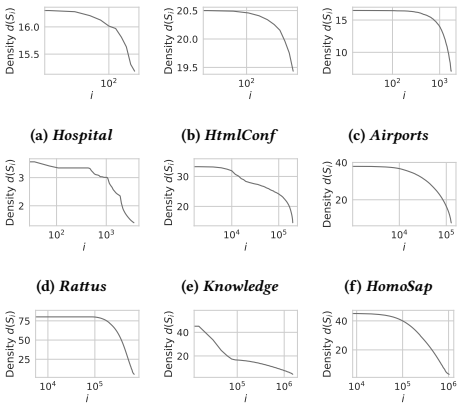
The combination of points (1) and (2) concludes the proof. □

Lemma 4. *Algorithm 3 gives an $O(1)$-approximation for atLeastHEdgesDSP on everywhere sparse multigraph input M.*

Proof. To adapt Algorithm 3 for use with a multigraph input, we should modify the peeling procedure in line 6 as follows: when we remove edges during the peeling procedure, we should remove all parallel edges that are incident to node v_i.

With arguments analogous to those presented in points (1) and (2) of the proof for Lemma 3, we can conclude that the theorem remains valid when applied to multigraphs. Furthermore, the running time complexity of Algorithm 3 remains unchanged. □

Proof of Theorem 7. Together with Lemma 4, the proof can be adapted from the proof of Algorithm 4, following the same line of reasoning that adding all parallel edges can only enhance the density of the resulting subgraph. The running time increases to $O(n+m\cdot\pi)$ due to the transformation of the graph to the multigraph. Note that π is often a small constant. □



(a) *Hospital* (b) *HtmlConf* (c) *Airports*

(d) *Rattus* (e) *Knowledge* (f) *HomoSap*

(g) *Epinions* (h) *DBLP* (i) *Twitter*

Figure 7: The density computed with `AtLeastHApprox` for increasing numbers of required edges.

B ILP FORMULATIONS

For the exact solution of atLeastHEdgesDSP, we solve the following ILP where guess the number of nodes $\ell(h) \leq k \leq n$ and return the solution with the maximum density. We use $y_u \in \{0, 1\}$ to denote if vertex $u \in V$ belongs to the solution $S \subseteq V$. Moreover, we use $x_{uv} \in \{0, 1\}$ for each edge $e = \{u, v\} \in E$ to encode if e is in the densest subgraph ($x_{uv} = 1$) or not ($x_{uv} = 0$). We can relax $x_{uv} \in \{0, 1\}$ to $x_{uv} \in [0, 1]$ because we maximize over x and due to Equation (7).

$$\max \sum_{\{u,v\} \in E} \frac{x_{uv}}{k} \tag{5}$$

$$\text{s.t.} \sum_{\{u,v\} \in E} x_{uv} \geq h \text{ and } \sum_{u \in V} = k \tag{6}$$

$$x_{uv} \leq y_u \text{ and } x_{uv} \leq y_v \quad \text{for all } \{u, v\} \in E \tag{7}$$

$$x_{uv} \in [0, 1] \quad \text{for all } \{u, v\} \in E \tag{8}$$

$$y_u \in \{0, 1\} \quad \text{for all } u \in V \tag{9}$$

We can solve the at least h colored edges DSP (alhcEdgesDSP) by adding the following additional constraint:

$$\sum_{\{u,v\} \in E_c} x_{uv} \geq h_c \quad \text{for all } c \in [\pi], \tag{10}$$

where $E_c \subseteq E$ is the subset of edges with color $c \in [\pi]$.

C DATASET DETAILS

- *AUCS* contains relations among the faculty and staff within the Computer Science department at Aarhus University [35]. The five layers correspond to the following relationship types: current working relationship, repeated leisure activities, regular lunch, co-authorship of publication, and facebook friendship.
- *Hospital* and *HtmlConf* are face-to-face contact networks between hospital patients and health care workers [44], and between visitors of a conference [26]. The networks spans five and three days, respectively, represented by the colors.
- *Airports* is an aviation transport network containing flight connections between European airports [8]. The layers represent different airline companies.
- *Rattus* contains different types of genetic interactions of *Rattus Norvegicus* [13]. The six layers are physical association, direct interaction, colocalization, association, additive genetic interaction defined by inequality, and suppressive genetic interaction defined by inequality.
- *Knowledge* is based on the *FB15K-237 Knowledge Base* data set [42]. The FB15K-237 data set contains knowledge base relation triples and textual mentions of the Freebase knowledge graph entity pairs. We represent the entities as nodes and different relations colored edges.
- *HomoSap* is a network representing different types of genetic interactions between genes in Homo Sapiens [18].
- *Epinions* is an online social network and general consumer review site. Members on the platform have the option to determine whether or not they trust one another [33].
- *DBLP* is a subgraph of the DBLP graph [34] containing only publications from A and A^* ranked conferences (according to the Core ranking). The edges describe collaborations between authors, and the edge colors represent the different conferences.
- *Twitter* is a subgraph of the Twitter graph representing users and retweets in the period of six months before the 2016 US Presidential Elections [40]. Each edge represents a retweet and is labeled as factual or misinformation.
- *FriendFeed* and *FfTwYt* are based on the former *FriendFeed* social media aggregation service that allowed users to consolidate and view updates from various social networking platforms and websites [14]. The *FriendFeed* dataset contains interactions among users collected over two months. The three layers represent commenting, liking and following. *FfTwYt* is a network in which users registered a Twitter and a Youtube account associated to their Friendfeed account.

Paper D

Fairness-aware PageRank via Edge Reweighting

Honglian Wang
KTH Royal Institute of Technology
Digital Futures
Stockholm, Sweden
honglian@kth.se

Haoyun Zhou
KTH Royal Institute of Technology
Digital Futures
Stockholm, Sweden
haoyunz@kth.se

Aristides Gionis
KTH Royal Institute of Technology
Digital Futures
Stockholm, Sweden
argioni@kth.se

Abstract

Link-analysis algorithms, such as PageRank, are instrumental in understanding the structural dynamics of networks by evaluating the importance of individual vertices based on their connectivity. Recently, with the rising importance of responsible AI, the question of fairness in link-analysis algorithms has gained traction.

In this paper, we present a new approach for incorporating group fairness into the PageRank algorithm by reweighting the transition probabilities in the underlying transition matrix. We formulate the problem of achieving fair PageRank by seeking to minimize the fairness loss, which is the difference between the original group-wise PageRank distribution and a target PageRank distribution. We further define a group-adapted fairness notion, which accounts for group homophily by considering random walks with group-biased restart for each group. Since the fairness loss is non-convex, we propose an efficient projected gradient-descent method for computing locally-optimal edge weights. Unlike earlier approaches, we do not recommend adding new edges to the network, nor do we adjust the restart vector. Instead, we keep the topology of the underlying network unchanged and only modify the relative importance of existing edges. We empirically compare our approach with state-of-the-art baselines and demonstrate the efficacy of our method, where very small changes in the transition matrix lead to significant improvement in the fairness of the PageRank algorithm.

CCS Concepts

• **Information systems** → **Social networks**; **Page and site ranking**; **Content ranking**.

Keywords

Link analysis, PageRank, Graph analysis, Fairness

ACM Reference Format:
Honglian Wang, Haoyun Zhou, and Aristides Gionis. 2026. Fairness-aware PageRank via Edge Reweighting. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining (WSDM '26), February 22–26, 2026, Boise, ID, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3773966.3777991

This work is licensed under a Creative Commons Attribution 4.0 International License.
WSDM '26, Boise, ID, USA
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2292-9/2026/02
https://doi.org/10.1145/3773966.3777991

1 Introduction

Machine-learning algorithms and data-driven decision making are playing increasingly crucial roles in our lives, impacting job markets, healthcare, government, industry, and education. As our dependence on automated decisions grows, however, so does the concern for fairness in algorithms. Ensuring that machine-learning algorithms are reliable and satisfy strong notions of fairness has become an issue of societal concern and, justifiably, has received significant attention in the literature [12, 19]. To date, algorithmic fairness has been studied for primarily supervised [15, 16, 27, 42] but also unsupervised learning [4, 7, 8]. In the context of network and link-analysis algorithms, a few different approaches have been proposed [20, 29, 36, 37, 43], however, the topic has received significantly less attention than the study of fairness in other settings.

Link-analysis algorithms, such as the PageRank family of methods, are fundamental tools for analyzing large-scale network data [3, 14], and have been used in a variety of applications, including identifying authoritative vertices [39], detecting communities [34], uncovering patterns of influence [23], and mitigating network vulnerabilities [18].

Fairness-aware PageRank. The problem of fairness-aware PageRank [37] can be broadly formulated as follows. We are interested in computing the PageRank scores on a graph whose vertices are partitioned into K groups, where the partition is defined by, say, a protected attribute. For simplicity, let us assume $K = 2$ and refer to the two groups of vertices as *red* and *blue*. The fairness of the PageRank distribution is then evaluated based on the total PageRank scores assigned to *red* and *blue* vertices. We would like to avoid situations where the PageRank mass is distributed unevenly (/unfairly) between the two groups. For example, given a target score tuple $\boldsymbol{\phi} = (\phi_{\text{red}}, \phi_{\text{blue}}) = (\phi_{\text{red}}, 1 - \phi_{\text{red}})$, we say that PageRank is $\boldsymbol{\phi}$-fair if the total PageRank score of the vertices in the *red* group is equal to ϕ_{red}; and consequently, the total PageRank score of the vertices in the *blue* group is equal to $1 - \phi_{\text{red}} = \phi_{\text{blue}}$;

Intervention strategies. To improve fairness for PageRank problems, the task is to design an intervention strategy so that the PageRank scores of the graph vertices become $\boldsymbol{\phi}$-fair, or "close enough" to $\boldsymbol{\phi}$-fair. Considering the PageRank algorithm, there are several possible ways to design intervention strategies, including: (*i*) modifying the PageRank restart probability; (*ii*) modifying the restart vector; (*iii*) adding edges to the graph; and (*iv*) modifying the edge weights of the graph in a controlled manner to ensure that the modified graph remains faithful to the original.

In this paper, we focus on intervention strategies of type (*iv*), namely, achieving fair PageRank by reweighting the graph edges. Our motivation is that such interventions are *intrinsic to the graph structure* and can be implemented with actionable operations. For

example, edge reweighting can be achieved by prioritizing/demoting content from one's connections in a social network. Strategies of type (*iii*) involving edge additions or deletions are also intrinsic to the graph structure but tend to be less practical. These strategies rely on link recommendations and have to account for the likelihood of accepting the recommended links, which introduces uncertainty and makes them challenging to implement.

Strategies of type (*i*) and (*ii*), i.e., modifying the PageRank restart probability or restart vector are *extrinsic to the graph structure* and rely on tuning hyper-parameters of the PageRank algorithm. In many cases, it is not meaningful, nor actionable, to modify those hyper-parameters. For example, in the personalized PageRank algorithm, the restart vector has a clear interpretation, which is to model the user interests and is not amenable to modification.

Locally- vs. globally-fair PageRank. Existing papers [20, 29, 37] make the observation that if fairness is ensured at each step of the PageRank random walk, then fairness is guaranteed for the final PageRank distribution. Ensuring fairness at each step (which we call locally-fair PageRank) translates to having equal probability of visiting all groups from each vertex. While this is a sound approach, the requirement of locally-fair PageRank can be too rigid and may lead to a disproportionate modification of the graph structure or the edge weights. Instead, in this paper, we pose the question of *whether it is possible to achieve globally-fair PageRank without having to impose the strict requirement of locally-fair PageRank.*

As we will see, we answer this question in the affirmative.

Our approach. We present a new approach to incorporate fairness into the PageRank algorithm by reweighting the transition probabilities of the PageRank matrix. More concretely, we seek to minimize the PageRank distance to a fair distribution while allowing only for reweighting the network edges within prespecified limits. Our approach handles multiple vertex groups. Unlike earlier approaches, we do not recommend that new edges be added to the network, nor do we adjust the restart vector. Instead, we keep the topology of the underlying network unchanged.

If perfect fairness cannot be achieved, our formulation aims to find a solution as close as possible to the target fairness objective.

Our approach provides an actionable environment to improve PageRank fairness, as the newly computed edge weights can be lead to concrete actions, e.g., reweighting and prioritizing the feed of users in the social-network platform.

We propose a *projected gradient-descent* method for computing optimal edge reweightings and show how it can be implemented efficiently. We empirically compare our approach with state-of-the-art baselines and demonstrate the efficacy of our method, where small changes in the transition matrix lead to significant improvement in the fairness of the PageRank algorithm.

Contributions. Our main contributions are:

- We incorporate group fairness into PageRank by reweighting transition probabilities to achieve solutions as close as possible to the target fairness objective with minimal modifications.
- We aim for globally-fair PageRank without the strict requirement of achieving local fairness at each vertex. This can lead to solutions with smaller modifications in the transition matrix.
- We introduce a group-adapted fairness notion that accounts for group homophily via group-biased restart distributions.
- We propose an efficient projected gradient-descent method for finding locally-optimal edge weights.
- We empirically compare our approach with state-of-the-art baselines and demonstrate the efficacy of our method, where very small changes in the transition matrix lead to significant improvement in the fairness of the PageRank algorithm.

Due to space limitations, proofs and additional material are deferred to the appendices in the full version of this paper [38].

2 Related work

In recent years, algorithmic fairness in graph mining has emerged as a prominent area of research [32]. This paper focuses on enhancing fairness in the content of the well-known PageRank algorithm, which has been widely used for ranking, classification, community detection, and more. The objective is to allocate PageRank scores to vertices in a manner that considers both the vertices' group affiliations and the overall structure of the network [32, 37].

In the standard PageRank algorithm [6, 14, 34], a random-walk process defines the transition from one vertex to its neighbors, while also with certain probability considering a "jump" to one of the vertices defined by a restart vector. When considering the group association of vertices, defined by a protected attribute, analytical studies in the literature [11, 13] have unveild the unfairness of PageRank. To address such potential group bias, FairWalk [29] proposes to modify the transition matrix of the random walk so that the transition probabilities from each vertex to all groups become equal. CrossWalk [20] further refines this idea by considering the local group structure and reweighting edges based on their distance from group boundaries. Both FairWalk and CrossWalk adjust the transition matrix using heuristic-based methods.

Tsioutsiouliklis et al. [37] introduce a novel fairness metric for PageRank and propose two families of algorithms: FSPR and LFPRs. Methods in the first family, FSPR, retain the original graph structure and edge weights, while seeking an optimal jump vector to minimize the disparity between the fair PageRank vector and the original PageRank vector, subject to fairness constraints. On the other hand, similar to FairWalk and CrossWalk, methods in the second family, LFPRs, aim to achieve fairness at the individual vertex level and include three variants: LFPR_N, LFPR_U, and LFPR_P. Each variant reweights the transition matrix in a different way while incorporating a fair jump vector. Although the authors suggest that FSPR and LFPRs can be extended to scenarios involving more than two groups, they do not provide details for such an extension.

Additionally, link recommendation has been explored as an alternative approach to promoting fairness in PageRank. In one such paper, Tsioutsiouliklis et al. [36] present analytical formulas for calculating the impact of adding edges on fairness. In a different direction, Zeng et al. [43] propose a set of de-biasing methods for fair network representation learning, using techniques based on sampling, projections, and graph neural networks.

Other papers consider the problem of infusing fairness into rankings without focusing specifically into the PageRank algorithm. For instance, Krasanakis et al. [21] propose a model that weights training samples to make classifiers fair and adapt it to estimate an unbiased personalization that yields fairer vertex ranks.

In this paper, we propose an edge-reweighting approach to enhance fairness in PageRank. Our method differs from previous studies in several ways: First, we consider only edge reweighting, which can lead to more actionable operations to achieve fairness. Second, we focus on global fairness, without enforcing the more rigid requirement of fair transitions from each vertex. As a result, our method can lead to significantly smaller modification of the transition matrix. Finally, our approach offers a clear and practical framework for handling effectively fairness over multiple groups.

3 Problem definition

3.1 Preliminaries

We consider a directed graph $G = (V, E)$ with a set V of n vertices and a set E of m edges. We assume that the set of vertices is partitioned into K groups $\{V_1 \cdots V_K\}$. Let ℓ_v indicate the group that a vertex $v \in V$ belongs to.

The PageRank algorithm takes G as input, and produces a PageRank vector $\mathbf{p}$ that indicates the relative importance of each vertex in the graph [6]. Despite the existence of multiple equivalent formulations, the PageRank algorithm is fully characterized by three parameters: the transition matrix $\mathbf{P}$, the restart (or jump) vector $\mathbf{v}$, and the restart probability γ. The PageRank vector $\mathbf{p}$ is the solution to the equation $\mathbf{p}^\mathsf{T} = (1-\gamma)\mathbf{p}^\mathsf{T}\mathbf{P} + \gamma\mathbf{v}^\mathsf{T}$.

Here, the transition matrix $\mathbf{P}$ is row stochastic, derived by normalizing each row of the adjacency matrix of G. The restart vector $\mathbf{v}$ specifies a distribution over all vertices, indicating where a random walk might restart. The parameter γ represents the probability that the random walk will restart at any given step. In the case of directed graph, there may exist sink vertices that have no outgoing edges, so their rows in the transition matrix would be all zeros. To handle sink vertices, we replace the corresponding zero rows of $\mathbf{P}$ with the restart vector $\mathbf{v}$.

By solving the equation of PageRank definition, we obtain the PageRank vector as $\mathbf{p}^\mathsf{T} = \gamma\mathbf{v}^\mathsf{T}(\mathbf{I} - (1-\gamma)\mathbf{P})^{-1}$.

3.2 Fair PageRank

Building on the earlier definition of fairness-aware PageRank [36, 37], we consider the vertices of a graph to be partitioned into distinct groups. These groups may represent social categories based on sensitive attributes such as gender, ethnicity, or sexual orientation. We begin by defining the fair PageRank problem, followed by a discussion of various approaches to achieving PageRank fairness, and conclude with a formal definition of the $\boldsymbol{\phi}$-PageRank ($\boldsymbol{\phi}$-PageRank) problem and the Restricted $\boldsymbol{\phi}$-PageRank (R-ϕPR) problem.

3.2.1 Group-wise fair PageRank. To establish the concept of PageRank fairness, we assume a *target PageRank score*, ϕ_k for each vertex group $k = 1, \ldots, K$, with $\sum_k \phi_k = 1$. This score represents the total PageRank mass that the algorithm is expected to allocate to the vertices in group V_k. A common approach is to set ϕ_k proportionally to the size of V_k, i.e., $\phi_k = |V_k|/n$. However, other choices may be more suitable depending on the specific application. In this work, as in prior studies, we assume that the target PageRank score ϕ_k for each group is provided as input to our problem formulation.

Let $\mathbf{1}_k \in \{0,1\}^n$ denote the indicator vector for group V_k, where $\mathbf{1}_k[i] = 1$ if vertex $i \in V_k$ and $\mathbf{1}_k[i] = 0$ otherwise. Let $\mathbf{p}$ be the PageRank vector computed by the PageRank algorithm using a transition matrix $\mathbf{P}$, with restart probability γ and restart vector $\mathbf{v}$. The total PageRank score for group V_k is then given by $\mathbf{1}_k^\mathsf{T}\mathbf{p}$.

Given an input graph G and its associated transition matrix $\mathbf{P}$, our goal is to find a new transition matrix $\hat{\mathbf{P}}$, defined on the same input graph G, such that the group-wise PageRank distribution computed over $\hat{\mathbf{P}}$ (defined shortly) closely matches the target group-wise PageRank distribution $\boldsymbol{\phi} = (\phi_1, \cdots, \phi_K)$.

Remark. For any given $\mathbf{P}$, γ, and $\mathbf{v}$, the total PageRank score for group V_k is lowered bounded by $\gamma\mathbf{1}_k^\mathsf{T}\mathbf{v}$ and upper bounded by $1-\gamma+\gamma\mathbf{1}_k^\mathsf{T}\mathbf{v}$. The lower bound represents the probability that a random walk restarts from group k, and it can be derived from the equation $\mathbf{1}_k^\mathsf{T}\mathbf{p} = (1-\gamma)\mathbf{1}_k^\mathsf{T}\mathbf{P}^\mathsf{T}\mathbf{p} + \gamma\mathbf{1}_k^\mathsf{T}\mathbf{v} \geq \gamma\mathbf{1}_k^\mathsf{T}\mathbf{v}$. The upper bound can be derived from the equation $1-\sum_{j\neq k}\mathbf{1}_j^\mathsf{T}\mathbf{v} \leq 1-\gamma\sum_{j\neq k}\mathbf{1}_j^\mathsf{T}\mathbf{v} = 1-\gamma+\gamma\mathbf{1}_k^\mathsf{T}$. Thus, for fixed γ and $\mathbf{v}$, it is clear that it may not always be possible to find a $\hat{\mathbf{P}}$ that exactly achieves the desired target distribution $\boldsymbol{\phi}$.

3.2.2 Rationale for PageRank fairness via edge reweighting. Consider the case where both $\mathbf{v}$ and γ can be freely adjusted. In this scenario, achieving the target fair distribution $\boldsymbol{\phi}$ is straightforward: we can simply set $\mathbf{v}[i] = \phi_{\ell_i}/|V_{\ell_i}|$ and $\gamma = 1$. However, adjusting $\mathbf{v}$ and γ is impractical as these parameters are not within our control in actual models; they are merely parameters of PageRank and do not reflect the actual graph structure. In contrast, modifying the transition matrix is more meaningful; this can be accomplished, for example, by appropriately prioritizing or demoting content from an individual's connections in a social network.

Therefore, we focus solely on adjusting the transition matrix $\mathbf{P}$ to achieve the desired fairness distribution. To maintain the structure of the input graph G, we impose the constraint that the new transition matrix $\hat{\mathbf{P}}$ must satisfy $\hat{\mathbf{P}}[i,j] = 0$ whenever $\mathbf{P}[i,j] = 0$. This condition ensures that no new edges are introduced.

3.2.3 Fairness-loss function and the $\boldsymbol{\phi}$-PageRank problem. To formalize our problem, we first define the fairness-loss function. It evaluates how much the actual fairness distribution deviates from the target distribution for a given transition matrix $\mathbf{P}$.

Definition 1 (PageRank Problem Instance)**.** *A PageRank problem instance consists of a graph G with vertices partitioned into groups $V_1, \ldots, V_K$, a row-stochastic transition matrix $\mathbf{P}$, a restart probability $0 < \gamma < 1$, a restart vector $\mathbf{v} \in \mathbb{R}^n$ satisfying $\|\mathbf{v}\|_1 = 1$, and group-wise target PageRank scores $\boldsymbol{\phi} = (\phi_1, \ldots, \phi_K)$ satisfying $\|\boldsymbol{\phi}\|_1 = 1$.*

Definition 2 (Fairness-loss function)**.** *Given a PageRank problem input as defined in Definition 1, the fairness-loss function is defined as:*

$$L(\mathbf{P}, \gamma, \mathbf{v}) = \frac{1}{K}\sum_{k=1}^{K}\left(\mathbf{1}_k^\mathsf{T}\mathbf{p} - \phi_k\right)^2, \tag{3.1}$$

where $\mathbf{p}$ is the PageRank vector corresponding to parameters $(\mathbf{P}, \gamma, \mathbf{v})$.

The proof of Proposition 1 can be found in Appendix A of the full version [38].

Proposition 1. *The fairness-loss function $L(\mathbf{P}, \gamma, \mathbf{v})$, as defined in Equation (3.1), is not convex with respect to $\mathbf{P}$.*

We introduce two problems that aim to achieve fair PageRank by minimizing the fairness-loss through edge reweighting.

Problem 1 ($\boldsymbol{\phi}$-PageRank)**.** *Given a PageRank problem input as defined in Definition 1, the goal is to find a new transition matrix* $\mathbf{P}^*$ *that minimizes the fairness-loss function* $L(\mathbf{P}^*, \gamma, \mathbf{v})$*, i.e.,*

$$\mathbf{P}^* = \arg\min_{\hat{\mathbf{P}} \in \mathcal{C}(\mathbf{P})} L(\hat{\mathbf{P}}, \gamma, \mathbf{v}), \tag{3.2}$$

where

$$\mathcal{C}(\mathbf{P}) = \{\hat{\mathbf{P}} \in \mathbb{R}^{n\times n}_{\geq 0} \mid \hat{\mathbf{P}}\mathbf{1} = \mathbf{1},\ \hat{\mathbf{P}}_{ij} = 0 \text{ if } \mathbf{P}_{ij} = 0\}$$

is the set of row-stochastic matrices that have non-negative entries, and the edge reweighting of G can take place only on existing edges.

Problem 2 (Restricted $\boldsymbol{\phi}$-PageRank)**.** *Given a PageRank problem input as defined in Definition 1, a relative modification bound* $\delta \geq 0$ *and an absolute modification bound* $\epsilon \geq 0$*, the goal is to find a new transition matrix* $\mathbf{P}^*$ *that minimizes the fairness-loss function* $L(\mathbf{P}^*, \gamma, \mathbf{v})$*, i.e.,*

$$\mathbf{P}^* = \arg\min_{\hat{\mathbf{P}} \in \mathcal{C}_R(\mathbf{P}) \cap \mathcal{C}(\mathbf{P})} L(\hat{\mathbf{P}}, \gamma, \mathbf{v}), \tag{3.3}$$

where $\mathcal{C}(\mathbf{P})$ *is defined as in Problem 1, and*

$$\mathcal{C}_R(\mathbf{P}) = \{\hat{\mathbf{P}} \in \mathbb{R}^{n\times n}_{\geq 0} \mid (1-\delta)\mathbf{P}[i,j] - \epsilon \leq \hat{\mathbf{P}}[i,j] \leq (1+\delta)\mathbf{P}[i,j] + \epsilon\}.$$

In other words, $\mathcal{C}_R(\mathbf{P})$ restricts the entries of the new transition matrix $\hat{\mathbf{P}}$ within a range of $(1-\delta)$ and $(1+\delta)$ of the corresponding entries in the original transition matrix $\mathbf{P}$, while also imposing an absolute modification bound ϵ. The relative bound is necessary because, for very small values in $\mathbf{P}$, even slight absolute changes can result in large relative variations. By choosing δ and ϵ such that $(1-\delta)\mathbf{P} - \epsilon > 0$, we can guarantee no edges are deleted.

The motivation for defining the Restricted $\boldsymbol{\phi}$-PageRank problem (Problem 2) is to consider only solutions that make small modifications in the transition matrix $\mathbf{P}$.

Without loss of generality, in this paper, we study both $\boldsymbol{\phi}$-PageRank and Restricted $\boldsymbol{\phi}$-PageRank with a uniform restart vector $\mathbf{v} = \left[\frac{1}{n}, \ldots, \frac{1}{n}\right]$.

3.3 Group-adapted fair PageRank

Real-world networks are mostly homophilic, i.e., vertices in the same group are more likely to be connected to each other [24, 35]. Hence, the total PageRank scores in group V_k are largely contributed by random walks (re)starting at vertices in group V_k [37]. Thus, we propose the notion of *group-adapted fairness*, which enhances PageRank by considering random walks that (re)start at each group and by penalizing the fairness loss for each such (re)start.

More specifically, let $\mathbf{v}_k = \frac{1}{|V_k|}\mathbf{1}_k$ be the vector indicating that the random walk restarts uniformly only from group k. We define the *group-adapted fairness-loss function* as follows:

Definition 3 (Group-adapted fairness loss)**.** *Given a PageRank problem input as defined in Definition 1, the group-adapted fairness-loss function is defined as*

$$L_g(\mathbf{P}, \gamma) = \frac{1}{K}\sum_{\ell=1}^{K} L(\mathbf{P}, \gamma, \mathbf{v}_\ell) = \frac{1}{K^2}\sum_{\ell=1}^{K}\sum_{k=1}^{K}\left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right)^2, \tag{3.4}$$

where $\mathbf{p}_\ell$ *is the PageRank vector with respect to parameters* $(\mathbf{P}, \gamma, \mathbf{v}_\ell)$*.*

In the group-adapted fairness loss, we iterate through all groups and sum up the fairness loss when restarting the random walk at each group. Minimizing the group-adapted fairness-loss function leads to the Group-adapted $\boldsymbol{\phi}$-PageRank problem.

Problem 3 (Group-adapted $\boldsymbol{\phi}$-PageRank)**.** *Given a PageRank problem input as defined in Definition 1, find a new transition matrix* $\mathbf{P}^*$ *that minimizes the group-adapted fairness-loss function*

$$\mathbf{P}^* = \arg\min_{\hat{\mathbf{P}} \in \mathcal{C}(\mathbf{P})} L_g(\hat{\mathbf{P}}, \gamma), \tag{3.5}$$

where $\mathcal{C}(\mathbf{P})$ *is defined as in Problem 1.*

Similarly, we define the restricted version of the problem.

Problem 4 (Restricted group-adapted $\boldsymbol{\phi}$-PageRank)**.** *Given a PageRank problem input as defined in Definition 1, a relative modification bound* $\delta \geq 0$ *and an absolute modification bound* $\epsilon \geq 0$*, find a new transition matrix* $\mathbf{P}^*$ *that minimizes the group-adapted fairness-loss function* $L_g(\mathbf{P}^*, \gamma)$*, i.e.,*

$$\mathbf{P}^* = \arg\min_{\hat{\mathbf{P}} \in \mathcal{C}_R(\mathbf{P}) \cap \mathcal{C}(\mathbf{P})} L_g(\hat{\mathbf{P}}, \gamma),$$

where $\mathcal{C}(\mathbf{P})$ *is defined as in Problem 1 and* $\mathcal{C}_R(\mathbf{P})$ *as in Problem 2.*

4 Algorithms

In this section, we present a projected gradient-descent algorithm for finding a locally-optimal transition matrix $\hat{\mathbf{P}}$ for the fair PageRank problems we defined in the previous section. We propose Algorithm 1 for problem Problem 1 and Problem 2, and Algorithm 3 for problem Problem 3 and Problem 4.

4.1 Projected gradient descent for ϕ-PageRank

We first derive the gradient of the fairness-loss function $L(\mathbf{P}, \gamma, \mathbf{v})$ with respect to the transition matrix $\mathbf{P}$. The proof of the following proposition can be found in Appendix B of the full version [38].

Proposition 2. *The gradient of the fairness-loss function* $L(\mathbf{P}, \gamma, \mathbf{v})$ *with respect to the transition matrix* $\mathbf{P}$ *is given by*

$$\frac{\partial}{\partial \mathbf{P}} L(\mathbf{P}, \gamma, \mathbf{v}) = \frac{2(1-\gamma)}{K}\sum_{k=1}^{K}\left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right)\mathbf{p}\mathbf{y}_k^\top, \tag{4.1}$$

where $\mathbf{p}$ *is the PageRank vector with respect to* $(\mathbf{P}, \gamma, \mathbf{v})$*, and* $\mathbf{y}_k = \mathbf{U}^{-T}\mathbf{1}_k = (\mathbf{I} - (1-\gamma)\mathbf{P})^{-1}\mathbf{1}_k$*, with* $\mathbf{U} = \mathbf{I} - (1-\gamma)\mathbf{P}^\top$*.*

Matrix inversion acceleration. The calculation of $\mathbf{y}_k = \gamma\mathbf{U}^{-T}\mathbf{1}_k$ involves the inverse of the matrix $\mathbf{U}^\top = \mathbf{I} - (1-\gamma)\mathbf{P}$, which is computationally expensive. To avoid computing matrix inversion, we approximation $\mathbf{U}^{-T}$ with the Neumann series $\mathbf{U}^{-T} = \sum_{i=0}^{\infty}(1-\gamma)^i\mathbf{P}^i$, and substituting into $\mathbf{y}_k$ gives $\mathbf{y}_k = \sum_{i=0}^{\infty}(1-\gamma)^i\mathbf{P}^i\mathbf{1}_k$. As the series terms decrease exponentially in magnitude, due to the decaying factor $(1-\gamma)$, we can approximate $\mathbf{y}_k$ efficiently by computing only the first few series terms. We show that truncating the Neumann series to its first 50 terms yields a relative error below 0.0003. The detailed analysis is provided in Appendix D of the full version [38].

Gradient interpretation. In Proposition 2 we have defined $\mathbf{y}_k = \mathbf{U}^{-T}\mathbf{1}_k$. The j-th coordinate of $\mathbf{y}_k$ is given by $\mathbf{y}_k[j] = \mathbf{1}_k^\top(\gamma\mathbf{U}^{-1}\mathbf{e}_j)$, which represents the sum of PageRank scores for the vertices in the

Algorithm 1 `FairGD`: Projected gradient descent for $\boldsymbol{\phi}$-PageRank and Restricted $\boldsymbol{\phi}$-PageRank

Input: $\mathbf{P}$, γ, $\mathbf{v}$, $\boldsymbol{\phi} = (\phi_1, \dots, \phi_K)$, group indicator vectors $\mathbf{1}_k$ for $k = \{1, \cdots, K\}$, learning rate α, iteration exponent t_1 and t_2, convergence criterion κ, maximum number of iterations N_{iter}, modification bound δ and ϵ (optional).
Output: Optimized transition matrix $\hat{\mathbf{P}}$

1: $\hat{\mathbf{P}} \leftarrow \mathbf{P}$, $iter \leftarrow 0$, $L_{prev} \leftarrow \infty$, $\mathbf{p} = \frac{1}{n}\mathbf{1}$
2: **while** $|L(\hat{\mathbf{P}}, \gamma, \mathbf{v}) - L_{prev}| > \kappa$ and $iter < N_{\text{iter}}$ **do**
3: $\quad iter \leftarrow iter + 1$
4: $\quad \mathbf{p} \leftarrow$ iterating $\mathbf{p}^\mathsf{T} = (1-\gamma)\mathbf{p}^\mathsf{T}\hat{\mathbf{P}} + \gamma\mathbf{v}^\mathsf{T}$ for t_1 times
5: $\quad L_{prev} \leftarrow L(\hat{\mathbf{P}}, \gamma, \mathbf{v})$
6: $\quad$ **for** $k = 1$ to K **do**
7: $\quad\quad \mathbf{y}_k \leftarrow \sum_{i=1}^{t_2}(1-\gamma)^i\hat{\mathbf{P}}^i\mathbf{1}_k$
8: $\quad\quad \hat{\mathbf{P}} \leftarrow \hat{\mathbf{P}} - \alpha\frac{2(1-\gamma)}{K}(\mathbf{1}_k^\mathsf{T}\mathbf{p} - \phi_k)\mathbf{p}\mathbf{y}_k^\mathsf{T}$
9: $\quad$ **end for**
10: $\quad \hat{\mathbf{P}} \leftarrow$ `Project`$(\hat{\mathbf{P}}, \delta, \epsilon)$
11: **end while**
12: **return** $\hat{\mathbf{P}}$

Algorithm 2 `Project`: projection to feasible set

Input: Transition matrix $\hat{\mathbf{P}}$, modification bound δ and ϵ (optional).
Output: Projected transition matrix $\hat{\mathbf{P}}$

1: **for** $i = 1, \cdots, n$ **do**
2: $\quad \mathbf{s} \leftarrow$ nonzero elements of $\hat{\mathbf{P}}_i$
3: $\quad$ **if** δ and ϵ are provided **then**
4: $\quad\quad \boldsymbol{\ell} = \max(0, \mathbf{s}(1-\delta) - \epsilon)$
5: $\quad\quad \mathbf{u} = \min(1, \mathbf{s}(1+\delta) + \epsilon)$
6: $\quad\quad \lambda^* \leftarrow$ `Root-Finding`$(\mathbf{s}, \boldsymbol{\ell}, \mathbf{u})$
7: $\quad\quad s_j^* = \text{clip}(\lambda^* + s_j, \ell_j, u_j)$ for all $j \in \{1, \cdots, |\mathbf{s}|\}$
8: $\quad$ **else**
9: $\quad\quad$ Rank elements in $\mathbf{s}$ such that $s_1 \geq \cdots \geq s_{|\mathbf{s}|}$
10: $\quad\quad k(\mathbf{s}) = \max\left\{k \in \{1, \dots, |\mathbf{s}|\} \;\middle|\; 1 + k\, s_k > \sum_{j \leq k} s_j\right\}$
11: $\quad\quad \tau(\mathbf{s}) = \frac{1}{k(\mathbf{s})}\left(\sum_{j \leq k(\mathbf{s})} s_j - 1\right)$
12: $\quad\quad s_j^* = \max(0, s_j - \tau(\mathbf{s}))$ for all $j \in \{1, \cdots, |\mathbf{s}|\}$
13: $\quad$ **end if**
14: $\quad$ non-zeros entries of $\hat{\mathbf{P}}_i \leftarrow \mathbf{s}^*$
15: **end for**

group V_k, with the random walk restarting at the unit vector $\mathbf{e}_j$. In other words, the random walk always (re)starts from vertex j.

Hence, the gradient of the fairness loss has the following intuitive interpretation: the $[i, j]$-th entry of the gradient matrix is a weighted sum of the products of $\mathbf{p}[i]$ and $\mathbf{y}_k[j]$. The coordinate $\mathbf{p}[i]$ represents the frequency with which a random walk, restarting at $\mathbf{v}$, visits vertex i, while the coordinate $\mathbf{y}_k[j]$ represents the frequency with which a random walk, restarting at vertex j, visits vertices in the group V_k. Therefore, the k-th term of the summation in Eq. (4.1) reflects the weighted contribution of the edge (i, j) to the overall PageRank score of the group V_k. The weight of the k-th term in Eq. (4.1) is proportional to the difference between the total PageRank scores of group V_k and the group-wise target score ϕ_k.

Gradient projection. After each gradient descent step, $\hat{\mathbf{P}}$ needs to be projected back into the feasible area subject to constraints $\mathcal{C}(\mathbf{P})$ or $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$. We perform the projection for each row of $\hat{\mathbf{P}}$ separately, and since both constraints forbid adding non-existing edges, we solve the projection only for its non-zero entries. Given any row of $\hat{\mathbf{P}}$, let $\mathbf{s}$ be the vector comprises the non-zero entries of the corresponding row vector, and $\mathbf{s}^*$ be the vector after projection. We use $N = |\mathbf{s}|$ to denote the cardinality of $\mathbf{s}$.

We first discuss the constraint $\mathcal{C}(\mathbf{P})$. Projecting $\mathbf{s}$ onto a vector $\mathbf{t}$ that lies in the feasible set $\mathcal{C}(\mathbf{P})$ is equivalent to solving the following problem:

$$\mathbf{s}^* = \underset{\mathbf{t} \in \mathcal{F}_1}{\arg\min} \|\mathbf{s} - \mathbf{t}\|_2^2,$$

where $\mathcal{F}_1 = \{\mathbf{t} \in \mathbb{R}^N \mid \mathbf{1}^\mathsf{T}\mathbf{t} = 1, \mathbf{t} \geq 0\}$ is a probability simplex. This is a well studied problem [9, 10, 25, 26], and the computational complexity for finding $\mathbf{s}^*$ is $\mathcal{O}(NlogN)$. In Algorithm 2, lines 9–12, we use the method detailed by Duchi et al. [10] to perform the projection of $\mathbf{s}$ after every gradient update step.

We then discuss the constraint $\mathcal{C}(\mathbf{P}) \cap \mathcal{C}_R(\mathbf{P})$. Solving the projection subject to this constraint is equivalent to finding a Euclidean projection of $\mathbf{s}$ onto the intersection of a probability simplex and a box. Namely,

$$\mathbf{s}^* = \underset{\mathbf{t} \in \mathcal{F}_2}{\arg\min} \|\mathbf{s} - \mathbf{t}\|_2^2,$$

where $\mathcal{F}_2 = \{\mathbf{t} \in \mathbb{R}^N \mid \mathbf{1}^\mathsf{T}\mathbf{t} = 1, \ell_i \leq t_i \leq u_i, \text{ for all } i \in \{1, \cdots, N\}\}$, with $\ell_i = \max\{0, (1-\delta)s_i - \epsilon\}$ and $u_i = \min\{1, (1+\delta)s_i + \epsilon\}$. This problem has already been studied by Adam and Mácha [1], and the time complexity for finding $\mathbf{s}^*$ is also $\mathcal{O}(NlogN)$. We provide the details in Algorithm 2, lines 4–7. Line 6 invokes Algorithm 4 (`Root-Finding`) as a subroutine, and the details are provided in Appendix E of the full version [38].

To sum up, our method is presented in Algorithms 1 and 2. Algorithm 1 (`FairGD`) is the overall projected gradient-descent scheme that iteratively updates the matrix $\hat{\mathbf{P}}$ until convergence. Algorithm 2 (`Project`) is the projection function, where the transition matrix $\hat{\mathbf{P}}$ is projected back to the feasible set after each iteration.

Algorithm 1 is designed to address both the $\boldsymbol{\phi}$-PageRank problem and the Restricted $\boldsymbol{\phi}$-PageRank problem. In the non-restricted case, we refer to the algorithm simply as `FairGD`. In the restricted case, parameterized by (δ, ϵ), we denote it as `FairGD`(δ, ϵ).

4.2 Group-adapted PageRank

The `FairGD` algorithm can be extended to address both the Group-adapted $\boldsymbol{\phi}$-PageRank and Restricted group-adapted $\boldsymbol{\phi}$-PageRank problems by introducing an outer loop that iterates over all groups. This enhanced algorithm, referred to as `AdaptGD`, is detailed in Algorithm 3 in Appendix E of the full version [38]. For the non-restricted case, we refer to the algorithm as `AdaptGD`. In the restricted case, parameterized by (δ, ϵ), we denote it as `AdaptGD`(δ, ϵ).

Proposition 3. *The gradient of the fairness-loss function $L_g(\mathbf{P}, \gamma)$ with respect to the transition matrix $\mathbf{P}$ is given by*

$$\frac{\partial}{\partial \mathbf{P}} L_g(\mathbf{P}, \gamma) = \frac{2(1-\gamma)}{K^2} \sum_{k=1}^{K} \sum_{\ell=1}^{K} \left(\mathbf{1}_k^\mathsf{T}\mathbf{p}_\ell - \phi_k\right)\left(\mathbf{p}_\ell \mathbf{1}_k^\mathsf{T}\mathbf{U}^{-1}\right). \quad (4.2)$$

where $\mathbf{p}_\ell$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v}_\ell)$.

4.3 Extensions

While not explicitly studied in the paper, we note that the proposed fairness loss can be generalized in several ways. First, when fairness scores are specified at the individual-node level, an auxiliary loss term can jointly enforce both group-level and individual-level constraints. Second, for graphs with hierarchical group structures, where each layer defines a partition with associated fairness goals, the overall objective can be formulated as the sum of layer-specific losses, enabling simultaneous satisfaction of fairness requirements across multiple scales. Finally, the restricted fairness losses can incorporate node-level constraints by specifying δ and ϵ for each node, with our algorithm remaining applicable.

5 Complexity and convergence analysis

The primary computational cost arises from computing PageRank and approximating $\mathbf{y}_k$. The PageRank vector is obtained by iterating the update equation $\mathbf{p}^\top = (1-\gamma)\,\mathbf{p}^\top\mathbf{P} + \gamma\,\mathbf{v}^\top$ for t_1 iterations, resulting in computational complexity $\mathcal{O}(t_1 n^2)$. The computation of $\mathbf{y}_k$ involves multiplying a vector by a matrix t_2 times. Evaluating $\mathbf{y}_k$ for all $k \in \{1, \dots, K\}$ therefore requires $\mathcal{O}(Kt_2 n^2)$ operations. Consequently, the overall complexity of Algorithm 1 is $\mathcal{O}\big((Kt_2 + t_1)n^2\big)$. Since AdaptGD performs an additional loop over K groups when computing $\mathbf{y}_k$, its complexity becomes $\mathcal{O}\big((K^2 t_2 + t_1)n^2\big)$.

Both loss objective functions, $L(\mathbf{P}, \gamma, \mathbf{v})$ and $L_g(\mathbf{P}, \gamma)$, are differentiable functions concerning $\mathbf{P}$, and their first derivatives are smooth. Since the feasible set $\mathcal{C}_R(\mathbf{P}) \cap \mathcal{C}(\mathbf{P})$ is convex and closed, the proposed projected gradient update method with constant step size converges to a stationary point, which is a local minimum [5]. We provide detailed analysis in Appendix F and Appendix G of the full version.

6 Experimental evaluation

In our experimental evaluation, we study the following questions:

Q1: To what extent can the FairGD and AdaptGD algorithms improve the fairness and group-adapted fairness of PageRank?

Q2: How do the FairGD and AdaptGD algorithms compare to the baselines in terms of changes to the transition matrix?

Q3: How does the ranking of the PageRank score within each group change? Do FairGD and AdaptGD preserve this ranking?

To answer the first question, we report the fairness loss and group-adapted fairness loss as defined in Equations (3.1) and (3.4). For the second question, we quantify the changes to the transition matrix by the relative change, given by $\Delta_{\mathbf{P}} = \|\hat{\mathbf{P}} - \mathbf{P}\|_F / \|\mathbf{P}\|_F$. For the third question, we compute the Spearman's rank correlation coefficient [33] between the original PageRank ranking and the ranking after re-weighting, and report the weighted average $\overline{\rho}$ across all groups (weighted by the group size). A larger $\overline{\rho}$ indicates fewer changes in the PageRank score's ranking.

6.1 Datasets

We evaluate our methods and the baselines on the following publicly-available datasets: Books,[1] Blogs [2], MIND [40], Twitter [31], and Slashdot [22]. Dataset statistics are given in Table 1, whereas more information is presented in Appendix H.1 of the full version [38].

[1] http://www-personal.umich.edu/~mejn/netdata/

Table 1: Datasets used in the experiments. $\boldsymbol{\phi}_O$ denotes the original group-wise PageRank scores of the datasets.

Dataset	#Vertices	#Edges	Modularity	$\boldsymbol{\phi}_O$
Books	105	882	0.4149	(0.42,0.10, 0.48)
Blogs	1224	19025	0.4111	(0.48, 0.52)
MIND	2269	11345	0.075	(0.26,0.74)
Twitter	18470	48365	0.4755	(0.42, 0.58)
Slashdot	82168	948464	0.2950	(0.30, 0.36, 0.17,0.17)

6.2 Proposed methods and baselines

The proposed methods are FairGD and AdaptGD. Additionally:

FairGD(δ, ϵ) is the restricted version of FairGD, with relative and absolute intervention bound δ and ϵ respectively.

AdaptGD(δ, ϵ) is the restricted version of our AdaptGD, with relative and absolute intervention bound δ and ϵ respectively.

We compare our methods with the following baselines: LFPR_U [37], LFPR_N [37], FairWalk [29], and CrossWalk [20]. More details on the baselines can be found in H.2 of the full version [38].

Remarks. Originally, FairWalk and CrossWalk assign equal weights to each group in the random walk. We modify them so that the probability of visiting each group during a random walk is proportional to the target group-wise PageRank scores. These two algorithms do not add edges, neither do they change the restart vector.

LFPR_N and LFPR_U are guaranteed to achieve perfect $\boldsymbol{\phi}$-fairness, provided the restart vector is $\boldsymbol{\phi}$-fair, i.e., $\mathbf{1}_k^\top \mathbf{v} = \phi_k$. LFPR_U modifies the graph to be fully connected. We report fairness loss $L(\hat{\mathbf{P}}, \gamma, \mathbf{v})$ and group-adapted fairness loss $L_g(\hat{\mathbf{P}}, \gamma)$ for these two baselines with the restart vector $\mathbf{v}$ set to a uniform vector.

6.3 Experimental setting

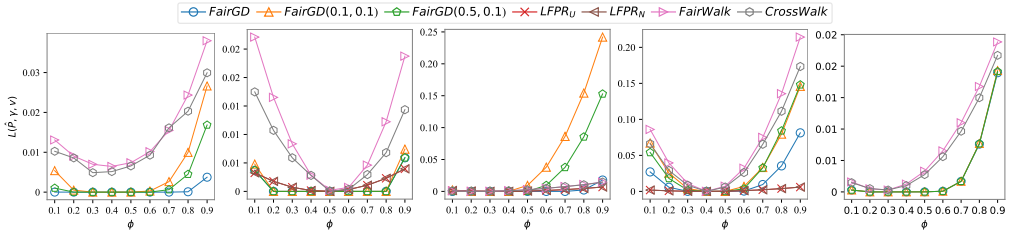
Parameters γ and $\mathbf{v}$. We follow the common assumption of PageRank algorithm and fix $\gamma = 0.15$. We fix $\mathbf{v}$ to be the uniform vector.

Group indicator definition. For datasets Books, Blogs, MIND, and Twitter, we use node labels as group indicators. For the Slashdot dataset, where node labels are unavailable, we apply the METIS graph partitioning algorithm [17] to divide the vertices into groups.

(Group-adapted) fairness loss evaluation. For the two-group datasets Blogs, MIND and Twitter datasets, we compare our approach with all baselines. For the remaining multi-group datasets, we only compare with FairWalk and CrossWalk as LFPR_U and LFPR_N do not extend directly to the multi-group scenario.

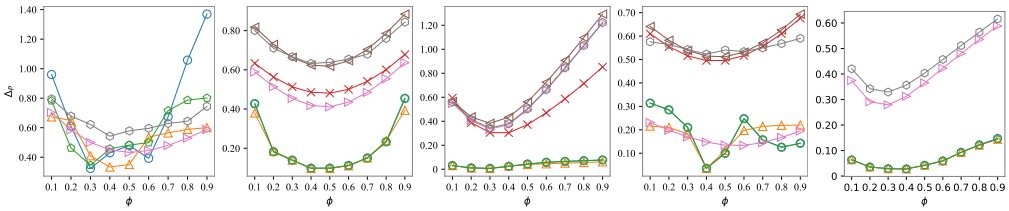
FairGD(δ, ϵ) and AdaptGD(δ, ϵ). For Group-adapted $\boldsymbol{\phi}$-PageRank and Restricted group-adapted $\boldsymbol{\phi}$-PageRank problems, we set $\delta = \{0.1, 0.5\}$ and $\epsilon = 0.1$. We perform a grid search over a logarithmically-spaced range of values between 10^{-4} and 10^4 to find the best learning rate.

Group-wise target PageRank score $\boldsymbol{\phi}$. For datasets containing only two groups, we define the target PageRank distribution as



(a) Books (b) Blogs (c) MIND (d) Twitter (e) Slashdot

Figure 1: Fairness loss $L(\hat{\mathbf{P}}, \gamma, \mathbf{v})$ of each method on five datasets for the $\boldsymbol{\phi}$-PageRank problem



(a) Books (b) Blogs (c) MIND (d) Twitter (e) Slashdot

Figure 2: Transition matrix change $\Delta_{\mathbf{P}}$ of each method on five datasets for the $\boldsymbol{\phi}$-PageRank problem

$\boldsymbol{\phi} = (\phi, 1-\phi)$. For datasets with multiple groups, we set $\boldsymbol{\phi} = \left(\phi, \frac{1-\phi}{K-1}, \cdots, \frac{1-\phi}{K-1}\right)$. We vary ϕ in the range $[0.1, 0.2, \ldots, 0.9]$.

The code and datasets used in this paper are available.[2]

6.4 Fairness loss and weight change $\Delta_{\mathbf{P}}$

We evaluate the fairness loss $L(\hat{\mathbf{P}}, \gamma, \mathbf{v})$ on `FairGD`, `FairGD`(0.1, 0.1), `FairGD`(0.5, 0.1), and all baselines across the five datasets; the results are shown in Figure 1. We also evaluate the changes, denoted as $\Delta_{\mathbf{P}}$, to the original transition matrix; the results are shown in Figure 2. The results demonstrate that `FairGD` and its restricted variants generally achieve better or comparable fairness loss compared to the baselines, while maintaining significantly smaller changes in the transition matrix weights.

On the two-group datasets, $\mathsf{LFPR}_{\mathsf{U}}$ and $\mathsf{LFPR}_{\mathsf{N}}$ achieve near-perfect fairness by modifying the graph so that each node fairly distributes its PageRank score to its neighbors according to the target fairness vector. On the Blogs dataset, `FairGD` and its two restricted variants exhibit very similar behavior, achieving 8 out of the 10 best fairness outcomes while also minimizing weight changes. For the Twitter dataset, `FairGD` and its variants outperform FairWalk and CrossWalk in terms of fairness loss, whereas FairWalk performs comparably with our proposed methods in weight change, surpassing all other baselines. On the MIND dataset, `FairGD` and all baselines achieve almost perfect fairness, but `FairGD` consistently outperforms them in weight change. However, for $\phi > 0.5$, the constrained variants `FairGD`(0.1, 0.1) and `FairGD`(0.5, 0.1) fail to maintain high fairness but consistently achieve smaller weight changes.

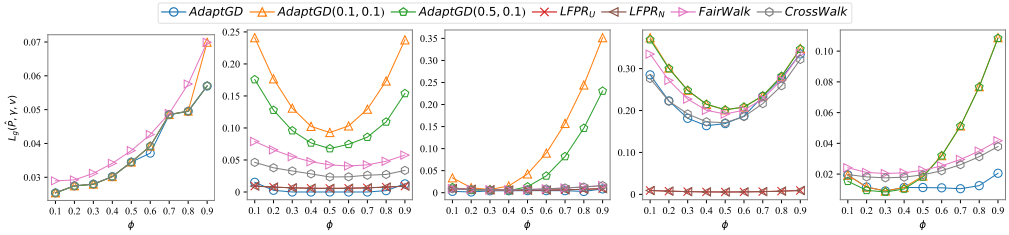
For multi-group datasets, `FairGD` and its two variants perform best on both fairness and weight change on the Slashdot dataset. On the Books dataset, `FairGD` attains the highest fairness score, followed by `FairGD`(0.5, 0.1) and `FairGD`(0.1, 0.1); the weight changes surpass those of FairWalk and CrossWalk for some ϕ values to achieve better fairness outcomes.

Overall, `FairGD` delivers superior results in terms of fairness loss and weight change when ϕ is close to the original PageRank score, ϕ_1 (the first dimension of $\boldsymbol{\phi}_o$ as reported in Table 1). As ϕ diverges from ϕ_1, both the fairness loss and weight change increase. The two restricted variants, `FairGD`(0.1, 0.1) and `FairGD`(0.5, 0.1), exemplify the trade-off between fairness loss and weight change by achieving smaller weight changes at the cost of higher fairness loss compared to `FairGD`. This refinement highlights the balance between fairness and weight change, emphasizing the flexibility of `FairGD` in adapting to various fairness goals.

6.5 Group-adapted fairness loss and $\Delta_{\mathbf{P}}$

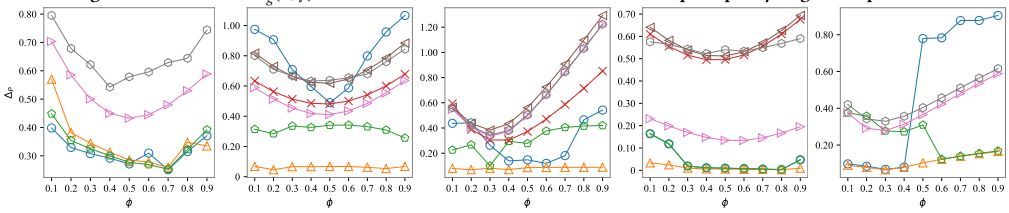
We evaluate `AdaptGD` and its restricted variants, `AdaptGD`(0.1, 0.1) and `AdaptGD`(0.5, 0.1), on the group-adapted fairness loss, as defined in Equation (3.4), and analyze the corresponding changes in the transition matrix weights. The results are presented in Figure 3

[2]https://github.com/HongLWang/Fair-Pagerank-via-Edge-Rewighting



(a) Books (b) Blogs (c) MIND (d) Twitter (e) Slashdot

Figure 3: Fairness loss $L_g(\hat{\mathbf{P}}, \gamma)$ of each baseline on five datasets for the Group-adapted ϕ-PageRank problem



(a) Books (b) Blogs (c) MIND (d) Twitter (e) Slashdot

Figure 4: Transition matrix change $\Delta_{\mathbf{P}}$ of each baseline on five datasets for the Group-adapted ϕ-PageRank problem

Table 2: Average Spearman's coefficient $\overline{\rho}$ between the original and revised PageRank weight for the MIND dataset

ϕ	FairWalk	CrossWalk	LFPR_U	LFPR_N	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	0.811	0.812	0.790	0.802	**0.970**	0.953	**0.970**	0.817	0.853	0.830
0.2	0.830	0.823	0.808	0.801	**0.997**	**0.997**	**0.997**	0.810	0.855	0.817
0.3	0.818	0.808	0.817	0.810	**0.998**	**0.998**	**0.998**	0.797	0.835	0.815
0.4	0.828	0.795	0.811	0.795	**0.970**	0.966	**0.970**	0.809	0.847	0.813
0.5	0.819	0.812	0.810	0.802	**0.970**	0.909	0.960	0.814	0.828	0.802
0.6	0.821	0.809	0.788	0.807	**0.966**	0.888	0.903	0.814	0.846	0.825
0.7	0.816	0.805	0.789	0.786	**0.958**	0.884	0.888	0.816	0.836	0.810
0.8	0.794	0.805	0.794	0.791	**0.930**	0.872	0.883	0.769	0.849	0.791
0.9	0.810	0.800	0.802	0.798	**0.884**	0.857	0.867	0.760	0.833	0.801

for the group-adapted fairness loss and in Figure 4 for the transition matrix weight changes. The results show that the trade-off between fairness loss and weight change is further exemplified and shown for AdaptGD and its variants.

Across the Books, Blogs, and MIND datasets, AdaptGD consistently achieves the most favorable fairness outcomes compared to competing baselines. In terms of weight change, AdaptGD attains optimal performance on the Books dataset and 9 out of 10 optimal outcomes on the MIND dataset. Although AdaptGD underperforms LFPR_U and LFPR_N in fairness loss on the Twitter dataset, it still matches or surpasses the other two baseline methods, while delivering substantially better weight change results. On the Slashdot dataset, AdaptGD outperforms all competing baselines in fairness outcome, and achieves the best weight change metrics for 4 out of 10 values of ϕ. Notably, on the Blogs and the Slashdot dataset, AdaptGD prioritizes improvements in fairness over weight change performance.

For all datasets, the two variants of AdaptGD exhibit comparable or slightly higher fairness loss relative to AdaptGD, while consistently maintaining lower or comparable weight change.

6.6 Ranking coefficient evaluation

We report the average Spearman's rank correlation coefficient $\overline{\rho}$ for each method across multiple datasets. For each group, we compute the Spearman's coefficient between the rankings of old and new PageRank values restricted to that group. These per-group coefficients are then aggregated using a weighted average, where each group's weight corresponds to its proportion of the total items. The results for the MIND dataset are shown in Table 2, while results for other datasets are provided in Appendix I of the full version [38]. In the tables, we depict with boldface the largest value and with underline the second largest.

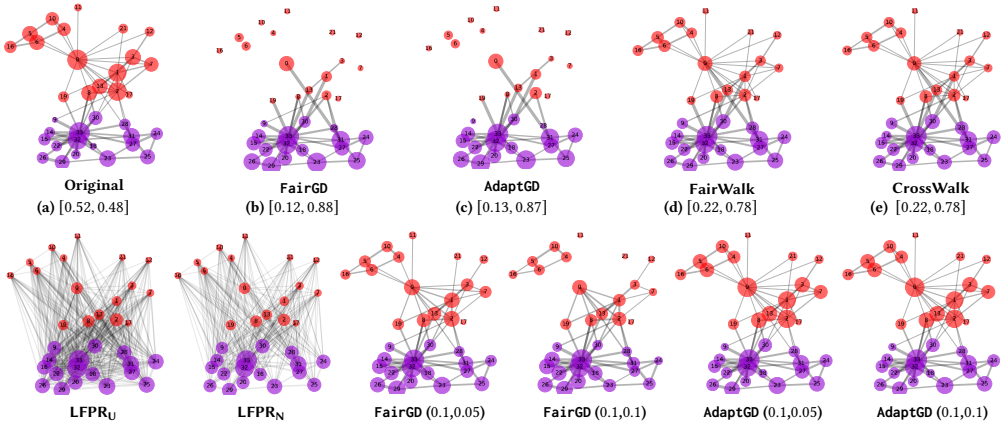


Figure 5: Graph structure of Karate before and after edge reweighting by each method. The target group-wise fairness vector is $[0.1, 0.9]$. Below each method name, the group-wise PageRank scores calculated using a uniform restart vector are displayed. The two values correspond to the red and purple groups, respectively.

Across all ϕ values, FairGD consistently attains the highest $\overline{\rho}$, followed by FairGD(0.5, 0.1) and FairGD(0.1, 0.1). These results indicate that FairGD exhibits very good preservation of the relative PageRank score rankings within individual groups. These results suggest that, in practical applications, when adjusting the content displayed to users based on items' PageRank scores to enhance fairness, fewer changes are required within each group. Instead, greater focus should be placed on adjusting the frequency or weighting of content across different groups.

6.7 Use case

We visualize the graph structure of a small real-world dataset, Karate [30, 41], before and after edge reweighting in Figure 5. The Karate dataset is a classic social network from Wayne Zachary's 1977 study of friendships among 34 members of a karate club, which eventually split into two factions after a disagreement. The dataset contains 34 nodes, with 17 nodes belonging to the "Mr. Hi" club and the remaining 17 nodes belonging to the "Officer" club.

In the visualization, vertex labels indicate node indices, node colors correspond to group membership, node sizes are proportional to their PageRank scores, and edge thickness reflects transition weights.

The original group-wise PageRank score for the Karate dataset is $[0.52, 0.48]$, while the fairness goal is set to $[0.1, 0.9]$. From Figure 5, we observe that FairGD achieves the closest group-wise fairness scores, followed by AdaptGD, LFPR_U, and LFPR_N. The unrestricted versions of FairGD and AdaptGD tend to remove a large proportion of intra-group edges within the red group, whereas their restricted counterparts delete fewer edges due to the modification bounds, at the cost of achieving a lower fairness improvement. FairWalk and CrossWalk remove no edges, resulting in graph structures very similar to the original. Finally, LFPR_U modifies the graph to be fully connected, achieving the same fairness score as LFPR_N, which adds fewer non-existing edges.

7 Conclusion

We presented a novel approach for incorporating group fairness into the PageRank algorithm by reweighting the entries of the underlying transition matrix. We focus on matrix reweighting interventions, as we argue that those are more actionable strategies, and further, reflect the network structure, rather than the hyperparameters of PageRank. Our goal is to minimize fairness loss (with respect to a target group-fairness distribution) while also minimizing the changes to the transition matrix. We achieve our objective by foregoing the idea of ensuring fairness locally, at every vertex, and instead aim to ensure fairness at a global level. Technically, our method is an efficient projected gradient-descent algorithm for finding locally-optimal edge weights. We presented a comprehensive empirical evaluation, demonstrating that our method offers significant advantages compared to strong baselines.

Acknowledgments

We sincerely thank Sebastian Dalleiger for providing constructive feedback. This research is supported by the ERC Advanced Grant REBOUND (834862), the Swedish Research Council project ExCLUS (2024-05603), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

8 Ethical considerations

This work focuses on reweighting the transition matrix of a graph to achieve target group-wise PageRank scores, with the goal of improving fairness across groups. While the proposed approach aims to improve fairness and mitigate bias in ranking systems, interventions that alter item rankings could be misused to deliberately amplify or suppress the visibility of certain content, potentially leading to manipulation or censorship. Furthermore, if applied to sensitive domains, such interventions could unintentionally disadvantage specific users or groups, even when fairness metrics improve. Our experiments use only publicly available datasets with no personal user or item-identifying information, reducing privacy risks. To mitigate potential misuse, we recommend deploying such methods with transparent governance, independent auditing of fairness outcomes, and clear documentation of intervention goals and criteria.

References

[1] Lukás Adam and Václav Mácha. 2022. Projections onto the canonical simplex with additional linear inequalities. *Optimization Methods and Software* 37, 2 (2022), 451–479.

[2] Lada Adamic and Natalie Glance. 2005. The political blogosphere and the 2004 US election: Divided they blog. In *Proceedings of the 3rd International Workshop on Link Discovery*. 36–43.

[3] Bahman Bahmani, Ravi Kumar, Mohammad Mahdian, and Eli Upfal. 2012. Pagerank on an evolving graph. In *Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining*. 24–32.

[4] Suman Bera, Deeparnab Chakrabarty, Nicolas Flores, and Maryam Negahbani. 2019. Fair algorithms for clustering. *Advances in Neural Information Processing Systems* 32 (2019).

[5] Dimitri P Bertsekas. 1997. Nonlinear programming. *Journal of the Operational Research Society* 48, 3 (1997), 334–334.

[6] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems* 30, 1 (1998), 107–117.

[7] Xingyu Chen, Brandon Fain, Liang Lyu, and Kamesh Munagala. 2019. Proportionally fair clustering. In *International Conference on Machine Learning*. 1032–1041.

[8] Flavio Chierichetti, Ravi Kumar, Silvio Lattanzi, and Sergei Vassilvitskii. 2017. Fair clustering through fairlets. *Advances in Neural Information Processing Systems* 30 (2017).

[9] Laurent Condat. 2016. Fast projection onto the simplex and the l 1 ball. *Mathematical Programming* 158, 1 (2016), 575–585.

[10] John Duchi, Shai Shalev-Shwartz, Yoram Singer, and Tushar Chandra. 2008. Efficient projections onto the l 1-ball for learning in high dimensions. In *Proceedings of the 25th international conference on Machine learning*. 272–279.

[11] Lisette Espín-Noboa, Claudia Wagner, Markus Strohmaier, and Fariba Karimi. 2022. Inequality and inequity in network-based ranking and recommendation algorithms. *Scientific reports* 12, 1 (2022), 2012.

[12] Runshan Fu, Manmohan Aseri, Param Vir Singh, and Kannan Srinivasan. 2022. "Un" fair machine learning algorithms. *Management Science* 68, 6 (2022), 4173–4195.

[13] Gourab Ghoshal and Albert-László Barabási. 2011. Ranking stability and super-stable nodes in complex networks. *Nature communications* 2, 1 (2011), 394.

[14] David F. Gleich. 2015. PageRank Beyond the Web. *SIAM Rev.* 57, 3 (Jan. 2015), 321–363. https://doi.org/10.1137/140976649 Publisher: Society for Industrial and Applied Mathematics.

[15] Moritz Hardt, Eric Price, and Nati Srebro. 2016. Equality of opportunity in supervised learning. *Advances in Neural Information Processing Systems* 29 (2016).

[16] Matthew Joseph, Michael Kearns, Jamie H Morgenstern, and Aaron Roth. 2016. Fairness in learning: Classic and contextual bandits. *Advances in Neural Information Processing Systems* 29 (2016).

[17] George Karypis and Vipin Kumar. 1997. METIS: A software package for partitioning unstructured graphs, partitioning meshes, and computing fill-reducing orderings of sparse matrices. (1997).

[18] Bill Kay, Hao Lu, Pravallika Devineni, Anika Tabassum, Supriya Chintavali, and Sangkeun Matt Lee. 2021. Identification of critical infrastructure via pagerank. In *2021 IEEE International Conference on Big Data (Big Data)*. IEEE, 3685–3690.

[19] Michael Kearns. 2017. Fair algorithms for machine learning. In *Proceedings of the 2017 ACM Conference on Economics and Computation*. 1–1.

[20] Ahmad Khajehnejad, Moein Khajehnejad, Mahmoudreza Babaei, Krishna P. Gummadi, Adrian Weller, and Baharan Mirzasoleiman. 2022. CrossWalk: Fairness-Enhanced Node Representation Learning. *Proceedings of the AAAI Conference on Artificial Intelligence* 36, 11 (2022), 11963–11970.

[21] Emmanouil Krasanakis, Symeon Papadopoulos, and Ioannis Kompatsiaris. 2021. Applying Fairness Constraints on Graph Node Ranks Under Personalization Bias. In *Complex Networks & Their Applications IX*. 610–622.

[22] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. 2009. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters. *Internet Mathematics* 6, 1 (2009), 29–123.

[23] Qi Liu, Biao Xiang, Nicholas Jing Yuan, Enhong Chen, Hui Xiong, Yi Zheng, and Yu Yang. 2017. An influence propagation view of pagerank. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 11, 3 (2017), 1–30.

[24] Miller McPherson, Lynn Smith-Lovin, and James M Cook. 2001. Birds of a feather: Homophily in social networks. *Annual review of sociology* 27, 1 (2001), 415–444.

[25] Christian Michelot. 1986. A finite algorithm for finding the projection of a point onto the canonical simplex of $\propto$ n. *Journal of Optimization Theory and Applications* 50, 1 (1986), 195–200.

[26] Panos M Pardalos and Naina Kovoor. 1990. An algorithm for a singly constrained class of quadratic programs subject to upper and lower bounds. *Mathematical Programming* 46, 1 (1990), 321–328.

[27] Dino Pedreshi, Salvatore Ruggieri, and Franco Turini. 2008. Discrimination-aware data mining. In *Proceedings of the 14th ACM SIGKDD international conference on Knowledge discovery and data mining*. 560–568.

[28] Kaare Brandt Petersen and Michael Syskind Pedersen. 2008. *The matrix cookbook (version 2012)*. Technical University of Denmark.

[29] Tahleen Rahman, Bartlomiej Surma, Michael Backes, and Yang Zhang. 2019. Fairwalk: Towards Fair Graph Embedding.

[30] Ryan A. Rossi and Nesreen K. Ahmed. 2015. The Network Data Repository with Interactive Graph Analytics and Visualization. In *AAAI*. https://networkrepository.com

[31] Ryan A. Rossi and Nesreen K. Ahmed. 2016. An Interactive Data Repository with Visual Analytics. *SIGKDD Explorations* 17, 2 (2016), 37–41.

[32] Akrati Saxena, George Fletcher, and Mykola Pechenizkiy. 2024. Fairsna: Algorithmic fairness in social network analysis. *Comput. Surveys* 56, 8 (2024), 1–45.

[33] Charles Spearman. 1961. The proof and measurement of association between two things. (1961).

[34] Shayan A Tabrizi, Azadeh Shakery, Masoud Asadpour, Maziar Abbasi, and Mohammad Ali Tavallaie. 2013. Personalized pagerank clustering: A graph clustering algorithm based on random walks. *Physica A: Statistical Mechanics and its Applications* 392, 22 (2013), 5772–5785.

[35] Bassel Tarbush and Alexander Teytelboym. 2012. Homophily in online social networks. In *Internet and Network Economics: 8th International Workshop, WINE 2012, Liverpool, UK, December 10-12, 2012. Proceedings 8*. Springer, 512–518.

[36] Sotiris Tsioutsiouliklis, Evaggelia Pitoura, Konstantinos Semertzidis, and Panayiotis Tsaparas. 2022. Link Recommendations for PageRank Fairness. In *Proceedings of the ACM Web Conference*. 3541–3551.

[37] Sotiris Tsioutsiouliklis, Evaggelia Pitoura, Panayiotis Tsaparas, Ilias Kleftakis, and Nikos Mamoulis. 2021. Fairness-Aware PageRank. In *Proceedings of the Web Conference*. 3815–3826.

[38] Honglian Wang, Haoyun Chen, and Aristides Gionis. 2025. Fairness-aware PageRank via Edge Reweighting. arXiv:2512.08055 [cs.SI] http://arxiv.org/abs/2512.08055

[39] Hanzhi Wang, Zhewei Wei, Junhao Gan, Sibo Wang, and Zengfeng Huang. 2020. Personalized pagerank to a target node, revisited. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 657–667.

[40] Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, et al. 2020. Mind: A large-scale dataset for news recommendation. In *Proceedings of the 58th annual meeting of the association for computational linguistics*. 3597–3606.

[41] Wayne W Zachary. 1977. An information flow model for conflict and fission in small groups. *Journal of anthropological research* (1977), 452–473.

[42] Rich Zemel, Yu Wu, Kevin Swersky, Toni Pitassi, and Cynthia Dwork. 2013. Learning fair representations. In *International Conference on Machine Learning*. 325–333.

[43] Ziqian Zeng, Rashidul Islam, Kamrun Naher Keya, James Foulds, Yangqiu Song, and Shimei Pan. 2021. Fair Representation Learning for Heterogeneous Information Networks. *Proceedings of the International AAAI Conference on Web and Social Media* 15 (2021), 877–887.

A Proof of Proposition 1

To demonstrate the non-convexity, it suffices to provide a single counterexample. Consider a fully connected graph G with three vertices: a, b, and c. Let the labels be ℓ_a = blue, and $\ell_b = \ell_c$ = red.

We also let

$$\hat{\mathbf{P}}_1 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \text{ and } \hat{\mathbf{P}}_2 = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \\ 1 & 0 & 0 \end{pmatrix}$$

be two row-stochastic transition matrices, and

$$\hat{\mathbf{P}}_3 = \frac{1}{2}\hat{\mathbf{P}}_1 + \frac{1}{2}\hat{\mathbf{P}}_2 = \begin{pmatrix} 0 & 1/2 & 1/2 \\ 1/2 & 0 & 1/2 \\ 1/2 & 1/2 & 0 \end{pmatrix}.$$

Given restart vector $\mathbf{v} = (1/3, 1/3, 1/3)$ and restart probability γ, we can calculate the PageRank vectors $\mathbf{p}_1$, $\mathbf{p}_2$, and $\mathbf{p}_3$, for $\hat{\mathbf{P}}_1$, $\hat{\mathbf{P}}_2$, and $\hat{\mathbf{P}}_3$, being $\mathbf{p}_1 = \left(\frac{\gamma^2-3\gamma+3}{3(2-\gamma)}, \frac{3-2\gamma}{3(2-\gamma)}, \frac{\gamma}{3}\right)$, $\mathbf{p}_2 = \left(\frac{\gamma^2-3\gamma+3}{3(2-\gamma)}, \frac{\gamma}{3}, \frac{3-2\gamma}{3(2-\gamma)},\right)$, and $\mathbf{p}_3 = \left(\frac{1}{3}, \frac{1}{3}, \frac{1}{3}\right)$, respectively.

Now, consider the group-wise target PageRank score, defined as $\phi_1 = \frac{\gamma^2-3\gamma+3}{3(2-\gamma)}$ and $\phi_2 = 1 - \phi_1$. It holds that $\frac{1}{2}L(\hat{\mathbf{P}}_1, \gamma, \mathbf{v}) + \frac{1}{2}L(\hat{\mathbf{P}}_2, \gamma, \mathbf{v}) = 0$. One can easily find a γ such that that $L(\frac{1}{2}\hat{\mathbf{P}}_1 + \frac{1}{2}\hat{\mathbf{P}}_2, \gamma, \mathbf{v}) = L(\hat{\mathbf{P}}_3, \gamma, \mathbf{v}) > 0$, which contradicts the definition of convexity. This completes the proof.

B Proof of Proposition 2

Proposition 2. *The gradient of the fairness-loss function $L(\mathbf{P}, \gamma, \mathbf{v})$ with respect to the transition matrix $\mathbf{P}$ is given by*

$$\frac{\partial}{\partial \mathbf{P}} L(\mathbf{P}, \gamma, \mathbf{v}) = \frac{2(1-\gamma)}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right) \mathbf{p}\mathbf{y}_k^\top, \tag{4.1}$$

where $\mathbf{p}$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v})$, and $\mathbf{y}_k = \mathbf{U}^{-T}\mathbf{1}_k = (\mathbf{I} - (1-\gamma)\mathbf{P})^{-1}\mathbf{1}_k$, with $\mathbf{U} = \mathbf{I} - (1-\gamma)\mathbf{P}^\top$.

Recalling that $\mathbf{p} = \gamma\mathbf{U}^{-1}\mathbf{v}$, we have the following:

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{P}} L(\mathbf{P}, \gamma, \mathbf{v}) &= \frac{\partial}{\partial \mathbf{P}} \frac{1}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right)^2 \\
&= \frac{2}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right) \\
&= \frac{2}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{p}\right) \\
&= \frac{2\gamma}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v}\right).
\end{aligned}$$

By the chain rule of matrix calculus [28, Equation (137)], we obtain:

$$\frac{\partial}{\partial \mathbf{P}[i,j]} \left(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v}\right) = \mathrm{Tr}\left[\left(\frac{\partial(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v})}{\partial \mathbf{U}}\right)^\top \frac{\partial \mathbf{U}}{\partial \mathbf{P}[i,j]}\right] \tag{B.1}$$

By considering the derivative of inverse [28, Equation (61)] we have:

$$\frac{\partial(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v})}{\partial \mathbf{U}} = -\mathbf{U}^{-T}\mathbf{1}_k\mathbf{v}^\top\mathbf{U}^{-T}.$$

The gradient of $\mathbf{U}$ with respect to $\mathbf{P}[i,j]$ is:

$$\frac{\partial \mathbf{U}}{\partial \mathbf{P}[i,j]} = -(1-\gamma)\frac{\partial \mathbf{P}^\top}{\partial \mathbf{P}[i,j]} = -(1-\gamma)\mathbf{E}_{ji},$$

where $\mathbf{E}_{ij}$ is the matrix whose $[i,j]$ entry is 1 and all other entries are 0. From Equation (B.1), we get:

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{P}[i,j]} \left(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v}\right) &= (1-\gamma)\mathrm{Tr}\left[\left(\mathbf{U}^{-T}\mathbf{1}_k\mathbf{v}^\top\mathbf{U}^{-T}\right)^\top \mathbf{E}_{ji}\right] \\
&= (1-\gamma)\left(\mathbf{U}^{-1}\mathbf{v}\mathbf{1}_k^\top\mathbf{U}^{-1}\right)[i,j] \\
&= \frac{1-\gamma}{\gamma}\left(\mathbf{p}\mathbf{1}_k^\top\mathbf{U}^{-1}\right)[i,j].
\end{aligned}$$

Substituting (B) into Equation (B) leads to

$$\frac{\partial}{\partial \mathbf{P}} L(\mathbf{P}, \gamma, \mathbf{v}) = \frac{2(1-\gamma)}{K} \sum_{k=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p} - \phi_k\right)\left(\mathbf{p}\mathbf{1}_k^\top\mathbf{U}^{-1}\right) \tag{B.2}$$

C Proof of Proposition 3

Proposition 3. *The gradient of the fairness-loss function $L_g(\mathbf{P}, \gamma)$ with respect to the transition matrix $\mathbf{P}$ is given by*

$$\frac{\partial}{\partial \mathbf{P}} L_g(\mathbf{P}, \gamma) = \frac{2(1-\gamma)}{K^2} \sum_{k=1}^{K}\sum_{\ell=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right)\left(\mathbf{p}_\ell\mathbf{1}_k^\top\mathbf{U}^{-1}\right). \tag{4.2}$$

where $\mathbf{p}_\ell$ is the PageRank vector with respect to $(\mathbf{P}, \gamma, \mathbf{v}_\ell)$.

Similarly to the proof in Appendix B, for the group-adapted fairness loss, we can compute its gradient over $\mathbf{P}$ as follows:

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{P}} L_g(\mathbf{P}, \gamma) &= \frac{\partial}{\partial \mathbf{P}} \frac{1}{K^2} \sum_{k=1}^{K}\sum_{l=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right)^2 \\
&= \frac{2}{K^2} \sum_{k=1}^{K}\sum_{l=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right) \\
&= \frac{2}{K^2} \sum_{k=1}^{K}\sum_{l=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{p}_\ell\right) \\
&= \frac{2\gamma}{K^2} \sum_{k=1}^{K}\sum_{l=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right) \frac{\partial}{\partial \mathbf{P}} \left(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v}_\ell\right).
\end{aligned} \tag{C.1}$$

According to Appendix B,

$$\begin{aligned}
\frac{\partial}{\partial \mathbf{P}[i,j]} \left(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{v}_\ell\right) &= (1-\gamma)\left(\mathbf{U}^{-1}\mathbf{v}_\ell\mathbf{1}_k^\top\mathbf{U}^{-1}\right)[i,j] \\
&= \frac{1-\gamma}{\gamma}\left(\mathbf{p}_\ell\mathbf{1}_k^\top\mathbf{U}^{-1}\right)[i,j].
\end{aligned} \tag{C.2}$$

Substituting Equation (C.2) to Equation (C.1), we get

$$\frac{\partial}{\partial \mathbf{P}} L_g(\mathbf{P}, \gamma) = \frac{2(1-\gamma)}{K^2} \sum_{k=1}^{K}\sum_{\ell=1}^{K} \left(\mathbf{1}_k^\top \mathbf{p}_\ell - \phi_k\right)\left(\mathbf{p}_\ell\mathbf{1}_k^\top\mathbf{U}^{-1}\right) \tag{C.3}$$

D Approximating $\mathbf{y}_k$

In this section, we prove that for approximating $\mathbf{y}_t$, truncating the Neumann series to its first 50 terms yields a relative error below 0.0003.

According to the Neumann series formula, we can calculate the matrix inverse of a matrix $\mathbf{I}-\mathbf{X}$ as: $(\mathbf{I}-\mathbf{X})^{-1} = \sum_{t=0}^{\infty}\mathbf{X}^{\mathsf{T}}$. Let $\mathbf{S}_t$ denote the partial sum of the Neumann series up to the t-th term, that is,

$$\mathbf{S}_t = \sum_{i=0}^{t}\mathbf{X}^{\mathsf{T}} = \mathbf{I}+\mathbf{X}+\mathbf{X}^2+\cdots+\mathbf{X}^k.$$

We denote the truncation error as $\mathbf{E}_t = (\mathbf{I}-\mathbf{X})^{-1}-\mathbf{S}_t$, then we can express $\mathbf{E}_t$ as the sum of the remaining terms:

$$\mathbf{E}_t = \sum_{i=t+1}^{\infty}\mathbf{X}^{\mathsf{T}}$$

By factoring out $\mathbf{X}^{t+1}$ from the series we get:

$$\mathbf{E}_t = \mathbf{X}^{t+1}\sum_{i=0}^{\infty}\mathbf{X}^{\mathsf{T}}.$$

Using the formula for the Neumann series, the infinite sum $\sum_{t=0}^{\infty}\mathbf{X}^{\mathsf{T}}$ is equal to $(\mathbf{I}-\mathbf{X})^{-1}$. Thus:

$$\mathbf{E}_t = \mathbf{X}^{t+1}(\mathbf{I}-\mathbf{X})^{-1}$$

By taking the matrix norm of both sides, we get

$$\|\mathbf{E}_t\| \le \|\mathbf{X}^{t+1}\|\cdot\|(\mathbf{I}-\mathbf{X})^{-1}\|$$

Given that $|\mathbf{X}|<1$, it follows that

$$\|\mathbf{E}_t\| \le \|\mathbf{X}\|^{t+1}\cdot\|(\mathbf{I}-\mathbf{X})^{-1}\|$$

Thus, we conclude that the relative error for approximating $(1-\mathbf{X})^{-1}$ using the first t term of the Neumann series is given by $\|\mathbf{X}\|^{t+1}$.

Since $\mathbf{U}^{\mathsf{T}} = 1-(1-\gamma)\mathbf{P}$, to evaluate the approximation error of $\mathbf{y}_t = \mathbf{U}^{-T}\mathbf{1}_k$, we set $\mathbf{X} = (1-\gamma)\mathbf{P}$ with $\gamma = 0.15$. Consequently, $\|(1-\gamma)\mathbf{P}\|_\infty = 0.85$. It follows that the relative error of the t-term approximation is 0.85^{t+1}. Setting $t = 50$ yields a relative error below 0.0003.

E Omitted algorithms

In this section, we provide pseudocode for Algorithms 2 and 3.

Algorithm 2 invokes Algorithm 4 as a subroutine to compute the optimal Lagrangian dual variable λ^*. The implementation in Algorithm 4 differs slightly in form from the procedure described by Adam and Mácha [1], yet both are mathematically equivalent. For completeness, we now present a formal justification of the correctness of Algorithm 4, establishing that it indeed returns the optimal dual variable.

According to Adam and Mácha [1], the solution to the projection defined in Section 4.1 is $s_i^* = \mathrm{clip}(s_i+\lambda^*, \ell_i, u_i)$, with $\mathrm{clip}(x,a,b)$ defined as

$$\mathrm{clip}(x,a,b) = \begin{cases} a & \text{if } x \le a, \\ x & \text{if } a < x < b, \\ b & \text{if } x \ge b, \end{cases}$$

and λ^* is the solution to the following equation

$$h(\lambda) := \sum_{i=1}^{K}\mathrm{clip}(s_i+\lambda, \ell_i, u_i) - 1 = 0.$$

Since h is piecewise linear and non-decreasing in λ, we can find λ^* via a simple method proposed by Adam and Mácha [1]:

(1) For any λ, $\mathrm{clip}(s_i, \ell_i, u_i)$ takes value ℓ_i, $\lambda+s_i$, or u_i, thus, we can rewrite $h(\lambda)$ as $h(\lambda) = \mathrm{Active}(\lambda)\cdot\lambda + C$ for some constant C, where $\mathrm{Active}(\lambda)$ is the number of elements such that $s_i+\lambda$ falls in range $[\ell_i, u_i)$.
(2) Let R be the increasing ranking of $2N$ items consisting of $\boldsymbol{\ell}-\mathbf{s}$ and $\mathbf{u}-\mathbf{s}$. For any λ in interval $[R_j, R_{j+1})$, the function $\mathrm{Active}(\lambda)$ remains constant.
(3) Let $\mathrm{Total}(\lambda) = \sum_{i=1}^{N}\mathrm{clip}(s_i+\lambda, \ell_i, u_i)$ and assume that $\mathcal{F}_2$ is non-empty. Then there must exist some $j \in \{1,\cdots,2N\}$ such that $\mathrm{Total}(R_j) \le 1$ and $\mathrm{Total}(R_{j+1}) > 1$, and we can conclude that λ^* lies in range $[R_j, R_{j+1})$. Since $\mathrm{Active}(R_j) = \mathrm{Active}(\lambda^*)$, it follows that $(\lambda^*-R_j)\cdot\mathrm{Active}(R_j) = 1-\mathrm{Total}(R_j)$ and thus we can compute λ^* as

$$\lambda^* = \frac{1-\mathrm{Total}(R_j)}{\mathrm{Active}(R_j)}.$$

Algorithm 3 `AdaptGD`: Projected gradient descent for Group-adapted $\boldsymbol{\phi}$-PageRank and Restricted Group-adapted $\boldsymbol{\phi}$-PageRank

Input: $\mathbf{P}$, γ, $\mathbf{v}$, $\boldsymbol{\phi} = (\phi_1,\ldots,\phi_K)$, group indicator vectors $\mathbf{1}_k$ for $k = \{1,\cdots,K\}$, learning rate α, iteration exponent t_1 and t_2, convergence criterion κ, maximum number of iterations N_{iter}, modification bound δ and ϵ (optional).
Output: Optimized transition matrix $\hat{\mathbf{P}}$

```
P̂ ← P, iter ← 0, L_prev ← ∞, p = (1/n)1
while |L_g(P̂, γ) − L_prev| > κ and iter < N_iter do
    iter ← iter + 1
    L_prev ← L_g(P̂, γ, v)
    for ℓ = 1 to K do
        p ← iterating p^T = (1 − γ)p^T P̂ + (γ/|V_ℓ|) 1_ℓ^T for t_1 times
        for k = 1 to K do
            y_k ← Σ_{i=1}^{t_2} (1 − γ)^i P̂^i 1_k
            P̂ ← P̂ − α (2(1−γ)/K^2) (1_k^T p_ℓ − φ_k) p_ℓ y_k
        end for
    end for
    P̂ ← Project(P̂, δ, ε)
end while
return P̂
```

F Convergence Analysis for minimizing $L(\mathbf{P})$

In this section, we first obtain an upper bound for the Lipschitz constant for the loss function, and then prove that the projected gradient descent method converges to a stationary point.

Given restart probability γ and restart vector $\mathbf{v}$, we can rewrite the loss function as :

$$L(\mathbf{P}) = \frac{1}{K}\sum_{k=1}^{K}\left(g_k(\mathbf{P})\right)^2$$

where $g_k(\mathbf{P}) = \mathbf{1}_k^{\mathsf{T}}\mathbf{p}-\phi_k$ and $\mathbf{p} = \gamma(\mathbf{I}-(1-\gamma)\mathbf{P}^{\mathsf{T}})^{-1}\mathbf{v}$.

Algorithm 4 `Root-Finding`: find the optimal dual variable

Input: vector $\mathbf{s}$, lower bound vector $\boldsymbol{\ell}$, upper bound vector $\mathbf{u}$
Output: λ^*

```
K = |s|
λ ← non-decreasing ranking of ℓ − s and u − s.
Active = 1, Total ← Σ_j ℓ_j
for i = 2, ···, 2K do
    Total ← Total + Active × (λ_i − λ_{i−1})
    if Total ≥ 1 then
        return λ* = (1−Total)/Active + λ_i
    else
        if λ_i belongs to ℓ − s then Active++ else Active−−
    end if
end for
```

F.1 Gradient of the Loss $L(\mathbf{P})$

Though we have already provided the first-gradient analysis of the loss function $L(\mathbf{P})$, we rewrite it here for ease of analysis. We first compute the gradient of L with respect to $\mathbf{P}_{ij}$:

$$\frac{\partial L}{\partial \mathbf{P}_{ij}} = \frac{2}{K}\sum_{k=1}^{K} g_k \frac{\partial h_k}{\partial \mathbf{P}_{ij}}$$

where $h_k(\mathbf{P}) = \mathbf{1}_k^\top \mathbf{p}$.

According to Equation (B.2), $\frac{\partial h_k}{\partial \mathbf{P}} = (1-\gamma)\mathbf{p}\mathbf{1}_k^\top \mathbf{U}^{-1}$, therefore,

$$\frac{\partial h_k}{\partial \mathbf{P}_{ij}} = (1-\gamma)\mathbf{p}_i(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j)$$

where $\mathbf{e}_j$ is the standard basis vector (1 at position j, zero elsewhere).

F.2 Second Derivative of the Loss $L(P)$

We now compute the second derivative of the loss:

$$\begin{aligned}\frac{\partial^2 L}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}} &= \frac{\partial}{\partial \mathbf{P}_{ab}}\left(\frac{\partial L}{\partial \mathbf{P}_{ij}}\right)\\ &= \frac{\partial}{\partial \mathbf{P}_{ab}}\left[\frac{2}{K}\sum_{k=1}^{K} g_k \frac{\partial h_k}{\partial \mathbf{P}_{ij}}\right]\\ &= \frac{2}{K}\sum_{k=1}^{K}\left[\frac{\partial g_k}{\partial \mathbf{P}_{ab}}\frac{\partial h_k}{\partial \mathbf{P}_{ij}} + g_k \frac{\partial^2 h_k}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}}\right]\end{aligned}$$

We calculate each term of the second derivative respectively.

First term:

$$\frac{\partial g_k}{\partial \mathbf{P}_{ab}} = \frac{\partial h_k}{\partial \mathbf{P}_{ab}} = (1-\gamma)\mathbf{p}_a(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_b)$$

Second term:

$$\begin{aligned}\frac{\partial^2 h_k}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}} &= (1-\gamma)\frac{\partial}{\partial \mathbf{P}_{ab}}\left[\mathbf{p}_i(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j)\right]\\ &= (1-\gamma)\left[\frac{\partial \mathbf{p}_i}{\partial \mathbf{P}_{ab}}(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) + \mathbf{p}_i\frac{\partial}{\partial \mathbf{P}_{ab}}(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j)\right]\end{aligned}$$

We have:

$$\frac{\partial \mathbf{p}_i}{\partial \mathbf{P}_{ab}} = (1-\gamma)[\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}]_i$$

and

$$\frac{\partial}{\partial \mathbf{P}_{ab}}(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) = (1-\gamma)\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{U}^{-1}\mathbf{e}_j$$

Combining the two terms, we get

$$\frac{\partial^2 h_k}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}} = (1-\gamma)^2\Big([\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}]_i \cdot (\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) + \mathbf{p}_i \cdot \mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{U}^{-1}\mathbf{e}_j\Big)$$

Thus, the Hessian entry of $L(\mathbf{P})$ at coordinate $((ij),(ab))$ is:

$$\begin{aligned}\frac{\partial^2 L}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}} = \frac{2}{K}\sum_{k=1}^{K}\Bigg[&(1-\gamma)^2\mathbf{p}_a\mathbf{p}_i(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_b)(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) \qquad \text{(F.1)}\\ &+ g_k(\mathbf{P})\cdot(1-\gamma)^2\Big([\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}]_i \cdot (\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) + \mathbf{p}_i \cdot \mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{U}^{-1}\mathbf{e}_j\Big)\Bigg]\end{aligned}$$

F.3 Lipschitz Constant for $\frac{\partial L}{\partial \mathbf{P}}$

Since $L(\mathbf{P})$ is twice differentiable, the first derivative of $L(\mathbf{P})$ is Lipschitz smooth with constant C if and only if the operator norm of the Hessian matrix $\frac{\partial^2 L}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}}$ is bounded by C for all $\mathbf{P}$ in the domain. In this section, we flat the Hessian matrix and show an upper bound for its operator norm, which serves as an upper bound to its Lipschitz constant.

In order to flatten the Hessian matrix, let $r \in [n^2]$ be the row indexes and $s \in [n^2]$ be the column indexes. Given any (i,j) pair, we can calculate a unique flattened row index $r = (i-1)n + j$ and given any (a,b) pair, we can calculate a unique flattened column index $s = (a-1)n + b$. Conversely, given any pair (r,s) we can calculate the unique corresponding four indices $i(r), j(r), a(s)$ *and* $b(s)$ as follows:

$$i(r) = \lfloor \frac{r-1}{n} \rfloor + 1, j(r) = r - (i(r)-1)n$$

$$a(s) = \lfloor \frac{s-1}{n} \rfloor + 1, b(s) = s - (a(s)-1)n$$

We can then rewrite Equation (F.1) as

$$\begin{aligned}\mathbf{H}_{rs} = \frac{2}{K}\sum_{k=1}^{K}\Bigg[&(1-\gamma)^2\mathbf{p}_{a(s)}\mathbf{p}_{i(r)}(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{b(s)})(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{j(r)})\\ &+ g_k(\mathbf{P})\cdot(1-\gamma)^2\Big([\mathbf{U}^{-1}\mathbf{E}_{b(s)a(s)}\mathbf{p}]_{i(r)} \cdot (\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{j(r)}\\ &+ g_k(\mathbf{P})\cdot(1-\gamma)^2\mathbf{p}_{i(r)} \cdot \mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{E}_{b(s)a(s)}\mathbf{U}^{-1}\mathbf{e}_{j(r)}\Big)\Bigg]\end{aligned}$$

Let

$$\mathbf{H}_{rs}^{(1)} = \frac{2}{K}(1-\gamma)^2\sum_{k=1}^{K}\mathbf{p}_{a(s)}\mathbf{p}_{i(r)}(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{b(s)})(\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{j(r)})$$

$$\mathbf{H}_{rs}^{(2)} = \frac{2}{K}(1-\gamma)^2\sum_{k=1}^{K} g_k(\mathbf{P})[\mathbf{U}^{-1}\mathbf{E}_{b(s)a(s)}\mathbf{p}]_{i(r)} \cdot (\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_{j(r)})$$

$$\mathbf{H}_{rs}^{(3)} = \frac{2}{K}(1-\gamma)^2\sum_{k=1}^{K} g_k(\mathbf{P})\mathbf{p}_{i(r)} \cdot \mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{E}_{b(s)a(s)}\mathbf{U}^{-1}\mathbf{e}_{j(r)}$$

Then it holds that

$$\mathbf{H}_{rs} = \mathbf{H}_{rs}^1 + \mathbf{H}_{rs}^2 + \mathbf{H}_{rs}^3$$

To be able to bound the operator norm of $\mathbf{H}$, we bound the operator norm of $\mathbf{H}^{(1)}$, $\mathbf{H}^{(2)}$, and $\mathbf{H}^{(3)}$ separately.

Before bounding the operator norm of each term, we first proof $\|\mathbf{U}^{-1}\|_2 \leq 1/\gamma$, which will be used multiple times in the following deriviation. The proof is the result of three observations: (1) the largest eigenvalue of $\mathbf{P}^\top$ is 1, (2) the smallest eigenvalue of $\mathbf{U}$ is γ and (3) the operator norm, $\|\mathbf{U}^{-1}\|_2$, of matrix $\mathbf{U}^{-1}$ is the reciprocal of the smallest absolute eigenvalue of $\mathbf{U}$. We provide details for each observation in the following:

(1) Let $\mathbf{1}$ be an all-one vector, i.e. $\mathbf{1} = (1, 1, ..., 1)^\top$. We can see $\mathbf{1}$ is a eigenvector of $\mathbf{P}$ since $\mathbf{P1} = 1$. We also know the largest eigenvalue is bounded by the infinity norm, which is $\|\mathbf{P}\|_\infty = \max_i \sum_j |\mathbf{P}_{ij}| = 1$. Thus, the maximum eigenvalue of $\mathbf{P}^\top$, which is equal to the maximum eigenvalue of $\mathbf{P}$, is equal to 1.
(2) Let σ denote each eigenvalue of $\mathbf{P}^\top$, and μ each eigenvalue of $\mathbf{U}$. Since $\mathbf{U} = \mathbf{I} - (1-\gamma)\mathbf{P}^\top$, it holds that $\mu = 1-(1-\gamma)\sigma$. Thus the smallest eigenvalue of $\mathbf{U}$ occurs when $\sigma = 1$, leading to $\mu_{min} = 1 - (1-\gamma) = \gamma$
(3) Combining (1) and (2), we get $\|\mathbf{U}^{-1}\| = \frac{1}{\mu_{min}} = \frac{1}{\gamma}$.

F.3.1 Bounding $\|\mathbf{H}^{(1)}\|_2$. Let $\mathbf{w}_k$ of a vector of length n^2 that depends on group k, specifically,

$$\mathbf{w}_k[r] = (1-\gamma)\mathbf{p}_{i(r)}(\mathbf{1}_k^\top \mathbf{U}^{-1} e_{j(r)})$$

Then we can represent $\mathbf{H}^{(1)}$ as a rank 1 matrix as follow:

$$\mathbf{H}^{(1)} = \frac{2}{K}\sum_{k=1}^{K} \mathbf{w}_k \mathbf{w}_k^\top$$

By the subadditivity of the spectral norm, and the identity $\|\mathbf{w}_k\mathbf{w}_k^\top\|_2 = \|\mathbf{w}_k\|_2^2$. We first bound the second norm of $\mathbf{w}_k$, it follows that

$$\begin{aligned}
\|\mathbf{w}_k\|_2^2 &= (1-\gamma)^2 \sum_{i=1}^{n} \mathbf{p}_i^2 \sum_{j=1}^{n} (\mathbf{1}_k^\top \mathbf{U}^{-1} e_j)^2 \\
&= (1-\gamma)^2 \sum_{i=1}^{n} \mathbf{p}_i^2 \|\mathbf{1}_k^\top \mathbf{U}^{-1}\|_2^2 \\
&\overset{(a)}{\leq} \frac{(1-\gamma)^2}{\gamma^2}\|\mathbf{1}_k\|_2^2
\end{aligned}$$

where (a) holds because $\sum_i \mathbf{p}_i^2 \leq 1$, and $\|\mathbf{U}^{-1}\| \leq \frac{1}{\gamma}$. We can then bound $\|\mathbf{H}^{(1)}\|_2$ as follow:

$$\|\mathbf{H}^{(1)}\|_2 = \frac{2}{K}\sum_{k=1}^{K}\|\mathbf{w}_k\|_2^2 \tag{F.2}$$

$$\leq \frac{2(1-\gamma)^2}{K\gamma^2}\sum_{k=1}^{K}\|\mathbf{1}_k\|_2^2 = \frac{2(1-\gamma)^2}{K\gamma^2}n \tag{F.3}$$

F.3.2 Bounding $\|\mathbf{H}^{(2)}\|_2$. Let $\mathbf{X} \in \mathbb{R}^{n\times n}$ be a matrix, and let $\mathbf{x} \in \mathbb{R}^{n^2}$ denote the vector obtained by flattening $\mathbf{X}$. We can then obtain a vector $\mathbf{y} = \mathbf{H}^{(2)}\mathbf{x} \in \mathbb{R}^{n^2}$, which, after reshaping, is equal to a matrix $\mathbf{Y} \in \mathbb{R}^{n\times n}$. Specifically,

$$\begin{aligned}
\mathbf{Y}_{i(r),j(r)} = \mathbf{y}_r &= \sum_{s=1}^{n^2} \mathbf{H}^{(2)}_{r,s}\mathbf{x} \\
&= \sum_{a(s)=1}^{n}\sum_{b(s)=1}^{n} \mathbf{H}^{(2)}_{(i(r),j(r)),(a(s),b(s))}\mathbf{X}_{a(s),b(s)} \\
&\overset{(a)}{=} \frac{2(1-\gamma)^2}{K}\sum_{k=1}^{K} g_k \left[\sum_{a,b}[\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}]_i \mathbf{X}_{ab}\right](\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j) \\
&= \frac{2(1-\gamma)^2}{K}\sum_{k=1}^{K} g_k \left[\sum_b (\mathbf{U}^{-1})_{i,b}\left(\sum_a \mathbf{p}_a \mathbf{X}_{a,b}\right)\right](\mathbf{1}_k^\top \mathbf{U}^{-1}\mathbf{e}_j)
\end{aligned}$$

where for equality (a) and the following deriviation, we replace $a(s), b(s)$ with a, b, and $i(r), j(r)$ with (i, j) for ease of notation whenever the context is clear.

Let $\mathbf{u} = \mathbf{X}^\top\mathbf{p}$, $\mathbf{z} = \sum_{k=1}^{K} g_k \mathbf{1}_k$, and $\mathbf{w} = \mathbf{A}^{-T}\mathbf{z}$, then we can rewrite $\mathbf{Y}$ as

$$\mathbf{Y} = \frac{2(1-\gamma)^2}{K}(\mathbf{U}^{-1}u)\mathbf{w}^\top$$

Since $(\mathbf{U}^{-1}u)\mathbf{w}^\top$ is a rank-1 matrix, the spectrual norm and the Frobenius norm of $(\mathbf{U}^{-1}u)\mathbf{w}^\top$ are equal. Thus we can bound the operator norm of $\mathbf{Y}$ as

$$\begin{aligned}
\|\mathbf{Y}\|_F &= \frac{2(1-\gamma)^2}{K}\|\mathbf{U}^{-1}\mathbf{u}\|_2\|\mathbf{w}\|_2 \\
&\leq \frac{2(1-\gamma)^2}{K}(\|\mathbf{U}^{-1}\|_2\|\mathbf{u}\|_2)(\|\mathbf{U}^{-1}\|_2\|\mathbf{z}\|_2) \\
&\overset{(a)}{\leq} \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{u}\|_2\|\mathbf{z}\|_2 \\
&\leq \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{X}^\top\mathbf{p}\|_2 \sum_{k=1}^{K}\|\mathbf{1}_k\|_2|g_k| \\
&\overset{(b)}{\leq} \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{X}\|_F\|\mathbf{p}\|_2 \sum_{k=1}^{K}\|\mathbf{1}_k\|_2 \\
&\overset{(c)}{\leq} \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{X}\|_F\sqrt{n}
\end{aligned}$$

where inequality (a) holds because $\|\mathbf{U}^{-1}\|_2 \leq \frac{1}{\gamma}$, (b) holds because $|g_k| \leq 1$ and (c) holds because $\|\mathbf{p}\| \leq 1$.

Therefore, it holds that

$$\begin{aligned}
\|\mathbf{H}^{(2)}\mathbf{x}\|_2 = \|\mathbf{y}\|_2 = \|\mathbf{Y}\|_F &\leq \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{X}\|_F\sqrt{n} \\
&= \frac{2(1-\gamma)^2}{K\gamma^2}\|\mathbf{x}\|_2\sqrt{n}
\end{aligned}$$

by dividing both side by $\|\mathbf{x}\|_2$ and taking the supremum over $\mathbf{x} \neq \mathbf{0}$, we get

$$\|\mathbf{H}^{(2)}\|_2 \leq \frac{2(1-\gamma)^2}{K\gamma^2}\sqrt{n} \tag{F.4}$$

F.3.3 Bounding $\|\mathbf{H}^{(3)}\|_2$. We can reorganize terms in $\mathbf{H}^{(3)}$ and it holds that

$$\mathbf{H}^{(3)}_{rs} = \frac{2(1-\gamma)^2}{K}\sum_{k=1}^{K} g_k \mathbf{p}_{i(r)}(\mathbf{U}^{-1})_{a(s),j(r)}(\mathbf{z}_k)_{b(s)}$$

where $\mathbf{z}_k = \mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}$.

Let $(\mathbf{C}_k)_{(i(r),j(r)),(a(s),b(s))} = \mathbf{p}_{i(r)}(\mathbf{U}^{-1})_{a(s),j(r)}(\mathbf{z}_k)_{b(s)}$, we can further rewrite $\mathbf{H}^2$ as follow:

$$\mathbf{H}^{(3)}_{rs} = \frac{2(1-\gamma)^2}{K}\sum_{k=1}^{K} g_k \mathbf{C}_k \tag{F.5}$$

Furthermore, since the three factors $\mathbf{p}_i(r), (\mathbf{U}^{-1})_{a(s),j(e)}, (\mathbf{z}_k)_{b(s)}$ depend on disjoint index pairs, $\mathbf{C}_k$ can be represented as a triple outer product and reshaped as

$$\mathbf{C}_k = \mathbf{p} \otimes \mathbf{U}^{-1} \otimes \mathbf{z}_k$$

and thus we can bound the operator norm of $\mathbf{C}_k$ as

$$\begin{aligned}\|\mathbf{C}_k\|_2 &= \|\mathbf{p}\|_2\|\mathbf{U}^{-1}\|_2\|\mathbf{z}_k\|_2 \\ &\le 1 \times \frac{1}{\gamma} \times \|\mathbf{U}^{-1}\|_2\|1_k\|_2 \\ &\le \frac{1}{\gamma^2}\|1_k\|_2\end{aligned}$$

Therefore, we can bound the operator norm of $\mathbf{H}^{(3)}$ as

$$\begin{aligned}\|\mathbf{H}^{(3)}\|_2 &\le \frac{2K(1-\gamma^2)}{\gamma^2}\sum_{k=1}^{K}\|1_k\|_2 \\ &= \frac{2K(1-\gamma^2)}{\gamma^2}\sum_{k=1}^{K}\sqrt{|V_k|} \\ &\overset{(a)}{\le} \frac{2K(1-\gamma^2)}{\gamma^2}\sqrt{K}\sqrt{\sum_{k=1}^{K}|V_k|} \\ &= \frac{2K(1-\gamma^2)}{\gamma^2}\sqrt{K}\sqrt{n} \\ &= \frac{2(1-\gamma^2)}{\gamma^2}\sqrt{\frac{n}{K}}\end{aligned} \tag{F.6}$$

where inequality (a) is due to Cauchy-Schwarz inequatlity.

F.3.4 The Lipschitz constant for the first derivative of $L(\mathbf{P})$. Combining Equation (F.2), Equation (F.4) and Equation (F.6), it follows that

$$\|\mathbf{H}\|_2 \le \|\mathbf{H}^{(1)}\|_2 + \|\mathbf{H}^{(2)}\|_2 + \|\mathbf{H}^{(3)}\|_2 \le \frac{2(1-\gamma)^2}{K\gamma^2}(n+\sqrt{n}+\sqrt{Kn})$$

Let $C = \frac{2(1-\gamma)^2}{K\gamma^2}(n+\sqrt{n}+\sqrt{Kn})$, it follows that $\frac{\partial L}{\partial \mathbf{P}}$ is C-Lipschitz.

F.4 Convergence analysis of the projected gradient descent method for minimizing $L(\mathbf{P})$

We have proved $L(P)$ is a differentiable function and its first derivative is C-Lipschitz, since its feasible set $\mathcal{C}_R(\mathbf{P}) \cap \mathcal{C}(\mathbf{P})$ is convex and closed, the projected gradient update with constant step size $0 \le \alpha \le 2/C$ converges to a stationary point, which is a local minimum of $L(\mathbf{P})$ over $\mathcal{C}_R(\mathbf{P}) \cap \mathcal{C}(\mathbf{P})$ [5].

G Convergence Analysis for minimizing $L_g(\mathbf{P})$

We can derive the Hessian matrix of $L_g(\mathbf{P})$ and its Lipschitz constant in a manner analogous to that of $L(\mathbf{P})$. Specifically, let $\mathbf{H}_g$ be the Hessian matrix of L_g, then it holds that

$$\begin{aligned}\mathbf{H}_g[(i,j),(a,b)] &= \frac{\partial^2 L_g}{\partial \mathbf{P}_{ij}\partial \mathbf{P}_{ab}} \\ &= \frac{2}{K^2}\sum_{k=1}^{K}\sum_{l=1}^{K}\Big[(1-\gamma)^2\mathbf{p}_{\ell a}\mathbf{p}_{\ell i}(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_b)(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_j) \\ &+ g_k(\mathbf{P})\cdot(1-\gamma)^2\Big([\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}_\ell]_i\cdot(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_j) + \mathbf{p}_{\ell i}\cdot\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{U}^{-1}\mathbf{e}_j\Big)\Big]\end{aligned} \tag{G.1}$$

Equation (G.1) follows directly from Equation (F.1) by replacing $\mathbf{p}$ with $\mathbf{p}_\ell$ throughout, introducing an additional summation over $l \in 1, \dots, K$, and dividing by the extra term K. Let $\mathbf{H}_g^k$ denote the inner sum of $\mathbf{H}_g$, that is,

$$\begin{aligned}\mathbf{H}_g^k &= \frac{2}{K}\sum_{l=1}^{K}\Big[(1-\gamma)^2\mathbf{p}_{\ell a}\mathbf{p}_{\ell i}(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_b)(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_j) \\ &+ g_k(\mathbf{P})\cdot(1-\gamma)^2\Big([\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{p}_\ell]_i\cdot(\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{e}_j) + \mathbf{p}_{\ell i}\cdot\mathbf{1}_k^{\mathsf{T}}\mathbf{U}^{-1}\mathbf{E}_{ba}\mathbf{U}^{-1}\mathbf{e}_j\Big)\Big]\end{aligned}$$

It is straightforward to see that the derivation of an upper bound on the operator norm of $\mathbf{H}$ remains valid for $\mathbf{H}_g^k$ when $\mathbf{p}$ is replaced by $\mathbf{p}_\ell$. That is, $\mathbf{H}$ and $\mathbf{H}_g^k$ share the same operator norm upper bound C. Given the subadditivity property of the operator norm, it holds that

$$\|\mathbf{H}_g\|_2 \le \frac{1}{K}\sum_{k=1}^{K}\|\mathbf{H}_g^k\|_2 = \|\mathbf{H}\|_2 = C$$

Therefore, we have established that the operator norm of L_g is bounded by C, which is equivalent to stating that the Lipschitz constant of its gradient is also bounded by C. Consequently, projected gradient descent converges to a local minimum when using a constant step size $0 \le \alpha \le 2/C$.

H Experimental settings

H.1 Datasets

Books[3] This is an undirected network of political books sold on Amazon. Each edge represents frequent co-purchases of two books. The books are categorized into three groups: liberal, neutral, and conservative. Since the network is undirected, each edge is converted into two directed edges.

Blogs [2]. A directed network of political blogs, where each edge represents a hyperlink between two blogs. The vertices are divided into two groups: liberal and conservative blogs.

MIND [40] A dataset built from user impression logs on Microsoft News. Each log captures a user's impressions of current news articles, along with their reading history. News articles are labeled, and we construct a directed graph from these logs, filtering out

[3] http://www-personal.umich.edu/~mejn/netdata/

news articles with a frequency of fewer than 50 occurrences. A directed edge is created from a historically read news article to a current one if both appear in the same user's log. The two vertex groups correspond to lifestyle-related and society-related news[4].

Twitter [31]. **A** directed network of political retweets, where each edge represents a retweet from one user to another. The vertex groups are left-leaning and right-leaning users.

Slashdot [22]. **A** directed network representing friend/foe relationships between users on Slashdot. Since this dataset lacks group labels, we apply the METIS graph partitioning algorithm [17] to divide the vertices into groups.

H.2 Baselines

We present here a description of the baselines we use in our evaluation. In our experiments, $\hat{\mathbf{P}}$ represents the revised transition matrix, and $\hat{\mathbf{v}}$ represents the revised restart vector for both our algorithms and all the baselines.

LFPR_U [37]. **A** residual-based algorithm that revises both the restart vector and the transition matrix to achieve perfect $\boldsymbol{\phi}$-fairness in PageRank scores. The intuition is that each node distributes a portion of its PageRank to the under-represented groups, and distributes the remaining portion to its neighbors uniformly.

The revised transition matrix is $\hat{\mathbf{P}} = \mathbf{P}_L + \mathbf{R}$, where:

$$\mathbf{P}_L = \begin{cases} \frac{1-\phi_1}{out_2(i)}, & \text{if } (i,j) \in E \text{ and } i \in L_1 \\ \frac{\phi_1}{out_1(i)}, & \text{if } (i,j) \in E \text{ and } i \in L_2 \\ 0 & \text{otherwise} \end{cases} \tag{H.1}$$

and $out_k(i)$ represents the number of out-neighbors of vertex i that belongs to group k, and L_k denotes the set of vertices whose portion of neighbors that belong to group k is smaller than the fairness target ϕ_k. $\mathbf{R}$ is given by

$$\mathbf{R} = \delta_1 \mathbf{x}^\mathsf{T} + \delta_2 \mathbf{y}^\mathsf{T} \tag{H.2}$$

where $\delta_1[i] = \phi_1 - \frac{(1-\phi_1)out_1(i)}{out_2(i)}$, $\delta_2[i] = (1-\phi_1) - \frac{\phi_1 out_2(i)}{out_1(i)}$. $\mathbf{x}$ and $\mathbf{y}$ are two indicator vectors where $\mathbf{x}(i) = 1$ if $i \in V_1$ and $\mathbf{y}(i) = 1$ if $i \in V_2$.

LFPR_N [37]. **A** neighborhood-based algorithm that achieves perfect $\boldsymbol{\phi}$-fairness in PageRank scores. The revised transition matrix is given by $\hat{\mathbf{P}} = \sum_{k=1}^{2} \phi_k \mathbf{P}_k$, where

$$\mathbf{P}_k = \begin{cases} \frac{1}{out_k(i)}, & \text{if } (i,j) \in E \text{ and } j \in V_k \\ \frac{1}{|V_k|}, & \text{if } out_k(i) = 0 \text{ and } j \in V_k \\ 0 & \text{otherwise} \end{cases} \tag{H.3}$$

FairWalk [29]. This algorithm reweights the edges such that a random walker has an equal probability of visiting each reachable group and an equal probability of visiting each vertex within each reachable group. FairWalk does not change the restart vector, and the revised transition matrix is given by:

$$\hat{\mathbf{P}}[i,j] = \frac{\phi_{\ell_j} \mathbf{P}[i,j]}{\sum_{m \in V_{\ell_j}} P[i,m] \sum_{k=1}^{K} I_k(i)\phi_k}, \tag{H.4}$$

[4]The original dataset contains 14 labels. We aggregate vertices with the labels {lifestyle, food and drink, sports, health} into group 1, and those with the labels {entertainment, travel, tv, news, finance, movies, autos, music, weather, video} into group 2.

where $I_k(i) = 1$ if vertex i has a out-neighbor that belongs to group k and 0 otherwise.

CrossWalk [20]. This algorithm extends FairWalk by taking into account the distance of each vertex to other groups in the random walk. CrossWalk upweights edges that (1) are closer to the group boundaries or (2) connect different groups in the graph. This algorithm does not change the restart vector, and we refer readers to Algorithm 1 in the paper for the revised transition matrix.

I Additional experimental results

I.1 Ranking Coefficient for the group-wise PageRank weights

In Tables 3 and 4, we report the weighted average Spearman's rank correlation coefficient, $\overline{\rho}$, for all methods on the Blogs and Slashdot datasets, respectively. Results for the Books and Twitter datasets are omitted, as the average coefficients are close to zero across all methods, indicating that none of the methods can preserve the original PageRank rankings within groups.

On the Blogs dataset, `AdaptGD`(0.1, 0.1) achieves the highest correlation coefficient, followed by LFPR_N and LFPR_U. `FairGD`(0.5, 0.1) and `AdaptGD` obtain the lowest values, while the remaining methods yield comparable performance. On the Slashdot dataset, algorithm `AdaptGD`(0.1, 0.1) again achieves the highest value, followed by `AdaptGD`(0.5, 0.1). Algorithms `AdaptGD`, `FairGD`, and its variants `FairGD`(0.1, 0.1) and `FairGD`(0.5, 0.1) produce comparable results, whereas FairWalk and CrossWalk perform the worst.

I.2 Ranking Coefficient for the edge weights

In Tables 5 to 7, we present the average Spearman's rank correlation coefficient, $\bar{\rho}$, for each method on the Books, Blogs and MIND dataset. For each node, we compute the Spearman's coefficient between the rankings of its original and updated outgoing edge weights, and then report the average coefficient across all nodes. We omit the results for the datasets Twitter and Slashdot, as the average coefficients are all approximately in the range [0.999–1) across all methods.

FairWalk performs consistently well across all three datasets, achieving the best results on the Books dataset and second-best results on the remaining datasets. `AdaptGD`(0.1, 0.1) and `AdaptGD` (0.5,0.1) attain the second-best performance on the Books and Blogs datasets, and the third-best on the MIND dataset. `FairGD` achieves the optimal outcome on the MIND dataset.

For the two-group datasets, LFPR_U and LFPR_N perform noticeably worse than the other methods. This outcome is expected, as both methods substantially modify the underlying graph structure.

In conclusion, FairWalk best preserves the edge weight ranking across datasets, while `AdaptGD`(0.1, 0.1) and `AdaptGD`(0.5, 0.1) also perform very well.

Table 3: Average Spearman's coefficient $\overline{\rho}$ between the PageRank weight of each method for the Blogs dataset

ϕ	FairWalk	CrossWalk	LFPR_U	LFPR_N	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	0.573	0.506	0.558	0.547	0.437	0.423	0.437	0.423	**0.594**	0.561
0.2	0.579	0.527	0.552	**0.591**	0.483	0.488	0.515	0.446	0.587	0.568
0.3	0.568	0.497	0.577	0.593	0.534	0.553	0.542	0.433	**0.598**	0.553
0.4	0.602	0.539	0.566	0.597	0.560	0.555	0.540	0.442	**0.616**	0.551
0.5	0.590	0.513	0.588	0.577	0.562	0.564	0.577	0.451	**0.609**	0.557
0.6	0.576	0.490	0.599	0.579	0.557	0.569	0.570	**0.801**	0.614	0.545
0.7	0.588	0.511	0.604	0.567	0.586	0.574	0.573	**0.754**	0.627	0.570
0.8	0.612	0.502	0.602	**0.613**	0.510	0.489	0.493	0.492	0.604	0.575
0.9	0.584	0.498	0.606	0.617	0.516	0.506	0.501	0.503	**0.620**	0.555

Table 4: Average Spearman's coefficient $\overline{\rho}$ between the PageRank weight of each method for the Slashdot dataset

ϕ	FairWalk	CrossWalk	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	0.123	0.124	0.221	0.215	0.217	0.212	**0.295**	0.244
0.2	0.140	0.131	0.219	0.217	0.216	0.213	**0.312**	0.251
0.3	0.146	0.135	0.245	0.239	0.241	0.238	**0.310**	0.251
0.4	0.135	0.143	0.229	0.231	0.235	0.243	**0.308**	0.271
0.5	0.121	0.120	0.218	0.212	0.213	0.223	**0.296**	0.261
0.6	0.131	0.119	0.221	0.228	0.228	0.211	**0.287**	0.254
0.7	0.129	0.117	0.226	0.229	0.232	0.212	**0.282**	0.248
0.8	0.128	0.117	0.235	0.230	0.235	0.224	**0.280**	0.250
0.9	0.125	0.122	0.230	0.224	0.229	0.224	**0.271**	0.254

Table 5: Average Spearman's coefficient $\tilde{\rho}$ between edge weight rankings for the Books dataset

ϕ	FairWalk	CrossWalk	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	**0.989**	0.974	0.897	0.919	0.889	0.853	0.975	0.976
0.2	**0.989**	0.975	0.976	0.919	0.940	0.855	0.976	0.975
0.3	**0.989**	0.975	0.963	0.946	0.976	0.869	0.977	0.977
0.4	**0.989**	0.974	0.969	0.959	0.964	0.871	0.977	0.977
0.5	**0.989**	0.973	0.965	0.961	0.972	0.862	0.978	0.976
0.6	**0.988**	0.976	0.973	0.945	0.950	0.879	0.979	0.977
0.7	**0.988**	0.975	0.942	0.943	0.932	0.865	0.977	0.977
0.8	**0.988**	0.974	0.921	0.939	0.922	0.865	0.976	0.977
0.9	**0.988**	0.974	0.907	0.938	0.920	0.909	0.977	0.977

Table 6: Average Spearman's coefficient $\tilde{\rho}$ between edge weight rankings on the Blogs dataset

ϕ	FairWalk	CrossWalk	LFPR_U	LFPR_N	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	**0.997**	0.991	-0.249	0.282	0.987	0.987	0.987	0.984	0.996	0.996
0.2	**0.997**	0.991	-0.142	0.284	0.992	0.991	0.992	0.984	0.995	0.996
0.3	**0.997**	0.991	-0.107	0.284	0.995	0.995	0.995	0.986	0.996	0.996
0.4	**0.997**	0.991	-0.074	0.284	0.995	0.995	0.995	0.986	0.996	0.996
0.5	**0.997**	0.991	0.157	0.486	0.995	0.995	0.995	0.988	0.996	0.996
0.6	**0.997**	0.991	0.161	0.486	0.995	0.995	0.995	0.995	0.996	0.996
0.7	**0.997**	0.991	0.186	0.486	0.995	0.995	0.995	0.995	0.996	0.996
0.8	**0.997**	0.991	0.226	0.486	0.991	0.990	0.990	0.984	0.995	0.996
0.9	**0.997**	0.991	0.290	0.486	0.987	0.988	0.987	0.984	0.995	0.996

Table 7: Average Spearman's coefficient $\tilde{\rho}$ between edge weight rankings on the MIND dataset.

ϕ	FairWalk	CrossWalk	LFPR_U	LFPR_N	FairGD	FairGD(0.1, 0.1)	FairGD(0.5, 0.1)	AdaptGD	AdaptGD(0.1, 0.1)	AdaptGD(0.5, 0.1)
0.1	0.999	0.998	0.318	0.842	≈ 1	≈ 1	≈ 1	0.998	0.998	0.999
0.2	0.999	0.998	0.306	0.842	≈ 1	≈ 1	≈ 1	0.998	0.999	0.998
0.3	0.999	0.998	0.306	0.842	≈ 1	≈ 1	≈ 1	0.998	0.999	0.998
0.4	0.999	0.998	0.298	0.842	≈ 1	≈ 1	≈ 1	0.998	0.999	0.999
0.5	0.999	0.998	0.298	0.842	≈ 1	0.999	≈ 1	0.998	0.998	0.998
0.6	0.999	0.998	0.298	0.842	≈ 1	≈ 1	0.999	0.998	0.998	0.998
0.7	0.999	0.998	0.295	0.842	≈ 1	≈ 1	0.999	0.998	0.998	0.998
0.8	0.999	0.998	0.295	0.842	≈ 1	0.999	0.999	0.997	0.998	0.998
0.9	0.999	0.998	0.294	0.842	≈ 1	0.999	0.999	0.996	0.998	0.998

Paper E

Minimizing Hitting Time between Disparate Groups with Shortcut Edges

Florian Adriaens
florian.adriaens@helsinki.fi
University of Helsinki
Helsinki, Finland

Honglian Wang
honglian@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

Aristides Gionis
argioni@kth.se
KTH Royal Institute of Technology
Stockholm, Sweden

ABSTRACT

Structural bias or segregation of networks refers to situations where two or more disparate groups are present in the network, so that the groups are highly connected internally, but loosely connected to each other. Examples include polarized communities in social networks, antagonistic content in video-sharing or news-feed platforms, etc. In many cases it is of interest to increase the connectivity of disparate groups so as to, e.g., minimize social friction, or expose individuals to diverse viewpoints. A commonly-used mechanism for increasing the network connectivity is to add *edge shortcuts* between pairs of nodes. In many applications of interest, edge shortcuts typically translate to *recommendations*, e.g., what video to watch, or what news article to read next. The problem of reducing structural bias or segregation via edge shortcuts has recently been studied in the literature, and random walks have been an essential tool for modeling navigation and connectivity in the underlying networks. Existing methods, however, either do not offer approximation guarantees, or engineer the objective so that it satisfies certain desirable properties that simplify the optimization task.

In this paper we address the problem of adding a given number of shortcut edges in the network so as to *directly* minimize the *average hitting time* and the *maximum hitting time* between two disparate groups. The objectives we study are more natural than objectives considered earlier in the literature (e.g., maximizing hitting-time *reduction*) and the optimization task is significantly more challenging. Our algorithm for minimizing average hitting time is a greedy bicriteria that relies on supermodularity. In contrast, maximum hitting time is not supermodular. Despite, we develop an approximation algorithm for that objective as well, by leveraging connections with average hitting time and the asymmetric k-center problem.

CCS CONCEPTS

• **Theory of computation** → Design and analysis of algorithms;
• **Mathematics of computing** → Discrete mathematics.

KEYWORDS

Random walks, Edge augmentation, Social networks, Polarization

This work is licensed under a Creative Commons Attribution International 4.0 License.

KDD '23, August 6–10, 2023, Long Beach, CA, USA
© 2023 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0103-0/23/08.
https://doi.org/10.1145/3580305.3599434

ACM Reference Format:
Florian Adriaens, Honglian Wang, and Aristides Gionis. 2023. Minimizing Hitting Time between Disparate Groups with Shortcut Edges. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23), August 6–10, 2023, Long Beach, CA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3580305.3599434

1 INTRODUCTION

The last decade has seen a surge in the development of methods for detecting, quantifying, and mitigating polarization and controversy in social media. An example of a polarized network is a conversation or endorsement graph associated with a controversial topic on Twitter. The nodes represent users, and the edges interactions between users in the form of posts, likes, endorsements, etc., related to a controversial topic. It has been observed that the interactions typically occur between like-minded individuals, resulting in the reinforcement of ones own beliefs [7, 12].

Structural bias arises in many types of networks beyond social networks, such as *content* and *information networks*. Recently, several methods have been developed for reducing *structural bias* [24] or *segregation* [13, 15] in *content networks*. An example of a content network is the network obtained by "what to watch next" recommendations in a video-sharing platform. Content in such networks can often be divided into two or more groups, each of which is highly connected internally, but loosely connected to each other. These groups could be the result of differentiating between "harmful" (or radicalized) content and "neutral" (or non-radicalized) content, or simply the result of different opinion (pro/contra) on a certain issue. It might be beneficial for users navigating these platforms to be exposed to diverse content—exposing themselves to multiple viewpoints—in order to become better informed.

A particular line of research focuses on adding new edges (denoted as *shortcut* edges) to a network, with the goal of reducing some quantifiable measure of polarization, controversy, structural bias, or segregation [13–15, 21, 24, 45]. Many of these measures are defined on the basis of *random walks* between groups of nodes. In content graphs, random walks are a natural and simple model for how a user navigates through content. For example, the Random Walk Controversy (RWC) score of Garimella et al. [21] has been reported to "discriminate controversial topics with great accuracy" in Twitter conversation graphs. The authors in that work suggest a method that adds a fixed number of shortcut edges to a graph in order to reduce the RWC score of the augmented graph [21].

However, most of these random walk-based augmentation problems do not have algorithms with provable approximation guarantees [21], or the optimized measure has been reformulated so that it exhibits a desirable property [13, 15, 24].

For example, the work of Haddadan et al. [24] considers maximizing the gain function $\Delta_g(F) = g(\emptyset) - g(F)$ in the context of bounded-length random walks in directed graphs. Here, F is a set of at most k new shortcut edges that will be added to an input graph, and the function g is the *average expected hitting time* for a random walk starting in one group to hit the other group (see Definition 1). They prove that Δ_g is a non-negative monotone submodular set function, and thus, the well-known greedy algorithm of [43] ensures a $(1 - 1/e)$ approximation guarantee.

Similarly, the work of Fabbri et al. [15] considers a gain function in the form of $\Delta_f(\cdot) = f(\emptyset) - f(\cdot)$ in the context of degree-regular directed graphs and with *rewiring* operations instead of shortcut edge additions. Here, the function f is the *largest hitting time* instead of the average (see Definition 1). Fabbri et al. show that the existence of a multiplicative approximation algorithm implies $\mathbf{P} = \mathbf{NP}$. This result leads them to propose heuristic algorithms. However, their inapproximability result does not follow from the use of random walks, nor from the rewire operations. Their reduction still applies if one would replace "largest hitting time" with "largest shortest path distance," for example. Hence, their inapproximability result is an artefact of maximizing a gain function in combination with the max operator in the function f (Definition 1).

The previous discussion motivates the following question: "*What can we say about directly* minimizing *the functions f and g, instead of indirectly optimizing them by maximizing associated gain functions?*"

This paper provides several first algorithmic results and ideas regarding this question for both functions f and g. We take an abstract view and define our problems on input graphs in their simplest form, without the additional constraints (bounded-length random walks or degree-regular graphs) considered in Haddadan et al. [24] and Fabbri et al. [15], respectively. We consider uniform simple random walks of unbounded length on undirected, unweighted and connected input graphs.

Results and techniques. For the first problem we study, *minimize average hitting time* (BMAH), we observe that the objective is supermodular and it can be optimized by a greedy bicriteria strategy [32], which offers an $(1+\epsilon)$-approximation at the cost of adding logarithmically more edges. In addition, we show how to speed up this algorithm by deriving an approximation for the objective, which relies on sampling a small number of bounded-length absorbing random walks and bounding the error term using eigenvalue techniques. The length of these walks directly depends on the *mixing time* and *average degree* of the red nodes, which are bounded quantities in most real-life networks. We then show that running the same greedy strategy of Liberty and Sviridenko [32] on the *approximative* values, still provides a $(2 + \epsilon)$-approximation factor.

For our second problem, *minimize maximum hitting time* (BMMH), the objective function is neither supermodular nor submodular, and thus, it is a more challenging task. Nevertheless, we present two different algorithms, both with approximation guarantees, albeit, weaker than in the case of average hitting time. The first algorithm utilizes a relation between maximum and average hitting time, and uses the greedy strategy designed for the BMAH problem. The second algorithm leverages a novel connection between BMMH and the asymmetric k-center problem [26], and utilizes optimal methods developed for the latter problem [4].

Table 1: Results summary. The approximation factor is computed with respect to an optimal solution that uses at most k edges. d_{m} is at most the maximum degree (see Section 6.2).

	Approx. factor	Number of shortcut edges
BMAH	$1 + \epsilon$	$O(k \log(n/\epsilon))$
BMAH	$2 + \epsilon$	$O(k \log(n/\epsilon))$
BMMH	$O(n^{3/4})$	$O(k \log n)$
BMMH	$O(\log^*(k)\, d_{\mathrm{m}})$	k

2 RELATED WORK

A large body of work has been devoted to designing methods for optimizing certain graph properties via edge additions, edge rewirings, or other graph-edit operations. These approaches include methods for increasing graph robustness [10], maximizing the centrality of a group of nodes [37], improving the betweenness centrality of a node [8], reducing the average shortest path distances over all pairs of nodes [39, 45], minimizing the diameter of the graph [1, 14], increasing resilience to adversarial attacks [35], and more. Some of these approaches provide algorithms with provable approximation guarantees, while the others are mostly practical heuristics.

The setting of the problem we consider has also been studied in the context of understanding phenomena of bias, polarization, and segregation in social networks [6, 11, 12, 17, 21, 23, 40, 48], and developing computational methods to mitigate those adverse effects [2, 19, 20, 22, 36, 42, 50–52]. Most of the works listed above seek to optimize a complex objective related to some dynamical diffusion process, such as information cascades [28], or opinion dynamics [18]. In all cases, the optimization functions studied in previous work are significantly different from the hitting-time objectives considered in this paper.

As discussed in the introduction, the work that is most closely related to ours, is the paper by Haddadan et al. [24], which seeks to reduce the structural bias between two polarized groups of a network via edge insertions. Their approach is based on hitting time for bounded-length random walks, but contrary to our proposal, they seek to *maximize the reduction* of hitting time caused by edge insertions, rather than *directly minimizing the hitting time*. Similar ideas have been proposed by other authors, e.g., Fabbri et al. [15] and more recenty Coupette et al. [13], in the context of reducing exposure to harmful content. We consider our work to be a compelling extension of such previous ideas, not only due to optimizing a more natural objective, but also for coping with a more challenging optimization problem, for which our solution offers new technical ideas and insights.

3 PRELIMINARIES

Graphs. We consider undirected connected graphs $G = (V, E)$ with $|V| = n$ nodes, where the nodes V are bipartitioned into two groups R and B. These groups as referred to as the *red* and *blue* nodes, respectively. We only consider bipartitions $V = \{R, B\}$ that are *valid*, meaning that R and B are disjoint and non-empty. For a non-valid bipartition, our problem statements (Section 4.1) are either trivial or ill-defined. An edge is an *inter-group* edge if it has one blue and one red endpoint. For a subset of nodes $X \subseteq V$, we let $E[X]$ be the subset

of edges of G that have both endpoints in X, and $G[X] = (X, E[X])$ be the subgraph of G *induced* by X. If F is a set of non-edges of $G = (V, E)$, then $G + F$ denotes the graph $(V, E \cup F)$. The degree of a node v in G, which is the number of neighbors of v in G, is denoted as $d_G(v)$. We let d_R be the average of the degrees of red nodes.

Random walks. Every random walk will be a uniform simple random walk, unless stated otherwise. By uniform simple random walk we mean that at each step one of the neighbors of the current node is selected with uniform probability. Given a graph $G = (V, E)$ and $A \subseteq V$, the random variable $\tau_u(A)$ indicates the first time that a random walk on G starting from u visits A. If $u \in A$, we set $\tau_u(A) = 0$. The expectation of this random variable is denoted as $H_G(u, A) = \mathbb{E}_G[\tau_u(A)]$, where the subscript emphasizes that the random walk is on G. With a slight abuse of notation, $H_G(u, v)$ is thus the (expected) *hitting time* for a walk starting from u to visit v for the first time. Note that $H_G(u, u) = 0$ and $H_G(u, A) = 0$ if and only if $u \in A$. It also holds that $H_G(u, A) \leq H_G(u, B)$ when $B \subseteq A$, and in particular $H_G(u, A) \leq \min_{a \in A} H_G(u, a)$.

Matrices and norms. We denote $\mathbf{A}$ as the adjacency matrix of a graph, and $\mathbf{D}$ is the degree diagonal matrix. Let $e_r \in \mathbb{R}^{r \times 1}$ be the vector of all-ones of size r, and let $\|\cdot\|_2$ denote the spectral norm.

Additional definitions. For a graph $G = (V, E)$ with a valid bipartition $V = \{R, B\}$, let $(R \times B) \setminus E$ be the set of inter-group non-edges in G. Define the following two set functions $f, g : 2^{(R \times B) \setminus E} \to \mathbb{R}_{>0}$ as the *maximum* and *average hitting time.*

Definition 1 (Maximum and average hitting time). *For a set of inter-group non-edges $F \subseteq (R \times B) \setminus E$, we define the* maximum hitting time from R to B on graph $G + F$ *by*

$$f(F) \triangleq \max_{r \in R} H_{G+F}(r, B),$$

and the average hitting time from R to B on graph $G + F$ *by*

$$g(F) \triangleq \frac{1}{|R|} \sum_{r \in R} H_{G+F}(r, B).$$

4 PROBLEMS AND OBSERVATIONS

In this section, we formally define the problems we study, and we discuss their properties.

4.1 Problems

We introduce and study the following two problems, which we denote as the BMMH (Budgeted Minimum Maximum Hitting time) and BMAH (Budgeted Minimum Average Hitting time) problems.

Problem 1 (Budgeted Minimum Maximum Hitting time problem (BMMH)). *Given an undirected connected graph $G = (V, E)$, with a valid bipartition $V = \{R, B\}$, and a budget $k \in \mathbb{N}$, we seek to find a set of inter-group non-edges $F \subseteq (R \times B) \setminus E$ with $|F| \leq k$ that minimizes $f(F)$, where f is the maximum hitting-time function defined in Definition 1.*

Problem 2 (Budgeted Minimum Average Hitting time problem (BMAH)). *Identical definition to Problem 1, but we aim to minimize the function g, instead of f, where g is the average hitting-time function defined in Definition 1.*

BMMH aims to augment G with k new inter-group edges, so as to minimize the *largest* hitting time to a blue node, for random walks starting from red nodes. Likewise, BMAH aims to minimize the *average* hitting time from red to blue nodes. Note that for every red node we allow multiple new edges to be incident to it, as long as the augmented graph remains simple.

4.2 Observations

The **NP**-hardness of both BMMH and BMAH follows from a reduction from the minimum set cover problem, very similar to the reduction detailed by Haddadan et al. [24] and omitted for brevity. We present several useful observations regarding BMMH and BMAH that will guide the design of approximation algorithms:

Observation 1. *Changing the blue endpoints of the edges in a feasible solution F does not change $f(F)$ or $g(F)$.*

Proof. Every walk starts from a red node and halts the moment a blue node is hit, hence the identity of the blue endpoints of edges in F do not matter. □

Both problems thus reduce to selecting a multiset of red endpoints, as we allow multiple edges in F to have the same red endpoint. If a red node is connected to all the blue nodes already, we can of course not add any more new edges incident to it.

Observation 2 (monotonicity). *If $Y \subseteq X$, then $f(X) \leq f(Y)$ and $g(X) \leq g(Y)$.*

Proof. Let F be a feasible solution. For every $r \in R$, the hitting time $H_{G+F}(r, B)$ is a monotonically decreasing set function of F, since the edges in F are inter-group edges. Every $e \in F$ only helps in reducing the hitting time from r to B. This can be proven more formally by a straightforward coupling argument between a walker on G and $G + F$ [24, 34]. Since individual hitting times are monotone, the functions f and g are monotone, as well. □

The following supermodularity property for individual hitting times is less obvious, and a variant of it was proven by Haddadan et al. [24] for bounded-length random walks on directed graphs. We give a different and shorter proof of supermodularity in our case of unbounded random walks, by using a coupling argument.

Observation 3 (supermodularity of hitting times). *For all $r \in R$ and for all $e_1 \triangleq e_2 \in (R \times B) \setminus E$ it holds that*

$$H_{G+e_2}(r, B) - H_{G+\{e_1, e_2\}}(r, B) \leq H_G(r, B) - H_{G+e_1}(r, B). \quad (1)$$

Proof. Pick any $r \in R$. We assume all walks start from r. Consider the right-hand side of Eq. (1). Couple walks on $G + e_1$ to walks on G by simply following the walker on G, until the walker hits the red endpoint $r_1 \in R$ of the edge e_1. If the walker hits r_1, follow the edge e_1 with probability $\frac{1}{d_G(r_1)+1}$. With probability $\frac{d_G(r_1)}{d_G(r_1)+1}$, keep following the walker on G. It is clear that the marginal distribution of this coupled walk is a uniform random walk on $G + e_1$.

As long as e_1 is not traversed, the two coupled walks are identical. If e_1 is traversed, the walks are identical up until the time prior to traversing e_1, when they are both in r_1 for the last time. On this moment, the walker on G still needs to travel $H_G(r_1, B)$ steps on average to hit B, while the walker on $G + e_1$ needs 1 more step

Algorithm 1 [Greedy] bicrit. $(1+\epsilon)$-approximation for BMAH.

Input: $G = (R \cup B, E)$, parameter $k \geq 1$ and error $\epsilon > 0$.

1: $F_0 \leftarrow \emptyset$
2: **while** $(i \leftarrow 1; i \leq \lceil k \ln(\frac{n^3}{\epsilon}) \rceil; i \leftarrow i+1)$ **do**
3: $\quad F_i \leftarrow F_{i-1} \cup \arg\min_{e \in (R \times B) \setminus (E \cup F_{i-1})} g(F_{i-1} \cup \{e\})$.

Output: F_i.

(conditioned on e_1 being traversed). Hence for all $r \in R$ we have

$$H_G(r,B) - H_{G+e_1}(r,B)$$
$$= (H_G(r_1,B) - 1)\Pr[\text{traversing } e_1 \text{ when walking on } G + e_1].$$

Similarly, we can write for the left-hand side of Eq. (1)

$$H_{G+e_2}(r,B) - H_{G+\{e_1,e_2\}}(r,B)$$
$$= (H_{G+e_2}(r_1,B) - 1)\Pr[\text{traversing } e_1 \text{ when walking on } G + \{e_1,e_2\}].$$

The result follows from Observation 2, since both $H_{G+e_2}(r_1,B) \leq H_G(r_1,B)$ and

$$\Pr[\text{traversing } e_1 \text{ when walking on } G + \{e_1,e_2\}]$$
$$\leq \Pr[\text{traversing } e_1 \text{ when walking on } G + e_1].$$

□

Observation 3 states that individual hitting times are supermodular, hence the objective function g of BMAH is also supermodular. However, the objective function f of BMMH is not guaranteed to be supermodular, as a result of the max operator. Observation 4 shows that even the weaker notion of weakly-α supermodularity [33] does not hold for f.

Observation 4. *The function g is supermodular, but the function f is not weakly-supermodular for any $\alpha \geq 1$.*

Proof. The supermodularity of g is direct, as it is an average of supermodular functions (Observation 3). We prove the second statement of the observation. A non-negative non-increasing set function γ on $[n]$ is said to be weakly-α supermodular with $\alpha \geq 1$ if and only if for all $S, T \subseteq [n] : \gamma(S) - \gamma(S \cup T) \leq \alpha \sum_{i \in T \setminus S} (\gamma(S) - \gamma(S \cup \{i\}))$ [33, Definition 2]. Consider the following instance for BMMH. The input graph is a path of five nodes. All the nodes are red, except the middle node is blue. Let $S = \emptyset$ and T be the two edges that connect both red endpoints to the blue center. Then, $f(S) - f(S \cup T) > 0$, but for all $i \in T \setminus S : f(S) - f(S \cup \{i\}) = 0$. □

5 ALGORITHMS FOR MINIMIZING AVERAGE HITTING TIME

We first discuss our algorithms for the BMAH problem, as we can use them as a building block for the BMMH problem.

5.1 A Greedy bicriteria algorithm

According to Observation 4 the objective function g of BMAH is supermodular. It also holds that $g \geq 1$, as it takes at least one step to hit a blue node starting from any red node. There are two main ways for minimizing a non-negative monotone-decreasing supermodular function in the literature. One approach gives bounds that depend on the *total curvature* [49] of the objective function. For example, the work of Sviridenko et al. [49, Section 6.2] details a randomized algorithm with guarantee $1 + \frac{c}{(1-c)e} + \frac{O(\epsilon)}{1-c}$, for any $\epsilon > 0$ and curvature $c \in [0,1)$. However, we do not utilize this algorithm since it is not straightforward how to bound the curvature of g, nor how to practically implement their algorithm.

Instead, we follow the bicriteria approach of Liberty and Sviridenko [32], where they show that the classic greedy algorithm gives a bicriteria $(1+\epsilon)$-approximation by allowing the addition of more than k elements. Their results are applicable if either the objective function is bounded away from zero (which holds in our case, since $g \geq 1$), or one has an approximate initial solution. An immediate consequence of their results is Lemma 1.

Lemma 1. *The* Greedy *strategy (Algorithm 1) returns a $(1+\epsilon)$-approximation to an optimal solution of BMAH (that adds at most k edges) by adding at most $\lceil k \ln(\frac{n^3}{\epsilon}) \rceil$ edges.*

Proof. We may assume $k \geq 1$. Let F^* denote an optimal solution to BMAH with at most k edges. We always have a crude approximate initial solution $F_0 = \emptyset$, since for any F^* it holds that $g(\emptyset) \leq n^3 \leq n^3 g(F^*)$. The first inequality holds because hitting times in connected graphs are upper bounded by n^3 [34]. The second inequality follows from $g \geq 1$. The lemma essentially follows by application of the results of Liberty and Sviridenko [32, Theorem 6]. We give a short proof here for completeness. Let F_τ be the output of Algorithm 1, where $\tau = \lceil k \ln(\frac{n^3}{\epsilon}) \rceil$. By using the supermodularity and monotonicity of g, we get the following recursion:

$$g(F_\tau) - g(F^*) \leq (g(\emptyset) - g(F^*))\,(1 - 1/k)^\tau$$
$$\leq (n^3 - 1) g(F^*)\,(1 - 1/k)^{k \ln(n^3/\epsilon)}$$
$$\leq (n^3 - 1) g(F^*) e^{-\ln(n^3/\epsilon)} \leq \epsilon g(F^*).$$

□

Note that finding the new edge that minimizes line 3 in Algorithm 1 does not require a search over $O(n^2)$ possible non-edges. By Observation 1 we only need to choose a red endpoint, reducing the search to $O(n)$ possible choices in each iteration.

5.2 Greedy+: Speeding up Greedy

In practice, lazy evaluation [41] will typically reduce the number of function evaluations that Algorithm 1 makes to g. Nonetheless, function evaluations are still expensive. They require either solving a linear system of equations with $O(n)$ variables, or directly computing the fundamental matrix $(\mathbf{I} - \mathbf{Q})^{-1}$ of an absorbing Markov chain with absorbing states B, where $\mathbf{Q}$ denotes the probability transition matrix between the transient red nodes [27].

5.2.1 $(1 \pm \epsilon)$-estimation of g. We show how to get a fast estimate $\hat{g}$ that satisfies w.h.p. $\hat{g}(\cdot) \in (1 \pm \epsilon) g(\cdot)$, for some error $\epsilon > 0$.

The high-level idea is to write g as a certain quadratic form of the fundamental matrix of an absorbing Markov chain with absorbing states being the blue nodes. The fundamental matrix of an absorbing Markov chain can be written as an infinite matrix power series [27]. We will estimate this infinite series by a truncated sum. The truncated sum can be estimated efficiently by simulating random walks of bounded length, while the error induced by the truncation can also be shown to be small. The point of truncation will depend on certain graph parameters, such as the mixing time

and average degree of the red nodes. Since most real-life graphs typically have bounded mixing time and bounded average degree, our approach is expected to work well in practice. This idea of truncating a matrix power series has been recently used by Peng et al. [46] and prior to that by Andoni et al. [3].

The following holds for any connected graph, so without loss of generality we prove our estimates for $g(\emptyset)$. Let $r = |R|$ throughout this proof. Let $h_R \in \mathbb{R}^{r\times 1}$ denote the vector of expected hitting times from the red nodes until absorption by the blue nodes. Let $\mathbf{Q} \in \mathbb{R}^{r\times r}$ be the transition matrix between red nodes, obtained by deleting the rows and columns of corresponding blue nodes in the random-walk transition matrix $\mathbf{P} = \mathbf{D}^{-1}\mathbf{A}$. It is well-known [27] that $h_R = (\mathbf{I}_r - \mathbf{Q})^{-1}e_r$. Since $g(\emptyset)$ is the average of h_R, we can write

$$g(\emptyset) = \frac{1}{r}e_r^T(\mathbf{I}_r - \mathbf{Q})^{-1}e_r = \frac{1}{r}e_r^T\sum_{i=0}^{+\infty}\mathbf{Q}^i e_r,$$

where the second equality follows from $\mathbf{Q}^i \to \mathbf{0}$, as $i \to +\infty$, since $|B| \geq 1$ by our assumption of valid bipartition [27, Theorem 1.11.1].

Now define $p_1 \triangleq \frac{1}{r}e_r^T\sum_{i=0}^{\ell}\mathbf{Q}^i e_r$ and $p_2 \triangleq \frac{1}{r}e_r^T\sum_{i=\ell+1}^{+\infty}\mathbf{Q}^i e_r$. We first show how to estimate p_1.

Lemma 2. *Let $\epsilon, \delta \in (0,1)$. For every $u \in R$, run $t \geq \frac{\ell^2}{\epsilon^2}\ln(\frac{2n}{\delta})$ absorbing random walks (B are the absorbing nodes) with bounded length ℓ and record how many steps are taken. Take the empirical average over these t trials, for every u, and stack the results in a vector $\hat{h}$. With probability at least $1-\delta$, $\hat{h} \in (1\pm\epsilon)\sum_{i=0}^{\ell}\mathbf{Q}^i e_r$. Hence, also with probability at least $1-\delta$, it holds that $\frac{1}{r}e_r^T\hat{h} \in (1\pm\epsilon)p_1$.*

Proof. We can give a probabilistic interpretation to p_1. We argue that it is the average of the hitting times—over all the red nodes as starting points—for absorbing random walks with bounded length ℓ. In other words, the walk halts when either it has taken more than ℓ steps, or it hits a blue node, whichever comes first. Consider such a bounded absorbing walk starting from $u \in R$. Let n_{uv} be the random variable counting the number of times the walk visits red node $v \in R$. Let n_{uv}^i be 1 if this walk is in state v after exactly i steps, and 0 otherwise. Clearly $n_{uv} = \sum_{i=0}^{\ell} n_{uv}^i$. So the expectation satisfies

$$\mathbb{E}[n_{uv}] = \sum_{i=0}^{\ell}\mathbb{E}[n_{uv}^i] = \sum_{i=0}^{\ell}(1\cdot\mathbf{Q}_{uv}^i + 0\cdot(1-\mathbf{Q}_{uv}^i)) = \sum_{i=0}^{\ell}\mathbf{Q}_{uv}^i.$$

Thus $\sum_{i=0}^{\ell}\mathbf{Q}^i e_r$ is a vector where the u-th entry is the hitting time of an absorbing (with absorbing states B) bounded (with length ℓ) random walk starting from red node u. And p_1 is the average of this vector. To find a $(1\pm\epsilon)$-estimation of p_1 it suffices to find a $(1\pm\epsilon)$-estimation of each entry of $\sum_{i=0}^{\ell}\mathbf{Q}^i e_r$. Now it is straightforward to estimate every u-th entry of $\sum_{i=0}^{\ell}\mathbf{Q}^i e_r$ within a factor of $(1\pm\epsilon)$. Run t walks from u and compute the empirical average of their lengths as the estimator. Such an estimation is exactly the topic of the RePBubLik paper [24, Lemma 4.3], and the claim in Lemma 2 follows easily from Hoeffding's inequality and the union bound. □

Next we show how to bound p_2 as a small fraction of $g(\emptyset)$, for a sufficiently large choice of ℓ.

Lemma 3. *If $\ell \geq \frac{\log(d_R/\epsilon(1-\lambda))}{\log(1/\lambda)} - 1$, then $p_2 \leq \epsilon g(\emptyset)$.*

Proof. Let $\mathbf{D}_R \in \mathbb{R}^{r\times r}$ be a diagonal matrix with diagonal entries being the degrees of the red nodes in G. All entries $d_1,\ldots,d_r$ are at least one, since G is assumed to be connected. Recall that $d_R = \frac{d_1+\ldots+d_r}{r}$ is the average of these entries.

Let $\mathbf{A}_R \in \mathbb{R}^{r\times r}$ be the adjacency matrix of $G[R]$. Note that $\mathbf{Q} = \mathbf{D}_R^{-1}\mathbf{A}_R$. Observe that the matrix $\mathbf{M} = \mathbf{D}_R^{-1/2}\mathbf{A}_R\mathbf{D}_R^{-1/2} = \mathbf{D}_R^{1/2}\mathbf{Q}\mathbf{D}_R^{-1/2}$ is real-valued, symmetric, and similar to $\mathbf{Q}$. So the real-valued eigenvalues $\lambda_1 \geq \lambda_2 \geq \ldots \geq \lambda_r$ of $\mathbf{M}$ are also the eigenvalues of $\mathbf{Q}$ (by similarity) and $\mathbf{M}$ has a orthonormal eigendecomposition $\mathbf{M} = \mathbf{U}\Lambda\mathbf{U}^T$. Let $\lambda = \max\{|\lambda_1|, |\lambda_r|\}$ be the spectral radius of $\mathbf{M}$ (and of $\mathbf{Q}$). By symmetry of $\mathbf{M}$, we have that $\|\mathbf{M}\|_2 = \lambda$. We know $\lambda < 1$, since $\mathbf{Q}$ is a *weakly-chained substochastic matrix*,[1] [5, Corollary 2.6]. Hence,

$$\begin{aligned}
p_2 = \|p_2\|_2 &\leq \frac{1}{r}\left\|\mathbf{D}_R^{-1/2}e_r\right\|_2\sum_{i=\ell+1}^{+\infty}\left\|\mathbf{M}^i\right\|_2\left\|\mathbf{D}_R^{1/2}e_r\right\|_2 \\
&\leq \frac{1}{r}\left\|\mathbf{D}_R^{-1/2}e_r\right\|_2\sum_{i=\ell+1}^{+\infty}\|\mathbf{M}\|_2^i\left\|\mathbf{D}_R^{1/2}e_r\right\|_2 \\
&\leq \frac{\lambda^{\ell+1}}{r(1-\lambda)}\sqrt{(d_1+\ldots+d_r)\left(\frac{1}{d_1}+\ldots+\frac{1}{d_r}\right)} \\
&\leq \frac{\lambda^{\ell+1}}{(1-\lambda)}d_R \leq \epsilon \leq \epsilon g(\emptyset).
\end{aligned}$$

The first and second inequality use the similarity between $\mathbf{Q}$ and $\mathbf{M}$, the orthonormal eigendecomposition of $\mathbf{M}$, and matrix norm properties. The third inequality follows from $\lambda < 1$. The fourth inequality holds as $\sum_i 1/d_i \leq \sum_i d_i$, as $d_i \geq 1$, and by plugging in the choice of $\ell \geq \frac{\log(d_R/\epsilon(1-\lambda))}{\log(1/\lambda)} - 1$. The final step follows from $g(\emptyset) \geq 1$. □

We combine Lemma 2 and Lemma 3 to obtain our final result.

Lemma 4. *Given an undirected connected graph $G = (V,E)$, with a valid bipartition $V = \{R,B\}$, we can find an estimate $\hat{g}(\emptyset) \in (1\pm\epsilon)g(\emptyset)$ with probability at least $1 - 1/n^c$ (for some $c \geq 1$) in time*

$$O\left(\frac{c\,n\,\ln(n)\,\log^3(d_R/\epsilon(1-\lambda))}{\epsilon^2\,\log^3(1/\lambda)}\right). \tag{2}$$

Proof. We use the estimate $\hat{g} = \frac{1}{r}e_r^T\hat{h}$ from Lemma 2 as our estimate of $g(\emptyset)$ with parameter $\epsilon' = \epsilon/2$. Recall that $g(\emptyset) = p_1 + p_2$. Since $\hat{g}$ is an estimation of p_1 by Lemma 2, it holds (w.h.p.) that $\hat{g} \leq (1+\epsilon/2)p_1 \leq (1+\epsilon)g(\emptyset)$ since $p_2 \geq 0$. On the other hand, $\hat{g} \geq (1-\epsilon/2)p_1 = (1-\epsilon/2)(g(\emptyset) - p_2) \geq (1-\epsilon/2)(1-\epsilon/2)g(\emptyset) \geq (1-\epsilon)g(\emptyset)$, where the second to last inequality uses Lemma 3.

The time complexity follows from Lemma 2 by plugging in $\delta = 1/n^c$ and the choice of ℓ from Lemma 3. We need to do this for every red node and there could be $O(n)$ of them. Every bounded random walk also needs at most ℓ steps. □

5.2.2 Greedy on the estimated values. Lemma 4 states that we can find good estimates $\hat{g}(\cdot) \in (1\pm\epsilon)g(\cdot)$ relatively fast and with good probability. A natural question to ask is whether running the greedy bicriteria approach from Algorithm 1 on $\hat{g}$ (instead of g) still ensures some form of approximation guarantee, given

[1] This is the case when for all i: either the i-th row of $\mathbf{Q}$ sums to a value < 1, or from i it is always possible to reach another j for which every row sums to a value < 1.

that $\hat{g}$ is not guaranteed to be supermodular. A similar question has been answered affirmatively for the classic greedy algorithm of Nemhauser et al. [43] for maximizing a nonnegative monotone submodular function under a cardinality constraint, by the work of Horel and Singer [25]. They showed that when $\epsilon < 1/k$, the greedy still gives a constant-factor approximation [25, Theorem 5].

We show an analogue result for the bicriteria greedy approach [33] to minimization of our monotone decreasing nonnegative supermodular objective function g, under a cardinality constraint.

Theorem 1. *Algorithm 2 returns a set F_τ that satisfies*

$$g(F_\tau) \leq (2+\epsilon)\mathrm{OPT}, \tag{3}$$

where $\mathrm{OPT} = g(O)$ *and O is an optimal solution of BMAH with at most k edges.*

Proof. By non-negativity, supermodularity and monotonicty of g, we can write for iteration i in Algorithm 2

$$\begin{aligned} g(F_i) - \mathrm{OPT} &\leq g(F_i) - g(F_i \cup O) \leq \sum_{e\in O} g(F_i) - g(F_i \cup \{e\}) \\ &\leq \sum_{e\in O} g(F_i) - \frac{1}{1+\epsilon}\hat{g}(F_i \cup \{e\}) \leq \sum_{e\in O} g(F_i) - \frac{1}{1+\epsilon}\hat{g}(F_{i+1}) \\ &\leq \sum_{e\in O} g(F_i) - \frac{1-\epsilon}{1+\epsilon} g(F_{i+1}) \leq k\left(g(F_i) - \frac{1-\epsilon}{1+\epsilon} g(F_{i+1})\right). \end{aligned}$$

In the third and fifth inequality we made use of $\hat{g}(\cdot) \in (1 \pm \epsilon) g(\cdot)$, and the fourth inequality utilizes the greedy choice in Algorithm 2. So we have the following recursion

$$g(F_{i+1}) \leq \left(\frac{1+\epsilon}{1-\epsilon}\right)(1-1/k) g(F_i) + \left(\frac{1+\epsilon}{1-\epsilon}\right)\frac{\mathrm{OPT}}{k}. \tag{4}$$

This recursion is of the form of $u_{i+1} \leq a u_i + b$, which after τ iterations satisfies $u_\tau \leq a^\tau u_0 + \frac{1-a^\tau}{1-a} b$. Now write $\epsilon = \delta/k$ for $\delta \in (0, 1/4]$. With our specific choice of $a = \frac{1+\epsilon}{1-\epsilon}(1-1/k)$ and $\tau = \lceil 2k \ln(\frac{n^3}{\epsilon}) \rceil$ we upper bound $a^\tau u_0 = a^\tau g(\emptyset)$ as

$$\begin{aligned} a^\tau g(\emptyset) &= \left(\left(\frac{k+\delta}{k-\delta}\right)(1-1/k)\right)^\tau g(\emptyset) \leq e^{(2\delta-1)2\ln(n^3/\epsilon)} g(\emptyset) \\ &\leq e^{-\ln(n^3/\epsilon)} g(\emptyset) = \frac{\epsilon}{n^3} g(\emptyset) \leq \epsilon \mathrm{OPT}. \end{aligned}$$

where the first inequality follows from Fact 1 and the last inequality from the discussion in the proof of Lemma 1.

Fact 1. *Let $x \in \mathbb{R}$ with $|x| \leq 1$. Let $k \geq 1$ be a positive integer. Then it holds that*

$$\left(\frac{k+x}{k-x}\left(1-\frac{1}{k}\right)\right)^k \leq e^{2x-1}. \tag{5}$$

Next we upper bound the second part $\frac{1-a^\tau}{1-a} b$. It can be verified that $0 \leq a < 1$ for parameters $\epsilon = \delta/k$, $\delta \leq 1/4$ and $k \geq 1$. Thus,

$$\frac{1-a^\tau}{1-a} b \leq \frac{1}{1-a} b = \frac{1+\epsilon}{1-2\epsilon k+\epsilon}\,\mathrm{OPT} \leq \frac{1}{1-2\delta}\,\mathrm{OPT} \leq 2\,\mathrm{OPT}.$$

□

Algorithm 2 [Greedy+] bicrit. $(2+\epsilon)$-approximation for BMAH.

Input: $G = (R \cup B, E)$, parameter $k \geq 1$, error $\epsilon \in (0, \frac{1}{4k}]$ and estimates $\hat{g}(\cdot) \in (1 \pm \epsilon) g(\cdot)$ as found by Lemma 4.

$F_0 \leftarrow \emptyset$

while $(i \leftarrow 1; i \leq \lceil 2k \ln(\frac{n^3}{\epsilon}) \rceil; i \leftarrow i+1)$ **do**

  $F_i \leftarrow F_{i-1} \cup \arg\min_{e \in (R\times B)\setminus(E \cup F_{i-1})} \hat{g}(F_{i-1} \cup \{e\})$.

Output: F_i.

6 ALGORITHMS FOR MINIMIZING MAXIMUM HITTING TIME

Next we turn our attention to the BMMH problem. We first show that a α-approximation for BMAH results in an $(2|R|^{3/4}\alpha)$-approximation for BMMH. We then give a completely different approach based on the asymmetric k-center problem [26].

6.1 Relating the average and the maximum hitting times

The BMMH objective function f is not supermodular (Observation 4), although we know that for every $r \in R$ each individual hitting time $H_{G+F}(r, B)$ is a supermodular and monotonically decreasing set function of F (Observation 2 and Observation 3).

We exploit this idea, by upper bounding f by an expression that involves g (Theorem 2). Since also $g \leq f$, we can use any approximation algorithm for minimizing g (for instance, the methods in Sections 5.1 and 5.2) and use the upper bound to derive an approximation guarantee for BMMH.

Theorem 2 relates the average and maximum hitting time from red to blue nodes. We credit Yuval Peres [47] for the proof of Theorem 2, in case the set B is a singleton, and extend the proof here when B contains more than one node. Theorem 2 gives a remarkable non-trivial $O(n^{3/4})$ upper bound on the ratio of the maximum and average hitting times between two groups that constitute a bipartition of the graph. Theorem 3 shows that there exists graphs for which this is tight. In contrast, if we disregard the bipartition, and take the average and the maximum hitting time over *all node-pairs* instead, then it is known that this ratio can grow as bad as $O(n)$, but not worse [16].

Theorem 2. *Given an undirected connected graph G with a valid bipartation $V = \{R, B\}$. Let $F \subseteq (R \times B) \setminus E$ be a set of non-edges. Then,*

$$g(F) \leq f(F) \leq 2|R|^{3/4} g(F). \tag{6}$$

Proof. It clearly suffices to prove this for $F = \emptyset$. Since G is connected, the hitting time between any two nodes is finite-valued. The first inequality is immediate since the maximum is not less than the average. Consider the second inequality. Let $\alpha > 0$ be some real number, which we will later choose carefully. Categorize the nodes into two groups, the ones with a "high" hitting time and the ones with a "low" hitting time to the blue nodes. More concretely, define

$$S = \{v \in V : H_G(v, B) \leq \alpha g(\emptyset)\},$$

and S^c as the complement of this set. Since

$$|R| g(\emptyset) = \sum_{r\in R} H_G(r, B) \geq \sum_{r \in S^c} H_G(r, B) > |S^c| \alpha g(\emptyset),$$

it follows that $|S^c| < |R|/\alpha$. The first inequality holds because $S^c \subseteq R$ (since $B \subseteq S$), and the second inequality uses the definition of S^c.

Now consider the graph G^* with $|S^c| + 1$ nodes, obtained by the following contraction in G: All nodes in S are contracted to a single supernode s. If $w \in S^c$ has several edges connecting it to S in G, we only keep one of these edges. This edge connects w to s in G^*. This contraction process has the property that for all $v \in S^c$ it holds $H_G(v,S) \le H_{G^*}(v,s)$, which is easily proved by a straightforward coupling argument. Now we can bound $H_{G^*}(v,s)$. Since G^* is connected, contains no multi-edges, and has $|S^c| + 1$ nodes, it follows that $H_{G^*}(v,s) \le |S^c|^3$ by a well-known result (see, for instance, Lawler [29], and Brightwell and Winkler [9]). By the strong Markov property [31, Appendix A.3], it holds for all $v \in V$:

$$\begin{aligned} H_G(v,B) &\le H_G(v,S) + \max_{u\in S} H_G(u,B) \\ &\le |S^c|^3 + \alpha g(\emptyset) < \left((|R|/\alpha)^3 + \alpha\right) g(\emptyset), \end{aligned} \tag{7}$$

where the last inequality holds because $|S^c| < |R|/\alpha$ and $g(\emptyset) \ge 1$. Now Eq. (7) is minimized for the choice $\alpha = |R|^{3/4}$, and hence $f(\emptyset) \le 2|R|^{3/4} g(\emptyset)$. □

To see the tightness of the upper bound in Theorem 2, consider the following family of graphs.

Theorem 3. *There exists graphs for which the ratio $f(\emptyset)/g(\emptyset) \in \Omega(n^{3/4})$.*

Proof. Consider the following star-path-clique configuration. We have a star consisting of n nodes, with center node x. Attach to x a $(n^{1/4}, n^{1/4})$-lollipop graph (see Brightwell and Winkler [9]). More precisely, attach to x a path of $n^{1/4}$ nodes and to the other endpoint of this path we attach a clique of also $n^{1/4}$ nodes. Set $B = \{x\}$ and $R = V \setminus B$. Clearly $f(\emptyset) \in \Theta(n^{3/4})$ since a random walk starting from a clique node needs a cubic number of expected steps with respect to the size of the lollipop to reach x [9]. On the other hand,

$$g(\emptyset) \in O\left(\frac{n \cdot 1 + n^{1/4} \cdot n^{3/4}}{n}\right) = O(1),$$

and the claim follows. □

Putting the pieces together, we get the following result.

Corollary 1. *An α-approximation for BMAH is a $(2|R|^{3/4}\alpha)$-approximation for BMMH.*

Proof. Consider the output F of α-approximation algorithm for BMAH. Let O_g (resp. O_f) be an optimal solution of BMAH (resp. BMMH). Using Theorem 2 it holds that $f(F) \le 2|R|^{3/4} g(F) \le 2|R|^{3/4}\alpha g(O_g) \le 2|R|^{3/4}\alpha g(O_f) \le 2|R|^{3/4}\alpha f(O_f)$. □

6.2 An asymmetric k-center approach

Hitting times in both undirected and directed graphs form a quasi-metric, in the sense that they satisfy all requirements to be a metric except symmetry, since it generally holds that $H_G(u,v) \triangleq H_G(v,u)$. However, hitting times do satisfy the triangle inequality [34]. We will try to exploit this, by linking the BMMH problem to the following variant of the asymmetric k-center problem.

Problem 3. *[Asymmetric k-center Problem with one fixed center] Given a quasi-metric space (V,d) and a point $x \in V$. Find a set of k centers $F \subseteq V$, $|F| \le k$ that minimize*

$$\max_{v\in V} d(v, F \cup \{x\}),$$

where for a set $A \subseteq V$ it holds $d(v,A) = \min_{a\in A} d(v,a)$.

Problem 3 is a variant of the classic asymmetric k-center problem [26]. It is not hard to see that Problem 3 is as hard to approximate as the classic problem. Additionally, existing algorithms [4, 44] for the classic problem can easily be modified to solve Problem 3.

We show a relationship between BMMH and Problem 3. Let $\mathcal{I}_1$ be an instance to BMMH. Lemma 5 states that the optimum of Problem 3 on a well-chosen quasi-metric space $\mathcal{I}_2$ (which depends on the instance $\mathcal{I}_1$) is a lower bound for the optimum of BMMH on instance $\mathcal{I}_1$. We define this quasi-metric space in Definition 2.

Definition 2 (modified hitting time quasi-metric). *Let $G = (R \cup B, E)$ be an input graph to BMMH. Associate with B a single new point b, and consider function $d : (R \cup \{b\}) \times (R \cup \{b\}) \to \mathbb{R}_{\ge 0}$ defined as*

$$d(u,v) \triangleq \begin{cases} H_G(u,v) & \text{if } u,v \in R, \\ H_G(u,B) & \text{if } u \in R \text{ and } v = b, \\ \max_{i\in B} H_G(i,v) & \text{if } u = b \text{ and } v \in R, \\ 0 & \text{if } u = v = b. \end{cases} \tag{8}$$

Proposition 1. *$(R \cup \{b\}, d)$ is a quasi-metric space.*

Proof. We prove the triangle inequality $d(x,y) \le d(x,z) + d(z,y)$, as the identity axiom is satisfied. The triangle inequality holds when $b \triangleq x,y,z$, because of standard triangle inequality of hitting times. So consider the case where b is present in the inequality. There are three cases involving $u,v \in R$ and b:

- $d(u,v) \le d(u,b) + d(b,v)$ or equivalently $H_G(u,v) \le H_G(u,B) + \max_{i\in B} H_G(i,v)$. This holds because of the strong Markov property when visiting the set B.
- $d(u,b) \le d(u,v) + d(v,b)$ or equivalently $H_G(u,B) \le H_G(u,v) + H_G(v,B)$. This holds because of triangle inequality of hitting times.
- $d(b,u) \le d(b,v) + d(v,u)$ or equivalently $\max_{i\in B} H_G(i,u) \le \max_{j\in B} H_G(j,v) + H_G(v,u)$. Let $i^* = \arg\max_{i\in B} H_G(i,u)$. This holds because $H_G(i^*,u) \le H_G(i^*,v) + H_G(v,u) \le \max_{j\in B} H_G(j,v) + H_G(v,u)$.

□

Lemma 5. *Let M^* be the optimum of BMMH for an input graph G. Let C^* be the optimum of Problem 3 on the quasi-metric space defined in Definition 2 with $x = b$. Then, it holds that*

$$C^* \le M^*.$$

Proof. Consider the red endpoints $C = \{c_1, \ldots, c_k\} \subseteq R$ of a set of edges F to BMMH that achieves the optimum value M^*. Note that some endpoints might be the same, so C is a multiset. Let $G^* = G + F$ be the augmentation of G with F.

Create a feasible solution to Problem 3 by taking C as the set of chosen centers. We claim that for every $v \in R \cup \{b\}$ it holds that $d(v, C \cup \{b\}) \le M^*$. We have three cases. If $v = b$, the claim is trivially satisfied. If $v \triangleq b$ and $H_G(v,B) \le M^*$, the claim also holds,

Algorithm 3 [ASyMM] $O(\beta\, d_{\mathrm{m}})$-approximation for BMMH.

Input: $G = (R \cup B, E)$, parameter $k \geq 1$.

1: Run any β-approximation algorithm [4, 44] for asymmetric k-center variant (Problem 3) on the quasi-metric space defined in Definition 2, with fixed center $x = b$.

2: Let $C = \{c_1, \ldots, c_k\} \subseteq R$ be the output of step 1. Return a set F of at most k shortcut edges by connecting every c_i to some node in B.

as from Definition 2 we have $d(v, C \cup \{b\}) \leq d(v, b) = H_G(v, B) \leq M^*$. So the remaining case is when $v \triangleq b$ and $H_G(v, B) > M^*$. Since $H_{G^*}(v, B) \leq M^*$ (by definition of M^*) there must exist a non-empty set of endpoints $I \subseteq C$, such that for all $i \in I$:

$$\Pr[\text{walk starting from } v \text{ uses a new edge incident to } i] > 0,$$

when walking on the graph G^*. Let I be the largest such set. Using the law of total expectation, decompose[2] $H_{G^*}(v, B)$ as

$$\begin{aligned} H_{G^*}(v,B) &= H_{G^*}(v,B \mid \text{walk uses no new edges}) \\ &\quad \cdot \Pr[\text{walk uses no new edges}] \\ &\quad + \sum_{i \in I} H_{G^*}(v,B \mid \text{walk uses a new edge incident to } i) \\ &\quad \cdot \Pr[\text{walk uses a new edge incident to } i] \leq M^*. \end{aligned} \tag{9}$$

By our case assumption $H_G(v, B) > M^*$, it holds that

$$H_{G^*}(v,B \mid \text{walk uses no new edges}) = H_G(v,B) > M^*.$$

Thus the only way Eq. (9) can hold is when

$$\min_{i \in I} H_{G^*}(v,B \mid \text{walk uses a new edge incident to } i) \leq M^*.$$

Let i^* be a center $i \in I$ that attains the minimum. The result follows since

$$\begin{aligned} d(v, C \cup \{b\}) &\leq d(v,C) = \min_{c \in C} H_G(v,c) \leq H_G(v,i^*) \\ &\leq H_{G^*}(v,B \mid \text{walk uses a new edge incident to } i^*) \leq M^*. \end{aligned}$$

□

Lemma 5 allows us to analyze Algorithm 3. We write $d_{\mathrm{m}} = \max_{c \in C} d_G(c)$ for the maximum degree of the nodes in C, and β for the approximation guarantee of an asymmetric k-center algorithm. The state-of-the-art approximation for asymmetric k-center is $\beta = O(\log^*(k))$ by two different algorithms of Archer [4].

Theorem 4. *Algorithm 3 is an $O(\beta\, d_m)$-approximation for BMMH.*

Proof. By using a β-approximation for Problem 3 and using Lemma 5 we get that for all $r \in R$ it is $H_G(r, C) \leq \min_{c \in C} H_G(r, c) \leq \beta C^* \leq \beta M^*$. Consider a random walk that starts from any $r \in R$. In expectation we need at most βM^* steps to encounter a node $c \in C$. We know that every $c \in C$ is incident to *some* edge connecting it to B. This is either a new shortcut edge $e \in F$ from Algorithm 3, or if we cannot add a new shortcut edge incident to c, this implies that c is already connected to all the blue nodes. In either case, when the walk is at c, we have a chance of at least $\frac{1}{d_{\mathrm{m}}+1}$ to hit the blue nodes in the next step. On the other hand, if the walk does not follow such an edge, the walk needs at most another βM^* steps in expectation to reach C again, and the process repeats. Of course, the walk might have encountered a blue node along the way, but this is only beneficial. So we conclude that for all $r \in R$:

$$\begin{aligned} H_{G+F}(r,B) \leq (\beta M^* + 1)\frac{1}{d_{\mathrm{m}}+1} + (2\beta M^* + 1)\frac{d_{\mathrm{m}}}{d_{\mathrm{m}+1}}\frac{1}{d_{\mathrm{m}}+1} + \\ (3\beta M^* + 1)\left(\frac{d_{\mathrm{m}}}{d_{\mathrm{m}+1}}\right)^2 \frac{1}{d_{\mathrm{m}}+1} + \ldots = 1 + (d_{\mathrm{m}}+1)\beta M^*. \end{aligned}$$

The last step follows by splitting the sum as a geometric series and an arithmetico-geometric series, which states that $\sum_{i=1}^{\infty} ir^i = \frac{r}{(1-r)^2}$ for $0 < r < 1$. □

7 EXPERIMENTAL EVALUATION

Although our work is mostly theoretical, we will compare the performance of our most scalable algorithm Greedy+ with several baselines on real-life datasets. In particular, we want to measure how Greedy+ reduces the objective functions of BMAH and BMMH when we add an incrementally larger set of shortcut edges to the graph, and we want to verify if other existing algorithms are effective for these two tasks.

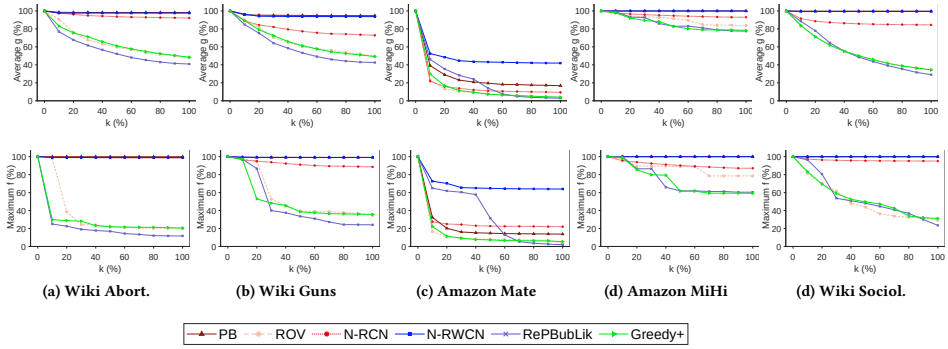
Baselines. We will compare with the fastest variant of the RePBubLik algorithm, which is the RePBubLik+ algorithm [24], as well as with three simplified variants of RePBubLik+. We set the *parochial* nodes [24] of the algorithm equal to R. The RePBubLik+ algorithm ranks the parochial nodes according to their random-walk centrality (only computed once), and includes a penalty factor that favours the insertion of new shortcut edges to red nodes that have not been shortcut before. The first variant, *PureRandom* [24] selects the endpoints of the new edges uniformly at random from R and B. The second variant is *Random Top-N Central Nodes (N-RCN)* [24], which sorts the top N red nodes in order of descending random-walk centrality and picks k source nodes uniformly from this set of N nodes. The third variant is *Random Top-N Weighted Central Nodes (N-RWCN)*, which accounts additionally accounts for the random-walk transition probabilities in the ordering [24]. Finally, we compare with the *ROV* algorithm [19]. ROV outputs k shortcut edges such that their addition aims to minimize the Random Walk Controversy (RWC) score [21] of the augmented graph. The RWC score tries to capture how well separated the two groups are with respect to a certain controversial topic. It considers candidate edges between high-degree vertices of both groups R and B. Then the top-k edges are picked with respect to the RWC score. All parameters of the aforementioned algorithms are set to their standard settings.

Experimental setup. All experiments are performed on an Intel core i5 machine at 1.8 GHz with 16 GB RAM. Our methods are implemented in Python 3.8 and we made publicly available.[3] We use the same datasets as RePBubLik [24], see Table 2.

Implementation of Greedy+. We set $\epsilon = \lambda = 0.1$ in our experiments. In order to draw a fair comparison with the other baselines, we run Greedy+ for k steps and not more. We also simplify the way we calculate the average g over all the red nodes in each iteration. Lemma 2 states that we need to run several walks starting from each $r \in R$ and then compute the empirical average, but we only do this

[2] In Eq. (9) we have assumed that Pr[walk uses no new edges] > 0. The analysis when this probability is zero is easier and follows the same reasoning.

[3] https://anonymous.4open.science/r/KDD-2023-Source-Code-0E2C/



(a) Wiki Abort. (b) Wiki Guns (c) Amazon Mate (d) Amazon MiHi (d) Wiki Sociol.

Figure 1: The y-axis shows the performance of the algorithms in reducing the functions g (first row) and f (second row) in terms of the number of newly added shortcut edges k. The x-axis shows the number of new edges k as a fraction of $|R|$.

Table 2: Datasets used in the experiments. For each network, we extracted the largest connected component. $|E|_{R\leftrightarrow B}$ denotes the number of inter-group edges, and $|E|_{tot}$ the total number of edges.

Data	$\lvert R\rvert$	$\lvert B\rvert$	$\lvert E\rvert_{R\leftrightarrow B}$	$\lvert E\rvert_{\text{tot}}$
Wiki Guns [38]	134	117	132	550
Wiki Abort. [38]	208	396	232	1585
Amazon Mate [30]	160	11	16	287
Amazon MiHi [30]	25	63	56	146
Wiki Sociol. [38]	648	2588	430	8745

for a randomly chosen subset of $|R|/10$ red nodes. This significantly sped up the process, without sacrificing too much in quality.

Results. Figure 1 shows the performance of the proposed algorithms and baselines. On the x-axis it shows the number of new shortcut edges k as a fraction of the total number of red nodes $|R|$. We observe that our algorithm `Greedy+` is competitive, performing very similar to `RePBubLik+`, for minimizing both objective functions f and g. Both these algorithms outperform the other baselines. The slightly better performance for `RePBubLik+` in some cases might be due to specific parameter settings, as well as the fact `RePBubLik+` is repeated 10 times whereas our algorithm only once. Intuitively `RePBubLik+` and `Greedy+` are in fact expected to perform very similar since they both are practical approximative versions (with different ways of estimating) of the same underlying greedy algorithm: picking the next edge that maximizes the `RePBubLik+` objective function is theoretically the same edge that `Greedy` (see Algorithm 1) would select. We note that the y-axis shows the *exact* function evaluations of f and g, which was done by solving a linear system with $O(n)$ variables (see Sect. 5.2), which was relatively time consuming and an important reason why we restricted ourselves to smaller datasets.

8 CONCLUSION

In this paper we studied the problem of minimizing average hitting time and maximum hitting time between two disparate groups in a network by adding new edges between pairs of nodes. In contrast to previous methods that modify the objective so that it becomes a submodular function and the optimization becomes straightforward, we minimize hitting time directly. Our approach leads to having a more natural objective at the cost of a more challenging optimization problem. For the two problems we define we present several observations and new ideas that lead to novel algorithms with provable approximation guarantees. For average hitting time we show that the objective is supermodular and we apply a known bicriteria greedy method; furthermore, we show how to efficiently approximate the computation of the greedy step by sampling bounded-length random walks. For maximum hitting time, we show that it relates to average hitting time, and thus, we can reuse the greedy method. In addition, we also demonstrate a connection with the asymmetric k-center problem.

ACKNOWLEDGMENTS

This research is supported by the Helsinki Institute for Information Technology HIIT, Academy of Finland projects AIDA (317085) and MLDB (325117), the ERC Advanced Grant REBOUND (834862), the EC H2020 RIA project SoBigData++ (871042), and the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

REFERENCES

[1] Florian Adriaens and Aristides Gionis. 2022. Diameter Minimization by Shortcutting with Degree Constraints. In *2022 IEEE International Conference on Data Mining (ICDM)*. 843–848. https://doi.org/10.1109/ICDM54844.2022.00095
[2] Victor Amelkin and Ambuj K Singh. 2019. Fighting opinion control in social networks via link recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 677–685.
[3] Alexandr Andoni, Robert Krauthgamer, and Yosef Pogrow. 2018. On solving linear systems in sublinear time. *arXiv preprint arXiv:1809.02995* (2018).

[4] Aaron Archer. 2001. Two $O(\log^* k)$-approximation algorithms for the asymmetric k-center problem. In *Integer Programming and Combinatorial Optimization: 8th International IPCO Conference Utrecht, The Netherlands, June 13–15, 2001 Proceedings 8*. Springer, 1–14.

[5] Parsiad Azimzadeh. 2018. A fast and stable test to check if a weakly diagonally dominant matrix is a nonsingular M-matrix. *Math. Comp.* 88, 316 (2018), 783–800.

[6] Eytan Bakshy, Solomon Messing, and Lada A Adamic. 2015. Exposure to ideologically diverse news and opinion on Facebook. *Science* 348, 6239 (2015), 1130–1132.

[7] Pablo Barberá. 2020. Social media, echo chambers, and political polarization. *Social media and democracy: The state of the field, prospects for reform* 34 (2020).

[8] Elisabetta Bergamini, Pierluigi Crescenzi, Gianlorenzo D'angelo, Henning Meyerhenke, Lorenzo Severini, and Yllka Velaj. 2018. Improving the betweenness centrality of a node by adding links. *Journal of Experimental Algorithmics (JEA)* 23 (2018), 1–32.

[9] Graham Brightwell and Peter Winkler. 1990. Maximum hitting time for random walks on graphs. *Random Structures & Algorithms* 1, 3 (1990), 263–276.

[10] Hau Chan and Leman Akoglu. 2016. Optimizing network robustness by edge rewiring: A general framework. *Data Mining and Knowledge Discovery* 30 (2016), 1395–1425.

[11] Xi Chen, Jefrey Lijffijt, and Tijl De Bie. 2018. Quantifying and minimizing risk of conflict in social networks. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1197–1205.

[12] Uthsav Chitra and Christopher Musco. 2019. Understanding filter bubbles and polarization in social networks. *arXiv preprint arXiv:1906.08772* (2019).

[13] Corinna Coupette, Stefan Neumann, and Aristides Gionis. 2023. Reducing Exposure to Harmful Content via Graph Rewiring. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*.

[14] Erik D Demaine and Morteza Zadimoghaddam. 2010. Minimizing the diameter of a network using shortcut edges. In *Algorithm Theory-SWAT 2010: 12th Scandinavian Symposium and Workshops on Algorithm Theory, Bergen, Norway, June 21-23, 2010. Proceedings 12*. Springer, 420–431.

[15] Francesco Fabbri, Yanhao Wang, Francesco Bonchi, Carlos Castillo, and Michael Mathioudakis. 2022. Rewiring what-to-watch-next recommendations to reduce radicalization pathways. In *Proceedings of the ACM Web Conference 2022*. 2719–2728.

[16] Yuval Filmus. 2015. https://cs.stackexchange.com/questions/42917/average-vs-worst-case-hitting-time. (2015).

[17] Seth Flaxman, Sharad Goel, and Justin M Rao. 2016. Filter bubbles, echo chambers, and online news consumption. *Public opinion quarterly* 80, S1 (2016), 298–320.

[18] Noah E Friedkin and Eugene C Johnsen. 1990. Social influence and opinions. *Journal of Mathematical Sociology* 15, 3-4 (1990), 193–206.

[19] Kiran Garimella, Gianmarco De Francisci Morales, Aristides Gionis, and Michael Mathioudakis. 2017. Reducing controversy by connecting opposing views. In *Proceedings of the tenth ACM international conference on web search and data mining*. 81–90.

[20] Kiran Garimella, Aristides Gionis, Nikos Parotsidis, and Nikolaj Tatti. 2017. Balancing information exposure in social networks. *Advances in neural information processing systems* 30 (2017).

[21] Kiran Garimella, Gianmarco De Francisci Morales, Aristides Gionis, and Michael Mathioudakis. 2018. Quantifying controversy on social media. *ACM Transactions on Social Computing* 1, 1 (2018), 1–27.

[22] Aristides Gionis, Evimaria Terzi, and Panayiotis Tsaparas. 2013. Opinion maximization in social networks. In *Proceedings of the 2013 SIAM International Conference on Data Mining*. SIAM, 387–395.

[23] Pedro Guerra, Wagner Meira Jr, Claire Cardie, and Robert Kleinberg. 2013. A measure of polarization on social media networks based on community boundaries. In *Proceedings of the international AAAI conference on web and social media*, Vol. 7. 215–224.

[24] Shahrzad Haddadan, Cristina Menghini, Matteo Riondato, and Eli Upfal. 2021. RePBubLik: Reducing Polarized Bubble Radius with Link Insertions. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining* (Virtual Event, Israel) *(WSDM '21)*. Association for Computing Machinery, New York, NY, USA, 139âĂŞ147. https://doi.org/10.1145/3437963.3441825

[25] Thibaut Horel and Yaron Singer. 2016. Maximization of Approximately Submodular Functions. In *Proceedings of the 30th International Conference on Neural Information Processing Systems* (Barcelona, Spain) *(NIPS'16)*. Curran Associates Inc., Red Hook, NY, USA, 3053âĂŞ3061.

[26] Oded Kariv and S Louis Hakimi. 1979. An algorithmic approach to network location problems. I: The p-centers. *SIAM journal on applied mathematics* 37, 3 (1979), 513–538.

[27] John G Kemeny and J Laurie Snell. 1983. *Finite Markov chains: with a new appendix "Generalization of a fundamental matrix"*. Springer.

[28] David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the spread of influence through a social network. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*. 137–146.

[29] Gregory F Lawler. 1986. Expected hitting times for a random walk on a connected graph. *Discrete Mathematics* 61, 1 (1986), 85–92.

[30] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. http://snap.stanford.edu/data.

[31] David A Levin and Yuval Peres. 2017. *Markov chains and mixing times*. Vol. 107. American Mathematical Soc.

[32] Edo Liberty and Maxim Sviridenko. 2017. Greedy Minimization of Weakly Supermodular Set Functions. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2017) (LIPIcs, Vol. 81)*. Dagstuhl, Germany, 19:1–19:11.

[33] Edo Liberty and Maxim Sviridenko. 2017. Greedy minimization of weakly supermodular set functions. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques (APPROX/RANDOM 2017)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.

[34] László Lovász. 1993. Random walks on graphs. *Combinatorics, Paul erdos is eighty* 2, 1-46 (1993), 4.

[35] Yao Ma, Suhang Wang, Tyler Derr, Lingfei Wu, and Jiliang Tang. 2021. Graph adversarial attack via rewiring. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1161–1169.

[36] Antonis Matakos, Cigdem Aslay, Esther Galbrun, and Aristides Gionis. 2020. Maximizing the diversity of exposure in a social network. *IEEE Transactions on Knowledge and Data Engineering* 34, 9 (2020), 4357–4370.

[37] Sourav Medya, Arlei Silva, Ambuj Singh, Prithwish Basu, and Ananthram Swami. 2018. Group centrality maximization via network design. In *Proceedings of the 2018 SIAM International Conference on Data Mining*. SIAM, 126–134.

[38] Cristina Menghini, Aris Anagnostopoulos, and Eli Upfal. 2020. Auditing Wikipedia's Hyperlinks Network on Polarizing Topics. *arXiv preprint arXiv:2007.08197* (2020).

[39] Adam Meyerson and Brian Tagiku. 2009. Minimizing average shortest path distances via shortcut edge addition. In *Approximation, Randomization, and Combinatorial Optimization. Algorithms and Techniques: 12th International Workshop, APPROX 2009, and 13th International Workshop, RANDOM 2009, Berkeley, CA, USA, August 21-23, 2009. Proceedings*. Springer, 272–285.

[40] Marco Minici, Federico Cinus, Corrado Monti, Francesco Bonchi, and Giuseppe Manco. 2022. Cascade-based echo chamber detection. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management*. 1511–1520.

[41] Michel Minoux. 2005. Accelerated greedy algorithms for maximizing submodular set functions. In *Optimization Techniques: Proceedings of the 8th IFIP Conference on Optimization Techniques Würzburg, September 5–9, 1977*. Springer, 234–243.

[42] Cameron Musco, Christopher Musco, and Charalampos E Tsourakakis. 2018. Minimizing polarization and disagreement in social networks. In *Proceedings of the 2018 world wide web conference*. 369–378.

[43] George L Nemhauser, Laurence A Wolsey, and Marshall L Fisher. 1978. An analysis of approximations for maximizing submodular set functionsâĂŤI. *Mathematical programming* 14 (1978), 265–294.

[44] Rina Panigrahy and Sundar Vishwanathan. 1998. An $o(\log^* n)$ Approximation Algorithm for the Asymmetric p-Center Problem. *Journal of Algorithms* 27, 2 (1998), 259–268.

[45] Nikos Parotsidis, Evaggelia Pitoura, and Panayiotis Tsaparas. 2015. Selecting shortcuts for a smaller world. In *Proceedings of the 2015 SIAM International Conference on Data Mining*. SIAM, 28–36.

[46] Pan Peng, Daniel Lopatta, Yuichi Yoshida, and Gramoz Goranci. 2021. Local Algorithms for Estimating Effective Resistance. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1329–1338.

[47] Yuval Peres. 2022. Personal communication. (2022).

[48] Manoel Horta Ribeiro, Raphael Ottoni, Robert West, Virgílio AF Almeida, and Wagner Meira Jr. 2020. Auditing radicalization pathways on YouTube. In *Proceedings of the 2020 conference on fairness, accountability, and transparency*. 131–141.

[49] Maxim Sviridenko, Jan Vondrák, and Justin Ward. 2017. Optimal approximation for submodular and supermodular optimization with bounded curvature. *Mathematics of Operations Research* 42, 4 (2017), 1197–1218.

[50] Sijing Tu, Cigdem Aslay, and Aristides Gionis. 2020. Co-exposure Maximization in Online Social Networks. In *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. 3232–3243.

[51] Liwang Zhu, Qi Bao, and Zhongzhi Zhang. 2021. Minimizing polarization and disagreement in social networks via link recommendation. *Advances in Neural Information Processing Systems* 34 (2021), 2072–2084.

[52] Liwang Zhu and Zhongzhi Zhang. 2022. A Nearly-Linear Time Algorithm for Minimizing Risk of Conflict in Social Networks. In *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2648–2656.